

O'REILLY®

2nd Edition

Python for Excel

A Modern Environment for Automation
and Data Analysis



Felix Zumstein

"Python for Excel is the clearest, most practical guide to the modern Excel-Python stack. It shows not just how the tools work, but why they meaningfully expand what Excel users can do."

George Mount, Microsoft Excel MVP, author of *Modern Data Analytics in Excel*

"Felix explains the material incredibly well and simply, making it easy to learn. The level of thought and detail that he demonstrates is evident throughout the book. Concise examples will get you working quickly. He calls out the gotchas to avoid. A delightfully well-written book."

Cody Paustell, programmer/developer, Rubberduck-VBA contributor

Python for Excel

Professionals in nearly every industry use Excel to create powerful tools that can adapt quickly to fast-changing environments. But Excel's automation language, VBA, stopped evolving a long time ago. Python, on the other hand, has exploded in popularity, so Python in Excel, Microsoft's official Python integration, came as no surprise. This thoroughly revised hands-on guide helps you unlock Excel's full potential using Python.

Analysts, engineers, data scientists, and other power users will learn how to extend Excel's powerful analytics engine with Python. To make the most out of this combination, author Felix Zumstein dives into various solutions, including Python in Excel, xlwings, and OpenPyXL.

- Learn how to use Python's core features
- Get started with modern tools, including Jupyter notebooks and Visual Studio Code
- Apply powerful pandas commands to acquire, clean, and analyze data
- Use advanced analytics via pandas in Excel, using Python in Excel
- Automate Excel and write custom functions with Python, using xlwings and xlwings Lite
- Manipulate Excel files with Python, using XlsxWriter and OpenPyXL

Felix Zumstein is an independent software developer and the founder of Zoomer Analytics GmbH. He pioneered the integration of Python and Excel with his open source project xlwings.

DATA / PYTHON

US \$59.99 CAN \$74.99

ISBN: 979-8-341-64029-0



5 5 9 9



Praise for *Python for Excel*

Python and Excel are themselves daunting tools to learn. But Felix Zumstein brings them together with clarity—demonstrating how to go beyond basic automation to create powerful analytical tools in Excel. If you would like to take your Excel spreadsheets to the next level, you need to read this book.

—Jordan Goldmeier, author, Microsoft MVP

Excel and Python are two of the most important tools in the business analytics toolbox, and together they are far greater than the sum of their parts. In this book, Felix Zumstein lays out his unparalleled mastery of the many ways to connect Python and Excel using open source, cross-platform solutions. It will be an invaluable tool for business analysts and data scientists alike, and any Python user looking to harness the power of Excel in their code.

—Daniel Guetta, professor of professional practice and
director of the Business Analytics Initiative at Columbia Business School
and coauthor of *Python for MBAs*

Python for Excel is the clearest, most practical guide to the modern Excel-Python stack. It shows not just how the tools work, but why they meaningfully expand what Excel users can do.

—George Mount, Microsoft Excel MVP,
author of *Modern Data Analytics in Excel*

Felix explains the material incredibly well and simply, making it easy to learn. The level of thought and detail that he demonstrates is evident throughout the book. Concise examples will get you working quickly. He calls out the gotchas to avoid. A delightfully well-written book.

—Cody Paustell, programmer/developer,
Rubberduck-VBA contributor

SECOND EDITION

Python for Excel

*A Modern Environment for
Automation and Data Analysis*

Felix Zumstein

O'REILLY®

Python for Excel

by Felix Zumstein

Copyright © 2026 Zoomer Analytics GmbH. All rights reserved.

Published by O'Reilly Media, Inc., 141 Stony Circle, Suite 195, Santa Rosa, CA 95401.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: Michelle Smith
Development Editor: Melissa Potter
Production Editor: Ashley Stussy
Copyeditor: Piper Content Partners
Proofreader: Rachel Rossi

Indexer: nSight, Inc.
Cover Designer: Susan Brown
Cover Illustrator: Karen Montgomery
Interior Designer: David Futato
Interior Illustrator: Kate Dullea

March 2021: First Edition
May 2026: Second Edition

Revision History for the Second Edition

2026-05-22: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9798341640290> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Python for Excel*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author, and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

979-8-341-64029-0

[LSI]

Table of Contents

Preface.....	xiii
--------------	------

Part I. Introduction to Python

1. Why Python for Excel?.....	3
Python Core Principles	4
General-Purpose Language	4
Readability Counts	4
Batteries Included	5
Cross-Platform	6
Scientific Computing	6
Modern Language Features	8
Working with Text	8
Array Support	8
Web APIs	9
Error Handling	9
Tooling	10
Package Manager	10
Testing	11
Version Control	11
Conclusion	13
2. Development Environment.....	15
Companion Repository	15
Using the Terminal	17
Installing Python	21
Installing uv	21

Installing Python and Packages	22
Python REPL	23
Jupyter Notebooks	24
Running Jupyter Notebooks	25
Notebook Cells	27
Edit versus Command Mode	29
Run Order Matters	30
Visual Studio Code	31
Installation and Configuration	33
Running a Python Script	35
Debugger	38
Running Jupyter Notebooks with VS Code	39
Conclusion	41
3. Getting Started with Python.....	43
Data Types	44
Objects	44
Numeric Types	45
Comments	47
Booleans	48
Strings	49
Indexing and Slicing	51
Indexing	51
Slicing	52
Data Structures	53
Lists	53
Dictionaries	55
Tuples	56
Sets	57
Control Flow	58
Code Blocks and the pass Statement	58
The if Statement and Conditional Expressions	59
The for and while Loops	60
List Comprehensions	63
Code Organization	64
Functions	64
Modules and the import Statement	66
The datetime Class	68
Code Formatting, Linting, and Type Hinting	70
PEP 8: Style Guide for Python Code	70
Ruff	72
Type Hints	74

Part II. Introduction to pandas

4. NumPy Foundations.....	79
Getting Started with NumPy	79
NumPy Array	79
Vectorization and Broadcasting	81
Universal Functions (ufunc)	83
Creating and Manipulating Arrays	84
Getting and Setting Array Elements	84
Useful Array Constructors	85
View Versus Copy	86
Conclusion	87
5. Data Analysis with pandas.....	89
DataFrame and Series	89
Index	92
Columns	94
Data Manipulation	96
Selecting Data	96
Setting Data	101
Missing Data	104
Duplicate Data	105
Arithmetic Operations	106
Working with Text	108
Combining DataFrames	109
Concatenating	109
Joining and Merging	110
Descriptive Statistics and Data Aggregation	113
Descriptive Statistics	113
Grouping	114
Pivot Tables	115
Plotting	117
Matplotlib	117
Plotly	118
Importing and Exporting DataFrames	121
Exporting CSV Files	121
Importing CSV Files	122
Modern Alternatives	125
Conclusion	126

Part III. Python in Excel

6. Getting Started with Python in Excel.....	129
Requirements	129
Writing Python Code in Excel	130
Python Cells	130
Output Types	132
The xl Function	134
Working with pandas DataFrames	137
Cell Organization	139
Python Editor	140
Plotting	142
Python in Excel Under the Hood	144
Initialization Script	144
Row-Major Order of Cell Execution	144
A DataFrame Roundtrip	145
Settings	147
Keyboard Shortcuts	147
Limitations	148
Conclusion	151
7. Time Series Analysis with pandas.....	153
DatetimeIndex	154
Creating a DatetimeIndex	154
Filtering a DatetimeIndex	157
Common Time Series Manipulations	158
Shifting and Percentage Changes	158
Rebasing and Correlation	160
Resampling	163
Rolling Windows	165
Loading Data with Power Query	166
Loading a Single File	166
Loading Files from a Folder	169
Conclusion	171

Part IV. xlwings

8. Excel Automation.....	175
Getting Started with xlwings	175
Using Excel as Data Viewer	176
The Excel Object Model	177

Running VBA Code	184
Converters, Options, and Collections	186
Working with DataFrames	186
Converters and Options	188
Charts, Pictures, and Defined Names	190
Reporting Case Study (Take 1)	194
Advanced xlwings Topics	199
Improving Performance	199
Missing Functionality	200
Conclusion	201
9. Python-Powered Excel Tools.....	203
Using Excel as Frontend	203
xlwings Add-in	204
Quickstart Command	206
Run Main	206
RunPython Function	208
Deployment	212
Python Dependency	212
Standalone Workbooks	213
xlwings Configuration	214
Conclusion	216
10. The Python Package Tracker.....	217
Taking It for a Spin	217
Core Functionality	220
Web APIs	220
Databases	223
Exceptions	231
Application Walkthrough	234
Frontend	234
Backend	238
Debugging	242
Conclusion	243
11. Custom Functions.....	245
Getting Started with Custom Functions	245
Prerequisites	246
Hello World	247
Working with DataFrames	251
Converters and Options	251
Fetching Data from the Web	257

Plotting	260
Debugging	262
Performance Optimization	264
Minimize Cross-Application Calls	264
Caching	265
The Script Decorator	267
Conclusion	269

Part V. xlwings Lite

12. Scripts with xlwings Lite.....	273
Getting Started with xlwings Lite	274
How xlwings Lite Works	274
Installation	275
A User Interface Tour	276
Running Your First Script	278
Sheet Buttons	279
Analyzing the NYC Taxi Dataset	280
Exploring the Dataset with DuckDB	280
Analyzing Trips and Distance by Hour	284
Platform Specifics	286
Making Web API Requests	286
Installing Python Packages	289
Limitations	290
Conclusion	292
13. Custom Functions with xlwings Lite.....	293
Modern Custom Functions	293
Hello World	294
Using Type Hints as Converters	295
Error Handling	298
Organizing Functions with Namespaces	299
Building an AI Function with OpenAI	301
Managing Credentials with Environment Variables	301
The AI Function	304
Performing Sentiment Analysis with Hugging Face	307
Machine Learning Terminology	308
The CLASSIFY Function	309
Plotting Weather Data with Open-Meteo	311
Working with Dates	311
Plotting Temperature over Time	313

Part VI. Reading and Writing Excel Files Without Excel

14. Excel File Manipulation with pandas.....	319
Reporting Case Study (Take 2)	319
Reading and Writing Excel Files	322
The read_excel Function and ExcelFile Class	323
Reading Sheets in Parallel	327
The to_excel Method and ExcelWriter Class	329
Limitations	331
Conclusion	332
15. Excel File Manipulation with Reader and Writer Packages.....	333
Reading and Writing Excel Files	333
Calamine-Based Excel Readers	335
python-calamine	335
fastexcel	336
XlsxWriter	337
Getting Started with XlsxWriter	337
Writing Large Files with XlsxWriter	340
OpenPyXL	341
Reading with OpenPyXL	341
Reading Large Files with OpenPyXL	343
Writing with OpenPyXL	343
Writing Large Files with OpenPyXL	346
Editing with OpenPyXL	346
More Reader and Writer Packages	347
Excelize	347
pyxlsb	348
xlrd, xlwt, and xlutils	350
Writing Formatted DataFrames to Excel	354
Formatting the Index and Headers of a DataFrame	355
Formatting the Data Part of a DataFrame	356
Reporting Case Study (Take 3)	357
Conclusion	358
A. The uv Package Manager.....	361
B. Copilot in Excel.....	367

C. Advanced Python Concepts..... 373

Index..... 381

Preface

“Yeah, so I helped the Excel team with this. Excited that it’s out!” This was the message that Guido van Rossum—the creator of Python and, at the time, a Microsoft employee—posted on X on August 22, 2023. His **post** included a link to the official announcement of Python in Excel. So how did we get here?

In the early days of Excel, Visual Basic for Applications (VBA) was the only choice to program and extend Microsoft Excel as an end user. VBA is a one-stop shop, as it allows you to:

- Write macros that run at the click of a button, e.g., to add a new worksheet with a report
- Write custom functions (also called *user-defined functions*) that you can call from worksheet cells
- Build your own add-ins to provide automation features and custom functions across workbooks

With the growing importance of the internet and mobile platforms, Microsoft had to make Excel work across these new environments. Today, Excel runs on Windows, macOS, Android, iOS, and—most importantly—on the web. However, only Windows and macOS support VBA. Rather than extending VBA support to the other platforms, Microsoft created new tools:

- *Power Query* can import and transform data and uses the M language.
- *Power Pivot* is an add-in for data modeling that uses Data Analysis Expressions (DAX) as its formula language.

- *Office Scripts* uses TypeScript and is the solution that comes closest to VBA. However, it doesn't support custom functions and requires a Microsoft 365 subscription.
- *Lambda functions* use Excel's own formula language to create custom functions.

So what used to be a one-stop shop is now a fragmented ecosystem where each tool uses a different programming language. Python in Excel adds yet another option to this list: it gives analysts access to a more powerful analytics and visualization engine, but it doesn't support Excel automation or custom functions.

Python, however, is a general-purpose programming language, and using it only for analytics and visualization is leaving a lot of its potential untapped. That's why this book doesn't stop at Python in Excel. Instead, I will show you how to use Python as a modern VBA replacement by leveraging its entire ecosystem.

How This Book Is Organized

This book is organized into six parts. The first two parts teach you the fundamentals of Python and pandas, so you'll need a bit of patience until we start using Python and Excel together in **Part III**. This is intentional: if you want the fast track, there are plenty of resources that jump straight into Python in Excel or pandas. However, as soon as you need to adapt or debug the code from these tutorials, it's easy to get stuck. By building a solid foundation in **Parts I** and **II**, you'll understand the concepts and tools you'll rely on in later chapters. This approach takes a bit longer upfront, but it will save you time and frustration in the long run.

Part I, "Introduction to Python"

Chapter 1 looks into the reasons why Python is such a good companion for Excel. **Chapter 2** introduces the tools we'll be using in this book: the uv package manager, the Terminal, Visual Studio Code, and Jupyter notebooks. **Chapter 3** will teach you enough Python to be able to follow the rest of this book.

Part II, "Introduction to pandas"

Chapter 4 gives you a very short introduction to NumPy, which is the backbone of many scientific packages, including pandas. pandas, the topic of **Chapter 5**, is one of the most popular Python libraries for data analysis. Its core data structure, the DataFrame, is similar to a spreadsheet and can serve as a programmatic alternative to an Excel workbook.

Part III, "Python in Excel"

Chapter 6 covers the basics of Python in Excel, the official Microsoft solution. Python in Excel is centered around the concept of a two-dimensional Jupyter notebook that unlocks better analytics and visualization by giving you access to the broader pandas ecosystem. **Chapter 7** teaches you how to use pandas with

time series. For this chapter, you can choose to follow the code samples with either Python in Excel or with a classic Jupyter notebook.

Part IV, “xlwings”

In **Chapter 8**, I’ll introduce the xlwings package to automate the Excel application from Python. For example, you will learn how to open and edit Excel workbooks programmatically. **Chapter 9** is about building interactive Excel tools, which allow you to click a button to run Python code. **Chapter 10** brings everything you’ve learned so far together to build a typical database-backed business application. In **Chapter 11**, you’ll write custom functions that you can call from Excel cells just like built-in functions such as SUM.

Part V, “xlwings Lite”

Chapter 12 introduces xlwings Lite that gives you access to the xlwings package without the need of a local Python installation. Instead, you install xlwings Lite from Excel’s add-in store. **Chapter 13** dives into writing custom functions with xlwings Lite. In this part, you already know the basics of Python, pandas, and xlwings. This allows us to explore a few interesting projects: we’ll analyze 3.5 million taxi rides, build an alternative to Microsoft’s COPILOT function, and run a pretrained machine learning model for sentiment analysis.

Part VI, “Reading and Writing Excel Files Without Excel”

This part is different from the rest of the book, as it doesn’t require an installation of Excel. You’ll learn how to read and write Excel workbooks directly on disk. **Chapter 14** introduces you to the reading and writing features that pandas offers. Under the hood, pandas depends on other packages, so called *engines*, that do the actual work. How you can use these engines directly is the topic of **Chapter 15**, where we’ll look at packages like OpenPyXL and XlsxWriter, among others.

What’s New in the Second Edition

It’s been five years since the first edition. In that time, Microsoft came out with Python in Excel, and xlwings Lite hit Excel’s add-in store. Besides that, artificial intelligence (AI) has become mainstream with the release of ChatGPT, Microsoft Copilot, and numerous other AI tools. Accordingly, this edition includes three new chapters and one new appendix:

- **Chapter 6** introduces Python in Excel.
- **Chapter 12** introduces xlwings Lite and its automation capabilities.
- **Chapter 13** shows how custom functions work with xlwings Lite.
- **Appendix B** introduces Copilot in Excel.

Other than that, I changed the order of chapters slightly:

- **Chapter 7, “Time Series Analysis with pandas”** was previously in the pandas part but has been moved to the Python in Excel part so that you can directly compare a classic Jupyter notebook with the Python in Excel implementation.
- **Part VI, “Reading and Writing Excel Files Without Excel”** has been moved to the very end. This has the advantage that the parts related to the focus of the book, interacting with Python from the Excel application, are kept together while the fundamentally different concept of directly manipulating files comes last.

This edition also adds a few new projects that make use of modern technologies, including DuckDB, Parquet files, pretrained machine learning models from Hugging Face, and OpenAI API. Besides that, the Python version has been updated from 3.8 to 3.14, and pandas has been updated from version 1.1.3 to 3.0.0. And finally, while the first edition used the Anaconda Python distribution, this edition uses the uv package manager instead to install Python and its packages.

Who This Book Is For

If you are an advanced Excel user who wants to push beyond the limits of Excel with a modern programming language, this book is for you. Most typically, this means that you spend hours every month downloading, cleaning, and copy/pasting big amounts of data into mission-critical spreadsheets. While there are different ways to overcome Excel’s limits, this book will focus on how to use Python for this task.

You should have a basic understanding of programming: it helps if you have already written a function or a for loop (no matter in which programming language) and have an idea about what an integer or a string is. However, you might still be able to follow this book even if you are just used to writing complex cell formulas or have experience with tweaking recorded VBA macros or Office Scripts. You are not expected to have any Python-specific experience, though, as I introduce all the tools that we will use, including an introduction to Python itself.

If you are a seasoned VBA developer, you will find regular comparisons between Python and VBA that will help you avoid common gotchas.

This book can also be helpful if you are a Python developer and need to learn about the different ways that Python can deal with the Excel application and Excel files to be able to pick the right package given the requirements of your business users.

Why I Wrote This Book

Through my work on xlwings, the Excel automation package that we will meet in **Part IV** and **Part V** of this book, I am in close contact with many users who use

Python for Excel—whether that’s via the [issue tracker](#) on GitHub, a question on [Stack Overflow](#), or at a physical event like a meetup or a conference.

On a regular basis, I am asked to recommend resources to get started with Python. While there is certainly no shortage of Python introductions, they are often either too general (nothing about data analysis) or too specific (full scientific introductions). However, Excel users tend to be somewhere in the middle: they certainly work with data, but a full scientific introduction may be too technical. They also often have specific requirements and questions that aren’t answered in any of the existing material. Some of these questions are:

- Which Python-Excel package do I need for which task?
- How do I move my Power Query database connection over to Python?
- What’s the equivalent of an Excel PivotTable in Python?

I wrote this book to get you from zero Python knowledge to automating your Excel-centric tasks and leveraging Python’s scientific ecosystem in Excel without any detours.

Python and Excel Versions

This book has been tested to work with Python 3.14 and expects you to use a modern version of Excel. Ideally, you would use the version that comes with Microsoft 365, or if you use Excel with a permanent license, you should use Excel 2021 or later. If a chapter requires a specific version of Excel, I will point this out at the beginning of the chapter.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

Constant width bold

Shows commands or other text that should be typed literally by the user.

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.



This element signifies a tip or suggestion.



This element signifies a general note.



This element indicates a warning or caution.

Using Code Examples

I am maintaining a [web page](#) with additional information to help you with this book. Make sure to check it out, especially if you run into an issue.

Supplemental material (code examples, Excel workbooks, etc.) is available for download on <https://oreil.ly/github-pfe2e>. I will provide detailed instructions on how to download this companion repository in [Chapter 2](#).

If you have a technical question or a problem using the code examples, please send an email to bookquestions@oreilly.com.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*Python for Excel*, Second Edition by Felix Zumstein (O’Reilly). Copyright 2026 Zoomer Analytics GmbH, 979-8-341-64029-0.”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O’Reilly Online Learning

O’REILLY® For more than 40 years, **O’Reilly Media** has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O’Reilly’s online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O’Reilly and 200+ other publishers. For more information, visit <http://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O’Reilly Media, Inc.
141 Stony Circle, Suite 195
Santa Rosa, CA 95401
800-889-8969 (in the United States or Canada)
707-827-7019 (international or local)
707-829-0104 (fax)
support@oreilly.com
<https://oreilly.com/about/contact.html>

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <https://oreil.ly/py4excel-2e>.

For more information about our books, courses, conferences, and news, see our website at <http://www.oreilly.com>.

Follow us on LinkedIn: <https://linkedin.com/company/oreilly>.

Watch us on YouTube: <http://youtube.com/oreillymedia>.

Acknowledgments

I am incredibly grateful to the many people who helped me write this book.

At O'Reilly, I would like to thank my editor, Melissa Potter, who helped bring this book into a readable form. I'd also like to thank Michelle Smith, Ashley Stussy, Jonathon Owen, and Daniel Elfanbaum.

I was very lucky that the book was reviewed by highly qualified tech reviewers, and I really appreciate the hard work they put in under a lot of time pressure. Thanks for all your help, Jordan Goldmeier, George Mount, Cody Paustell, Andreas Clenow, Werner Brönnimann, Björn Stiel, and Eric Moreira!

A big thank you goes to all my colleagues, friends, and early release readers who read the earliest forms of my drafts: Adam Rodriguez, Mano Beeslar, Simon Schiegg, Rui Da Costa, Jürg Nager, Christophe de Montrichard, Felipe Maion, Ray Doue, Kolyu Minevski, Scott Drummond, Volker Roth, and David Ruggles.

PART I

Introduction to Python

Why Python for Excel?

Many Excel users start to reconsider their spreadsheet tools when they hit a limitation. A classic example is when Excel workbooks contain so much data and so many formulas that they become slow or in the worst case, crash. Don't let it get that far! If you work on mission-critical workbooks where errors can result in financial or reputational damage, or if you spend hours updating Excel workbooks manually, you should add a programming language to your tool belt. A programming language allows you to automate error-prone copy/paste processes and can turn unreadable Excel formulas into maintainable functions. It can also connect your spreadsheet to external systems such as a database, allowing you to trim down your workbook and make it fast again. This book uses Python as the programming language for Excel, so let's take a quick look at the origins of our two main characters!

Given how fast technology moves, Excel and Python have both been around for a remarkably long time: Excel was first launched in 1985 by Microsoft—and this may come as a surprise—it was only available for Apple Macintosh. Windows users had to wait until 1987 when Excel 2.0 came out. Microsoft wasn't the first player in the spreadsheet market, though: VisiCorp came out with VisiCalc in 1979, followed by Lotus Software in 1983 with Lotus 1-2-3. Even Microsoft had another application in the game: three years before Excel, they released Multiplan, a spreadsheet program for MS-DOS.

Python was born in 1991, just six years after Excel. While Excel became popular early on, Python had a longer journey before it got traction as a scripting language and in web development. In 2005, Python started to become a serious alternative for scientific computing when *NumPy*, a package for array-based computing and linear algebra, was released. NumPy combined two predecessor packages and therefore streamlined all development efforts around scientific computing into a single project. Today, it forms the basis of countless scientific packages, including *pandas*, which

came out in 2008 and is largely responsible for the widespread adoption of Python in the world of finance and data science. Today, thanks to the growing scientific ecosystem, Python has become one of the most commonly used languages for machine learning (ML) and artificial intelligence (AI).

In this chapter, I will point out what the advantages of Python are compared to Excel's traditional programming language, VBA (Visual Basic for Applications). I will focus on Python's core principles, scientific computing capabilities, and its modern language features. To conclude, I will introduce a few tools that you will have access to as a Python developer.

Python Core Principles

Like every programming language, Python has a few core principles that shaped it into what it is today:

- General-purpose language: Python can be used for many different purposes.
- Readability counts: Python's syntax is easy to read.
- Batteries included: Python comes with a comprehensive standard library.
- Cross-platform: Python runs on all major platforms.

Let's explore each of these principles and draw comparisons with VBA.

General-Purpose Language

Python is a general-purpose programming language, meaning it wasn't designed for a specific domain like web development or data analysis: Python can be used for many different purposes, including Excel automation, ML, and large-scale web applications. This also means that you can turn your Python-powered Excel tools into fully-fledged web applications without having to rewrite the code with the business logic. In contrast, VBA was designed as a domain-specific language to automate and extend Microsoft Office applications. Consequently, VBA code can't be used outside the Microsoft ecosystem—if you want to turn your Excel tool into a web app, you'll need to rewrite it in another language.

Beyond being general-purpose, Python was designed with another key principle in mind: readability.

Readability Counts

If your code is readable, it is easy to follow and understand. This makes it easier to spot errors and maintain the code going forward. And maintenance is where most time is spent in programming rather than writing the code in the first place. That's why one line in [The Zen of Python](#) is “readability counts.” The Zen of Python is a

concise summary of Python’s core design principles, and we will learn how to print it in the next chapter. Let’s have a look at the following VBA code snippet:

```
If i < 5 Then
    Debug.Print "i is less than 5"
ElseIf i <= 10 Then
    Debug.Print "i is greater than 5 but less than or equal to 10"
Else
    Debug.Print "i is greater than 10"
End If
```

In VBA, you can reformat the code into the following, which is completely equivalent:

```
If i < 5 Then
    Debug.Print "i is less than 5"
ElseIf i <= 10 Then
    Debug.Print "i is greater than 5 but less than or equal to 10"
Else
    Debug.Print "i is greater than 10"
End If
```

In the first version, the visual indentation aligns with the logic of the code. This makes it easy to read and understand. In the second version, it’s much harder to spot the ElseIf and Else conditions when you glance over it.

To enforce readability, Python doesn’t accept code that is formatted like the second example: it forces you to align the visual indentation with the logic of the code. To do this, Python relies on indentation to define code blocks as you use them in `if` statements. Instead of indentation, the majority of programming languages use curly braces to define code blocks, and VBA uses keywords such as `End If`, as we’ve just seen.

You will learn all about Python’s indentation rules in [Chapter 3](#). Now, however, I’ll explain what it means when they say that Python comes with batteries included.

Batteries Included

Python comes with a rich set of built-in functionality delivered by its *standard library*. The Python community likes to refer to it by saying that Python comes with “batteries included.” Whether you need to uncompress a ZIP file, read the values of a CSV file, or want to send an email, Python’s standard library has you covered, and you can achieve all this in usually just a few lines of code. With VBA, you are often required to write a lot of code or install an add-in for these basic tasks. Another issue with VBA is that it isn’t truly cross-platform. Let’s see why this matters!

Cross-Platform

Even though Excel VBA runs on Windows and macOS, it's easy to introduce functionality that only runs on Windows. In the official VBA documentation or on community forums, you will often see code like this:

```
Set fso = CreateObject("Scripting.FileSystemObject")
```

Whenever you have a `CreateObject` call or are being told to add a reference in the VBA editor, you are usually dealing with code that will only run on Windows.

In contrast, Python runs on all major platforms, including Windows, macOS, and Linux. That's important, because even if you develop your programs on a laptop that runs on Windows or macOS, it's likely that one day you will want to run your application on a server or in the cloud. Servers allow you to access your application over the web or run your Python script fully automated as part of a pipeline. In fact, I will show you how to run Python code on a server in the next chapter by introducing you to hosted Jupyter notebooks. The vast majority of servers run on Linux, an operating system that is stable, secure, and open source. And since Python programs run on all major operating systems, this will take out much of the pain when you transition from your local machine to a server or cloud setup.

With Python's core principles covered, let's now find out why scientific computing has become so big in Python.

Scientific Computing

As we've seen before, an important reason for Python's success is the fact that it was created as a general-purpose programming language. The capabilities for scientific computing were added later on in the form of third-party packages. This has the unique advantage that a data scientist and a web developer can work with the same programming language. Being able to build scientific applications out of a single language reduces friction, implementation time, and costs. Scientific packages like NumPy and pandas give us access to a very concise way of formulating mathematical problems. As an example, let's have a look at a famous financial formula from Modern Portfolio Theory, used to calculate the portfolio variance:

$$\sigma^2 = w^T C w$$

The portfolio variance is denoted by σ^2 , while w is the weight vector of the individual assets and C is the portfolio's covariance matrix. If w and C are Excel ranges, you can calculate the portfolio variance in VBA like so:

```
variance = Application.MMult(Application.MMult(Application.Transpose(w), C), w)
```


Compare this to the almost mathematical notation in Python, assuming that `w` and `C` are pandas DataFrames or NumPy arrays (I will formally introduce them in [Part II](#)):

```
variance = w.T @ C @ w
```

But it's not just about aesthetics and readability: NumPy and pandas use compiled Fortran and C code under the hood, which is much faster than VBA when working with big matrices. Other packages that we'll use in this book are written in Rust or C++ for performance reasons, but no matter what low-level language is used, Python provides the tools to create user-friendly packages. This allows you to take advantage of high-performance libraries without needing to understand their underlying implementation.

Thanks to the growing scientific ecosystem, Python has become the de facto lingua franca for ML and AI. All important ML frameworks—such as scikit-learn, TensorFlow, PyTorch, or XGBoost—are available as Python packages. And AI platforms such as ChatGPT offer a Python library to interact with their service programmatically.

Open Source Software (OSS)

Python is open source software (OSS). If software is distributed under an open source license, it means that its source code is freely available at no cost, allowing everybody to contribute new functionality, bug fixes, or documentation. Python itself and almost all third-party Python packages are open source and most commonly maintained by developers in their spare time. This is not always an ideal state: if your company is relying on certain packages, you have an interest in the continued development and maintenance of these packages by professional programmers. Fortunately, the scientific Python community has recognized that some packages are too important to leave their fate in the hands of a few volunteers.

That's why in 2012, **NumFOCUS**, a nonprofit organization, was created to sponsor various Python packages and projects in the area of scientific computing. NumFOCUS sponsors many popular Python packages, including pandas and NumPy, but nowadays they also support projects from various other languages, including R, Julia, and JavaScript. Similarly, Python is overseen by the Python Software Foundation that welcomes new members and donors.

Missing support for scientific computing is an obvious limitation in VBA. But even if you look at the core language features, VBA has fallen behind.

Modern Language Features

Since Excel 2000, the VBA language hasn't been actively developed anymore. This, however, doesn't mean it's dead: Microsoft is still shipping updates to VBA to be able to automate new Excel features. For example, Excel 2016 added support to automate Power Query. Since VBA stopped evolving such a long time ago, it's missing out on modern language concepts that were introduced in all major programming languages over the years. In this section, we're going to look at some of them, including text and error handling, array support, and web APIs.

Working with Text

VBA has very limited tools to work with strings, i.e., text data. Let's have a look at the following sentence:

This was printed by a language called "VBA".

To print this with the language name stored in a variable, you would write the following code in VBA:

```
lang = "VBA"
message = "This was printed by a language called "" & lang & ""."
Debug.Print message
```

This is a very short string, but it's already hard to know how many quotes you have to put where. Python, on the other hand, has powerful f-strings, which we will cover in [Chapter 3](#). The same example looks like this in Python:

```
lang = "Python"
message = f'This was printed by a language called "{lang}".'
print(message)
```

Just like with text, handling arrays in VBA comes with its own set of challenges, as we will see next.

Array Support

Working with arrays in VBA is hard and verbose. For example, multiplying all values in an array by 2 looks something like this:

```
Dim numbers(1 To 4) As Integer
Dim doubled() As Integer
Dim i As Integer

' Initialize array
numbers(1) = 1
numbers(2) = 2
numbers(3) = 3
numbers(4) = 4
```

```

' Double each element
ReDim doubled(LBound(numbers) To UBound(numbers))
For i = LBound(numbers) To UBound(numbers)
    doubled(i) = numbers(i) * 2
Next i

```

In Python, you can write the same in two lines of code (I will show you how this works in [Chapter 4](#)):

```

import numpy as np

numbers = np.array([1, 2, 3, 4])
doubled = numbers * 2

```

Python makes working with arrays easy. The same is true for web APIs, our next topic.

Web APIs

Microsoft stopped investing in VBA around the time the internet became mainstream. It's therefore not surprising that downloading data from a web API is a cumbersome task in VBA, especially if you want it to work across Windows and macOS.

In contrast, Python has various packages that make it easy to talk to a web API. Since web APIs usually send their response in the JSON (JavaScript Object Notation) format, accessing the information of interest is straightforward, as Python—unlike VBA—has built-in support to read JSON. We will see how all this works in [Chapter 9](#).

Nowadays, Python also supports asynchronous programming via its Asyncio library. This makes network requests such as talking to a web API or accessing databases highly efficient. We will briefly look into Asyncio in [Chapter 12](#).

While Python makes it easy to work with web APIs, error handling is another area where Python has evolved significantly beyond VBA's approach.

Error Handling

If you'd like to handle an error in VBA, it goes something like this:

```

Sub PrintReciprocal(number)
    Dim result As Double

    On Error GoTo ErrorHandler
    result = 1 / number
    Debug.Print "The reciprocal is: " & result
    Exit Sub

ErrorHandler:
    Debug.Print "There was an error: " & Err.Description
End Sub

```

VBA error handling involves the use of *labels* like `ErrorHandler`. You instruct the code to jump to these labels via the `GoTo` statements. Go to statements were famously criticized by computer scientist Edsger Dijkstra in his 1968 letter “**Go To Statement Considered Harmful**”, as they make the flow of the code hard to follow and therefore difficult to maintain, often resulting in so-called *spaghetti code*. That’s why modern languages use more advanced error handling—Python uses the `try/except` statements that I will introduce in [Chapter 10](#).

With the modern language features covered, we can move on to the last section in this chapter: it explains the various tools that you get access to as a Python developer.

Tooling

Modern programming languages like Python rely on a set of tools that make software development a lot more reliable. In this section, I’ll give you a short introduction to a few of these, including package management, testing, and version control.

Package Manager

While Python’s standard library covers an impressive amount of functionality, it—as discussed before—doesn’t come with any domain-specific functionality such as data science libraries or web frameworks. This is where **PyPI** comes in. PyPI stands for *Python Package Index* and is a giant repository where everybody (including you!) can upload Python packages that add additional functionality to Python.



PyPI versus PyPy

PyPI is pronounced “pie pea eye.” This is to differentiate PyPI from PyPy, which is pronounced “pie pie” and is a fast alternative implementation of Python.

A package manager allows you to install additional packages like `pandas` with a simple command. What sounds easy is in reality a very complex task as a package often depends on other packages and these again on yet another set of packages. The package manager makes sure that these dependencies and subdependencies are installed with compatible versions and that you can reproduce your exact set of dependencies on another machine. For example, when you install `pandas`, `NumPy` is automatically installed as a dependency. In this book, we will use the `uv` package manager, which I will introduce in the next chapter.

In VBA, there is no package manager, so you either have to use add-ins or copy/paste VBA code whenever you create a new Excel file. However, both of these solutions have issues: on the one hand, add-ins are global, so you have to use the same version of the add-in for every workbook. This can cause compatibility issues with

workbooks you created with an older version of the add-in. On the other hand, copy/pasting VBA code is a nightmare for maintainability: if you fix the code in one workbook, you have to apply the fix to all other workbooks, too.

While the package manager makes it easy to add new functionality to your projects, another important aspect of professional software development is testing, which makes sure your code works as expected.

Testing

When you tell an Excel developer to test their workbooks, they will most likely perform a few random checks, e.g., click a button and see if the macro still does what it is supposed to do. This is, however, a risky strategy: Excel makes it easy to introduce errors that are hard to spot. For example, you can overwrite a formula with a hardcoded value. Or you forget to adjust a formula in a hidden column.

When you tell a Python developer or an AI assistant to test their code, they will write *unit tests*. As the name suggests, it's a mechanism to test individual components of your program. Unit tests make sure that a specific function of a program works properly by comparing the expected output with the actual output of a function. For example, if you have a function that sums up two values, you'd call it with, e.g., 2 and 3, and the unit test compares the result of the function with the expected value of 5. If the function returns anything other than 5, the test fails, and you know that the function has a bug. Most programming languages offer a way to run unit tests automatically. Running automated tests will increase the reliability of your codebase and can prevent you from introducing bugs while making changes to your codebase.

Pretty much all programming languages offer a test framework to write unit tests without much effort—but not Excel. Fortunately, by connecting Excel with Python, you get access to Python's unit testing frameworks, most notably **pytest**. While a more in-depth presentation of unit tests is beyond the scope of this book, I invite you to have a look at my **blog post**, in which I walk you through the topic.

Unit tests are often set up to run automatically when you commit your code to your version control system. The next section explains what a version control system is and why working with Excel files can make version control difficult to use.

Version Control

A *version control system* (VCS) tracks changes in your source code over time, enabling you to see who changed what, when, and why, and allows you to revert to old versions at any point in time. The most popular version control system nowadays is **Git**. It was originally created to manage the Linux source code and since then has conquered the programming world. Excel offers a version history if you save your file on OneDrive

or SharePoint and are subscribed to Microsoft 365, but this doesn't cover VBA code. Therefore, the most popular version control system for Excel still looks like this:

```
sales_report_v1.xlsx  
sales_report_v2_2026_02_02.xlsx  
sales_report_final_edits_Bob.xlsx  
sales_report_final_final.xlsx
```

If, unlike in this sample, the Excel developer sticks to a certain convention in the file name, there's nothing inherently wrong with that. But keeping a version history of your files locally locks you out of important aspects of source control in the form of easier collaboration, peer reviews, sign-off processes, and audit logs. And if you want to make your workbooks more secure and stable, you don't want to miss out on these things. Most commonly, Git is used in connection with a web-based platform like GitHub or GitLab. These platforms allow you to work with so-called *pull requests*, which allow developers to formally request that their changes are merged into the main codebase. A pull request offers the following information:

- Who is the author of the changes
- When were the changes made
- What is the purpose of the changes
- Which code was added, changed, or removed

This allows a coworker or an AI assistant to review the changes and flag any potential issues. VBA can't be easily version-controlled with Git as it is stored in the Excel workbook, which Git can't read. However, when you write Python code in standalone files, you can use Git to its full potential. Not all code in this book lives in a standalone Python file, though. For example, Python in Excel, which we'll meet in [Chapter 6](#), stores the Python code in Excel cells, so you can't easily commit it to Git. But you can use Git for many other use cases that we'll meet in this book, like the Excel tool we'll build in [Chapter 9](#).

Conclusion

In this chapter, we met Python and Excel, two very popular technologies that have been around for decades—a long time compared to much of the other software that we use today.

Python comes with powerful features that are missing in Excel: the standard library, package managers, libraries for scientific computing, and cross-platform compatibility. By learning how to combine Excel with Python, you get the best of both worlds: Excel provides a familiar and easy-to-use user interface while Python allows you to save time through automation, leverage scientific packages, and gain the flexibility to scale your application beyond Excel.

Now that you know why Python is such a powerful companion for Excel, it's time to set up your development environment so you can write your first lines of Python code!

Development Environment

To write VBA code, it's enough to launch Excel and open the VBA editor. To use a full Python installation, it's a bit more work. If this book were only about Python in Excel ([Part III](#)) and xlwings Lite ([Part V](#)), we wouldn't have to install anything locally. But using Python from Excel covers only one side of the story—we don't want to miss out on the other side, which is using Excel from Python ([Part IV](#) and [Part VI](#)). Besides that, both Python in Excel and xlwings Lite have certain limitations that a full Python installation doesn't have. This chapter will make you familiar with all the tools you need as a Python developer. We will start with a quick introduction to the Terminal, the most fundamental programming tool. We'll continue with the installation of uv, a modern package manager, before using it to install Python itself. With Python installed, we will run our first code snippets in Jupyter notebooks and Visual Studio Code (VS Code): *Jupyter notebooks* allow us to work with data, code, and charts in an interactive way, which makes them a serious competitor to Excel workbooks. VS Code is a powerful text editor that works great for writing, running, and debugging Python code. As this book is about Excel, I am focusing on Windows and macOS in this chapter. Before anything else, let's download the companion repository!

Companion Repository

To install Python and run the examples in this book, you first need to download the companion repository from [GitHub](#). Click on the green Code button, then select Download ZIP, as shown in [Figure 2-1](#).

This downloads a file called *python-for-excel-2e.zip*. Once downloaded, extract the ZIP file into your user directory by following the instructions for your operating system.

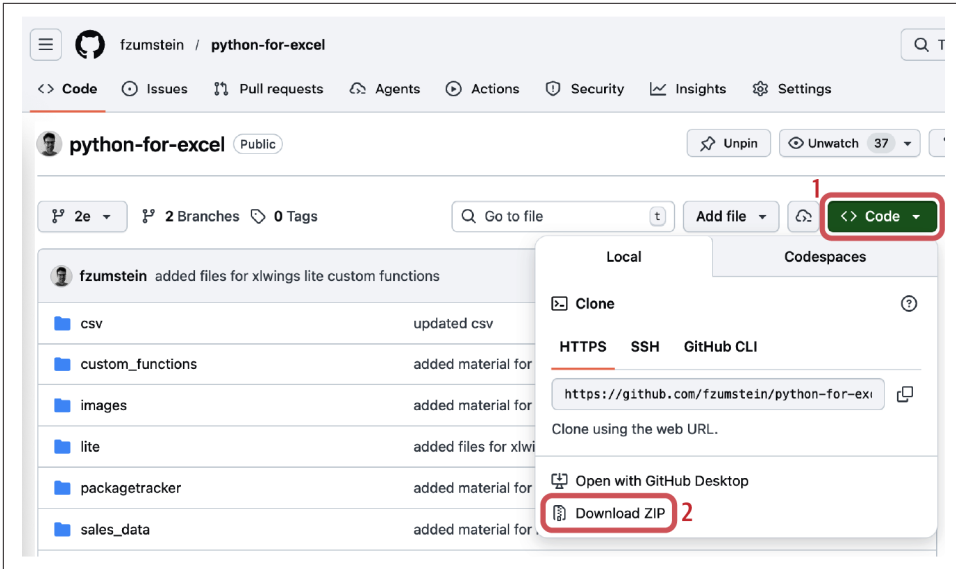


Figure 2-1. Downloading the companion repository



Please Follow the Steps Exactly

Please follow these steps to extract the ZIP file exactly—this ensures that you end up having the companion repository in the correct folder.

Windows

Right-click the downloaded ZIP file and select Extract All. In the dialog box, change the path so that it only shows your user directory, in my case this is `C:\Users\felix`, then click on Extract (see [Figure 2-2](#)). Your companion repository is now ready in your user directory: `C:\Users\<username>\python-for-excel-2e`.

macOS

In Finder, double-click the ZIP file to extract the files into a folder called *python-for-excel-2e*. Note that some browsers like Safari automatically extract ZIP files during download. Now move that folder into the correct directory:

1. Open a second Finder window via Command-N (make sure Finder is your active window).
2. In that second Finder window, use the keyboard shortcut Command-Shift-H to go to your user directory.
3. Drag and drop the extracted *python-for-excel-2e* folder to your user directory: your companion repository is now ready under `/Users/<username>/python-for-excel-2e`.

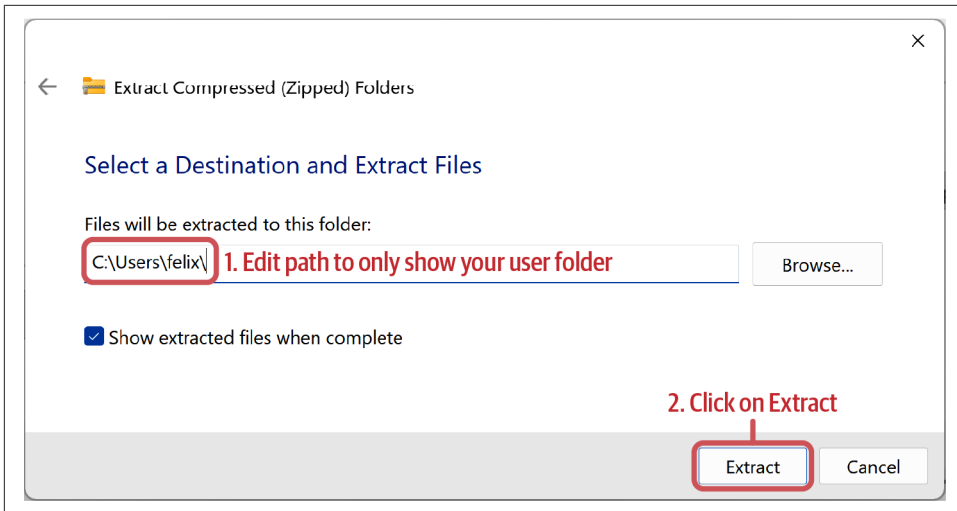


Figure 2-2. Extracting the ZIP file into your user directory

Alternatively, if you know how to work with Git version control, feel free to clone the repository to your user directory instead of downloading the ZIP file manually.

With the companion repository sitting in your user directory, let me give you a quick introduction to the Terminal, as we will use it to install Python.

Using the Terminal

The Terminal is the most fundamental programming tool, and we will use it extensively. If you have never used a Terminal, don't worry: you only need to know a handful of commands to work through this book. Once you get used to it, using the Terminal is often faster and more convenient than clicking your way through graphical user menus. Before we start, let me remind you that you should only type in the commands shown in **bold**. Text in regular font represents Terminal output that you won't need to type. Let's now open a Terminal:

Windows

Click on the Windows Start menu, type in **Command Prompt** in the search box, and click on the Command Prompt entry that appears. Alternatively, use the keyboard shortcut Windows+R to open the Run box. Then type **cmd** into the box and hit Enter. The input line of the Command Prompt will look something like this:

```
C:\Users\username>
```

macOS

Press Command-Space or open the Launchpad, then type **Terminal** and hit Enter. Alternatively, open the Finder and navigate to *Applications > Utilities*, where you will find the Terminal app that you can double-click. Once the Terminal appears, its input line looks something like this:

```
username@MacBook ~ %
```

The tilde stands for the user directory, which is usually */Users/username*. To see the full path of your current directory, type **pwd**, followed by Enter. **pwd** stands for *print working directory*.



Terminal

Going forward, whenever I write “Terminal,” I am referring to the Command Prompt on Windows and the Terminal on macOS. Recent versions of Windows even default to opening the Command Prompt inside a new app, which is, indeed, called Terminal.

In a Terminal, try out the commands outlined in [Table 2-1](#). I’ll explain each command in more detail after the table.

Table 2-1. Commands for the Terminal

Command	Windows	macOS
List files in current directory	<code>dir</code>	<code>ls -la</code>
Change directory (relative)	<code>cd path\to\dir</code>	<code>cd path/to/dir</code>
Change directory (absolute)	<code>cd C:\path\to\dir</code>	<code>cd /path/to/dir</code>
Change to D drive	<code>D:</code>	(doesn’t exist)
Change to parent directory	<code>cd ..</code>	<code>cd ..</code>
Scroll through previous commands	↑ (up-arrow)	↑ (up-arrow)

List files in current directory

On Windows, type in **dir** for *directory*, then hit Enter. This will print the content of the directory you are currently in.

On macOS, type in **ls -la**, followed by Enter. **ls** is short for *list directory contents*, and **-la** will print the output in the *long listing* format and include *all* files, including hidden ones.

File Extensions

Unfortunately, Windows and macOS hide file extensions by default in the Windows Explorer or macOS Finder, respectively. This can make it harder to work with Python scripts and the Terminal, as they will require you to refer to files including their extensions. When working with Excel, showing file extensions also helps you understand whether you're dealing with the default *xlsx* file, a macro-enabled *xlsm* file, or any of the other Excel file formats. Here is how you make file extensions visible:

Windows

Open the File Explorer and click on the View tab. Under Show, click on “File name extensions” to activate it.

macOS

Open the Finder and go to Preferences by hitting Command-, (Command-comma). On the Advanced tab, check the box next to “Show all filename extensions.”

Change directory

Type **cd Down** and hit the Tab key. If you are in your user directory and using an English version of Windows or macOS, hitting the Tab key will most likely autocomplete **cd Down** to **cd Downloads** (**cd** stands for *change directory*). If you are in a different folder, simply start to type the beginning of one of the directory names you saw with the previous command (**dir** or **ls -la**) before hitting the Tab key to autocomplete. Then hit Enter to change into the autocompleted directory. If you are on Windows and need to change your drive, you first need to type the drive name before you can change into the correct directory:

```
C:\Users\username> D:  
D:\> cd data  
D:\data>
```

Note that by starting your path with a directory or file name that is within your current directory, you are using a *relative path*, e.g., **cd Downloads**. If you would like to go outside of your current directory, you can type an *absolute path*, e.g., **cd C:\Users** on Windows or **cd /Users** on macOS (on macOS, absolute paths start with a forward slash).



Permissions on macOS

On macOS, when you run a command such as `ls` for the first time in a specific folder, let's assume the *Downloads* folder, you will be shown the following pop-up window: "Terminal would like to access files in your *Downloads* folder." Make sure to click on "Allow." If you clicked "Don't Allow" by mistake, go to Apple menu > System Settings > Privacy & Security > Files & Folders > Terminal. Click on the dropdown so that you can enable the toggle for the *Downloads* Folder.

Change to parent directory

To go to your parent directory, i.e., one level up in your directory hierarchy, type `cd ..` followed by Enter (make sure that there is a space between `cd` and the dots). You can combine this with a directory name: if you want to go up one level and change to the *Desktop*, enter `cd ../Desktop`. On macOS, replace the backslash with a forward slash.

Change to the Directory of the Companion Repository

Most Terminal commands in this book need to be run from the directory of the companion repository. In these cases, to prevent confusion, I'll always include the appropriate `cd` command:

Windows

```
cd %userprofile%\python-for-excel-2e
```

`%userprofile%` is a so-called environment variable, which points to your user directory, e.g., `C:\Users\<username>`. If you already are in the correct directory, you can ignore this line. The Terminal, i.e., Command Prompt, shows the full path on its input line, so it's easy to check which directory you are in. In my case, it looks like this:

```
C:\Users\felix\python-for-excel-2e>
```

macOS

```
cd ~/python-for-excel-2e
```

As I mentioned earlier in this section, `~` points to your home directory, e.g., `/Users/<username>`. If you already are in the correct directory, you can ignore this line. Since macOS only shows the name of the current directory, you may want to run the `pwd` command that we met earlier to check the full path of your current directory. In my case, this looks like this:

```
felix@MacBook python-for-excel-2e % pwd
/Users/felix/python-for-excel-2e
```

Usually, when you start a new Terminal, you are in your user folder by default. In that case, you can also just run `cd python-for-excel-2e` instead of `cd %user profile%\python-for-excel-2e` to change into the directory of the companion repository.

Scroll through previous commands

Use the up-arrow key to scroll through the previous commands. This will save you many keystrokes if you need to run the same commands over and over again. If you scroll too far, use the down-arrow key to scroll back.

And that's it! You are now able to fire up a Terminal and run commands in the desired directory. Let's use your new skills right away to install Python!

Installing Python

In this section, you will install Python by using uv, a modern package manager. We'll start by installing uv itself, then use it to install Python and the packages we'll use in this book. By the end of this section, you'll be running an interactive Python session.

Installing uv

While uv is primarily a package manager (see [Chapter 1](#)), it covers a lot more functionality, including the installation of Python itself. To install uv, use the following instructions according to your operating system. If you hit an issue, consult the original [installation instructions](#) for up-to-date information.

Windows

In a Terminal, i.e., Command Prompt, type the following command:

```
powershell -ep ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

macOS

In the Terminal, type the following command:

```
curl -Lsf https://astral.sh/uv/install.sh | sh
```

Hit Enter to run the command. Once done, you should see the following output:

```
[...]
everything's installed!
```

Now close and reopen the Terminal to finish the installation of uv. To properly close the Terminal on macOS, make sure to use the keyboard shortcut Command-Q.

After reopening the Terminal, type `uv` and hit Enter: this should print the help page of uv. If it doesn't, make sure to properly restart the Terminal before trying again. Let's now use uv to install Python and the third-party packages we'll use!

Installing Python and Packages

Now that we have `uv` installed, let's use it to install Python and all the packages we will use in this book. In a Terminal, first change into the directory of the companion repo, then install Python and the packages by running the following commands (hit Enter after each line):

Windows (Terminal)

```
cd %userprofile%\python-for-excel-2e
uv sync
```

macOS (Terminal)

```
cd ~/python-for-excel-2e
uv sync
```

The `uv sync` command downloads and installs Python and all required packages, which can take a few moments. Once done, `uv` will print something along the following lines:

```
Using CPython 3.14.2
Creating virtual environment at: .venv
Resolved 132 packages in 4ms
Installed 127 packages in 700ms
+ anyio==4.12.1
+ appnope==0.1.4
[...]
+ xlwings==0.33.20
+ xlwt==1.3.0
```



Your Python Version Will Be Different

In your case, you will likely see a different Python version, e.g., 3.14.3 or 3.14.4. These are newer versions that include bug fixes.

`uv` relies on the following files from the companion repo to know what it has to install:

- `.python-version` (defines the Python version)
- `pyproject.toml` (project metadata)
- `uv.lock` (contains all packages with their exact version)



Troubleshooting

If uv gives you the error `No pyproject.toml found in current directory or any parent directory`, your Terminal is not in the correct directory. Go back to “[Companion Repository](#)” on [page 15](#) to make sure you have extracted the repository into the correct folder and verify that the Terminal is in the correct path (usually `C:\Users\<username>\python-for-excel-2e` on Windows and `/Users/<username>/python-for-excel-2e` on macOS).

During the installation, uv creates the folder `.venv` in the root of the companion repository, which is a *virtual environment* (see the sidebar). To get a more detailed introduction to uv and see how you can create your own projects, have a look at [Appendix A](#).

Virtual Environment

A virtual environment is an isolated workspace with its own Python interpreter and set of packages. Imagine you’re working on two projects at the same time: project A might use Python 3.13 with pandas 2.3.2, while Project B requires Python 3.14 with pandas 3.0.0. Since code written for pandas 2.3.2 might need modifications to be compatible with pandas 3.0.0, using a single version of pandas for all your projects doesn’t work. Virtual environments solve this issue by keeping each project’s dependencies separate.

With Python and our packages installed, let’s start an interactive Python session!

Python REPL

An interactive Python session is called a *REPL*, which stands for *read-eval-print loop*: Python reads your input, evaluates it, and prints the result, then waits for your next input. The following command shows you how to start a Python REPL with uv. Remember, if you are already in the directory of the companion repository, you don’t need to run the first line with the `cd` command:

Windows (Terminal)

```
cd %userprofile%\python-for-excel-2e
uv run python
Python 3.14.2 (main, Dec 17 2025, [...]) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

macOS (Terminal)

```
cd ~/python-for-excel-2e
uv run python
Python 3.14.2 (main, Jan 14 2026, 23:37:46) [Clang 21.1.4 ] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>>
```



uv run

uv makes it easy to use the correct virtual environment if you follow these rules:

1. Change into the directory of your project; in our case, that's the directory of the companion repository.
2. Run your commands with `uv run`. As we've just learned, `uv run python` starts an interactive interpreter and `uv run hello_world.py` runs a Python script, as we will see in a moment.

Let's play around a bit! Note that `>>>` in an interactive session means that Python expects your input; you don't have to type this in. In contrast, the lines without `>>>` show the output of the command on the previous line. Follow along by typing in each line that starts with `>>>` and confirm with the Enter key:

```
>>> 3 + 4
7
>>> "python " * 3
'python python python '
```

Remember The Zen of Python that I mentioned in the previous chapter? Run the following code (i.e., hit Enter after typing it in) and you'll get a witty version of the guiding principles of Python:

```
>>> import this
```

To exit out of your Python session, type **quit** followed by the Enter key. Alternatively, hit Ctrl-D on macOS or Ctrl+Z on Windows. On Windows, you need to additionally press Enter.

Running code in a Python REPL can be helpful at times, but there's a more comfortable way to run code interactively: Jupyter notebooks!

Jupyter Notebooks

In the previous section, I showed you how to start an interactive Python session in a Terminal. This is useful if you want a bare-bones environment to test out something simple. For the majority of your work, however, you want an environment that is

easier to use. For example, going back to previous commands and displaying charts is hard with a Python REPL. That's why *Jupyter notebooks* have emerged as one of the most popular ways to run scientific Python code. You interact with Jupyter notebooks in the browser and they allow you to tell a story by combining executable Python code with formatted text, pictures, and charts. They are beginner-friendly and thus especially useful for the first steps of your Python journey. They are, however, also hugely popular for teaching, prototyping, and researching, as they facilitate reproducible research.

Jupyter notebooks have become a serious competitor to Excel as they cover roughly the same use case as a workbook: you can quickly prepare, analyze, and visualize data. The difference from Excel is that Jupyter notebooks don't mix data and business logic: the Jupyter notebook holds your code and charts, whereas you typically consume data from an external file or a database. Having Python code visible in your notebook makes it easy to see what's going on, whereas in Excel the formulas are hidden behind a cell's value.

In this section, I'll show you the basics of how to run and navigate a Jupyter notebook. Along the way, you will learn about different types of notebook cells and why the run order of cells matters. Let's get started with our first notebook!

Running Jupyter Notebooks

Using a Terminal, launch a Jupyter notebook server (this may take a while to start):

Windows (Terminal)

```
cd %userprofile%\python-for-excel-2e
uv run jupyter notebook
```

macOS (Terminal)

```
cd ~/python-for-excel-2e
uv run jupyter notebook
```

This will automatically open your browser and show the Jupyter dashboard with the files in the directory from where you were running the command. On the top right of the Jupyter dashboard, click on New, then select Python 3 from the drop-down list (see [Figure 2-3](#)).

This will open a new browser tab with your first empty Jupyter notebook, as shown in [Figure 2-4](#).

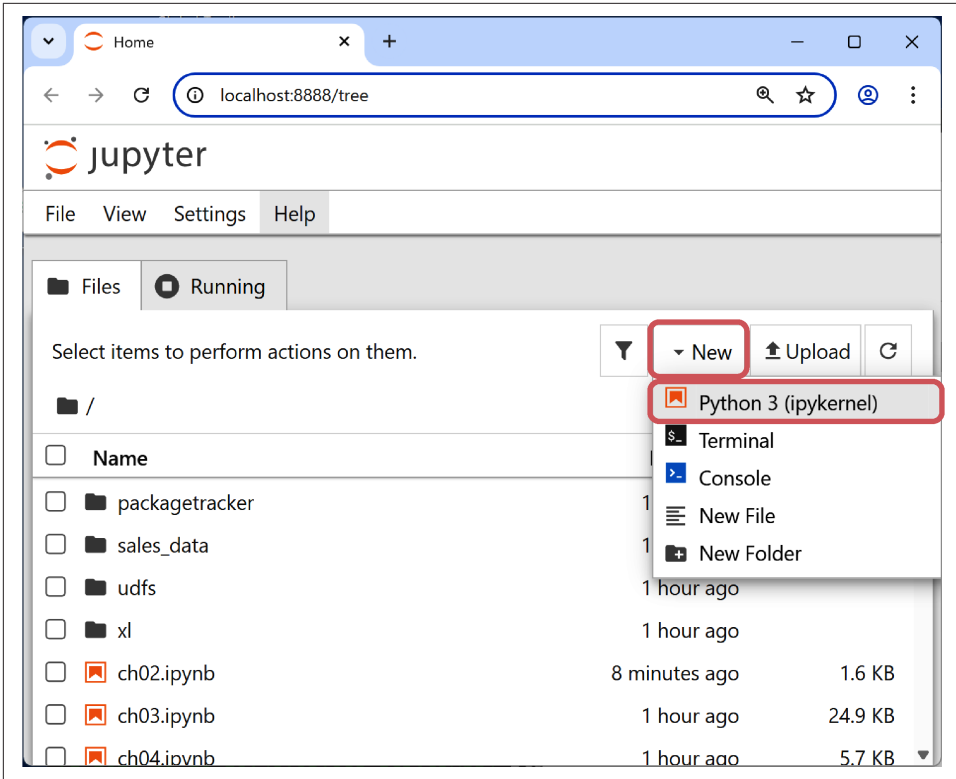


Figure 2-3. The Jupyter dashboard

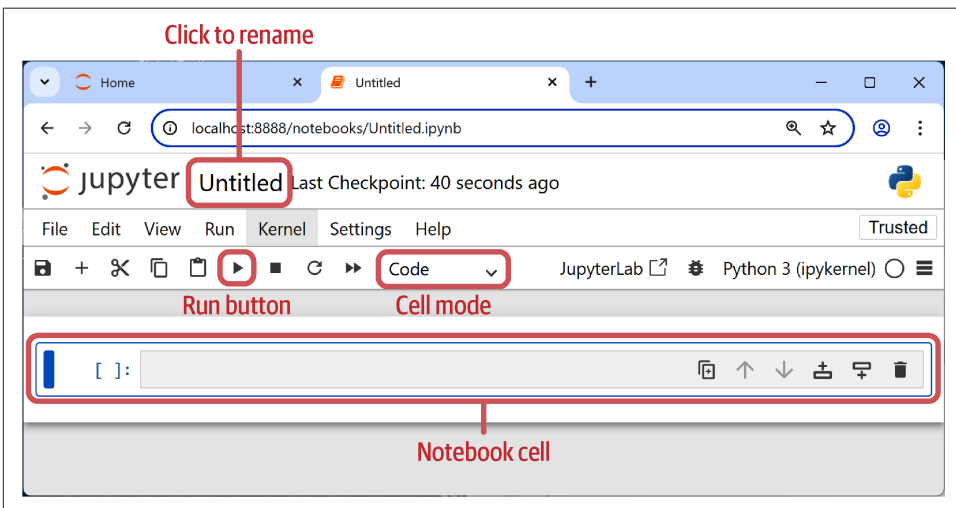


Figure 2-4. An empty Jupyter notebook

Jupyter Notebooks in the Cloud

Jupyter notebooks have become so popular that they are offered as a hosted solution by various cloud providers. I am introducing three services here that are free to use. The great thing about running notebooks in the cloud is that you don't need to install anything locally.

Binder

Binder is a service provided by Project Jupyter, the organization behind Jupyter notebooks. Binder is meant to try out the Jupyter notebooks from public Git repositories. It's read-only—you can't store anything on Binder, and hence you don't need to sign up or log in to use it.

Google Colab

Google Colab (short for Colaboratory) is Google's notebook platform. Unfortunately, the majority of the Jupyter notebook keyboard shortcuts don't work, but you can access files on your Google Drive, including Google Sheets.

Kaggle Notebooks

Kaggle is a platform for data science and ML with a huge collection of datasets. Kaggle has been part of Google since 2017.

The easiest way to run the Jupyter notebooks of the companion repository in the cloud is by going to its **Binder URL**. You will be working on a read-only copy of the companion repository, so feel free to edit and break stuff as you like! However, since **Part IV** requires a local installation of Excel, you also need to run the notebooks locally for that part.

It's a good habit to click on Untitled next to the Jupyter logo to rename your workbook into something more meaningful, e.g., *first_notebook*. The lower part of **Figure 2-4** shows a notebook cell—move on to the next section to learn more about them!

Notebook Cells

In **Figure 2-4**, you see an empty cell with a blinking cursor. If the cursor doesn't blink, click into the cell with your mouse, i.e., to the right of []. Now repeat the exercise from the last section: type in **3 + 4** and run the cell by either clicking on the Run button in the menu bar at the top or—much easier—by hitting Shift+Enter. This will run the code in the cell, show the result below the cell, and jump to the next cell. In this case, it inserts an empty cell below as we only have one cell so far. While a cell is calculating, it shows [*], and when it's done, the asterisk turns into a number, e.g., [1]. Below the cell you will have the corresponding output labeled with the same number. Every time you run a cell, the number increases by one, helping

you understand the order in which cells were executed. Going forward, I will show the code samples in this format:

```
In [1]: 3 + 4
```

```
Out[1]: 7
```

As you can see, I am additionally showing the labels In and Out in front of the cell number to make it clearer what the input and output is. This notation allows you to follow along easily by typing `3 + 4` into a notebook cell. When running it by hitting Shift+Enter, you will get what I show as output under Out[1]. If you read this book in an electronic format supporting colors, you will notice that the input cell shows text, numbers, and so on with different colors to make it easier to read. This is called *syntax highlighting*.

Running Code Examples

To run the examples in this book, you have two options: create a new Jupyter notebook and type the code manually or open the respective Jupyter notebook file, e.g., `ch02.ipynb`, by clicking on it in the Jupyter dashboard (see [Figure 2-3](#)). Using the Jupyter notebooks from the companion repository allows you to run and edit the examples without typing them from scratch.

Cells can have different types, two of which are of interest to us:

Code

This is the default type. Use it whenever you want to run Python code.

Markdown

Markdown is a syntax that uses standard text characters for formatting and can be used to include nicely formatted explanations and instructions in your notebook.

To change a cell's type to Markdown, select the cell, then choose Markdown in the cell mode dropdown (see [Figure 2-4](#)). I'll show you a keyboard shortcut to change the cell mode in [Table 2-2](#). After changing an empty cell into a Markdown cell, type in the following text, which explains a few Markdown rules:

```
# This is a first-level heading
```

```
## This is a second-level heading
```

```
You can make your text *italic* or **bold** or `monospaced`.
```

```
* This is a bullet point
```

```
* This is another bullet point
```

After hitting Shift+Enter, the text will be rendered into nicely formatted HTML. At this point, your notebook should look like what's in [Figure 2-5](#). Markdown cells also allow you to include images, videos, or formulas; see the [Jupyter notebook docs](#).

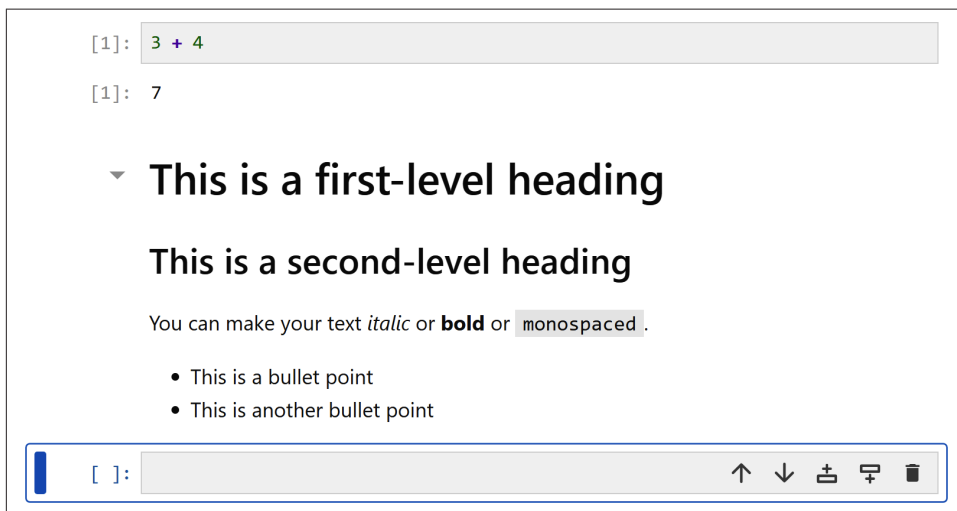


Figure 2-5. The notebook after running a code cell and a Markdown cell

Now that you know about the code and Markdown cell types, it's time to learn an easier way to navigate between cells: the next section introduces the edit and command mode along with a few keyboard shortcuts.

Edit versus Command Mode

When you interact with cells in a Jupyter notebook, you are either in the *edit mode* or *command mode*:

Edit mode

Clicking into a cell starts the edit mode: the cursor in the cell is blinking. Instead of clicking into a cell, you can also hit Enter when the cell is selected.

Command mode

To switch into command mode, hit the Escape key; the blinking cursor will disappear. The most important keyboard shortcuts that you can use while being in command mode are shown in [Table 2-2](#).

Table 2-2. Keyboard shortcuts (command mode)

Shortcut	Action
Shift+Enter	Run the cell and proceed to the next cell (also works in edit mode)
↑ (up-arrow)	Move cell selector up
↓ (down-arrow)	Move cell selector down
b	Insert a new cell <i>below</i> the current cell
a	Insert a new cell <i>above</i> the current cell
dd	Delete the current cell (type the letter d two times)
m	Change cell type to Markdown
y	Change cell type to code

Knowing these keyboard shortcuts allows you to work with notebooks very efficiently. In the next section, I'll show you a common gotcha that you need to be aware of when using Jupyter notebooks: the importance of running cells in order.

Run Order Matters

As easy and user-friendly as notebooks are to get started, they also make it easy to get into confusing states if you don't run cells sequentially. Assume you have the following notebook cells that are run from top to bottom:

```
In [2]: a = 1
```

```
In [3]: a
```

```
Out[3]: 1
```

```
In [4]: a = 2
```

Cell Out[3] prints the value 1 as expected. However, if you now rerun *only* In [3] again, you will end up in the following situation (rerunning cell In [3] will turn it into cell In [5]):

```
In [2]: a = 1
```

```
In [5]: a
```

```
Out[5]: 2
```

```
In [4]: a = 2
```

Out[5] now shows the value 2, which is probably not what you would expect when you read the notebook from top to bottom, especially if cell [4] is further down. To prevent such cases, I would recommend rerunning not just a single cell but all of its previous cells, too. Jupyter notebooks offer an easy way to accomplish this under the menu Run > Run All Above Selected Cell.

marimo Notebooks

The issue with the run order isn't the only limitation with Jupyter notebooks. **marimo notebooks** are a modern notebook solution that addresses several of these shortcomings: they are reactive (dependent cells update automatically), stored as plain Python files (making version control easy), and can be run as scripts or deployed as web apps. This book sticks to Jupyter notebooks as they are still more widely used, but if you want to go all in with notebooks, make sure to give marimo a try!

Now that you know how to work with Jupyter notebooks, you'll learn about how to write and run regular Python files. To do this, we'll use Visual Studio Code, a powerful text editor with great Python support.

Visual Studio Code

In this section, we'll install and configure *Visual Studio Code* (VS Code), a free text editor from Microsoft. After introducing its most important components, we'll write a first Python script and run it in a few different ways. To begin with, however, let me explain the difference between Python scripts and Jupyter notebooks and why I chose VS Code for this book.

While Jupyter notebooks are amazing for interactive workflows like researching, teaching, and experimenting, they are less ideal if you want to write Python scripts geared toward a production environment. Also, more complex projects that involve multiple files and developers are hard to manage with only Jupyter notebooks. In this case, you want to use a proper text editor to write and run regular Python files. In theory, you could use just about any text editor, but in reality, you want one that “understands” Python—that is, a text editor that supports at least the following features:

Syntax highlighting

The editor colors words differently based on whether they represent a function, a string, a number, etc. This makes it much easier to read the code.

Autocomplete

Autocomplete or *IntelliSense*, as Microsoft calls it, automatically suggests code completions so that you have to type less and consult documentation less frequently.

And soon enough, you have additional needs:

Run code

Switching back and forth between the text editor and an external Terminal to run your code can be a hassle.

Debugger

A debugger allows you to step through the code line by line.

Version control

Having built-in support for Git version control saves you from using an external tool.

AI assistant

AI coding assistants like GitHub Copilot can read and edit your files directly in the editor.

There is a wide spectrum of tools that can help you with all that, and as usual, every developer has different needs and preferences. Some may indeed want to use a no-frills text editor together with an external Terminal. Others may prefer an *integrated development environment* (IDE): IDEs try to put everything you'll ever need into a single tool, which can make them bloated.

I chose VS Code for this book as it has quickly become one of the most popular code editors after its initial release in 2015. What makes VS Code such a popular tool? In essence, it's the right mix between a bare-bones text editor and a full-blown IDE: VS Code is a mini IDE that comes with the essential tools out of the box, but everything else needs to be added via extensions:

Cross-platform

VS Code runs on Windows, macOS, and Linux. There are also cloud-hosted versions like [GitHub Codespaces](#).

Integrated tools

VS Code comes with a debugger, a Terminal, and support for Git version control and has GitHub Copilot built in.

Extensions

Additional functionality, e.g., Python support, is added via extensions that you can install with a simple click.

Lightweight

Depending on your operating system, the VS Code installer is around 100–150 MB.



Visual Studio Code versus Visual Studio

Don't confuse Visual Studio Code with Visual Studio, the IDE! While you can use Visual Studio for Python development, it's a heavy installation and is traditionally used to work with .NET languages like C#.

To find out if you agree with my praise for VS Code, there is no better way than trying it out yourself. The next section gets you started!

Installation and Configuration

Download the installer from the [VS Code home page](#). For the installation, use the following instructions according to your operating system. In case of an issue, please refer to the [original documentation](#):

Windows

Double-click the installer, accept the license agreement, then accept all defaults. Once installed, open VS Code via the Windows Start menu, where you will find it under Visual Studio Code.

macOS

Double-click the ZIP file to unpack the app (some browsers, like Safari, do this automatically). Then drag and drop *Visual Studio Code.app* into the *Applications* folder: you can now start it from the Launchpad. You need to confirm the pop-up message saying “Visual Studio Code.app is an app downloaded from the internet. Are you sure you want to open it?” by clicking the Open button.

When you open VS Code for the first time, it looks like [Figure 2-6](#). Note that I have switched from the default dark theme to a light theme to make the screenshots easier to read. I have also closed the documents with the welcome message and release notes that VS Code shows when you first open it.

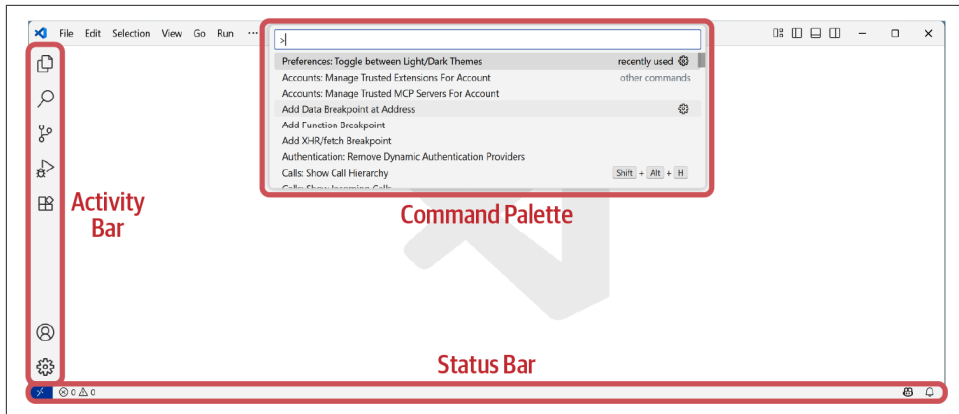


Figure 2-6. Visual Studio Code

Here is an overview of the main menus and interface elements highlighted in **Figure 2-6**:

Activity Bar

On the left-hand side, you see the Activity Bar with the following icons from top to bottom:

- Explorer
- Search
- Source Control
- Run and Debug
- Extensions

Status Bar

At the bottom of the editor, you have the Status Bar. Once you have completed the configuration and edit a Python file, you will see, e.g., which Python interpreter is being used.

Command Palette

You can show the Command Palette via F1 or with the keyboard shortcut Ctrl+Shift+P (Windows) or Command-Shift-P (macOS). If you are unsure about something, your first stop should always be the Command Palette, as it gives you easy access to almost everything you can do with VS Code. For example, if you are looking for keyboard shortcuts, type in **keyboard shortcuts**, select the entry “Help: Keyboard Shortcuts Reference,” and hit Enter.

VS Code is a great text editor out of the box, but to make it work nicely with Python, you need to install two extensions:

- Python by Microsoft (core language support)
- Jupyter by Microsoft (Jupyter notebook support)

To install them, click on the Extensions icon on the Activity Bar and search for “Python.” Install the Python extension that shows Microsoft as the author. Once this is installed, search for “Jupyter,” again authored by Microsoft, and click Install. After installing an extension, VS Code will show a welcome document—feel free to close it. Now finalize the configuration according to your platform:

Windows

Open the Command Palette and type **default profile**. Select the entry that reads “Terminal: Select Default Profile” and hit Enter. In the drop-down menu, select Command Prompt and confirm by hitting Enter. This prevents VS Code from running into a permission error when activating the virtual environment in PowerShell, which is the default Terminal.

macOS

Open the Command Palette and type **shell command**. Select the entry that reads “Shell Command: Install code command in PATH,” and hit Enter. This is required so that you can start VS Code conveniently from the Terminal.

Now that VS Code is installed and configured, let’s use it to write and run our first Python script!

Running a Python Script

While you can open VS Code via the Start menu on Windows or Launchpad on macOS, it’s often faster to open VS Code from the Terminal, where you are able to launch it via the code command. Therefore, start a new Terminal and use `cd` to change into the directory you want to open (in our case that’s the directory of the companion repository), then instruct VS Code to open the current directory, which is represented by the dot:

Windows (Terminal)

```
cd %userprofile%\python-for-excel-2e
code .
```

macOS (Terminal)

```
cd ~/python-for-excel-2e
code .
```

Starting VS Code this way opens the companion repository directly in the Explorer on the Activity Bar. Note that VS Code will ask, “Do you trust the authors of the files in this folder?” Confirm this dialog by clicking the “Yes, I trust the authors” button. If

you don't want to use the code command, just start VS Code normally and open the companion repository via File > Open Folder.

When you hover over the file list in the Explorer on the Activity Bar, you will see the New File button appear, as shown in **Figure 2-7**. Click on New File and call your file *hello_world.py*, then hit Enter. Once it opens in the editor, write the following line of code:

```
print("hello world!")
```

Remember that Jupyter notebooks print the result of the last line automatically. When you run a regular Python script, you need to tell Python explicitly what to print, which is why you need to use the `print` function here. While you are in a Python file, the Status Bar will now show your Python environment, “py4xl”; this name is defined in *pyproject.toml*. Since VS Code automatically selects virtual environments in the project's root folder, you don't have to select the environment manually. Your setup should now look like the one in **Figure 2-7** (keep in mind that your Python version might be different).

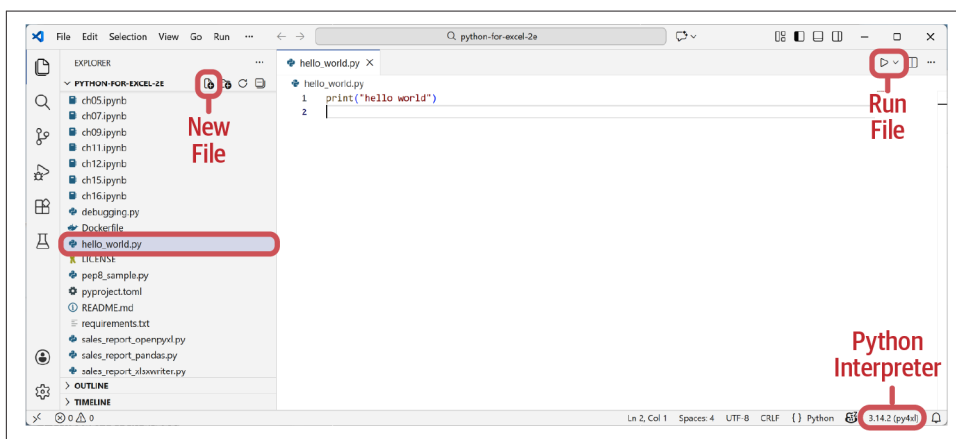


Figure 2-7. VS Code with *hello_world.py* open

Before we can run the script, make sure to save it by hitting Ctrl+S on Windows or Command-S on macOS. With Jupyter notebooks, we could hit Shift+Enter to run a cell. With VS Code, you can run your code from either the Terminal or by clicking the Run button. Running Python code from the Terminal is how you usually run scripts on a server, so it's important to know how this works.

Terminal

In a Terminal, in the directory of *hello_world.py*, i.e., in the directory of the companion repository, run the script by using `uv run`:

Windows (Terminal)

```
cd %userprofile%\python-for-excel-2e
uv run hello_world.py
```

macOS (Terminal)

```
cd ~/python-for-excel-2e
uv run hello_world.py
```

This will show the following output:

```
hello world!
```

If you are not in the same directory as your Python file, you need to use the full path to your Python file:

Windows (Terminal)

```
uv run %userprofile%\python-for-excel-2e\hello_world.py
```

macOS (Terminal)

```
uv run ~/python-for-excel-2e/hello_world.py
```



Long File Paths on the Terminal

A convenient way to deal with long file paths is to drag and drop the file onto your Terminal. This will write the full path to wherever the cursor is in the Terminal. Note that it will write the absolute path to the Terminal, so it won't use %USERPROFILE% or ~.

Instead of using an external Terminal, you can also use the Terminal that's integrated into VS Code.

Terminal in VS Code

You don't need to switch away from VS Code to work with the Terminal: VS Code has an integrated Terminal that you can show via the keyboard shortcut Ctrl+` or via View > Terminal. Since it opens in the project folder, you don't need to change the directory first:

Windows and *macOS (Terminal)*

```
uv run hello_world.py
```

The integrated Terminal allows you to run a Python script directly from VS Code. For an even easier experience, VS Code offers the Run button.

Run button

When you edit a Python file, you will see a Play icon at the top right—this is the Run Python File button, as shown in [Figure 2-7](#). Clicking it will open the Terminal at the bottom automatically and run the code.



Opening Files in VS Code

VS Code has an unconventional default behavior when you single-click a file in the Explorer on the Activity Bar: the file is opened in *preview mode*, which means that the next file that you single-click will open in the same tab unless you have edited the file. If you want to switch off the single-click behavior (so only a double-click will open it), go to Preferences > Settings (Ctrl+, on Windows or Command-, on macOS). Under Workbench > “List: Open Mode,” set the dropdown to “doubleClick.”

If you encounter an issue when running a Python script, a debugger can help you find the cause.

Debugger

If you’ve ever used the VBA debugger in Excel, I have good news for you: debugging with VS Code is a very similar experience. Let’s start by opening the file *debugging.py* from the companion repository in VS Code. Click into the margin to the left of line number 4 so that a red dot appears—this is your breakpoint where code execution will be paused. Now click on the dropdown of the play button at the top right and select “Python Debugger: Debug Python File.” This runs the code until it hits the breakpoint. The line with the breakpoint will be highlighted and code execution pauses (see [Figure 2-8](#)).

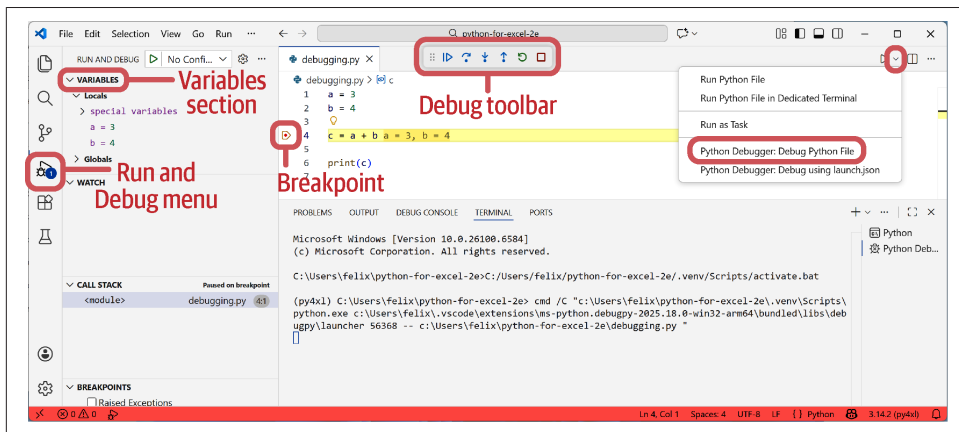


Figure 2-8. VS Code with the debugger stopped at the breakpoint

If the Variables section doesn't show up automatically on the left, make sure to click on the Run and Debug menu to see the values of the variables. Alternatively, you can also hover over a variable in the source code and get a tooltip with its value. At the top, you will see the Debug Toolbar that gives you access to the following buttons from left to right: Continue, Step Over, Step Into, Step Out, Restart, and Stop. When you hover over them, you will see their keyboard shortcuts.

Let's see what each of these buttons does:

Continue

This continues to run the program until it either hits the next breakpoint or reaches the end of the program. If it reaches the end of the program, the debugging process will stop.

Step Over

The debugger will advance one line. *Step Over* means that the debugger will not visually step through lines of code that are outside of your current scope. For example, it runs functions in one step instead of going through each line inside them.

Step Into

If you have code that calls a function or class, etc., *Step Into* will cause the debugger to step into that function or class. If the function or class is in a different file, the debugger will open this file for you.

Step Out

If you stepped into a function with Step Into, *Step Out* causes the debugger to return to the next higher level until, eventually, you are back on the highest level from where you called Step Into initially.

Restart

This will stop the current debug process and start a new one from the beginning.

Stop

This will stop the current debug process.

Now that you know what each button does, click on Step Over to advance one line and see how variable `c` appears in the Variables section, then finish this short debugging exercise by clicking on Continue.

At this point, you know how to create, run, and debug Python scripts in VS Code. However, VS Code can also run Jupyter notebooks—here's how this works.

Running Jupyter Notebooks with VS Code

Instead of spinning up a Jupyter notebook server in the Terminal, you can run notebooks directly in VS Code thanks to the Jupyter extension that we installed

earlier. To create a new Jupyter notebook, open the Command Palette and search for “Create: New Jupyter Notebook.” If you want to open an existing file instead, open it like any other file from the file explorer in VS Code. With Jupyter notebooks, VS Code shows the `py4x1` virtual environment on the top right of the notebook as the Python interpreter (in the context of a Jupyter notebook, this is called the *Jupyter kernel*; see Figure 2-9). If VS Code shows `Select Kernel` instead of the environment name, click on it to select the correct Python environment, `py4x1`.

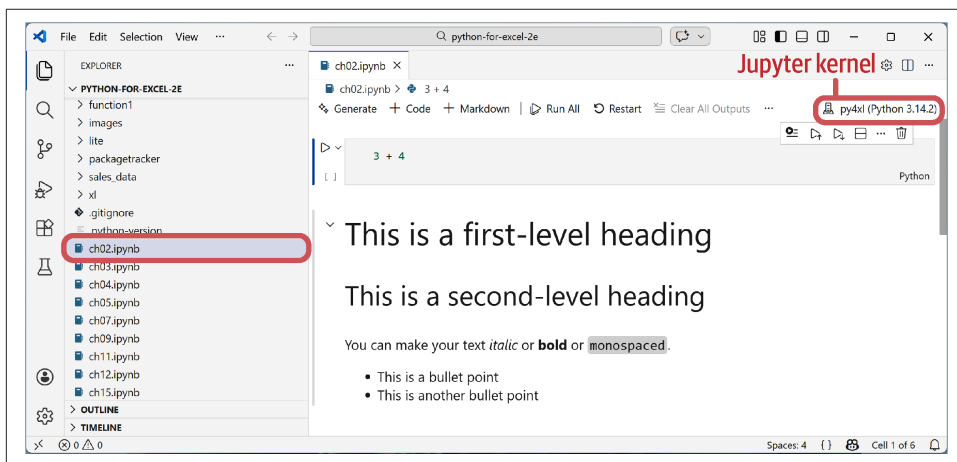


Figure 2-9. Jupyter notebooks in VS Code

Running Jupyter notebooks in VS Code makes things easier as you don’t need to run the Jupyter notebook server manually in a Terminal. The main advantage, however, is that you get access to the tools that VS Code offers such as code completion, GitHub Copilot, and the debugger.

Alternative Text Editors and IDEs

Tools are something individual, and just because this book is based on VS Code doesn’t mean you shouldn’t have a look at other options. Some of these options include the following:

- **Zed** is a free and fast text editor by the creators of the retired Atom editor.
- **Sublime** is a fast commercial text editor.
- **Notepad++** is a free text editor for Windows.
- **PyCharm** is an IDE with a free community edition and a professional edition that adds support for scientific tools and web development.
- **Spyder** is similar to MATLAB’s IDE and comes with a variable explorer.
- **Positron** is a free IDE for data science that supports both Python and R.

- **JupyterLab** is a web-based IDE developed by the Jupyter notebook team.
- **Wing Python IDE** is available in both free and paid versions.
- **PyDev** is originally based on the Eclipse IDE but nowadays is also available as a VS Code extension.

Besides these solutions, there's a growing number of AI-centric IDEs such as **Cursor**. Given how rapidly this landscape is evolving, I recommend searching the web for the latest products.

Conclusion

In this chapter, apart from helping you install Python, I've introduced you to all the essential tools that we will work with in this book: the Terminal, uv, Jupyter notebooks, and VS Code. We also ran a tiny bit of Python code in a Python REPL, in a Jupyter notebook, and as a script in VS Code.

I do recommend you get comfortable with the Terminal, as it will give you a lot of power once you get used to it. Ultimately, the Terminal is where you run your Python code—whether directly, through VS Code's integrated Terminal, or via the VS Code play button.

With a working development environment, you are now ready to tackle the next chapter, where you'll learn enough Python to be able to follow the rest of the book.

Getting Started with Python

With Python and VS Code installed, you have everything in place to get started. Although this chapter doesn't go much further than the basics, it still covers a lot of ground. If you are at the beginning of your coding career, there may be a lot to digest. However, most concepts will get clearer once you use them in later chapters as part of a practical example, so there's no need to worry if you don't understand something fully the first time around. If you are a seasoned VBA developer, I will try to make your transition to Python a little easier by drawing parallels between the two languages and pointing out any caveats.

I'll start this chapter with Python's basic data types, such as integers and strings. After that, I will introduce indexing and slicing, a core concept in Python that gives you access to specific elements of a sequence. Up next are data structures like lists and dictionaries that can hold multiple objects. I'll continue with the `if` statement and the `for` and `while` loops before getting to an introduction of functions and modules that allow you to organize and structure your code. To wrap this chapter up, I will show you how to format your Python code properly. As you have probably guessed by now, this chapter is as technical as it can get. Running the examples for yourself in a Jupyter notebook is therefore a good idea to make everything a bit more interactive and playful. Either type the examples yourself or run them by using the notebook *ch03.ipynb* from the companion repository.



Creating a New Jupyter Notebook

If you don't remember how to create a new Jupyter notebook, go back to [“Running Jupyter Notebooks” on page 25](#) to see how to run a classic Jupyter notebook in a browser, or [“Running Jupyter Notebooks with VS Code” on page 39](#) to see how to create a new Jupyter notebook directly in VS Code.

Data Types

Python, like every other programming language, treats numbers, text, booleans, etc., differently by assigning them a different *data type*. The data types that we will use most often are integers, floats, booleans, and strings. In this section, I am going to introduce them one after another with a few examples. To be able to understand data types, though, I first need to explain what an object is.

Objects

In Python, *everything* is an object, including numbers, strings, functions, and everything else that we'll meet in this chapter. Objects can make complex things easy and intuitive by giving you access to a set of variables and functions. The variables hold the object's data (often called *state*), while the functions let you perform actions. So before anything else, let me say a few words about variables and functions!

Variables

In Python, a *variable* is a name that you assign to an object by using the equal sign. In the first line of the following example, the name `a` is assigned to the object `3`:

```
In [1]: a = 3
        b = 4
        a + b
```

```
Out[1]: 7
```

This works the same for all objects, which is simpler compared to VBA, where you use the equal sign for data types like numbers and strings and the `Set` statement for objects like workbooks or worksheets. In Python, you change a variable's type simply by assigning it to a new object. This is referred to as *dynamic typing*:

```
In [2]: a = 3
        print(a)
        a = "three"
        print(a)
```

```
Out[2]: 3
        three
```

Unlike VBA, Python is case-sensitive, so `a` and `A` are two different variables. Variable names must follow certain rules:

- They must start with either a letter or an underscore.
- They must consist of letters, numbers, and underscores.

After this short introduction to variables, let's see how we can make function calls!

Functions

I will introduce functions with a lot more detail later in this chapter. For now, you should simply know how to call built-in functions like `print` that we used in the previous code sample. To call a function, you add parentheses to the function name and provide the arguments within the parentheses, which is pretty much equivalent to the mathematical notation:

```
function_name(argument1, argument2, ...)
```

Let's now look at how variables and functions work in the context of objects!

Attributes and methods

In the context of objects, variables are called *attributes* and functions are called *methods*: attributes give you access to the data of an object, and methods allow you to perform an action. To access attributes and methods, you use the dot notation like this: `myobject.attribute` and `myobject.method()`. Note that you use parentheses when calling methods but not when accessing attributes.

Let's make this a bit more tangible: if you write a car racing game, you will most likely use an object that represents a car. The car object could have a `speed` attribute that allows you to get the current speed via `car.speed`, and you might be able to accelerate the car by calling the `accelerate` method `car.accelerate(10)`, which would increase the speed by ten miles per hour.

The type of an object, and with that its behavior, is defined by a *class*, so the previous example would require you to write a `Car` class. The process of getting a car object out of a `Car` class is called *instantiation*, and you instantiate an object by calling the class in the same way as you call a function: `car = Car()`. We won't write our own classes in this book, but if you are interested in it, look at [Appendix C](#).

We will use a method in the next section to make a string, i.e., a piece of text, uppercase, and we will get back to the topic of objects and classes when we talk about `datetime` objects toward the end of this chapter. Now, however, let's move on with those objects that have a numeric data type!

Numeric Types

The data types `int` and `float` represent *integers* and *floating-point numbers*, respectively. To find out what data type an object has, use the built-in function `type`:

```
In [3]: type(4)
Out[3]: int
In [4]: type(4.4)
Out[4]: float
```

If you want to force a number to be a `float` instead of an `int`, use a trailing decimal point or the `float` *constructor* (a constructor is a function that creates a specific object):

```
In [5]: type(4.)
```

```
Out[5]: float
```

```
In [6]: float(4)
```

```
Out[6]: 4.0
```

The last example can also be turned around: using the `int` constructor, you can turn a `float` into an `int`. If the fractional part is not zero, it will be truncated:

```
In [7]: int(4.9)
```

```
Out[7]: 4
```



Excel Cells Always Store Floats

You may need to convert a `float` to an `int` when you read in a number from an Excel cell and provide it as an argument to a Python function that expects an integer. The reason is that numbers in Excel cells are always stored as floats behind the scenes, even if Excel shows you what looks like an integer.

Python has a few more numeric types that I won't discuss in this book: there are the `decimal`, `fraction`, and `complex` data types. If floating-point inaccuracies are an issue (see sidebar), use the `decimal` type for exact results. These cases are rare, though. As a rule of thumb: if Excel would be good enough for the calculations, use floats.

Floating-Point Inaccuracies

By default, Excel often shows rounded numbers: type `=1.125-1.1` into a cell, and you will see `0.025`. While this might be what you expect, it is not what Excel stores internally. Change the display format to show at least 16 decimals, and it will change to `0.0249999999999999`. This is the effect of *floating-point inaccuracy*: computers live in a binary world, i.e., they calculate only with zeros and ones. Certain decimal fractions like `0.1` can't be stored as a finite binary floating-point number, which explains the result from the subtraction. In Python, you will see the same effect, but Python doesn't hide the decimals from you:

```
In [8]: 1.125 - 1.1
```

```
Out[8]: 0.0249999999999999
```


Mathematical operators

Calculating with numbers requires the use of mathematical operators like the plus or minus sign. Except for the power operator, there shouldn't be any surprise if you come from Excel:

```
In [9]: 3 + 4 # Sum
Out[9]: 7

In [10]: 3 - 4 # Subtraction
Out[10]: -1

In [11]: 3 / 4 # Division
Out[11]: 0.75

In [12]: 3 * 4 # Multiplication
Out[12]: 12

In [13]: 3**4 # The power operator (Excel uses 3^4)
Out[13]: 81

In [14]: 3 * (3 + 4) # Use of parentheses
Out[14]: 21
```

Now that you know the basics of working with numbers, let's see how to comment your code!

Comments

In the previous section, I was describing the type of operation by using *comments* (e.g., `# Sum`). Comments help other people (and yourself, a few weeks after writing the code) understand what's going on in your program. My advice is to only comment those things that are not already evident from reading the code. For example, rather than explaining *what* the code does, describe the *why*. Consider the following examples:

```
# Set discount to 0.1 ❶
discount = 0.1

# Large orders get a 10% discount ❷
discount = 0.1

discount_for_large_order = 0.1 ❸
```

- ❶ The comment duplicates what the code already shows, which isn't helpful.
- ❷ This comment explains *why* the discount is set to 0.1.

- ③ Whenever possible, the code should be self-documenting, e.g., by using descriptive variable names, making a comment obsolete.

AI-generated code often comes with a lot of comments—it doesn't hurt to remove those that aren't adding any value. Anything starting with a # sign is a comment in Python and is ignored when you run the code:

```
In [15]: # This is a sample we've seen before.  
        # Every comment line has to start with a #  
        3 + 4
```

```
Out[15]: 7
```

```
In [16]: 3 + 4 # This is an inline comment
```

```
Out[16]: 7
```

Most editors have a keyboard shortcut to comment/uncomment lines. In Jupyter notebooks and VS Code, it is Ctrl+/ (Windows) or Command-/ (macOS). Note that Markdown cells in Jupyter notebooks won't accept comments—if you start a line with a # there, Markdown will interpret this as a heading.

With comments covered, let's move straight to booleans!

Booleans

The boolean types in Python are True or False, exactly like in VBA. The boolean operators and, or, and not, however, are all lowercase, while VBA shows them capitalized. Boolean expressions are similar to how they work in Excel formulas, except for equality and inequality operators:

```
In [17]: 3 == 4 # Equality (Excel uses 3 = 4)
```

```
Out[17]: False
```

```
In [18]: 3 != 4 # Inequality (Excel uses 3 <> 4)
```

```
Out[18]: True
```

```
In [19]: 3 < 4 # Smaller than. Use > for bigger than.
```

```
Out[19]: True
```

```
In [20]: 3 <= 4 # Smaller or equal. Use >= for bigger or equal.
```

```
Out[20]: True
```

```
In [21]: # You can chain logical expressions  
        # In VBA, this would be: 10 < 12 And 12 < 17  
        # In Excel formulas, this would be: =AND(10 < 12, 12 < 17)  
        10 < 12 < 17
```

```
Out[21]: True
```

```
In [22]: not True # "not" operator
```

```
Out[22]: False
In [23]: False and True # "and" operator
Out[23]: False
In [24]: False or True # "or" operator
Out[24]: True
```

Every Python object evaluates to either True or False. The majority of objects are True, but there are some that evaluate to False, including None (see sidebar), False, and 0 or empty data types, e.g., an empty string or an empty list (I'll introduce strings and lists in a moment).

None

None is a built-in constant and represents “the absence of a value” according to the official docs. For example, if a function does not explicitly return anything, it returns None. It is also a good choice to represent empty cells in Excel as we will see in [Part III](#) and [Part IV](#).

To double-check if an object is True or False, use the bool constructor:

```
In [25]: bool(2)
Out[25]: True
In [26]: bool(0)
Out[26]: False
In [27]: bool("some text") # We'll get to strings in a moment
Out[27]: True
In [28]: bool("")
Out[28]: False
In [29]: bool(None)
Out[29]: False
```

With booleans in our pocket, there is one more basic data type left: text data, better known as *strings*.

Strings

If you have ever worked with strings in VBA that are longer than one line and contain variables and literal quotes, you probably wished it was easier. Fortunately, this is an area where Python is particularly strong. Strings can be expressed by using either double quotes (") or single quotes ('). The only condition is that you have to start

and end the string with the same type of quotes. You can use + to join (“concatenate”) strings or * to repeat them. Since we were repeating strings in the previous chapter, here is a sample using the plus sign:

```
In [30]: "A double quote string. " + 'A single quote string.'
```

```
Out[30]: 'A double quote string. A single quote string.'
```

Depending on what you want to write, using single or double quotes can help you to easily print literal quotes without the need to escape them. If you still need to escape a character, you precede it with a backslash:

```
In [31]: print("Don't wait! " + 'Learn how to "speak" Python.')
```

```
Out[31]: Don't wait! Learn how to "speak" Python.
```

```
In [32]: print("It's easy to \"escape\" characters with a leading \\\.")
```

```
Out[32]: It's easy to "escape" characters with a leading \.
```

When you are mixing strings with variables, you usually work with *f-strings*, short for *formatted string literal*. Simply put an *f* in front of your string and use variables in between curly braces:

```
In [33]: # Note how Python allows you to conveniently assign multiple
# values to multiple variables in a single line
first_adjective, second_adjective = "free", "open source"
f"Python is {first_adjective} and {second_adjective}."
```

```
Out[33]: 'Python is free and open source.'
```

As I mentioned at the beginning of this section, strings are objects like everything else, and they offer a few methods (i.e., functions) to perform an action on that string. For example, this is how you transform between upper and lowercase letters:

```
In [34]: "PYTHON".lower()
```

```
Out[34]: 'python'
```

```
In [35]: "python".upper()
```

```
Out[35]: 'PYTHON'
```

Getting Help

How do you know what attributes certain objects like strings offer and what arguments their methods accept? The answer depends on the tool you use: with classic Jupyter notebooks, hit the Tab key after typing the dot that follows an object, for example, "python". **<Tab>**. This will make a dropdown appear with all the attributes and methods that this object offers. If your cursor is in a method, for example, within the parentheses of "python".upper(), hit Shift+Tab to get the description of that method. VS Code will display this information automatically as a tooltip. If you run a Python REPL, use dir("python") to get the available attributes and

`help("python".upper)` to print the description of the `upper` method. Other than that, it's always a good idea to get back to Python's [online documentation](#). If you are looking for the documentation of third-party packages like `pandas`, it's helpful to search for them on [PyPI](#), Python's package index, where you will find the links to the respective home pages and documentation. Nowadays, AI assistants like ChatGPT or Microsoft Copilot are also a great way to get support if you are lost (see [Appendix B](#)).

When working with strings, you often need to select a part: for example, you may want to get the USD part out of the EURUSD exchange rate notation. The next section shows you Python's powerful indexing and slicing mechanism that allows you to do this easily.

Indexing and Slicing

Indexing and slicing give you access to specific elements of a sequence. Since strings are sequences of characters, we can use them to learn how it works. In the next section, we will meet additional sequences like lists and tuples that support indexing and slicing too.

Indexing

[Figure 3-1](#) introduces the concept of *indexing*. Python is zero-based, which means that the first element in a sequence is referred to by index 0. Negative indices from -1 allow you to refer to elements from the end of the sequence.

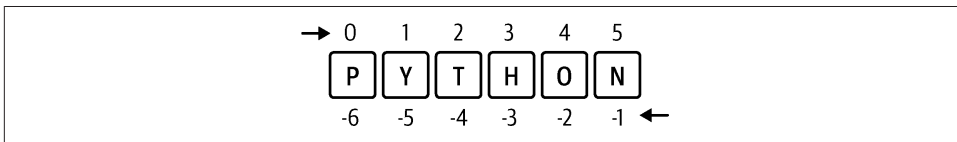


Figure 3-1. Indexing from the beginning and end of a sequence



Common Error Traps for VBA Developers

If you are coming from VBA, indexing is a common error trap. VBA uses one-based indexing for most collections like sheets (`Sheets(1)`) but uses zero-based indexing for arrays (`MyArray(0)`), although that default can be changed. Another difference is that VBA uses parentheses for indexing, while Python uses square brackets.

The syntax for indexing is as follows:

```
sequence[index]
```

Accordingly, you access specific elements from a string like this:

```
In [36]: language = "PYTHON"
In [37]: language[0]
Out[37]: 'P'
In [38]: language[1]
Out[38]: 'Y'
In [39]: language[-1]
Out[39]: 'N'
In [40]: language[-2]
Out[40]: 'O'
```

If you want to extract more than just a single character, you have to use slicing.

Slicing

If you want to get more than one element from a sequence, use the *slicing* syntax, which works as follows:

```
sequence[start:stop:step]
```

Python uses half-open intervals: the `start` index is included, while the `stop` index is not. If you don't specify the `start` or `stop` arguments, it will include everything from the beginning or to the end of the sequence, respectively. `step` determines the direction and the step size: for example, 2 will return every second element and -3 will return every third element starting from the end of the sequence. The default step size is one:

```
In [41]: language[:3] # Same as language[0:3]
Out[41]: 'PYT'
In [42]: language[1:3]
Out[42]: 'YT'
In [43]: language[-3:] # Same as language[-3:6]
Out[43]: 'HON'
In [44]: language[-3:-1]
Out[44]: 'HO'
In [45]: language[::2] # Every second element
Out[45]: 'PTO'
In [46]: language[-1:-4:-1] # A negative step goes from end to beginning
Out[46]: 'NOH'
```

So far we've looked at just a single index or slice operation, but Python also allows you to *chain* multiple index and slice operations together. For example, if you want to get the second character out of the last three characters, you could do it like this:

```
In [47]: language[-3:][1]
Out[47]: '0'
```

This is the same as `language[-2]`, so in this case, it wouldn't make much sense to use chaining, but it will make more sense when we use indexing and slicing with lists, one of the data structures that I am going to introduce in the next section.

Data Structures

Python provides powerful data structures that make it easy to work with collections of objects. In this section, I am going to introduce lists, dictionaries, tuples, and sets. While each of these data structures has slightly different characteristics, they are all able to hold multiple objects. In VBA, you may have used collections or arrays to hold multiple values. VBA even offers a data structure called dictionary that works conceptually the same as Python's dictionary. It is, however, only available on the Windows version of Excel. Let's get started with lists, the data structure that you will probably use most.

Lists

Lists are capable of holding multiple objects of different data types. They are so versatile that you will use them all the time. You create a list as follows:

```
[element1, element2, ...]
```

Here are two lists, one with the names of Excel files and the other one with a few numbers:

```
In [48]: file_names = ["one.xlsx", "two.xlsx", "three.xlsx"]
         numbers = [1, 2, 3]
```

Like strings, lists can easily be merged ("concatenated") with the plus sign. This also shows you that lists can hold different types of objects:

```
In [49]: file_names + numbers
Out[49]: ['one.xlsx', 'two.xlsx', 'three.xlsx', 1, 2, 3]
```

As lists are objects like everything else, they can also have other lists as their elements. I will refer to them as *nested lists*:

```
In [50]: nested_list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

If you rearrange this to span over multiple lines, you can easily recognize that this is a very nice representation of a matrix, or a range of spreadsheet cells. Note that the

square brackets implicitly allow you to break the lines (see sidebar). Via indexing and slicing, you get the elements you want:

```
In [51]: cells = [[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]]

In [52]: cells[1] # Second row
Out[52]: [4, 5, 6]

In [53]: cells[1][1:] # Second row, second and third column
Out[53]: [5, 6]
```

Line Continuation

Sometimes, a line of code can get so long that you will need to break it up into two or more lines to keep your code readable. Technically, you can either use parentheses or a backslash to break up the line:

```
In [54]: a = (1 + 2
              + 3)

In [55]: a = 1 + 2 \
              + 3
```

Python's style guide, however, prefers that you use *implicit line breaks* if possible: whenever you are using an expression that contains parentheses, square brackets, or curly braces, use them to create a line break without having to introduce an additional character. I will say more about Python's style guide toward the end of this chapter.

You can add elements to lists:

```
In [56]: users = ["Linda", "Brian"]

In [57]: users.append("Jennifer") # Most commonly you add to the end
                                     users

Out[57]: ['Linda', 'Brian', 'Jennifer']

In [58]: users.insert(0, "Kim") # Insert "Kim" at index 0
                                     users

Out[58]: ['Kim', 'Linda', 'Brian', 'Jennifer']
```

To delete an element, use either `pop` or `del`. While `pop` is a method, `del` is implemented as a statement in Python:

```
In [59]: users.pop() # Removes and returns the last element by default
Out[59]: 'Jennifer'

In [60]: users
```



```
Out[60]: ['Kim', 'Linda', 'Brian']
```

```
In [61]: del users[0] # del removes an element at the given index
```

Some other useful things you can do with lists are:

```
In [62]: len(users) # Length
```

```
Out[62]: 2
```

```
In [63]: "Linda" in users # Check if users contains "Linda"
```

```
Out[63]: True
```

```
In [64]: print(sorted(users)) # Returns a new sorted list
          print(users) # The original list is unchanged
```

```
Out[64]: ['Brian', 'Linda']
          ['Linda', 'Brian']
```

```
In [65]: users.sort() # Sorts the original list
          users
```

```
Out[65]: ['Brian', 'Linda']
```

Note that you can use `len` and `in` with strings as well:

```
In [66]: len("Python")
```

```
Out[66]: 6
```

```
In [67]: "free" in "Python is free and open source."
```

```
Out[67]: True
```

To get access to elements in a list, you refer to them by their index, i.e., position—that’s not always practical. Dictionaries, the topic of the next section, allow you to get access to elements via a key (often a name).

Dictionaries

Dictionaries map keys to values. You will come across key-value combinations all the time. The easiest way to create a dictionary is as follows:

```
{key1: value1, key2: value2, ...}
```

While lists allow you to access elements by index, i.e., position, dictionaries allow you to access elements by key. As with indices, keys are accessed via square brackets. The following code samples will use the dictionary `exchange_rates`, which maps currency pairs (“key”) to exchange rates (“value”):

```
In [68]: exchange_rates = {"EURUSD": 1.1152, "GBPUSD": 1.2454, "AUDUSD": 0.6161}
```

```
In [69]: exchange_rates["EURUSD"] # Access the EURUSD exchange rate
```

```
Out[69]: 1.1152
```

To change existing values or add new key-value pairs, do the following:

```
In [70]: exchange_rates["EURUSD"] = 1.2 # Change an existing value
         exchange_rates
```

```
Out[70]: {'EURUSD': 1.2, 'GBPUSD': 1.2454, 'AUDUSD': 0.6161}
```

```
In [71]: exchange_rates["CADUSD"] = 0.714 # Add a new key/value pair
         exchange_rates
```

```
Out[71]: {'EURUSD': 1.2, 'GBPUSD': 1.2454, 'AUDUSD': 0.6161, 'CADUSD': 0.714}
```

To merge two or more dictionaries, use the pipe operator. If the second dictionary contains keys from the first one, the values from the first will be overridden. You can see this happening in the following example by looking at the GBPUSD exchange rate:

```
In [72]: exchange_rates | {"SGDUSD": 0.7004, "GBPUSD": 1.2222}
```

```
Out[72]: {'EURUSD': 1.2,
          'GBPUSD': 1.2222,
          'AUDUSD': 0.6161,
          'CADUSD': 0.714,
          'SGDUSD': 0.7004}
```

Many objects can serve as keys; the following is an example with integers:

```
In [73]: currencies = {1: "EUR", 2: "USD", 3: "AUD"}
```

```
In [74]: currencies[1]
```

```
Out[74]: 'EUR'
```

By using the `get` method, dictionaries allow you to use a default value in case the key doesn't exist:

```
In [75]: # currencies[100] would raise an exception since the key doesn't exist.
         # Using "get" prevents the exception and delivers the default instead.
         currencies.get(100, "N/A")
```

```
Out[75]: 'N/A'
```

Now that you know how to work with dictionaries, let's move on to the next data structure: tuples. They are similar to lists except for one big difference.

Tuples

Tuples are similar to lists with the difference that they are *immutable*: once created, their elements can't be changed. While you can often use tuples and lists interchangeably, tuples are the obvious choice for a collection that never changes throughout the program. Technically, tuples are created by separating values with commas:

```
mytuple = element1, element2, ...
```

However, using parentheses often makes it easier to read:

```
In [76]: currencies = ("EUR", "GBP", "AUD")
```

Tuples allow you to access elements the same way as lists, but they won't allow you to change elements. Instead, merging ("concatenating") tuples will create a new tuple behind the scenes, then bind your variable to this new tuple:

```
In [77]: currencies[0] # Accessing the first element
Out[77]: 'EUR'

In [78]: # Concatenating tuples will return a new tuple
         currencies + ("SGD",)

Out[78]: ('EUR', 'GBP', 'AUD', 'SGD')
```

I explain the difference between *mutable* versus *immutable* objects in detail in [Appendix C](#), but for now, let's have a look at the last data structure of this section: sets.

Sets

Sets are collections that have no duplicate elements. You create sets by using curly braces:

```
{element1, element2, ...}
```

Let's see an example:

```
In [79]: currencies = {"USD", "EUR", "SGD"}
         currencies

Out[79]: {'EUR', 'SGD', 'USD'}
```

You can use sets to get the unique values of a list or a tuple by using the set constructor like so:

```
In [80]: set(["USD", "USD", "SGD", "EUR", "USD", "EUR"])

Out[80]: {'EUR', 'SGD', 'USD'}
```

Other than that, you can apply set theory operations like union and intersection:

```
In [81]: portfolio1 = {"USD", "EUR", "SGD", "CHF"}
         portfolio2 = {"EUR", "SGD", "CAD"}

In [82]: # Same as portfolio2.union(portfolio1)
         portfolio1.union(portfolio2)

Out[82]: {'CAD', 'CHF', 'EUR', 'SGD', 'USD'}

In [83]: # Same as portfolio2.intersection(portfolio1)
         portfolio1.intersection(portfolio2)

Out[83]: {'EUR', 'SGD'}
```

For a full overview of set operations, see the [official docs](#). Before moving on, let's quickly revise the four data structures we just met in [Table 3-1](#). It shows a sample for each data structure in the notation I used in the previous paragraphs, the so-called *literals*. Additionally, I am also listing their constructors that offer an alternative to

using the literals and are often used to convert from one data structure to another. For example, to convert a tuple to a list, do:

```
In [84]: currencies = ("USD", "EUR", "CHF")
         currencies

Out[84]: ('USD', 'EUR', 'CHF')

In [85]: list(currencies)

Out[85]: ['USD', 'EUR', 'CHF']
```

Table 3-1. Data structures

Data structure	Literals	Constructor
List	[1, 2, 3]	list((1, 2, 3))
Dictionary	{"a": 1, "b": 2}	dict(a=1, b=2)
Tuple	(1, 2, 3)	tuple([1, 2, 3])
Set	{1, 2, 3}	set((1, 2, 3))

At this point, you know all important data types, including basic ones like floats and strings, and data structures like lists and dictionaries. Next, you'll learn how to make your code respond to different conditions and how to write loops.

Control Flow

This section presents the `if` statement as well as the `for` and `while` loops. The `if` statement allows you to execute certain lines of code only if a condition is met, and the `for` and `while` loops will execute a block of code repeatedly. To use these control flow statements, however, you need to understand one of Python's most noteworthy particularities: significant white space. So let's start with this!

Code Blocks and the `pass` Statement

A *code block* defines a section in your source code that is used for something special. For example, you use a code block to define the lines over which your program is looping, or it makes up the definition of a function. In Python, you define code blocks by indenting them, not by using keywords (like in VBA) or curly braces (like in most other languages). This is referred to as *significant white space*. The Python community has settled on four spaces as indentation, but you usually insert them by hitting the Tab key: both Jupyter notebooks and VS Code will automatically convert tabs into four spaces. Let me show you how code blocks are formally defined by using the `if` statement:

```
if condition:
    pass # Do nothing
```

The line preceding the code block always terminates with a colon. Since the end of the code block is reached when you no longer indent the line, you need to use the `pass` statement if you want to create a dummy code block that does nothing. In VBA, this would correspond to the following:

```
If condition Then
    ' Do nothing
End If
```

Note that “Do nothing” is a comment in both versions and could be left out. Now that you know how to define code blocks, I can properly introduce the `if` statement.

The `if` Statement and Conditional Expressions

To introduce the `if` statement, let me reproduce the example from “[Readability Counts](#)” on page 4 in [Chapter 1](#) but this time in Python:

```
In [86]: i = 20
        if i < 5:
            print("i is less than 5")
        elif i <= 10:
            print("i is greater than 5 but less than or equal to 10")
        else:
            print("i is greater than 10")
```

```
Out[86]: i is greater than 10
```

If you would do the same as we did in [Chapter 1](#), i.e., indent the `elif` and `else` statements, you would get a `SyntaxError`. Python won’t let you indent your code differently from the logic. Compared to VBA, the keywords are lowercase, and instead of using `ElseIf` in VBA, Python uses `elif`. Note that in Python, a simple `if` statement doesn’t require any parentheses around it, and to test if a value is `True`, you don’t need to do that explicitly. Here is what I mean by that:

```
In [87]: is_important = True
        if is_important:
            print("This is important.")
        else:
            print("This is not important.")
```

```
Out[87]: This is important.
```

The same works if you want to check if a sequence like a list is empty or not:

```
In [88]: values = []
        if values:
            print(f"The following values were provided: {values}")
        else:
            print("There were no values provided.")
```

```
Out[88]: There were no values provided.
```

Conditional expressions, also called *ternary operators*, allow you to use a more compact style for simple if/else statements:

```
In [89]: is_important = False
         print("important") if is_important else print("not important")

Out[89]: not important
```

With if statements and conditional expressions in our pocket, let's move on to for and while loops.

The for and while Loops

If you need to do something repeatedly, a for loop can often do the work for you. for loops iterate over the items of a sequence like a list, a tuple, or a string (remember, strings are sequences of characters). As an introductory example, let's create a for loop that takes each element of the `currencies` list, assigns it to the variable `currency`, and prints it—one after another until there are no more elements in the list:

```
In [90]: currencies = ["USD", "HKD", "AUD"]

         for currency in currencies:
             print(currency)
```

```
Out[90]: USD
         HKD
         AUD
```

As a side note, VBA's For Each statement is close to how Python's for loop works. The previous example could be written like this in VBA:

```
Dim currencies As Variant
Dim ccy As Variant 'currency is a reserved word in VBA

currencies = Array("USD", "HKD", "AUD")

For Each ccy In currencies
    Debug.Print ccy
Next
```

In Python, if you need a variable that keeps track of the number of iterations in a for loop ("counter variable"), the `range` or `enumerate` built-in functions can help you with that. Let's first look at `range`, which provides a sequence of numbers: you call it by either providing a single stop argument or by providing a start and stop argument, with an optional step argument. `start` is inclusive, `stop` is exclusive, and `step` determines the step size, with 1 being the default:

```
range(stop) # Option 1
range(start, stop, step) # Option 2
```

`range` evaluates lazily, which means that it generates numbers on the fly, and without explicitly asking for it, you won't see the whole sequence. This lazy evaluation is memory-efficient, as `range` doesn't need to store all numbers at once:

```
In [91]: range(5)
Out[91]: range(0, 5)
```

Converting the range to a list makes the values visible:

```
In [92]: list(range(5)) # stop argument
Out[92]: [0, 1, 2, 3, 4]

In [93]: list(range(2, 5, 2)) # start, stop, step arguments
Out[93]: [2, 4]
```

Normally, you want to use `range` directly, though, to take advantage of its memory-efficiency:

```
In [94]: for i in range(3):
          print(i)

Out[94]: 0
          1
          2
```

If you need a counter variable while looping over a sequence, use `enumerate`. It returns a sequence of (index, element) tuples. By default, the index starts at zero and increments by one. You can use `enumerate` in a loop like this:

```
In [95]: for i, currency in enumerate(currencies):
          print(i, currency)

Out[95]: 0 USD
          1 HKD
          2 AUD
```

Looping over tuples and sets works the same as with lists. When you loop over dictionaries, however, Python will loop over the keys:

```
In [96]: exchange_rates = {"EURUSD": 1.1152, "GBPUSD": 1.2454, "AUDUSD": 0.6161}
          for currency_pair in exchange_rates:
              print(currency_pair)

Out[96]: EURUSD
          GBPUSD
          AUDUSD
```

Use the `items` method to get the key and the value as a tuple:

```
In [97]: for currency_pair, exchange_rate in exchange_rates.items():
          print(currency_pair, exchange_rate)
```

```
Out[97]: EURUSD 1.1152
         GBPUSD 1.2454
         AUDUSD 0.6161
```

To exit a loop immediately, use the `break` statement:

```
In [98]: for i in range(15):
         if i == 2:
             break
         else:
             print(i)
```

```
Out[98]: 0
         1
```

You skip the remainder of an iteration with the `continue` statement, which means that the loop continues with the next element `i`:

```
In [99]: for i in range(4):
         if i == 2:
             continue
         else:
             print(i)
```

```
Out[99]: 0
         1
         3
```

When comparing for loops in VBA with Python, there is a subtle difference: in VBA, the counter variable increases beyond your upper limit after finishing the loop:

```
For i = 1 To 3
    Debug.Print i
Next i
Debug.Print i
```

This prints:

```
1
2
3
4
```

In Python, after a for loop finishes, the counter variable stays at its last value:

```
In [100]: for i in [1, 2, 3]:
         print(i)
         print(i)
```

```
Out[100]: 1
         2
         3
         3
```

Instead of looping over a sequence, you can also use *while loops* to run a loop while a certain condition is true:


```
In [101]: n = 0
          while n <= 2:
              print(n)
              n += 1
```

```
Out[101]: 0
          1
          2
```



Augmented Assignment

I have used the *augmented assignment* notation in the last example: `n += 1`. This is the same as if you would write `n = n + 1` and also works with other mathematical operators; for example, `n = n - 2` can be written as `n -= 2`.

Quite often, you will need to collect certain elements in a list for further processing. In this case, Python offers an alternative to writing loops: list comprehensions.

List Comprehensions

List comprehensions are technically a way to create a list, but they often replace a for loop, which is why I am introducing them here. As an introductory example, in the following list of USD currency pairs, you need to select those currencies where USD is quoted as the second currency. You could write the following for loop:

```
In [102]: currency_pairs = [
          "USDJPY",
          "USDGBP",
          "USDCHF",
          "USDCAD",
          "AUDUSD",
          "NZDUSD",
          ]

In [103]: usd_quote = []
          for pair in currency_pairs:
              if pair[3:] == "USD":
                  usd_quote.append(pair[:3])
          usd_quote
```

```
Out[103]: ['AUD', 'NZD']
```

This is often easier to write with a *list comprehension*. A list comprehension is a concise way of creating a list. You can grab its syntax from the following example, which does the same as the previous for loop:

```
In [104]: [pair[:3] for pair in currency_pairs if pair[3:] == "USD"]

Out[104]: ['AUD', 'NZD']
```

If you don't have any condition to satisfy, simply remove the `if` part. For example, to invert all the currency pairs so that the first currency comes second, and vice versa, you would do:

```
In [105]: [f"{pair[3:]}{pair[:3]}" for pair in currency_pairs]
Out[105]: ['JPYUSD', 'GBPUSD', 'CHFUSD', 'CADUSD', 'USDAUD', 'USDNZD']
```

At this point, you are able to write simple scripts, as you know most of the basic building blocks of Python. In the next section, you will learn how to organize your code to keep it maintainable when your projects start to get bigger.

Code Organization

In this section, we'll bring our code into a maintainable structure: I'll start by introducing functions with all the details that you will commonly need before I'll show you how to split your code into different Python modules. The knowledge about modules will then allow us to look into the `datetime` module that is part of the standard library.

Functions

Even if you will use Python for simple scripts only, you are still going to write functions regularly: they allow you to reuse the same lines of code from anywhere in your program. We'll start this section by defining a function before we see how to call it!

Defining functions

To write your own function in Python, you have to use the keyword `def`, which stands for function *definition*. Unlike VBA, Python doesn't differentiate between a function and a Sub procedure. In Python, the equivalent of a Sub procedure is simply a function without a return statement. Functions in Python follow the syntax for code blocks, i.e., you end the first line with a colon and indent the body of the function:

```
def function_name(required_argument, optional_argument=default_value, ...):
    return result
```

Required arguments

Required arguments do not have a default value. Multiple arguments are separated by commas.

Optional arguments

You make an argument optional by supplying a default value. `None` is often used to make an argument optional if there is no meaningful default.

Return value

The `return` statement defines the result that the function returns. If you remove it, the function returns `None`.

To be able to play around with a function, let's define one that is able to convert the temperature from Fahrenheit or Kelvin to degrees Celsius:

```
In [106]: def convert_to_celsius(degrees, source="fahrenheit"):
          if source.lower() == "fahrenheit":
              return (degrees - 32) * (5 / 9)
          elif source.lower() == "kelvin":
              return degrees - 273.15
          else:
              # Exceptions will be introduced in Chapter 10
              raise ValueError(f"Don't know how to convert from {source}")
```

I am using the string method `lower`, which transforms the provided strings to lowercase. This allows us to accept the source string with any capitalization, and the comparison will still work. With the `convert_to_celsius` function defined, let's see how we can call it!

Calling functions

As briefly mentioned at the beginning of this chapter, you call a function by adding parentheses to the function name, enclosing the function arguments:

```
result = function_name(positional_arg, arg_name=value, ...)
```

Positional arguments

If you provide a value as a positional argument (`positional_arg`), the values are matched to the arguments according to their position in the function definition.

Keyword arguments

By providing the argument in the form `arg_name=value`, you're providing a keyword argument. This has the advantage that you can provide the arguments in any order. It is also more explicit to the reader and can make it easier to understand. For example, if the function is defined as `f(a, b)`, you could call the function like this: `f(b=1, a=2)`. This concept also exists in VBA as *named arguments* with the following syntax: `f(b:=1, a:=2)`.

Let's play around with the `convert_to_celsius` function to see how this works in practice:

```
In [107]: convert_to_celsius(100, "fahrenheit") # Positional arguments
Out[107]: 37.77777777777778

In [108]: convert_to_celsius(100) # Will use the default source ("fahrenheit")
Out[108]: 37.77777777777778
```

```
In [109]: convert_to_celsius(source="kelvin", degrees=0) # Keyword arguments
Out[109]: -273.15
```

Now that you know how to define and call functions, let's see how to organize them with the help of modules.

Modules and the import Statement

When you work on bigger projects, you will have to split your code into different files to make it easier to maintain. As we have already seen in the previous chapter, Python files have the extension `.py`, and you usually refer to your main file as a *script*. If you want your main script to access functionality from other files, you need to *import* that functionality first. In this context, Python source files are called *modules*. To get a better feeling for how this works and what the different import options are, open the file `temperature.py` (Example 3-1) in VS Code. You'll find it in the root directory of the companion repository. If you need a refresher on how to open files in VS Code, have another look at [Chapter 2](#).

Example 3-1. temperature.py

```
TEMPERATURE_SCALES = ("fahrenheit", "kelvin", "celsius")

def convert_to_celsius(degrees, source="fahrenheit"):
    if source.lower() == "fahrenheit":
        return (degrees - 32) * (5 / 9)
    elif source.lower() == "kelvin":
        return degrees - 273.15
    else:
        # Exceptions will be introduced in Chapter 10
        raise ValueError(f"Don't know how to convert from {source}")

print("This is the temperature module.")
```

To be able to import the `temperature` module, `temperature.py` needs to be in the same directory as the Jupyter notebook—this is already the case in the companion repository. To import, you only use the name of the module, without the `.py` ending. After running the `import` statement, you will have access to all the objects in that Python module via the dot notation. For example, use `temperature.convert_to_celsius()` to perform your conversion:

```
In [110]: import temperature
Out[110]: This is the temperature module.
In [111]: temperature.TEMPERATURE_SCALES
Out[111]: ('fahrenheit', 'kelvin', 'celsius')
```

```
In [112]: temperature.convert_to_celsius(120, "fahrenheit")
```

```
Out[112]: 48.88888888888889
```

Note that I used uppercase letters for `TEMPERATURE_SCALES` to express that it is a constant—I will say more about that toward the end of this chapter. When you execute the cell with `import temperature`, Python will run the *temperature.py* file from top to bottom. You can easily see this happening since importing the module will fire the print function at the bottom of *temperature.py*.



Modules Are Only Imported Once

If you run the `import temperature` cell again, you will notice that it does not print anything anymore. This is because Python modules are only imported once per session. If you change code in a module that you import, you need to restart your Python interpreter to pick up all the changes. To restart a Jupyter notebook, click on the Restart button at the top of a notebook in VS Code, or go to Kernel > Restart Kernel in classic Jupyter notebooks.

In reality, you usually don't print anything in modules. This was only to show you the effect of importing a module more than once. Most commonly, you put functions and classes in your modules (for more on classes, see [Appendix C](#)). If you don't want to type `temperature` every time you use an object from the `temperature` module, change the `import` statement like this:

```
In [113]: import temperature as tp
```

```
In [114]: tp.TEMPERATURE_SCALES
```

```
Out[114]: ('fahrenheit', 'kelvin', 'celsius')
```

Assigning an alias such as `tp` to your module can make it easier to use while it's still always clear where an object comes from. Many third-party packages suggest a specific convention when using an alias. For example, `pandas` uses the alias `pd`. There is one more option to import objects from another module:

```
In [115]: from temperature import TEMPERATURE_SCALES, convert_to_celsius
```

```
In [116]: TEMPERATURE_SCALES
```

```
Out[116]: ('fahrenheit', 'kelvin', 'celsius')
```



The `__pycache__` Folder

When you import the `temperature` module, you will see that Python creates a folder called `__pycache__` with files that have the `.pyc` extension. These are bytecode-compiled files that the Python interpreter creates when you import a module. For our purposes, we can simply ignore this folder, as it is a technical detail of how Python runs your code.

When using the `from ... import ...` syntax, you import specific objects only. By doing this, you are importing them directly into the *namespace* of your main script: that is, without looking at the `import` statements, you won't be able to tell whether the imported objects were defined in your current Python script or Jupyter notebook or if they come from another module. This could cause conflicts: if your main script has a function called `convert_to_celsius`, it would override the one that you are importing from the `temperature` module. If, however, you use one of the two previous methods, your local function and the one from the imported module could live next to each other as `convert_to_celsius` and `temperature.convert_to_celsius`.



Don't Name Your Scripts Like Existing Packages

A common source for errors is naming your Python file the same as an existing Python package or module. If you create a file to play around with `pandas`, don't call that file `pandas.py`, as this can cause conflicts.

Now that you know how the import mechanism works, let's use it to import the `datetime` module! This will also allow you to learn a few more things about objects and classes.

The `datetime` Class

Working with date and time is a common operation in Excel, but it comes with limitations: for example, date-formatted cells don't support smaller units than milliseconds, and time zones are not supported at all. In Excel, date and time are stored as a simple float called the *date serial number*. The Excel cell is then formatted to display it as date and/or time. For example, January 1, 1900, has the date serial number of 1, which means that this is also the earliest date that you can work with in Excel. Time gets translated into the decimal part of the float, e.g., 01/01/1900 10:10:00 is represented by 1.4236111111.

In Python, to work with date and time, you import the `datetime` module, which is part of the standard library. The `datetime` module contains a class with the same name that allows us to create `datetime` objects. Since having the same name for the

module and the class can be confusing, I will use the following import convention throughout this book: `import datetime as dt`. This makes it easy to differentiate between the module (`dt`) and the class (`datetime`).

Up to this point, we were mostly using *literals* to create objects like lists or dictionaries. Literals refer to the syntax that Python recognizes as a specific object type—in the case of a list, this would be something like `[1, 2, 3]`. However, the majority of objects have to be created by calling their class: this process is called *instantiation*, and objects are therefore also called *class instances*. Calling a class works the same way as calling a function, i.e., you add parentheses to the class name and provide the arguments as we did with functions. To instantiate a `datetime` object, you need to call the class like this:

```
import datetime as dt
dt.datetime(year, month, day, hour, minute, second, microsecond, timezone)
```

Let's go through a couple of examples to see how you work with `datetime` objects in Python. For the purpose of this introduction, we'll ignore time zones and work with time-zone-naive `datetime` objects. If you want to know more about time zones, though, have a look at [Appendix C](#):

```
In [117]: # Import the datetime module as "dt"
import datetime as dt

In [118]: # Instantiate a datetime object called "timestamp"
timestamp = dt.datetime(2020, 1, 31, 14, 30)
timestamp

Out[118]: datetime.datetime(2020, 1, 31, 14, 30)

In [119]: # Datetime objects have attributes to access specific components.
# Here, we are extracting the day:
timestamp.day

Out[119]: 31

In [120]: # The difference of two datetime objects returns a timedelta object
timestamp - dt.datetime(2020, 1, 14, 12, 0)

Out[120]: datetime.timedelta(days=17, seconds=9000)

In [121]: # Add or subtract timedelta objects to shift datetime values
timestamp + dt.timedelta(days=1, hours=4, minutes=11)

Out[121]: datetime.datetime(2020, 2, 1, 18, 41)
```

To *format* `datetime` objects into strings, you usually use f-strings. Alternatively, `datetime` objects have a `strftime` method that can do the same. I will show both options in the code examples that follow. To *parse* a string and convert it into a date time object, use the `strptime` function (you can find an overview of the accepted format codes in the [datetime docs](#)):

```

In [122]: # Format a datetime object via f-string.
          # Note that the format specifier starts with a colon.
          f"{timestamp:%d/%m/%Y %H:%M}"

Out[122]: '31/01/2020 14:30'

In [123]: # Using strftime instead of an f-string
          timestamp.strftime("%d/%m/%Y %H:%M")

Out[123]: '31/01/2020 14:30'

In [124]: # Parse a string into a datetime object
          dt.datetime.strptime("12.1.2020", "%d.%m.%Y")

Out[124]: datetime.datetime(2020, 1, 12, 0, 0)

```

After this short introduction to the `datetime` module, let's move on to the last topic of this chapter, which is about formatting your code properly and discovering bugs without even running your code.

Code Formatting, Linting, and Type Hinting

In this section, you'll learn how to format your code according to the official Python style guide. To do the formatting automatically, I will show you the Ruff package, which is both a code formatter and linter. While the formatter edits your code to, e.g., always use the same amount of empty lines after a function, a linter checks your code for style and programming errors. To conclude, I'll introduce you to type hints: they help editors like VS Code provide better code autocompletion and allow type checkers to catch type-related errors. To get started, let's find out what PEP 8 means!

PEP 8: Style Guide for Python Code

While going through the code samples in this chapter, you may have noticed that I was sometimes using variable names with underscores or in all caps. To explain my formatting choices, let me introduce you to Python's official style guide. Python uses so-called Python Enhancement Proposals (PEPs) to discuss the introduction of new language features. One of these, the Style Guide for Python Code, is often referred to by its number: PEP 8. PEP 8 is a set of style recommendations for the Python community; if everybody who works on the same code adheres to the same style guide, the code becomes much more readable.

Example 3-2 shows a short Python file that introduces the most important conventions.

Example 3-2. pep8_sample.py

```

"""This script shows a few PEP 8 rules.""" ❶

import datetime as dt ❷

```



```

TEMPERATURE_SCALES = ( ❸
    "fahrenheit",
    "kelvin",
    "celsius",
)
❹

class TemperatureConverter: ❺
    pass # Doesn't do anything at the moment ❻

def convert_to_celsius(degrees, source="fahrenheit"): ❼
    """This function converts degrees Fahrenheit or Kelvin
    into degrees Celsius. ❽
    """
    if source.lower() == "fahrenheit": ❾
        return (degrees - 32) * (5 / 9) ❿
    elif source.lower() == "kelvin":
        return degrees - 273.15
    else:
        # Exceptions will be introduced in Chapter 10
        raise ValueError(f"Don't know how to convert from {source}")

celsius = convert_to_celsius(44, source="fahrenheit") ⓫
non_celsius_scales = TEMPERATURE_SCALES[:-1] ⓫

print("Current time: " + dt.datetime.now().isoformat())
print(f"The temperature in Celsius is: {celsius}")

```

- ❶ Explain what the script or module does with a *docstring* at the top. A docstring is a special type of string, enclosed with triple quotes. Apart from serving as a string for documenting your code, a docstring also makes it easy to write strings over multiple lines and is useful if your text contains a lot of double-quotes or single-quotes, as you won't need to escape them. They are also useful to write multiline SQL queries, as we will see in [Chapter 10](#).
- ❷ All imports are at the top of the file, one per line. List the imports of the standard library first, then those of third-party packages, and finally those from your own modules. This sample only makes use of the standard library.
- ❸ Use capital letters with underscores for constants. Use a maximum line length of 79 characters. If possible, take advantage of parentheses, square brackets, or curly braces for implicit line breaks.
- ❹ Separate classes and functions with two empty lines from the rest of the code.

- ❺ Despite the fact that many classes from the standard library, like `datetime`, are lowercase, your own classes should use `CapitalizedWords` (also called “*Camel-Case*”). For more on classes, see [Appendix C](#).
- ❻ Inline comments should be separated by at least two spaces from the code. Code blocks should be indented by four spaces.
- ❼ Functions and function arguments should use lowercase names with underscores if they improve readability. Don’t use spaces between the argument name and its default value unless you are using type hints (I will introduce type hints at the end of this section).
- ❽ A function’s docstring should also list and explain the function arguments and its return value. I haven’t done this here to keep the sample short.
- ❾ Don’t use spaces around the colon.
- ❿ Use spaces around mathematical operators. If operators with different priorities are used, you may consider adding spaces around those with the lowest priority only.
- ⓫ Use lowercase names for variables. Make use of underscores if they improve readability. When assigning a variable name, use spaces around the equal sign. However, when calling a function, don’t use spaces around the equal sign used with keyword arguments.
- ⓫ With indexing and slicing, don’t use spaces around the square brackets.

This is a simplified summary of PEP 8, so it’s a good idea to have a look at the original [PEP 8](#). PEP 8 clearly states that it is a recommendation and that your own style guides will take precedence. After all, consistency is the most important factor.

In practice, I didn’t format [Example 3-2](#) by hand. Instead, I used a package called `Ruff` to do the work for me—let me show you how it works!

Ruff

In order to save you from manually applying the PEP 8 rules, let me introduce you to `Ruff`, a Python package that is both a code formatter and linter. Let’s start with the formatter before looking at the linter!

Formatter

The Ruff formatter is an opinionated formatter, meaning it offers only a few options you can change. This has the nice effect that the code looks consistent across all projects that use it. The Ruff formatter largely follows PEP 8 with a few exceptions—for instance, it uses a default line length of 88 characters, while PEP 8 uses 79. Since Ruff is a drop-in replacement for **Black** (the package that made automatic code formatting popular in Python), you can also say that Ruff produces Black-style formatting.

To try out Ruff, open a Terminal and run `uv run ruff format` in the directory of your companion repository:

Windows (Terminal)

```
cd %userprofile%\python-for-excel-2e
uv run ruff format pep8_sample.py
```

macOS (Terminal)

```
cd ~/python-for-excel-2e
uv run ruff format pep8_sample.py
```

It will show 1 file left unchanged, as the file is already properly formatted. Now introduce a change to `pep8_sample.py`, for example, add a few spaces in front of "fahrenheit" like so:

```
TEMPERATURE_SCALES = (
    "fahrenheit",
    "kelvin",
    "celsius",
)
```

Make sure to save the file, then run `uv run ruff format pep8_sample.py` one more time—ruff will remove these additional spaces again! After this quick introduction to the formatter, let's continue with the linter!

Linter

A linter checks your source code for programming and style errors without running your code. To run Ruff's linter, use `ruff check` (as usual, you can ignore the `cd` command if you are already in the directory of the companion repository):

Windows (Terminal)

```
cd %userprofile%\python-for-excel-2e
uv run ruff check pep8_sample.py
```

macOS (Terminal)

```
cd ~/python-for-excel-2e
uv run ruff check pep8_sample.py
```

It will show `All checks passed!` as the file doesn't have any errors. However, if you, for example, insert `import os` at the top of `pep8_sample.py` and run the command again, Ruff will return the following error message: `os imported but unused`.



Ruff VS Code Extension

If you are working with VS Code, instead of running the Ruff commands manually, it's arguably more convenient to install the Ruff extension:

1. On the Activity Bar, go to Extensions and search for “Ruff” by Astral Software and click on Install. Once installed, linting issues are marked directly in the editor with squiggly lines.
2. To format your code, open the Command Palette via `Ctrl+Shift+P` (Windows) or `Command-Shift-P` (macOS) and search for “Ruff: Format Document.”

To find errors that a linter can't catch, e.g., supplying the wrong data type as a function argument, you can use type hints together with a type checker. Another benefit of type hints is that they enable editors such as VS Code to offer much better code completion—reason enough to look at them in a little more detail!

Type Hints

In VBA, you often see code that prefixes each variable with an abbreviation for the data type, like `strEmployeeName` or `wbWorkbookName`. While nobody will stop you from doing this in Python, it isn't commonly done. You also won't find an equivalent to VBA's `Option Explicit` or `Dim` statement to declare the type of a variable. Instead, Python offers *type hints*. Type hints are also called *type annotations* and allow you to declare the data type of a variable. They are optional and have no effect on how the code is run by the Python interpreter. Third-party packages, however, can make use of these type hints during runtime. For example, in [Chapter 13](#), we will see how `xlwings` makes use of type hints to convert Excel values into specific Python objects. The main benefits of type hints are:

- They improve autocompletion in editors like VS Code.
- They let type checkers find type-related errors (like linters, type checkers don't need to run your code).

To understand how type annotations work in Python, open the file `pep8_sample.py` in VS Code. On line 22, place your cursor after `degrees` and type a dot. You'll notice that VS Code doesn't show any suggestions for methods or attributes. Remove that dot again and add type hints to the `convert_to_celsius` function definition:

```
def convert_to_celsius(degrees: float, source: str = "fahrenheit") -> float:
```

Type hints are added by writing a colon and the expected type after each parameter, and the return type is specified after the closing parenthesis using `->`. Now try typing the dot after `degrees` again on line 22. This time, VS Code shows a list of attributes and methods that work for floats, as shown in [Figure 3-2](#).

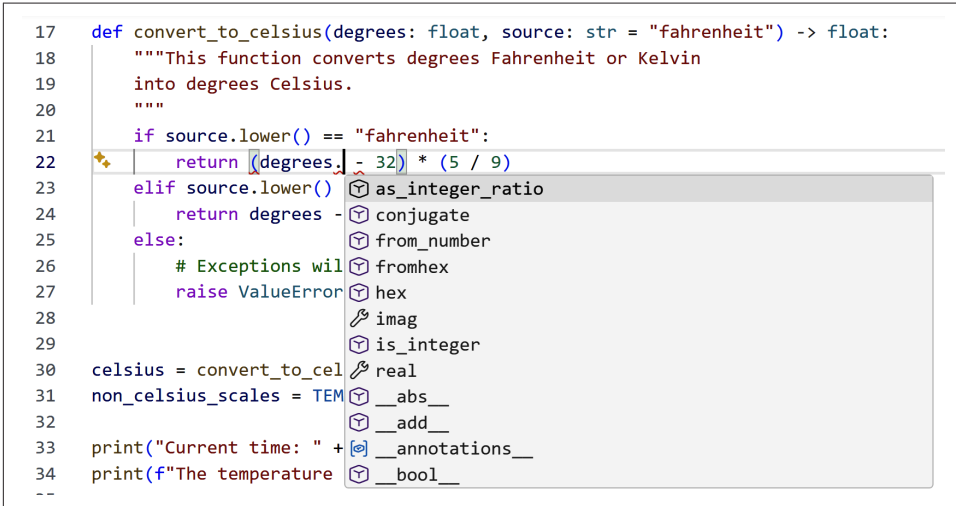


Figure 3-2. Code autocompletion after adding type hints

There are various type-checkers available for Python code. For example, Pylance, a VS Code extension by Microsoft, has one included. Pylance was automatically installed during the installation of the Python extension in [Chapter 2](#). However, by default, the Pylance type-checker is off, so if you want to enable it, open the VS Code settings via `Ctrl+,` (Windows) or `Command+,` (macOS), then search for `@id:python.analysis.typeCheckingMode`; this will pull up the respective page on the settings with instructions on how to enable the type-checker. Another type-checker is [Ty](#), which is from the same team as [Ruff](#). To try it out, open a Terminal and, if you are not already in the root directory of your companion repository, change into it first:

Windows (Terminal)

```
cd %userprofile%\python-for-excel-2e
uv run ty check pep8_sample.py
```

macOS (Terminal)

```
cd ~/python-for-excel-2e
uv run ty check pep8_sample.py
```

You should get All checks passed!. However, if you change the following line:

```
celsius = convert_to_celsius(44, source="fahrenheit")
```

to this:

```
celsius = convert_to_celsius("forty-four", source="fahrenheit")
```

and then run the command again, Ty will complain with the following error:

```
Argument to function convert_to_celsius is incorrect.
```

Despite the benefits that type hints bring, some Python programmers argue that they make the code harder to read. In this book, I will use type hints occasionally, where they make sense.

Conclusion

This chapter was a packed introduction to Python. We met the most important building blocks of the language, including data structures, functions, and modules. We also touched on some of Python's particularities like meaningful white space and code formatting guidelines, better known as PEP 8. To continue with this book, you won't need to know all the details: as a beginner, just knowing about lists and dictionaries and indexing and slicing, as well as how to work with functions, modules, for loops, and if statements, will get you far.

Compared to VBA, I find Python more consistent and powerful but at the same time easier to learn. If you are a VBA die-hard fan and this chapter didn't convince you just yet, I am pretty sure the next part will: there, I will give you an introduction to array-based calculations before starting our data analysis journey with the pandas library. Let's get started with **Part II** by learning a few basics about NumPy!

Introduction to pandas

NumPy Foundations

As you may recall from [Chapter 1](#), NumPy is the core package for scientific computing in Python, providing support for array-based calculations and linear algebra. As NumPy is a key dependency of pandas, I am going to introduce its basics in this chapter: after explaining what a NumPy array is, we will look into vectorization and broadcasting, two important concepts that allow you to write concise mathematical code and that you will find again in pandas. After that, we're going to see why NumPy offers special functions called *universal functions* before we wrap this chapter up by learning how to get and set values of an array and by explaining the difference between a view and a copy of an array. Even if we will hardly use NumPy directly in this book, knowing its basics will make it easier to learn pandas in the next chapter.

Getting Started with NumPy

In this section, you'll learn about one- and two-dimensional NumPy arrays and what's behind the technical terms *vectorization*, *broadcasting*, and *universal function*.

NumPy Array

To perform array-based calculations with nested lists, you have to write some sort of loop. For example, you can use a nested list comprehension to add a number to every element in `matrix`:

```
In [1]: matrix = [[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]]

In [2]: [[element + 1 for element in row] for row in matrix]

Out[2]: [[2, 3, 4], [5, 6, 7], [8, 9, 10]]
```

This isn't very readable, and more importantly, looping through each element is slow with big arrays. Depending on your use case and the size of the arrays, working with NumPy arrays instead of Python lists can make your calculations from a couple of times to around a hundred times faster. NumPy achieves this performance by using code written in C and Fortran—these are compiled programming languages that are much faster than Python. A NumPy array is an N-dimensional array for *homogeneous data*. *Homogeneous* means that all elements in the array need to have the same data type. Most commonly, you are dealing with one- and two-dimensional arrays of floats as schematically displayed in [Figure 4-1](#).

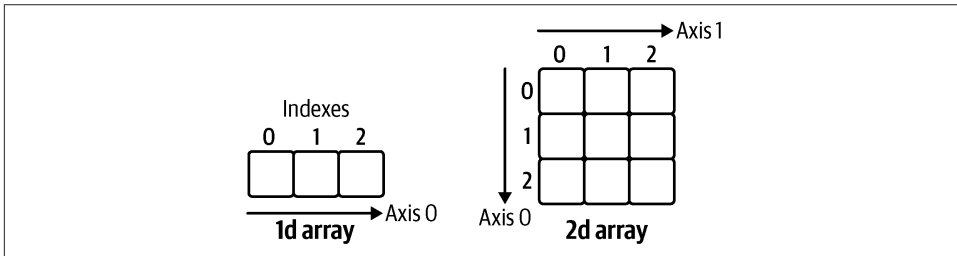


Figure 4-1. A one-dimensional and two-dimensional NumPy array

Let's create a one- and two-dimensional array to work with throughout this chapter:

```
In [3]: # Import NumPy with the "np" alias
import numpy as np

In [4]: # Constructing an array with a simple list results in a 1d array
array1 = np.array([10, 100, 1000.0])
array1

Out[4]: array([ 10., 100., 1000.])

In [5]: # Constructing an array with a nested list results in a 2d array
array2 = np.array([[1.0, 2.0, 3.0],
                   [4.0, 5.0, 6.0]])
array2

Out[5]: array([[1., 2., 3.],
               [4., 5., 6.]])
```

Even if `array1` has two integers (10 and 100) and only one float (1,000), the homogeneity of NumPy arrays forces the data type of the array to be `float64`, which is capable of accommodating all elements. To learn about an array's data type, access its `dtype` attribute:

```
In [6]: array1.dtype

Out[6]: dtype('float64')
```

Did you notice that `dtype` displays `float64` instead of the plain `float` type you saw in the last chapter? This is because NumPy uses its own numerical data types (`float64` stands for 64-bit floating-point number). This usually isn't an issue though, as in most cases you can use a NumPy float anywhere you would use a native Python float. If you end up with an error when passing a NumPy data type to a function of another library, you can simply use the corresponding constructor to convert the NumPy data type to a Python data type (I will say more about accessing an element from an array shortly):

```
In [7]: array1[0]
Out[7]: np.float64(10.0)

In [8]: # Convert from a NumPy float64 to a native Python float
        float(array1[0])

Out[8]: 10.0
```

For a full list of NumPy's data types, see the [NumPy docs](#). With NumPy arrays, you can write simple code to perform array-based calculations, as we will see next.

Vectorization and Broadcasting

If you build the sum of two NumPy arrays, NumPy will perform an element-wise operation so you don't have to loop through the elements yourself. This is called *vectorization*. It allows you to write concise code, practically representing the mathematical notation:

```
In [9]: array2 + array2
Out[9]: array([[ 2.,  4.,  6.],
               [ 8., 10., 12.]])
```

If you use two arrays with different shapes in an arithmetic operation, NumPy extends—if possible—the smaller array automatically across the larger array so that their shapes become compatible. This is called *broadcasting*. The first example illustrates how this works by summing up an array with a scalar, and the second example multiplies a one-dimensional array with a two-dimensional array:

```
In [10]: array2 + 1
Out[10]: array([[2., 3., 4.],
               [5., 6., 7.]])

In [11]: array2 * array1
Out[11]: array([[ 10., 200., 3000.],
               [ 40., 500., 6000.]])
```



Scalar

Scalar refers to a single-element data type like a float or a string. This is to differentiate them from data structures with multiple elements like lists, dictionaries, or NumPy arrays.

To perform matrix multiplications or dot products, use the @ operator. This sample shows a matrix multiplication of array2 with its transpose:

```
In [12]: array2 @ array2.T # array2.T is a shorthand for array2.transpose()
Out[12]: array([[14., 32.],
               [32., 77.]])
```

Don't be intimidated by the terminology I've introduced in this section, such as scalar, vectorization, or broadcasting! All this works the same in Excel if you use named ranges together with dynamic arrays. **Figure 4-2** shows the previous examples together with the respective array-based Excel formula. As you can see, the formulas look the same with NumPy and Excel, with only the last one being slightly different. The screenshot is taken from *array_calculations.xlsx*, which you will find in the *xl* directory of the companion repository.

	A	B	C	D	E	F	G	H	I	J
1					• NumPy notation: array2 + 1					
2		array1 (Named range)			• Excel formula: =array2 + 1					
3		10	100	1000	2	3	4			
4					5	6	7			
5										
6					• NumPy notation: array2 * array2					
7		array2 (Named range)			• Excel formula: =array2 * array2					
8		1	2	3	1	4	9			
9		4	5	6	16	25	36			
10										
11					• NumPy notation: array2 * array1					
12					• Excel formula: =array2 * array1					
13		10	200	3000						
14		40	500	6000						
15										
16					• NumPy notation: array2 @ array2.T					
17					• Excel formula: =MMULT(array2, TRANSPOSE(array2))					
18		14	32							
19		32	77							

Figure 4-2. Array-based calculations in Excel

You know now that arrays perform arithmetic operations element-wise, but how can you apply a function on every element in an array? This is what universal functions are for.

Universal Functions (ufunc)

Universal functions (ufunc) work on every element in a NumPy array. For example, if you use Python's standard square root function from the `math` module on a NumPy array, you will get an error:

```
In [13]: import math

In [14]: math.sqrt(array2) # This will raise an Error

-----
TypeError                                Traceback (most recent call last)
Cell In[15], line 1
----> 1 math.sqrt(array2) # This will raise an Error
```

TypeError: only 0-dimensional arrays can be converted to Python scalars

You could, of course, write a nested loop to compute the square root of every element and build a new NumPy array from the result:

```
In [15]: # We're using a nested list comprehension to do the looping
         np.array([[math.sqrt(element) for element in row] for row in array2])

Out[15]: array([[1.          , 1.41421356, 1.73205081],
                [2.          , 2.23606798, 2.44948974]])
```

This makes sense in cases where NumPy doesn't offer a ufunc and the array is small enough. However, if NumPy has a ufunc, you should use it, as it will be much faster with big arrays—apart from being easier to type and read:

```
In [16]: np.sqrt(array2)

Out[16]: array([[1.          , 1.41421356, 1.73205081],
                [2.          , 2.23606798, 2.44948974]])
```

Some of NumPy's ufuncs, like `sum`, are additionally available as array methods: if you want the sum of each column in a two-dimensional array, do the following:

```
In [17]: array2.sum(axis=0) # Returns a 1d array

Out[17]: array([5., 7., 9.])
```

The argument `axis=0` refers to the axis along the rows, while `axis=1` refers to the axis along the columns, as depicted in [Figure 4-1](#). Removing the `axis` argument sums up the whole array:

```
In [18]: array2.sum()

Out[18]: np.float64(21.0)
```

You will meet more NumPy ufuncs in later chapters, as they can also be used with pandas DataFrames.

So far, we've always worked with the entire array. The next section shows you how to manipulate parts of an array and introduces a few helpful array constructors.

Creating and Manipulating Arrays

I'll start this section by showing you how to get and set specific elements of an array before introducing a few useful array constructors, including one to create pseudorandom numbers. To wrap it up, I'll explain the difference between a view and a copy of an array.

Getting and Setting Array Elements

In the last chapter, I showed you how to access specific elements of a list via indexing and slicing. As an example, to get the first element of the first row from a nested list called `matrix`, you write `matrix[0][0]`. NumPy arrays, however, expect you to provide the index and slice arguments for both dimensions in a single pair of square brackets:

```
numpy_array[row_selection, column_selection]
```

For one-dimensional arrays, this simplifies to `numpy_array[selection]`. When you select a single element, you will get back a scalar; otherwise, you will get back a one- or two-dimensional array. Remember that slice notation uses a start index (included) and an end index (excluded) with a colon in between, as in `start:end`. By removing the start and end index, you are left with a colon, which therefore stands for all rows or all columns in a two-dimensional array. I have visualized a few examples in [Figure 4-3](#), but you may also want to give [Figure 4-1](#) another look, as the indices and axes are labeled there. Remember, by slicing a column or row of a two-dimensional array, you end up with a one-dimensional array, not with a two-dimensional column or row vector!

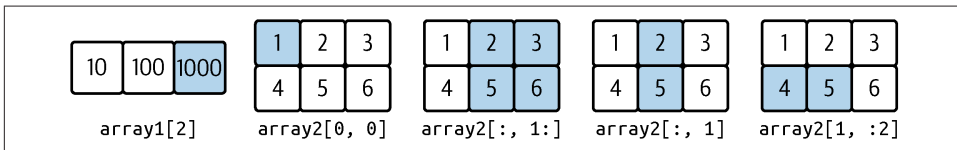


Figure 4-3. Selecting elements of a NumPy array

To play around with the examples shown in [Figure 4-3](#), run the following code:

```
In [19]: array1[2] # Returns a scalar
Out[19]: np.float64(1000.0)

In [20]: array2[0, 0] # Returns a scalar
Out[20]: np.float64(1.0)

In [21]: array2[:, 1:] # Returns a 2d array
Out[21]: array([[2., 3.],
               [5., 6.]])
```

```
In [22]: array2[:, 1] # Returns a 1d array
Out[22]: array([2., 5.])

In [23]: array2[1, :2] # Returns a 1d array
Out[23]: array([4., 5.])
```

So far, I have constructed the sample arrays by hand, i.e., by providing numbers in a list. In practice, though, NumPy offers several functions to create arrays more easily!

Useful Array Constructors

NumPy offers a few ways to construct arrays that will also be helpful to create pandas DataFrames, as we will see in [Chapter 5](#). One way to easily create arrays is to use the `arange` function. This stands for *array range* and is similar to the built-in function `range` that we met in the previous chapter—with the difference that `arange` returns a NumPy array. Combining it with `reshape` allows us to quickly generate an array with the desired dimensions:

```
In [24]: np.arange(2 * 5).reshape(2, 5) # 2 rows, 5 columns
Out[24]: array([[0, 1, 2, 3, 4],
               [5, 6, 7, 8, 9]])
```

Another common need, for example, for Monte Carlo simulations, is to generate arrays of normally distributed pseudorandom numbers. NumPy makes this easy:

```
In [25]: rng = np.random.default_rng() # Random number generator
         rng.standard_normal((2, 3)) # 2 rows, 3 columns

Out[25]: array([[ -0.48133152, -1.39335439,  1.01016164],
               [ 0.04042171, -0.67380965,  1.3339146 ]])
```



Your Numbers Will Be Different

As these numbers are pseudorandom numbers, they will look different on your end. If you always want to work with the same pseudorandom variables to get reproducible results, you can set a *seed*, which ensures that you get the same random numbers each time you run your code.

For example, `rng = np.random.default_rng(seed=123)` will always produce the same sequence of random numbers (you can use any integer as the seed value).

Other helpful constructors are `np.ones` and `np.zeros` to create arrays with ones and zeros, respectively, and `np.eye` to create an identity matrix. We'll come across some of these constructors again in the next chapter, but for now, let's continue with the difference between a view and a copy of a NumPy array.

View Versus Copy

NumPy arrays return *views* when you slice them. This means that you are working with a subset of the original array without copying the data. Setting a value on a view will therefore also change the original array:

```
In [26]: # This is the original array
         array2

Out[26]: array([[1., 2., 3.],
               [4., 5., 6.]])

In [27]: # Slicing means that subset is a view
         subset = array2[:, :2]
         subset

Out[27]: array([[1., 2.],
               [4., 5.]])

In [28]: # Changing a value in subset...
         subset[0, 0] = 1000

In [29]: # ...changes the subset...
         subset

Out[29]: array([[1000.,  2.],
               [  4.,  5.]])

In [30]: # ...but also the original array
         array2

Out[30]: array([[1000.,  2.,  3.],
               [  4.,  5.,  6.]])
```

If that's not what you want, create your subset as a copy:

```
subset = array2[:, :2].copy()
```

Working on a copy will leave the original array unchanged.

Conclusion

In this chapter, I showed you how to work with NumPy arrays and explained the concepts of vectorization and broadcasting. Putting these technical terms aside, using arrays should feel quite intuitive given that they follow the mathematical notation very closely. While NumPy is an incredibly powerful library, there are two main issues when you intend to use it for data analysis:

Homogeneous data types

As seen at the beginning of this chapter, NumPy arrays require all elements to have the same data type. This, for example, means that you can't perform any arithmetic operations when your array contains a mix of text and numbers.

Missing labels

NumPy arrays don't provide labels for rows or columns, which makes it harder to keep track of what each column or row represents.

To solve these issues, the pandas library provides the data structures Series and DataFrame. The next chapter will teach you how to use them!

Data Analysis with pandas

This chapter will introduce you to pandas, the *Python Data Analysis Library* or—how I like to put it—the Python-based spreadsheet with superpowers. The pandas library makes tasks that are particularly painful in Excel easier, faster, and less error-prone. Some of these tasks include getting datasets from external sources and working with statistics, time series, and interactive charts. If you're familiar with Power Query in Excel, pandas covers similar territory but with more flexibility. pandas' most important superpowers are vectorization and data alignment. As we've already seen in the previous chapter with NumPy arrays, vectorization allows you to write concise, array-based code, while data alignment makes sure that there is no data mismatch when you work with multiple datasets.

This chapter covers the entire data analysis journey: it starts with cleaning and preparing data before it shows you how to make sense out of bigger datasets via aggregation, descriptive statistics, and visualization. At the end of the chapter, we'll see how we can import and export data with pandas. But first things first—let's get started with an introduction to pandas' main data structures: DataFrame and Series!

DataFrame and Series

DataFrame and Series are the core data structures in pandas. In this section, I am introducing the main components of a DataFrame: index, columns, and data. A *DataFrame* is similar to a two-dimensional NumPy array, but it comes with column and row labels, and each column can hold different data types. By extracting a single column or row from a DataFrame, you get a one-dimensional Series. Again, a *Series* is similar to a one-dimensional NumPy array with labels. When you look at the structure of a DataFrame in [Figure 5-1](#), it won't take a lot of imagination to see that DataFrames are going to be your Python-based spreadsheets.

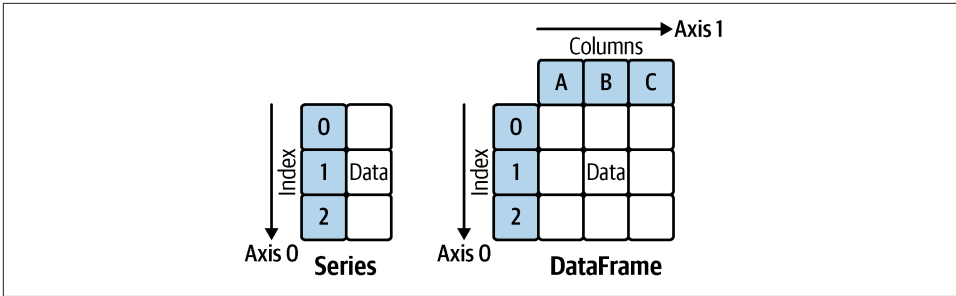


Figure 5-1. A pandas Series and DataFrame

To show you how easy it is to transition from a spreadsheet to a DataFrame, consider the following Excel table in [Figure 5-2](#), which shows students of an online course with their scores. You will find the corresponding file *students.xlsx* in the *xl* folder of the companion repo.

	A	B	C	D	E	F
1	user_id	name	born	country	score	continent
2	1001	Mark	1966	Italy	4.5	Europe
3	1000	John	1988	USA	6.7	America
4	1002	Tim	1980	USA	3.9	America
5	1003	Jenny	2009	Germany	9	Europe

Figure 5-2. *students.xlsx*

To make this Excel table available in Python, start by importing pandas, then use its `read_excel` function, which returns a DataFrame:

```
In [1]: import pandas as pd
In [2]: pd.read_excel("xl/students.xlsx", engine="calamine")
Out[2]:
```

	user_id	name	born	country	score	continent
0	1001	Mark	1966	Italy	4.5	Europe
1	1000	John	1988	USA	6.7	America
2	1002	Tim	1980	USA	3.9	America
3	1003	Jenny	2009	Germany	9.0	Europe



Calamine Engine

When you use `pd.read_excel` with `engine="calamine"`, you can read Excel files much faster than with the default engine. I will give a more detailed introduction to the calamine engine in [“Calamine: A Faster Reader Engine” on page 322](#).

If you run this in a Jupyter notebook, the DataFrame will be nicely formatted as an HTML table, which makes it even closer to how the table looks in Excel. I will spend

the whole of [Chapter 14](#) on reading and writing Excel files with pandas, so this was only an introductory example to show you that spreadsheets and DataFrames are, indeed, very similar. Let's now re-create this DataFrame with code instead of reading it from an Excel file: one way of creating a DataFrame is to provide the data as a nested list, along with values for columns and index:

```
In [3]: data = [
        ["Mark", 1966, "Italy", 4.5, "Europe"],
        ["John", 1988, "USA", 6.7, "America"],
        ["Tim", 1980, "USA", 3.9, "America"],
        ["Jenny", 2009, "Germany", 9.0, "Europe"],
    ]
    students = pd.DataFrame(
        data=data,
        columns=["name", "born", "country", "score", "continent"],
        index=[1001, 1000, 1002, 1003],
    )
    students
```

```
Out[3]:
```

	name	born	country	score	continent
1001	Mark	1966	Italy	4.5	Europe
1000	John	1988	USA	6.7	America
1002	Tim	1980	USA	3.9	America
1003	Jenny	2009	Germany	9.0	Europe



Keyword Argument Versus Variable Name

In the previous example, I am calling the DataFrame class with `data=data`. While you will see this often in Python code, it may look a bit confusing at first sight: the part on the left of the equal sign is the name of the function parameter, while the part on the right of the equal sign is the name of the variable that you pass into the function as argument. If you don't like this, you can change the variable name to something like `source_data` on the previous line so that calling the DataFrame class would read `pd.DataFrame(data=source_data, ...)`. For a refresher about keyword arguments, see [Chapter 3](#).

By calling the `info` method, you will get some basic information, most importantly the number of data points (Non-Null Count) and the data types (Dtype) for each column:

```
In [4]: students.info()

Out[4]: <class 'pandas.DataFrame'>
Index: 4 entries, 1001 to 1003
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   name        4 non-null     str
```

```

1   born      4 non-null    int64
2   country   4 non-null    str
3   score     4 non-null    float64
4   continent 4 non-null    str
dtypes: float64(1), int64(1), str(3)
memory usage: 252.0 bytes

```



String Column Data Type

If you're using an older version of pandas (before pandas 3.0), string columns will show the data type object instead of str.

If you are just interested in the data type of your columns, run `df.dtypes` instead. Let us now have a closer look at the index and columns of a DataFrame.

Index

The row labels of a DataFrame are called an *index*. If you don't have a meaningful index, don't specify one. pandas will then automatically create an integer index starting at zero. We saw this in the very first example when we read the DataFrame from the Excel file. An index will allow pandas to look up data faster and is essential for certain functionality, such as working with time series data. You access the index object like this:

```
In [5]: students.index
```

```
Out[5]: Index([1001, 1000, 1002, 1003], dtype='int64')
```

If it makes sense, give the index a name. Let's follow the table in Excel and give it the name `user_id`:

```
In [6]: students.index.name = "user_id"
students
```

```
Out[6]:
```

	name	born	country	score	continent
user_id					
1001	Mark	1966	Italy	4.5	Europe
1000	John	1988	USA	6.7	America
1002	Tim	1980	USA	3.9	America
1003	Jenny	2009	Germany	9.0	Europe

Unlike the primary key of a database, a DataFrame index can have duplicates, but looking up values may be slower in that case. To turn an index into a regular column, use `reset_index`, and to set a new index, use `set_index`. If you don't want to lose your existing index when setting a new one, make sure to reset it first:

```
In [7]: # "reset_index" turns the index into a column, replacing the
        # index with the default index. This corresponds to the DataFrame
        # from the beginning that we loaded from Excel.
students.reset_index()
```

```
Out[7]:
```

	user_id	name	born	country	score	continent
0	1001	Mark	1966	Italy	4.5	Europe
1	1000	John	1988	USA	6.7	America
2	1002	Tim	1980	USA	3.9	America
3	1003	Jenny	2009	Germany	9.0	Europe

```
In [8]: # "reset_index" turns "user_id" into a regular column and
# "set_index" turns the column "name" into the index
students.reset_index().set_index("name")
```

```
Out[8]:
```

	user_id	born	country	score	continent
name					
Mark	1001	1966	Italy	4.5	Europe
John	1000	1988	USA	6.7	America
Tim	1002	1980	USA	3.9	America
Jenny	1003	2009	Germany	9.0	Europe

By doing `df.reset_index().set_index("name")`, you are using *method chaining*: since `reset_index()` returns a DataFrame, you can directly call another DataFrame method without having to write out the intermediate result first.



DataFrame Methods Return Copies

Whenever you call a method on a DataFrame `df` in the form `df.method_name()`, you will get back a copy of the DataFrame with that method applied, leaving the original DataFrame untouched. We have just done that by calling `df.reset_index()`. If you wanted to change the original DataFrame, you would have to assign the return value back to the original variable:

```
df = df.reset_index()
```

Since we are not doing this, our variable `df` is still holding its original data. The next samples also call DataFrame methods, i.e., don't change the original DataFrame. As usual, there are no rules without exception: `df.insert()` inserts a new column in the original DataFrame, not a copy.

To change the index, use the `reindex` method:

```
In [9]: students.reindex([999, 1000, 1001, 1004])
```

```
Out[9]:
```

	name	born	country	score	continent
user_id					
999	NaN	NaN	NaN	NaN	NaN
1000	John	1988.0	USA	6.7	America
1001	Mark	1966.0	Italy	4.5	Europe
1004	NaN	NaN	NaN	NaN	NaN

This is a first example of data alignment at work: `reindex` will take over all rows that match the new index and will introduce rows with missing values (NaN) where no information exists. Index elements that you remove will be dropped. I will introduce

NaN properly later in this chapter. Finally, to sort an index, use the `sort_index` method:

```
In [10]: students.sort_index()

Out[10]:
```

	name	born	country	score	continent
user_id					
1000	John	1988	USA	6.7	America
1001	Mark	1966	Italy	4.5	Europe
1002	Tim	1980	USA	3.9	America
1003	Jenny	2009	Germany	9.0	Europe

If, instead, you want to sort the rows by one or more columns, use `sort_values`:

```
In [11]: students.sort_values(["continent", "score"])

Out[11]:
```

	name	born	country	score	continent
user_id					
1002	Tim	1980	USA	3.9	America
1000	John	1988	USA	6.7	America
1001	Mark	1966	Italy	4.5	Europe
1003	Jenny	2009	Germany	9.0	Europe

The sample shows how to sort first by continent, then by score. If you wanted to sort by only one column, you could also provide the column name as a string:

```
students.sort_values("score")
```

This section has covered the basics of how indices work. Let's now turn our attention to their counterpart, the DataFrame columns!

Columns

To get information about the columns of a DataFrame, run the following code:

```
In [12]: students.columns

Out[12]: Index(['name', 'born', 'country', 'score', 'continent'], dtype='str')
```

If you don't provide any column names when constructing a DataFrame, pandas will number the columns with integers starting at zero. With columns, however, this is almost never a good idea as columns represent variables and should therefore be easy to name. You can assign a name to the column headers in the same way we did with the index:

```
In [13]: students.columns.name = "properties"
students

Out[13]:
```

properties	name	born	country	score	continent
user_id					
1001	Mark	1966	Italy	4.5	Europe
1000	John	1988	USA	6.7	America
1002	Tim	1980	USA	3.9	America
1003	Jenny	2009	Germany	9.0	Europe

If you don't like the column names, rename them:

```
In [14]: students.rename(columns={"name": "First Name", "born": "Born"})
```

```
Out[14]: properties First Name  Born  country  score  continent
user_id
1001           Mark  1966    Italy    4.5    Europe
1000           John  1988     USA    6.7    America
1002            Tim  1980     USA    3.9    America
1003           Jenny  2009   Germany    9.0    Europe
```

If you want to delete columns, use the following syntax (the sample shows how to drop columns and indices at the same time):

```
In [15]: students.drop(columns=["name", "country"], index=[1000, 1003])
```

```
Out[15]: properties  born  score  continent
user_id
1001      1966    4.5    Europe
1002      1980    3.9    America
```

The columns and the index of a DataFrame are both represented by an Index object, so you can change your columns into rows, and vice versa, by transposing your DataFrame:

```
In [16]: students.T # Shorthand for students.transpose()
```

```
Out[16]: user_id      1001      1000      1002      1003
properties
name           Mark      John      Tim      Jenny
born           1966     1988     1980     2009
country        Italy      USA      USA     Germany
score           4.5      6.7      3.9      9.0
continent      Europe   America  America  Europe
```



Our DataFrame Is Still Unchanged!

It's worth remembering that our DataFrame `students` is still unchanged, as we have never reassigned the result from a method call back to the original `students` variable.

If you would like to reorder the columns of a DataFrame, you could use the `reindex` method that we used previously, but selecting the columns in the desired order is often more intuitive:

```
In [17]: students.loc[:, ["continent", "country", "name", "born", "score"]]
```

```
Out[17]: properties continent  country  name  born  score
user_id
1001      Europe    Italy  Mark  1966    4.5
1000      America    USA   John  1988    6.7
1002      America    USA   Tim   1980    3.9
1003      Europe    Germany Jenny 2009    9.0
```

This last example needs a few explanations: everything about `loc` and how data selection works is the topic of the next section.

Data Manipulation

Real-world data is messy, so before working with it, you need to clean it and bring it into a digestible form. We'll begin this section by looking at how to select data from a `DataFrame`, how to change it, and how to deal with missing and duplicate data. We'll then perform a few calculations with `DataFrames` and see how you work with text data. Quite a few concepts in this section are related to what we have already seen with NumPy arrays in [Chapter 4](#).

Selecting Data

Let's start with accessing data by label and position before looking at other methods, including boolean indexing.

Selecting by label

The most common way of accessing the data of a `DataFrame` is by referring to its labels. Use the attribute `loc`, which stands for *location*, to specify which rows and columns you want to retrieve:

```
df.loc[row_selection, column_selection]
```

`loc` supports the slice notation and therefore accepts a colon to select all rows or columns, respectively. Additionally, you can provide lists with labels as well as a single column or row name. Have a look at [Table 5-1](#) to see a few examples of how you select different parts from our sample `DataFrame` `students`.

Table 5-1. Data selection by label

Selection	Return data type	Example
Single value	Scalar	<code>students.loc[1000, "country"]</code>
One column (1d)	Series	<code>students.loc[:, "country"]</code>
One column (2d)	DataFrame	<code>students.loc[:, ["country"]]</code>
Multiple columns	DataFrame	<code>students.loc[:, ["country", "born"]]</code>
Range of columns	DataFrame	<code>students.loc[:, "name":"country"]</code>
One row (1d)	Series	<code>students.loc[1000, :]</code>
One row (2d)	DataFrame	<code>students.loc[[1000], :]</code>
Multiple rows	DataFrame	<code>students.loc[[1003, 1000], :]</code>
Range of rows	DataFrame	<code>students.loc[1000:1002, :]</code>



Label Slicing Has Closed Intervals

Using slice notation with labels is inconsistent with respect to how everything else in Python and pandas works: they *include* the upper bound.

Applying our knowledge from [Table 5-1](#), let's use `loc` to select scalars, Series, and DataFrames:

```
In [18]: # Using scalars for both row and column selection returns a scalar
         students.loc[1001, "name"]

Out[18]: 'Mark'

In [19]: # Using a scalar on either the row or column selection returns a Series
         students.loc[[1001, 1002], "born"]

Out[19]: user_id
         1001    1966
         1002    1980
         Name: born, dtype: int64

In [20]: # Selecting multiple rows and columns returns a DataFrame
         students.loc[:1002, ["name", "country"]]

Out[20]: properties  name country
         user_id
         1001      Mark   Italy
         1000      John    USA
         1002       Tim    USA
```

It's important that you understand the difference between a DataFrame and a Series: even with a single column, DataFrames are two-dimensional, while Series are one-dimensional. Both DataFrame and Series have an index, but only the DataFrame has column headers. When you select a column from a DataFrame, the column header becomes the Series name. Many functions or methods will work on both Series and DataFrame, but when you perform arithmetic calculations, the behavior differs: with DataFrames, pandas aligns the data according to the column headers—more about that a little later in this chapter.



Shorthand for Column Selection

Since selecting columns is such a common operation, pandas offers a shorthand. Instead of:

```
df.loc[:, column_selection]
```

you can write:

```
df[column_selection]
```

For example, `df["country"]` returns a Series from our sample DataFrame, and `df[["name", "country"]]` returns a DataFrame.

Selecting by position

Selecting a subset of a DataFrame by position works essentially the same as with NumPy arrays (see [Chapter 4](#)). With DataFrames, however, you have to use the `iloc` attribute, which stands for *integer location*:

```
df.iloc[row_selection, column_selection]
```

When using slices, you deal with the standard half-open intervals. [Table 5-2](#) gives you the same cases we looked at previously in [Table 5-1](#).

Table 5-2. Data selection by position

Selection	Return data type	Example
Single value	Scalar	<code>students.iloc[1, 2]</code>
One column (1d)	Series	<code>students.iloc[:, 2]</code>
One column (2d)	DataFrame	<code>students.iloc[:, [2]]</code>
Multiple columns	DataFrame	<code>students.iloc[:, [2, 1]]</code>
Range of columns	DataFrame	<code>students.iloc[:, :3]</code>
One row (1d)	Series	<code>students.iloc[1, :]</code>
One row (2d)	DataFrame	<code>students.iloc[[1], :]</code>
Multiple rows	DataFrame	<code>students.iloc[[3, 1], :]</code>
Range of rows	DataFrame	<code>students.iloc[1:3, :]</code>

Here is how you use `iloc`—again with the same samples that we used with `loc` before:

```
In [21]: students.iloc[0, 0] # Returns a Scalar
Out[21]: 'Mark'

In [22]: students.iloc[[0, 2], 1] # Returns a Series
Out[22]: user_id
          1001    1966
          1002    1980
          Name: born, dtype: int64

In [23]: students.iloc[:3, [0, 2]] # Returns a DataFrame
Out[23]: properties  name  country
          user_id
          1001      Mark    Italy
          1000      John     USA
          1002       Tim     USA
```

Selecting data by label or position is not the only means to access a subset of your DataFrame. Another important way is to use boolean indexing.

Selecting by boolean indexing

Boolean indexing refers to selecting subsets of a DataFrame with the help of a Series or a DataFrame whose data consists of only True or False. Boolean Series are used to select specific columns or rows of a DataFrame, while boolean DataFrames are used to select specific values across a whole DataFrame. Most commonly, you will use boolean indexing to filter the rows of a DataFrame (similar to Excel's FILTER formula). For example, this is how you filter your DataFrame so it only shows people who live in the USA and were born before 1985:

```
In [24]: tf = (students["born"] < 1985) & (students["country"] == "USA")
         tf # This is a Series with only True/False
```

```
Out[24]: user_id
         1001    False
         1000    False
         1002     True
         1003    False
         dtype: bool
```

```
In [25]: students.loc[tf, :]
```

```
Out[25]: properties name  born country  score continent
         user_id
         1002      Tim  1980     USA    3.9  America
```

There are two things I need to explain here. First, due to technical limitations, you can't use Python's boolean operators from [Chapter 3](#) with DataFrames. Instead, you need to use the symbols as shown in [Table 5-3](#).

Table 5-3. Boolean operators

Python	pandas
and	&
or	
not	~

Second, if you have more than one condition, make sure to put every boolean expression in between parentheses so operator precedence doesn't get in your way: for example, & has higher operator precedence than ==.

Therefore, without parentheses, the expression from the sample would be interpreted as:

```
students["born"] < (1985 & students["country"]) == "USA"
```

If you want to filter the index, you can refer to it as `students.index`:

```
In [26]: students.loc[students.index > 1001, :]
```

```
Out[26]: properties  name  born  country  score  continent
         user_id
         1002      Tim  1980      USA    3.9    America
         1003     Jenny  2009   Germany    9.0    Europe
```

Whereas you would use the `in` operator with Python's basic data structures, e.g., lists, you need to use `isin` when dealing with a Series. This is how you filter your DataFrame to participants from Italy and Germany:

```
In [27]: students.loc[students["country"].isin(["Italy", "Germany"]), :]
```

```
Out[27]: properties  name  born  country  score  continent
         user_id
         1001      Mark  1966     Italy    4.5    Europe
         1003     Jenny  2009   Germany    9.0    Europe
```

While you use `loc` to provide a boolean Series, DataFrames offer a special syntax without `loc` to select values given the full DataFrame of booleans:

```
df[boolean_df]
```

This is especially helpful if you have DataFrames that consist of only numbers. Providing a DataFrame of booleans returns the DataFrame with NaN wherever the boolean DataFrame is False. Again, a more detailed discussion of NaN will follow shortly. Let's start by creating a new sample DataFrame called `rainfall` that consists of only numbers:

```
In [28]: # Yearly rainfall sample (in millimeters)
         # across 3 cities over 2 years (2023-2024)
         rainfall = pd.DataFrame(
             {
                 "Alphaville": [300.1, 100.2],
                 "Betaport": [400.3, 300.4],
                 "Gammafield": [1000.5, 1100.6],
             },
             index=[2023, 2024],
         )
         rainfall
```

```
Out[28]:      Alphaville  Betaport  Gammafield
2023         300.1      400.3      1000.5
2024         100.2      300.4      1100.6
```

```
In [29]: rainfall < 400
```

```
Out[29]:      Alphaville  Betaport  Gammafield
2023         True      False      False
2024         True       True      False
```

```
In [30]: rainfall[rainfall < 400]
```

```
Out[30]:      Alphaville  Betaport  Gammafield
2023         300.1         NaN         NaN
2024         100.2      300.4         NaN
```

Note that in this example, I have used a dictionary to construct a new DataFrame—this is often convenient if the data already exists in that form. Working with booleans in this way is most commonly used to filter out specific values such as outliers.

Now that you know various ways to *select* data, it's time to learn how you *change* data.

Setting Data

The easiest way to change the data of a DataFrame is by assigning values to certain elements using the `loc` or `iloc` attributes. This is the starting point of this section before we turn to other ways of manipulating existing DataFrames: replacing values and adding new columns.

Setting data by label or position

As pointed out earlier in this chapter, when you call DataFrame methods like `df.reset_index()`, the method will be applied to a copy, leaving the original DataFrame unchanged. However, assigning values via the `loc` and `iloc` attributes changes the original DataFrame. Since I want to leave our DataFrame `students` untouched, I am working with a copy here that I am calling `students2`. If you want to change a single value, do the following:

```
In [31]: # Copy the DataFrame first to leave the original untouched
         students2 = students.copy()
```

```
In [32]: students2.loc[1000, "name"] = "JOHN"
         students2
```

```
Out[32]: properties  name  born  country  score  continent
         user_id
         1001      Mark  1966    Italy    4.5    Europe
         1000      JOHN  1988     USA    6.7    America
         1002       Tim  1980     USA    3.9    America
         1003      Jenny  2009  Germany    9.0    Europe
```

You can also change multiple values at the same time. One way to change the score of the users with ID 1000 and 1001 is to use a list:

```
In [33]: students2.loc[[1000, 1001], "score"] = [3, 4]
         students2
```

```
Out[33]: properties  name  born  country  score  continent
         user_id
         1001      Mark  1966    Italy    4.0    Europe
         1000      JOHN  1988     USA    3.0    America
         1002       Tim  1980     USA    3.9    America
         1003      Jenny  2009  Germany    9.0    Europe
```

Changing data by position via `iloc` works the same way. Let's now move on to see how you change the data by using boolean indexing.

Setting data by boolean indexing

Boolean indexing, which we used to filter rows, can also be used to assign values in a DataFrame. Imagine that you need to anonymize all names of people who were born after 1980 or who are from the USA:

```
In [34]: tf = (students2["born"] > 1980) | (students2["country"] == "USA")
students2.loc[tf, "name"] = "xxxx"
students2
```

```
Out[34]: properties  name  born  country  score  continent
user_id
1001          Mark  1966    Italy     4.0    Europe
1000          xxxx  1988     USA     3.0    America
1002          xxxx  1980     USA     3.9    America
1003          xxxx  2009   Germany    9.0    Europe
```

Sometimes, you have a dataset where you need to replace certain values across the board, i.e., not specific to certain columns. In that case, make use of the special syntax again and provide the whole DataFrame with booleans like this (the sample uses the `rainfall` DataFrame):

```
In [35]: # Copy the DataFrame first to leave the original untouched
rainfall2 = rainfall.copy()
rainfall2
```

```
Out[35]:      Alphaville  Betaport  Gammafield
2023         300.1      400.3      1000.5
2024         100.2      300.4      1100.6
```

```
In [36]: # Set the values to 0 wherever they are below 400
rainfall2[rainfall2 < 400] = 0
rainfall2
```

```
Out[36]:      Alphaville  Betaport  Gammafield
2023           0.0      400.3      1000.5
2024           0.0           0.0      1100.6
```

If you just want to replace a value with another one, there is an easier way to do it, as I will show you next.

Setting data by replacing values

If you want to replace a certain value across your entire DataFrame or selected columns, use the `replace` method:

```
In [37]: students2.replace("USA", "U.S.")
```

```
Out[37]: properties  name  born  country  score  continent
user_id
1001          Mark  1966    Italy     4.0    Europe
1000          xxxx  1988     U.S.     3.0    America
1002          xxxx  1980     U.S.     3.9    America
1003          xxxx  2009   Germany    9.0    Europe
```


If, instead, you only wanted to act on the country column, you could use a dictionary in this form:

```
students2.replace({"country": {"USA": "U.S."}})
```



Attributes Versus Methods

This is a good moment to clarify the difference between object attributes and methods, as introduced in [Chapter 3](#). *Methods* are functions that belong to an object and are called with parentheses, often with arguments. For example, in cell [11], `students` is the object and `sort_values` is the method, which you call as `students.sort_values(["continent", "score"])`. *Attributes*, on the other hand, are variables associated with an object and hold data. They are accessed without parentheses. For example, in cell [12], `students` is the object and `columns` is the attribute that holds the names of the columns: `students.columns`.

In our case, since USA only turns up in the country column, this yields the same result as the previous sample. To wrap this section up, let's see how you can add additional columns to a DataFrame.

Setting data by adding a new column

To add a new column to a DataFrame, assign values to a new column name. For example, you could add a new column to a DataFrame by using a scalar or list:

```
In [38]: students2["discount"] = 0
students2["price"] = [49.9, 49.9, 99.9, 99.9]
students2
```

```
Out[38]: properties  name  born  country  score  continent  discount  price
user_id
1001          Mark  1966    Italy     4.0    Europe         0    49.9
1000          xxxx  1988     USA     3.0    America         0    49.9
1002          xxxx  1980     USA     3.9    America         0    99.9
1003          xxxx  2009   Germany    9.0    Europe         0    99.9
```

Adding a new column often involves vectorized calculations:

```
In [39]: # Let's start with a fresh copy
students3 = students.copy()
# How old are the students in the year 2026?
students3["age"] = 2026 - students3["born"]
students3
```

```
Out[39]: properties  name  born  country  score  continent  age
user_id
1001          Mark  1966    Italy     4.5    Europe     60
1000          John  1988     USA     6.7    America     38
```

1002	Tim	1980	USA	3.9	America	46
1003	Jenny	2009	Germany	9.0	Europe	17

I will tell you more about calculating with DataFrames in a moment, but before we get there, do you remember that I have used NaN a few times already? The next section will finally give you more context around the topic of missing data.

Missing Data

Missing data can bias the results of your data analysis, thereby making your conclusions less robust. Nevertheless, it's very common to have gaps in your datasets that you will have to take care of. In Excel, you usually have to deal with empty cells, but pandas uses NumPy's `np.nan` for missing data, displayed as NaN. NaN is the floating-point standard for *Not-a-Number*. For timestamps, `pd.NaT`, which stands for *Not-a-Time*, is used instead (we will work with date and time in [Chapter 7](#)). Using `None` or `np.nan`, you can introduce missing values:

```
In [40]: students4 = students.copy() # Let's start with a fresh copy
students4.loc[1000, "score"] = None
students4.loc[1003, :] = None
students4
```

```
Out[40]: properties  name    born country  score continent
user_id
1001             Mark  1966.0    Italy    4.5     Europe
1000             John  1988.0     USA    NaN     America
1002             Tim  1980.0     USA    3.9     America
1003             NaN    NaN      NaN    NaN      NaN
```

To clean a DataFrame, you often want to remove rows with missing data. This is as simple as:

```
In [41]: students4.dropna()

Out[41]: properties  name    born country  score continent
user_id
1001             Mark  1966.0    Italy    4.5     Europe
1002             Tim  1980.0     USA    3.9     America
```

If, however, you only want to remove rows where *all* values are missing, use the `how` parameter:

```
In [42]: students4.dropna(how="all")

Out[42]: properties  name    born country  score continent
user_id
1001             Mark  1966.0    Italy    4.5     Europe
1000             John  1988.0     USA    NaN     America
1002             Tim  1980.0     USA    3.9     America
```

To get a boolean DataFrame or Series depending on whether the value is missing or not, use `isna`:

```
In [43]: students4.isna()
```

```
Out[43]: properties  name  born  country  score  continent
user_id
1001             False  False   False   False    False
1000             False  False   False   True    False
1002             False  False   False   False   False
1003              True   True    True    True    True
```

To fill missing values, use `fillna`. For example, to replace NaN in the score column with its mean:

```
In [44]: students4.fillna({"score": students4["score"].mean()})
```

```
Out[44]: properties  name  born  country  score  continent
user_id
1001           Mark  1966.0   Italy    4.5    Europe
1000           John  1988.0    USA    4.2    America
1002            Tim  1980.0    USA    3.9    America
1003            NaN     NaN     NaN    4.2     NaN
```

Missing data isn't the only condition that requires us to clean our dataset. The same is true for duplicate data, so let's see how we can deal with them!

Duplicate Data

Like missing data, duplicates negatively impact the reliability of your analysis. Let's start by introducing a duplicate row into our students DataFrame:

```
In [45]: students5 = students.copy() # Let's start with a fresh copy
students5.loc[1004,:] = students5.loc[1001, :]
students5
```

```
Out[45]: properties  name  born  country  score  continent
user_id
1001           Mark  1966.0   Italy    4.5    Europe
1000           John  1988.0    USA    6.7    America
1002            Tim  1980.0    USA    3.9    America
1003           Jenny  2009.0  Germany    9.0    Europe
1004           Mark  1966.0   Italy    4.5    Europe
```

To find out if a certain column contains duplicates or to get its unique values, use the following two commands:

```
In [46]: students5["name"].is_unique
```

```
Out[46]: False
```

```
In [47]: students5["name"].unique()
```

```
Out[47]: <ArrowStringArray>
['Mark', 'John', 'Tim', 'Jenny']
Length: 4, dtype: str
```

To understand which rows are duplicates, use the `duplicated` method, which returns a boolean Series: by default, it uses the parameter `keep="first"`, which keeps the first occurrence and marks only successive duplicates with `True`. By setting the parameter `keep=False`, it will return `True` for all nonunique rows, making it easy to get a DataFrame with all duplicate rows:

```
In [48]: # By default, "duplicated" marks only duplicates as True, i.e.,
         # the first occurrence shows False
         students5.duplicated()
```

```
Out[48]: user_id
         1001    False
         1000    False
         1002    False
         1003    False
         1004     True
         dtype: bool
```

```
In [49]: # To get all duplicate rows, use keep=False
         students5.loc[students5.duplicated(keep=False), :]
```

```
Out[49]: properties  name    born country  score continent
         user_id
         1001      Mark  1966.0    Italy    4.5    Europe
         1004      Mark  1966.0    Italy    4.5    Europe
```

And finally, to get rid of duplicate rows, use the `drop_duplicates` method:

```
In [50]: students5.drop_duplicates()
```

```
Out[50]: properties  name    born country  score continent
         user_id
         1001      Mark  1966.0    Italy    4.5    Europe
         1000      John  1988.0     USA    6.7    America
         1002       Tim  1980.0     USA    3.9    America
         1003      Jenny  2009.0  Germany    9.0    Europe
```

Once you have cleaned your DataFrames by removing missing and duplicate data, you might want to perform some arithmetic operations—the next section gives you an introduction to how this works.

Arithmetic Operations

Like NumPy arrays, DataFrames and Series make use of broadcasting. As a reminder, broadcasting extends a scalar or Series automatically across a DataFrame so that their shapes become compatible for element-wise operations. For example, to add a number to every value in the `rainfall` DataFrame, simply do the following:

```
In [51]: rainfall
```

```
Out[51]:      Alphaville  Betaport  Gammafield
         2023      300.1      400.3      1000.5
         2024      100.2      300.4      1100.6
```

```
In [52]: rainfall + 100
```

	Alphaville	Betaport	Gammafield
2023	400.1	500.3	1100.5
2024	200.2	400.4	1200.6

However, the true power of pandas is its automatic *data alignment* mechanism: when you use arithmetic operators with more than one DataFrame, pandas automatically aligns them by their columns and row indices. Let's create a second DataFrame with some of the same row and column labels. We then build the sum:

```
In [53]: more_rainfall = pd.DataFrame(
        data={
            "Alphaville": [100, 300],
            "Deltawood": [200, 400],
        },
        index=[2024, 2025],
    )
more_rainfall
```

	Alphaville	Deltawood
2024	100	200
2025	300	400

```
In [54]: rainfall + more_rainfall
```

	Alphaville	Betaport	Deltawood	Gammafield
2023	NaN	NaN	NaN	NaN
2024	200.2	NaN	NaN	NaN
2025	NaN	NaN	NaN	NaN

The index and columns of the resulting DataFrame are the union of the indices and columns of the two DataFrames: the fields that have a value in both DataFrames show the sum, while the rest of the DataFrame shows NaN. This follows the floating-point standard, which defines any arithmetic operation involving NaN to return NaN. This may be something you have to get used to if you come from Excel, where empty cells are automatically turned into zeros when you use them in arithmetic operations. To get the same behavior as in Excel, use the add method with a `fill_value` to replace NaN values with zeros:

```
In [55]: rainfall.add(more_rainfall, fill_value=0)
```

	Alphaville	Betaport	Deltawood	Gammafield
2023	300.1	400.3	NaN	1000.5
2024	200.2	300.4	200.0	1100.6
2025	300.0	NaN	400.0	NaN

This works the same for the other arithmetic operators, as shown in [Table 5-4](#).

Table 5-4. Arithmetic operators

Operator	Method
*	mul
+	add
-	sub
/	div
**	pow

It is, however, more common to calculate with individual DataFrame columns instead of whole DataFrames. When working with specific columns in the form of Series, you can take advantage of vectorized operations. For example, if you want to find out the ratio of rain between Alphaville and Betaport, you can do it like this:

```
In [56]: rainfall3 = rainfall.copy() # Let's start with a fresh copy
rainfall3["Alphaville/Betaport"] = (
    rainfall3["Alphaville"] / rainfall3["Betaport"]
)
rainfall3
```

```
Out[56]:
```

	Alphaville	Betaport	Gammafield	Alphaville/Betaport
2023	300.1	400.3	1000.5	0.749688
2024	100.2	300.4	1100.6	0.333555

While this section was about DataFrames with numbers and how they behave in arithmetic operations, the next section introduces text manipulation.

Working with Text

To perform operations on text data, i.e., on columns that have the `str` data type, use the `str` attribute to get access to Python's string methods. We have already met a few string methods in [Chapter 3](#), but it won't hurt to have a look at the available methods in the [Python docs](#). For example, to remove leading and trailing white space, use the `strip` method; or to make all first letters capitalized, use the `capitalize` method. Chaining these together will clean up messy text columns that are often the result of manual data entry:

```
In [57]: # Let's create a new DataFrame
names = pd.DataFrame({"name": [" mArk ", "JOHN ", "Tim", " jenny"]})
names
```

```
Out[57]:
```

	name
0	mArk
1	JOHN
2	Tim
3	jenny

```
In [58]: names_cleaned = names["name"].str.strip().str.capitalize()
names_cleaned
```

```
Out[58]: 0    Mark
         1    John
         2    Tin
         3    Jenny
         Name: name, dtype: str
```

Or, to find all names that start with a J:

```
In [59]: names_cleaned.str.startswith("J")
```

```
Out[59]: 0    False
         1     True
         2    False
         3     True
         Name: name, dtype: bool
```

So far, we've mostly worked with one DataFrame at a time. The next section shows you various ways to combine multiple DataFrames into one, a very common task for which pandas offers powerful tools.

Combining DataFrames

Combining different datasets in Excel can be a cumbersome task and typically involves the VLOOKUP formula. Fortunately, pandas makes combining DataFrames easy due to the data alignment capabilities of DataFrames. Combining and merging DataFrames can be done in various ways; this section walks you through the `concat`, `join`, and `merge` functions.

Concatenating

To simply glue multiple DataFrames together, the `concat` function is your best friend. As you can tell by the name of the function, this process has the technical name *concatenation*. By default, `concat` glues DataFrames together along the rows and aligns the columns automatically. In the following example, I create another DataFrame, `more_students`, and attach it to the bottom of our sample DataFrame `students`:

```
In [60]: more_students = pd.DataFrame(
        data={
            "born": [1988, 1982],
            "country": ["France", "Canada"],
            "score": [4.1, 6.1],
            "name": ["Becky", "Leanne"],
        },
        index=[1000, 1011],
    )
more_students
```

```
Out[60]:
```

	born	country	score	name
1000	1988	France	4.1	Becky
1011	1982	Canada	6.1	Leanne

```
In [61]: pd.concat([students, more_students])
```

```
Out[61]:
```

	name	born	country	score	continent
1001	Mark	1966	Italy	4.5	Europe
1000	John	1988	USA	6.7	America
1002	Tim	1980	USA	3.9	America
1003	Jenny	2009	Germany	9.0	Europe
1000	Becky	1988	France	4.1	NaN
1011	Leanne	1982	Canada	6.1	NaN

Note that the index now contains duplicates, as `concat` glues the data together on the indicated axis (rows) and only aligns the data on the other one (columns). If you want to glue two DataFrames together along the columns, set `axis=1`:

```
In [62]: more_categories = pd.DataFrame(
    data={
        "quizzes": [3, 5],
        "logins": [4, 6]
    },
    index=[1000, 2000]
)
more_categories
```

```
Out[62]:
```

	quizzes	logins
1000	3	4
2000	5	6

```
In [63]: pd.concat([students, more_categories], axis=1)
```

```
Out[63]:
```

	name	born	country	score	continent	quizzes	logins
1001	Mark	1966.0	Italy	4.5	Europe	NaN	NaN
1000	John	1988.0	USA	6.7	America	3.0	4.0
1002	Tim	1980.0	USA	3.9	America	NaN	NaN
1003	Jenny	2009.0	Germany	9.0	Europe	NaN	NaN
2000	NaN	NaN	NaN	NaN	NaN	5.0	6.0

A useful feature of `concat` is that it accepts more than two DataFrames. We will use this in [Chapter 7](#) to make a single DataFrame out of multiple CSV files:

```
pd.concat([df1, df2, df3, ...])
```

While `concat` allows you to stack DataFrames both vertically and horizontally, `join` and `merge` are similar to database joins: they combine DataFrames horizontally by matching rows based on a common column.

Joining and Merging

When you *join* two DataFrames, you combine them horizontally by matching rows based on their indices. If you have worked with relational databases before, it's the

same concept as the JOIN clause in SQL queries. **Figure 5-3** shows how the four join types (that is the inner, left, right, and outer join) work by using two sample DataFrames, df1 and df2.

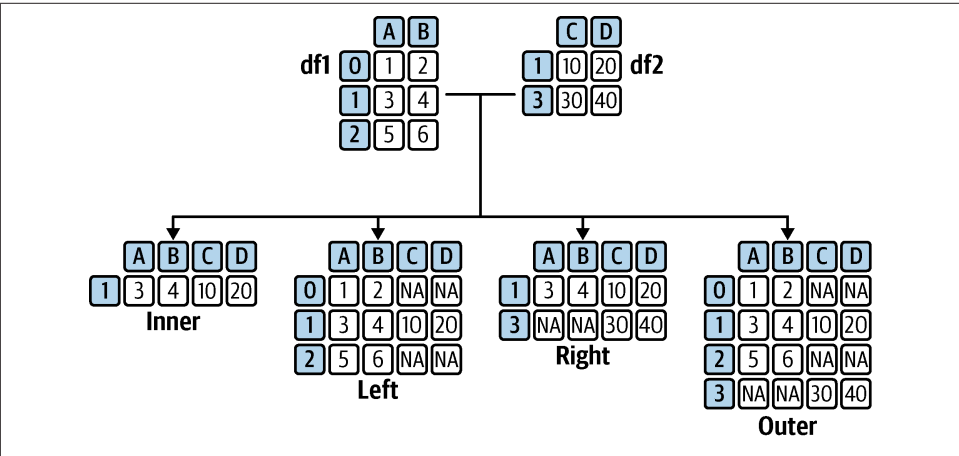


Figure 5-3. Join types

An *inner join* returns a DataFrame with only those rows where the indices overlap. A *left join* takes all the rows from the left DataFrame df1 and matches the rows from the right DataFrame df2 on the index. Where df2 doesn't have a matching row, pandas will fill in NaN. The left join corresponds to the VLOOKUP case in Excel. The *right join* takes all rows from the right table df2 and matches them with rows from df1 on the index. And finally, the *outer join*, which is short for *full outer join*, takes the union of indices from both DataFrames and matches the values where it can. **Table 5-5** is the equivalent of **Figure 5-3** in text form.

Table 5-5. Join types

Type	Description
inner	Only rows whose index exists in both DataFrames
left	All rows from the left DataFrame, matching rows from the right DataFrame
right	All rows from the right DataFrame, matching rows from the left DataFrame
outer	The union of row indices from both DataFrames

Let's see how this works in practice, bringing the examples from **Figure 5-3** to life:

```
In [64]: df1 = pd.DataFrame({"A": [1, 3, 5], "B": [2, 4, 6]})
         df1

Out[64]:   A  B
0    1  2
```

```

1  3  4
2  5  6

In [65]: df2 = pd.DataFrame({"C": [10, 30], "D": [20, 40]}, index=[1, 3])
df2

Out[65]:
   C  D
1  10 20
3  30 40

In [66]: df1.join(df2, how="inner")

Out[66]:
   A  B  C  D
1  3  4 10 20

In [67]: df1.join(df2, how="left")

Out[67]:
   A  B  C  D
0  1  2  NaN NaN
1  3  4 10.0 20.0
2  5  6  NaN NaN

In [68]: df1.join(df2, how="right")

Out[68]:
   A  B  C  D
1  3.0 4.0 10 20
3  NaN NaN 30 40

In [69]: df1.join(df2, how="outer")

Out[69]:
   A  B  C  D
0  1.0 2.0  NaN  NaN
1  3.0 4.0 10.0 20.0
2  5.0 6.0  NaN  NaN
3  NaN NaN 30.0 40.0

```

If you want to join on one or more DataFrame columns instead of relying on the index, use `merge` instead of `join`. `merge` accepts the `on` argument to provide one or more columns as the *join condition*: these columns, which have to exist on both DataFrames, are used to match the rows:

```

In [70]: # Add a column called "X" to both DataFrames
df1["X"] = ["a", "b", "c"]
df2["X"] = ["c", "b"]

In [71]: df1

Out[71]:
   A  B  X
0  1  2  a
1  3  4  b
2  5  6  c

In [72]: df2

Out[72]:
   C  D  X
1  10 20  c
3  30 40  b

```

```
In [73]: df1.merge(df2, how="inner", on=["X"])
```

```
Out[73]:   A  B  X   C   D
         0  3  4  b  30  40
         1  5  6  c  10  20
```

```
In [74]: df1.merge(df2, how="left", on=["X"])
```

```
Out[74]:   A  B  X   C   D
         0  1  2  a  NaN  NaN
         1  3  4  b  30.0 40.0
         2  5  6  c  10.0 20.0
```

Since `join` and `merge` accept quite a few optional arguments to accommodate more complex scenarios, I invite you to have a look at the [official documentation](#) to learn more about them.

You know now how to manipulate one or more DataFrames, which brings us to the next step in our data analysis journey: making sense of data.

Descriptive Statistics and Data Aggregation

One way to make sense of bigger datasets is to compute a descriptive statistic like the sum or the mean on either the whole dataset or on meaningful subsets. This section starts by calculating the mean on the entire DataFrame before it introduces two ways to aggregate data into subsets: the `groupby` method and the `pivot_table` function.

Descriptive Statistics

Descriptive statistics allows us to summarize datasets by using quantitative measures. For example, the number of data points is a simple descriptive statistic. Measures of central tendency like mean, median, or mode are other popular examples. DataFrames and Series allow you to access descriptive statistics conveniently via methods like `sum`, `mean`, and `count`, to name just a few. You will meet many of them throughout this book, and the full list is available via the [pandas documentation](#). By default, they return a Series along axis 0, which means you get the statistic of the columns:

```
In [75]: rainfall
```

```
Out[75]:   Alphaville  Betaport  Gammafield
         2023      300.1     400.3     1000.5
         2024      100.2     300.4     1100.6
```

```
In [76]: rainfall.mean()
```

```
Out[76]: Alphaville      200.15
         Betaport       350.35
         Gammafield    1050.55
         dtype: float64
```

If you want the statistic per row, provide the `axis` argument:

```
In [77]: rainfall.mean(axis=1)

Out[77]: 2023    566.966667
         2024    500.400000
         dtype: float64
```

By default, missing values are not included in descriptive statistics like `sum` or `mean`. This is in line with how Excel treats empty cells, so using Excel's `AVERAGE` formula on a range with empty cells will give you the same result as the `mean` method applied on a Series with the same numbers and `NaN` values instead of empty cells.

Getting a statistic across all rows of a DataFrame is sometimes not good enough and you need more granular information—the mean per category, for example. Let's see how it's done!

Grouping

Using our sample DataFrame `students` again, let's find out the average score per country. To do this, you first group the rows by country and subsequently apply the `mean` method, which will calculate the mean *per group*. Note how we are chaining the selection of the `score` column directly to the `groupby` method:

```
In [78]: students.groupby(["country"])[ "score" ].mean()

Out[78]: country
         Germany    9.0
         Italy     4.5
         USA       5.3
         Name: score, dtype: float64
```

Instead of `mean`, you can use most of the descriptive statistics that pandas offers, and if you want to use your own function, use the `agg` method. The `agg` method accepts a function, which is often written as a lambda expression (see sidebar). For example, here is how you get the difference between the maximum and minimum value per group:

```
In [79]: selection = students[["born", "score", "continent"]]
         selection.groupby(["continent"]).agg(lambda x: x.max() - x.min())

Out[79]: properties  born  score
         continent
         America      8    2.8
         Europe     43    4.5
```

Lambda Expressions

Python allows you to define a function in a single line via *lambda expressions*. Lambda expressions are anonymous functions, which means that they are functions without names. Consider this function:

```
def function_name(arg1, arg2, ...):  
    return result
```

This function can be rewritten as a lambda expression like this:

```
lambda arg1, arg2, ...: result
```

In essence, you replace `def` with `lambda`, remove the `return` keyword and the function name, and put everything on one line. This can be convenient as we don't need to define a function for something that's just being used a single time.

A popular way to get statistics per group in Excel is to use PivotTables. They introduce a second dimension and are great to look at your data from different perspectives. pandas has a pivot table functionality, too, as we will see next.

Pivot Tables

If you are using PivotTables in Excel, you will have no trouble applying pandas' `pivot_table` function, as it works in largely the same way. The data in the following DataFrame is organized similarly to how records are typically stored in a database; each row shows a sales transaction for a specific fruit in a certain region:

```
In [80]: fruits = pd.DataFrame(  
    {  
        "Fruit": [  
            "Oranges",  
            "Apples",  
            "Oranges",  
            "Bananas",  
            "Bananas",  
            "Oranges",  
        ],  
        "Region": [  
            "North",  
            "South",  
            "South",  
            "South",  
            "North",  
            "North",  
        ],  
        "Revenue": [12.30, 10.55, 22.00, 5.90, 31.30, 13.10],  
    })
```

```
)
fruits

Out[80]:
```

	Fruit	Region	Revenue
0	Oranges	North	12.30
1	Apples	South	10.55
2	Oranges	South	22.00
3	Bananas	South	5.90
4	Bananas	North	31.30
5	Oranges	North	13.10

To create a pivot table, you provide the DataFrame as the first argument to the `pivot_table` function. The `index` and `columns` arguments define which column of the DataFrame will become the pivot table's row and column labels, respectively. `values` are going to be aggregated into the data part of the resulting DataFrame by using the `aggfunc`, a function that can be provided as a string or NumPy ufunc. And finally, `margins` correspond to Grand Total in Excel:

```
In [81]: pivot = pd.pivot_table(
    fruits,
    index="Fruit",
    columns="Region",
    values="Revenue",
    aggfunc="sum",
    margins=True,
    margins_name="Total",
)
pivot

Out[81]:
```

Region	North	South	Total
Fruit			
Apples	NaN	10.55	10.55
Bananas	31.3	5.90	37.20
Oranges	25.4	22.00	47.40
Total	56.7	38.45	95.15

In summary, pivoting your data means to take the unique values of a column (Region, in our case) and turn them into the column headers of the pivot table, thereby aggregating the values from another column. This makes it easy to read off summary information across the dimensions of interest. In our pivot table, you instantly see that there were no apples sold in the north region and that in the south region, most revenues come from oranges.

Working with aggregated statistics helps you understand your data, but nobody likes to read a page full of numbers. To make information easily understandable, nothing works better than creating visualizations. While Excel uses the term *charts*, pandas generally refers to them as *plots*. I will use these terms interchangeably in this book.

Plotting

Plotting allows you to visualize the findings of your data analysis and may well be the most important step in the whole process. For plotting, we're going to use two libraries: we start by looking at Matplotlib, pandas' default plotting library, before we focus on Plotly, a modern plotting library that gives us a more interactive experience in Jupyter notebooks.

Matplotlib

Matplotlib is one of the oldest and most popular plotting libraries in Python. With it, you can generate plots in a variety of formats, including vector graphics for high-quality printing. When you call the `plot` method of a `DataFrame`, pandas will produce a Matplotlib plot by default.

In the following example, we are creating a first Matplotlib plot via pandas (see [Figure 5-4](#)):

```
In [82]: import numpy as np

In [83]: # This shows you how to generate a DataFrame with random numbers.
# 1234 is a random number seed that makes rng.random always return
# the same numbers. Remove it if you want true pseudorandom numbers.
rng = np.random.default_rng(seed=1234) # Random number generator
data = rng.random((4, 4)) * 100000

sales = pd.DataFrame(
    data=data,
    index=["Q1", "Q2", "Q3", "Q4"],
    columns=["East", "West", "North", "South"],
)
sales.index.name = "Quarters"
sales.columns.name = "Region"
sales
```

Region	East	West	North	South
Quarters				
Q1	97669.976670	38019.573502	92324.623376	26169.242386
Q2	31909.705841	11809.123297	24176.629325	31853.392878
Q3	96407.924518	26364.980428	44100.612205	60987.080942
Q4	86362.129657	86375.767077	67488.131335	65987.434796

```
In [84]: # sales.plot is a shorthand for sales.plot.line()
# "style" makes the lines distinguishable in a black and white print
sales.plot(style=["-", "--", "-.", ":"])

Out[84]: <Axes: xlabel='Quarters'>
```

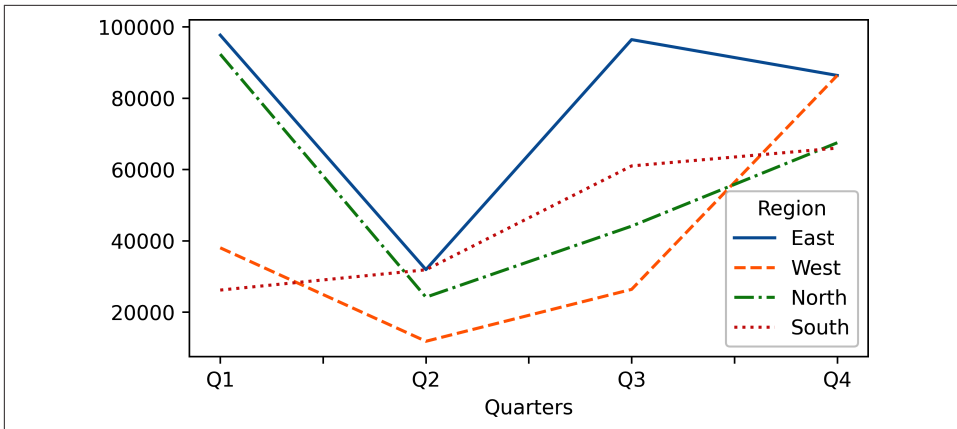


Figure 5-4. Matplotlib plot

In this example, I have used a NumPy array to construct a pandas DataFrame. Providing NumPy arrays allows you to leverage NumPy's constructors that we met in [Chapter 4](#); here, we use NumPy to generate a pandas DataFrame based on pseudo-random numbers.

Matplotlib was originally designed for static plots rather than for a modern web interface. That's why we're going to use Plotly next, a plotting library designed for the web.

Plotly

Plotly is a JavaScript-based library and can be used as a pandas plotting backend with great interactivity: you can easily zoom in, click on the legend to show or hide a data series, and get tooltips with more info about the data point you're hovering over.

To use Plotly directly with your DataFrames, you need to set it as a plotting backend in pandas. This enables you to plot [Figures 5-5](#) and [5-6](#):

```
In [85]: # Set the plotting backend to Plotly
         pd.set_option("plotting.backend", "plotly")

In [86]: sales.plot()
```

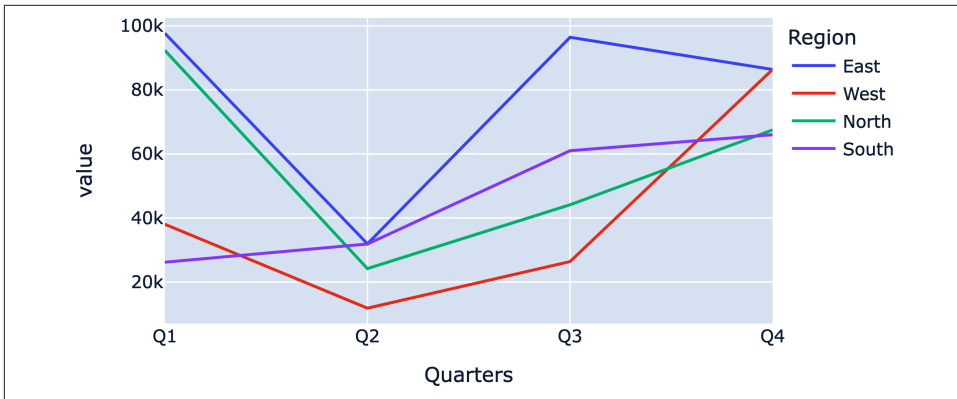



Figure 5-5. Plotly line plot

```
In [87]: # Display the same data as bar plot
sales.plot.bar(barmode="group")
```

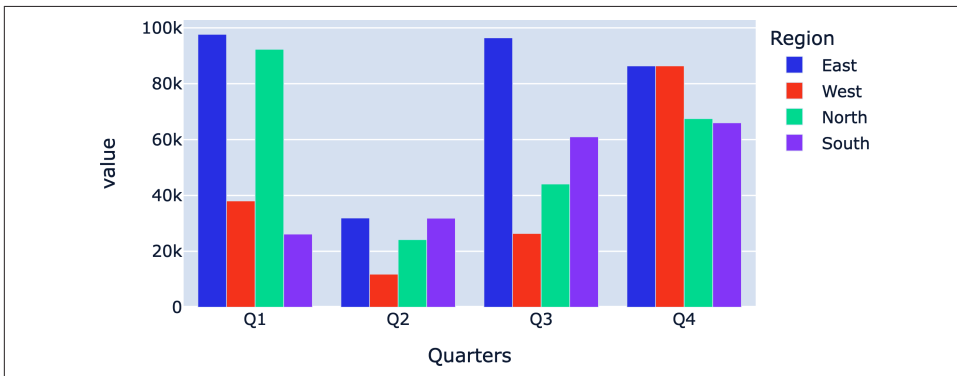


Figure 5-6. Plotly bar plot



Differences in Plotting Backends

If you use Plotly as a plotting backend, you'll need to check the accepted arguments of the plot methods directly on the Plotly docs. For example, you can take a look at the `barmode=group` argument in [Plotly's bar charts documentation](#).

pandas and the underlying plotting libraries offer a wealth of chart types and options to format the charts in almost any desired way. As an overview, [Table 5-6](#) shows the available plot types.

Table 5-6. *pandas* plot types

Type	Description
line	Line chart, default when running <code>df.plot()</code>
bar	Vertical bar chart
barh	Horizontal bar chart
hist	Histogram
box	Box plot
kde	Density plot, can also be used via <code>density</code>
area	Area chart
scatter	Scatter plot
hexbin	Hexagonal bin plots
pie	Pie chart

Other Plotting Libraries

The scientific visualization landscape in Python is very active, and besides Matplotlib and Plotly, there are many other high-quality options to choose from:

- **Seaborn** is built on top of Matplotlib and allows you to create advanced statistical plots with only a few lines of code. We'll use seaborn in [Chapter 6](#) and [Chapter 7](#).
- **Bokeh** is similar to Plotly in technology and functionality: it's based on JavaScript and therefore also works great for interactive charts in Jupyter notebooks.
- **Altair** is a library for statistical visualizations based on the [Vega project](#). Altair is also JavaScript-based and offers some interactivity like zooming.
- **hvPlot** is built on top of [HoloViews](#), which generates plots via Bokeh, Matplotlib, or Plotly.
- **Plotnine** is based on Matplotlib and uses a similar syntax as R's famous plotting library, ggplot2.

Note that Bokeh, Altair, and hvPlot can be used as a pandas plotting backend, just as we used Plotly in this chapter.

We will create more plots in [Chapter 7](#) to analyze time series, but before we get there, let's wrap this chapter up by learning how we can import and export data with pandas!

Importing and Exporting DataFrames

So far, we constructed DataFrames from scratch using nested lists, dictionaries, or NumPy arrays. These techniques are important to know, but in practice, data typically lives in files or databases, and you simply need to load the data into a DataFrame. To do this, pandas offers various reader functions. In Excel, data import is the type of work you would usually handle with Power Query.

After analyzing and changing your dataset, you might want to push the results back into a database or export it to a CSV file or—in the spirit of this book—present it in an Excel workbook to your manager. To export DataFrames, pandas offers several methods that all start with `to`: Table 5-7 shows an overview of the most common import and export methods.

Table 5-7. Importing and exporting DataFrames

Data format/system	Import: pandas (pd) function	Export: DataFrame (df) method
CSV files	<code>pd.read_csv</code>	<code>df.to_csv</code>
JSON	<code>pd.read_json</code>	<code>df.to_json</code>
HTML	<code>pd.read_html</code>	<code>df.to_html</code>
Clipboard	<code>pd.read_clipboard</code>	<code>df.to_clipboard</code>
Excel files	<code>pd.read_excel</code>	<code>df.to_excel</code>
SQL Databases	<code>pd.read_sql</code>	<code>df.to_sql</code>
Parquet files	<code>pd.read_parquet</code>	<code>df.to_parquet</code>

We'll meet some of these methods in later chapters: in Chapter 10, we will work with SQL databases, and in Chapter 12, we will use Parquet files. And since I am going to dedicate the whole of Chapter 14 to the topic of reading and writing Excel files with pandas, I will show you how to import and export CSV files in the next section. Let's start with exporting an existing DataFrame!

Exporting CSV Files

If you need to give a DataFrame to a colleague who might not use Python or pandas, passing it in the form of a CSV file is often a good idea: they are human-readable, and pretty much every program knows how to import them. To export our sample DataFrame `students` to a CSV file, use the `to_csv` method:

```
In [88]: students.to_csv("students.csv")
```

If you wanted to store the file in a different directory, supply the full path as a raw string, e.g., `r"C:\path\to\desired\location\msft.csv"`.



Use Raw Strings for File Paths on Windows

In strings, the backslash is used to escape certain characters. That's why on Windows, you either need to use double backslashes (C:\\path\\to\\file.csv) or prefix the string with an `r` to turn the path into a *raw string* that interprets the characters literally. This isn't an issue on macOS or Linux, as they use forward slashes in paths.

By providing only the file name, as I do, it will produce the file *students.csv* in the current directory, which, in our case, is the same directory as the notebook. It looks like this:

```
user_id,name,born,country,score,continent
1001,Mark,1966,Italy,4.5,Europe
1000,John,1988,USA,6.7,America
1002,Tim,1980,USA,3.9,America
1003,Jenny,2009,Germany,9.0,Europe
```

Now that you know how to use the `df.to_csv` method, let's see how importing a CSV file works!

Importing CSV Files

Importing a CSV file is as easy as providing its path to the `read_csv` function. The file *MSFT.csv* in the *csv* folder of the companion repository contains daily historical stock prices for Microsoft from the Quandl Wiki EOD Stock Prices dataset (see sidebar):

```
In [89]: msft = pd.read_csv("csv/MSFT.csv")
```

Quandl Wiki EOD Stock Prices Dataset

Quandl was an online platform for time series data. The Quandl community curated end-of-day (EOD) stock prices for 3,000 US companies and released them into the public domain. Unfortunately, when Quandl was acquired by Nasdaq in 2018, the database was discontinued. The data, however, can still be accessed via [Nasdaq Data Link](#), which requires a free account. Since it's one of the very few historical stock datasets that can be distributed freely, we're using it in this book.

Often, you will need to supply a few more parameters to `read_csv` than just the file name. For example, `sep` allows you to tell pandas what separator or delimiter the CSV file uses in case it isn't the comma. We will use a few more parameters in [Chapter 7](#), but for the full overview, have a look at the [pandas documentation](#).

Now that we are dealing with bigger DataFrames with many thousands of rows, typically the first thing is to run the `info` method to get a summary of the DataFrame.

Next, you may want to take a peek at the first or last few rows of the DataFrame using the `head` and `tail` methods. These methods return five rows by default, but this can be changed by providing the desired number of rows as an argument. You can also run the `describe` method to get some basic statistics:

```
In [90]: msft.info()
```

```
Out[90]: <class 'pandas.DataFrame'>
RangeIndex: 8076 entries, 0 to 8075
Data columns (total 14 columns):
#   Column          Non-Null Count  Dtype
---  -
0   ticker          8076 non-null   str
1   date            8076 non-null   str
2   open            8076 non-null   float64
3   high            8076 non-null   float64
4   low             8076 non-null   float64
5   close           8076 non-null   float64
6   volume          8076 non-null   float64
7   ex-dividend     8076 non-null   float64
8   split_ratio     8076 non-null   float64
9   adj_open        8076 non-null   float64
10  adj_high        8076 non-null   float64
11  adj_low         8076 non-null   float64
12  adj_close       8076 non-null   float64
13  adj_volume      8076 non-null   float64
dtypes: float64(12), str(2)
memory usage: 993.9 KB
```

```
In [91]: # I am selecting a few columns to fit the printed page width
# You can also just run: msft.head()
msft[["date", "adj_close", "adj_volume"]].head()
```

```
Out[91]:
```

	date	adj_close	adj_volume
0	1986-03-13	0.064720	1.031789e+09
1	1986-03-14	0.067031	3.081600e+08
2	1986-03-17	0.068187	1.331712e+08
3	1986-03-18	0.066454	6.776640e+07
4	1986-03-19	0.065298	4.789440e+07

```
In [92]: msft[["date", "adj_close", "adj_volume"]].tail(2)
```

```
Out[92]:
```

	date	adj_close	adj_volume
8074	2018-03-26	93.78	55031149.0
8075	2018-03-27	89.47	53704562.0

```
In [93]: msft[["adj_close", "adj_volume"]].describe()
```

```
Out[93]:
```

	adj_close	adj_volume
count	8076.000000	8.076000e+03
mean	18.768430	6.223490e+07
std	17.760955	3.909512e+07
min	0.060097	2.304000e+06
25%	1.908662	3.907406e+07
50%	18.594782	5.531360e+07

75%	24.060766	7.561875e+07
max	96.770000	1.031789e+09

`adj_close` stands for *adjusted close price* and corrects the stock price for corporate actions such as stock splits. `adj_volume` is the number of stocks that were traded, again, adjusted for corporate actions. I have summarized the various DataFrame exploration methods we've seen in this chapter in [Table 5-8](#).

Table 5-8. DataFrame exploration methods and attributes

DataFrame (df) method/attribute	Description
<code>df.info()</code>	Provides number of data points, index type, dtype, and memory usage
<code>df.describe()</code>	Provides basic statistics including count, mean, std, min, max, and percentiles
<code>df.head(n=5)</code>	Returns the first <i>n</i> rows of the DataFrame
<code>df.tail(n=5)</code>	Returns the last <i>n</i> rows of the DataFrame
<code>df.dtypes</code>	Returns the dtype of each column

The `read_csv` function also accepts a URL instead of a local CSV file. This is how you read the CSV file directly from the companion repo:

```
In [94]: # The line break in the URL is only to make it fit on the page
         url = (
             "https://raw.githubusercontent.com/fzumstein/"
             "python-for-excel/2e/csv/MSFT.csv"
         )
         msft = pd.read_csv(url)

In [95]: msft[["date", "adj_close", "adj_volume"]].head(2)

Out[95]:
```

	date	adj_close	adj_volume
0	1986-03-13	0.064720	1.031789e+09
1	1986-03-14	0.067031	3.081600e+08

We'll continue with this dataset and the `read_csv` function in [Chapter 7](#), where we will turn the date column into a `DatetimeIndex` to perform time series analysis. Reading CSV files, as we did in this section, is a highly optimized function, which runs very fast. This isn't true, though, for the majority of core functionality in pandas, which has led to the creation of various modern alternatives.

Modern Alternatives

In this section, I'll introduce a few pandas alternatives. To find out why they exist, we first need to understand the two biggest limitations of pandas:

Data must fit into memory

pandas loads your entire DataFrame into memory, so the maximum DataFrame size is dictated by the available RAM on your computer.

Mostly single-threaded

pandas is mostly single-threaded, i.e., it only takes advantage of one CPU core. There are a few functions that run on multiple cores, like `pd.read_csv`, but most core operations, such as `groupby` are restricted to a single core. In consequence, pandas can be slow with larger DataFrames.

These limitations have led to the creation of a few new solutions for data analysis. The most popular ones are Polars and DuckDB that gained a lot of traction over the last couple of years:

Polars

At a high-level, **Polars** is very similar to pandas, as it's also built around the concept of a DataFrame. However, as Polars is programmed in the Rust programming language, it is more memory-efficient and most of the core operations are done in parallel, using all of your available CPU cores. This makes Polars very fast. It can also work with data that doesn't fit into memory ("out-of-core processing").

DuckDB

DuckDB is an analytical database written in C++. DuckDB is an in-process database, so it doesn't require any setup other than installing the DuckDB Python package. DuckDB primarily relies on SQL as its primary interface to analyze data. There is, however, also a relational API, which allows you to chain methods instead of writing SQL strings. DuckDB also allows you to query pandas DataFrames, which makes a transition from pandas to DuckDB very easy. Like Polars, DuckDB is very fast and can handle datasets larger than memory.

In **Chapter 12**, you'll use DuckDB yourself to analyze a dataset of 3.5 million taxi trips.

Conclusion

This chapter was packed with new concepts and tools to analyze datasets in pandas. We learned how to load CSV files, how to deal with missing or duplicate data, and how to make use of descriptive statistics. We also saw how easy it is to turn your DataFrames into interactive plots. While it may take a while to digest everything, it probably won't take long before you will understand the immense power you are gaining by adding pandas to your tool belt. Along the way, we compared pandas to the following Excel functionality:

FILTER formula

See “[Selecting by boolean indexing](#)” on page 99.

VLOOKUP formula

See “[Joining and Merging](#)” on page 110.

Pivot Table

See “[Pivot Tables](#)” on page 115.

Power Query

This is a combination of “[Importing and Exporting DataFrames](#)” on page 121, “[Data Manipulation](#)” on page 96, and “[Combining DataFrames](#)” on page 109.

You may have noticed that I left out anything with date and time in this chapter. The reason is that we will dedicate [Chapter 7](#) entirely to time series analysis—an exciting topic since time series analysis is one of the main reasons why pandas conquered the financial industry by storm.

Time series analysis is also one of the most compelling use cases for Python in Excel, which is the topic of [Part III](#). So let's kick this new part off with a chapter about the basics of Python in Excel before continuing with time series analysis in pandas.

PART III

Python in Excel

Getting Started with Python in Excel

At this point, you've worked with pandas DataFrames and Jupyter notebooks quite a bit, which puts you in an excellent position to explore Python in Excel. Why? Because Python in Excel allows you to turn Excel cells into Jupyter notebook cells. This means that Python in Excel is mainly about getting access to a better analytics and charting engine in the form of pandas DataFrames and Python plots. It is, however, not competing with VBA or Office Scripts, as it doesn't support Excel automation or custom functions, i.e., functions you can call directly from Excel cells, e.g., `=SUM()`.

This chapter will introduce you to the basic concepts of Python in Excel: you will learn how to write Python code in an Excel cell and how to turn an Excel range into a pandas DataFrame and back again into native Excel values. You will also meet specific tools that come with Python in Excel, e.g., the Python editor. But first, let's make sure you've got what it takes to run Python in Excel!

Requirements

Despite its name, Python in Excel runs in Microsoft's Azure cloud and not in Excel itself: every time you run a cell with Python code, Excel sends the relevant spreadsheet data to the cloud where the Python code runs and from where the results are sent back again to Excel. This leads to the following requirements:

- Python in Excel requires an internet connection.
- Python in Excel requires a Microsoft 365 subscription and doesn't work with those versions of Office that come with a permanent license (e.g., Office 2024).



Python Source Code

While Python in Excel runs your code in the cloud, the code itself is stored inside the Excel file.

To confirm that you have access to Python in Excel, go to the Formulas tab in the ribbon menu: do you see the Python group as shown in [Figure 6-1](#)? If you don't see it, double-check whether you fulfill the requirements by consulting Microsoft's article [Python in Excel availability](#) and update your Excel installation. If you do: great, you're all set and ready to write some code!

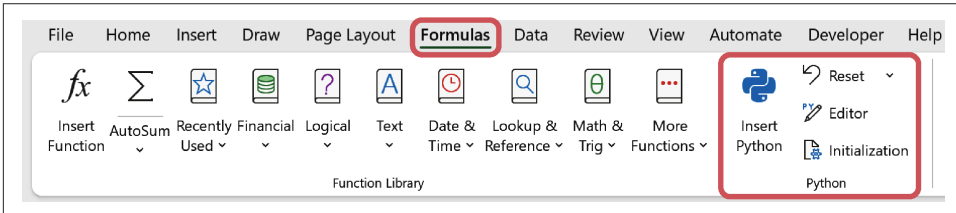


Figure 6-1. The Python menu in the Formulas tab of the Excel ribbon

Writing Python Code in Excel

In this section, you'll learn that Python cells have two output types (*Python object* and *Excel value*) and that the `x1` function is used to read in Excel cell values. We will also create plots and use the Python editor. To do all that, however, you first need to know how to turn a regular Excel cell into a Python cell.

Python Cells

To write Python code in an Excel cell, start by turning it into a Python cell: open Excel and create a new workbook, then select cell A1. Now write `=py` and hit the Tab key. This turns cell A1 into a Python cell, as indicated by the green "PY" icon in both the Excel cell and in the formula bar (see [Figure 6-2](#)).

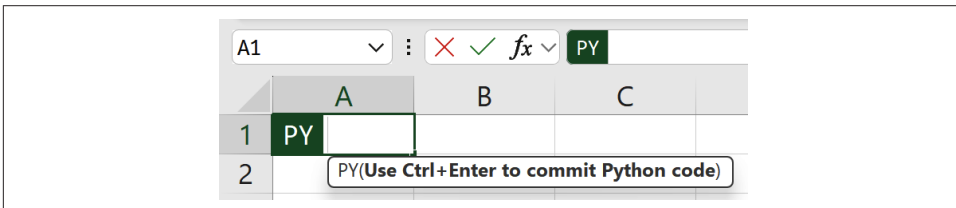


Figure 6-2. A Python cell

Here are some additional ways to turn an Excel cell into a Python cell:

- Instead of hitting the Tab key, write an opening parenthesis: **=py(**.
- Use the keyboard shortcut Ctrl+Alt+Shift+P (Windows) or Command-Option-Shift-P (macOS).
- Go to the Formulas tab in the Excel ribbon and click on the Insert Python button.



How to Remove a Python Cell

If you want to switch a Python cell back to a regular Excel cell, select the cell and hit the Delete key or simply start typing something new. In order for it to work, you need to make sure you're not editing the cell. If you are editing the cell, hit the Escape key first to switch from editing the cell to selecting it.

Now that cell A1 is a Python cell, write some code in it! We'll start with the same example that we used back in [Chapter 2](#) when introducing Jupyter notebooks:

```
3 + 4
```

After typing **3 + 4**, hit Enter. In a Python cell, hitting Enter will add a new line rather than run the code. So to run the code, hit Ctrl+Enter (Windows) or Command-Enter (macOS). The cell will now show a [PY] icon followed by the result, as shown in [Figure 6-3](#).

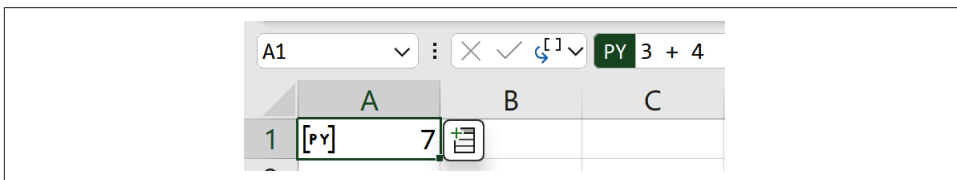


Figure 6-3. Python in Excel showing the output of 3 + 4

Similar to Jupyter notebooks, Python in Excel automatically shows the output of the last line of code in that cell. However, unlike Jupyter notebooks, if you assign the value to a variable, it keeps showing the output. So edit cell A1 as follows:

```
x = 3 + 4
```

When you hit Ctrl+Enter (Windows) or Command-Enter (macOS) again, you'll see that the cell still shows 7. Now add another Python cell by writing **=py** into cell A2, followed by the Tab key. Write the following code to create a Python list:

```
["Hello", "World"]
```

Hit Ctrl+Enter (Windows) or Command-Enter (macOS) to run the cell: it will display `[PY] list`. This is because Python in Excel only shows the value of the Python object

for simple objects such as numbers, strings, `datetime` objects, `None`, and booleans. For everything else, including data structures like lists or pandas DataFrames, Python in Excel will show its data type rather than the values so that they fit into one cell. But how can we see what's inside this list? The next section has the answer!

Output Types

Python in Excel has two output types for Python cells: *Python object* and *Excel value*. Let's break them down!

Python object

Python object is the default output type for Python cells and fits into a single cell. It shows the [PY] icon and represents the object of the last line of that Python cell. In our case, it shows 7 in cell A1 and `list` in cell A2. To get a preview of an object, click on the [PY] icon to show the preview card, as shown in [Figure 6-4](#). Alternatively, select the cell and use the keyboard shortcut Ctrl+Shift+F5 (Windows) or Command-Shift-F5 (macOS).

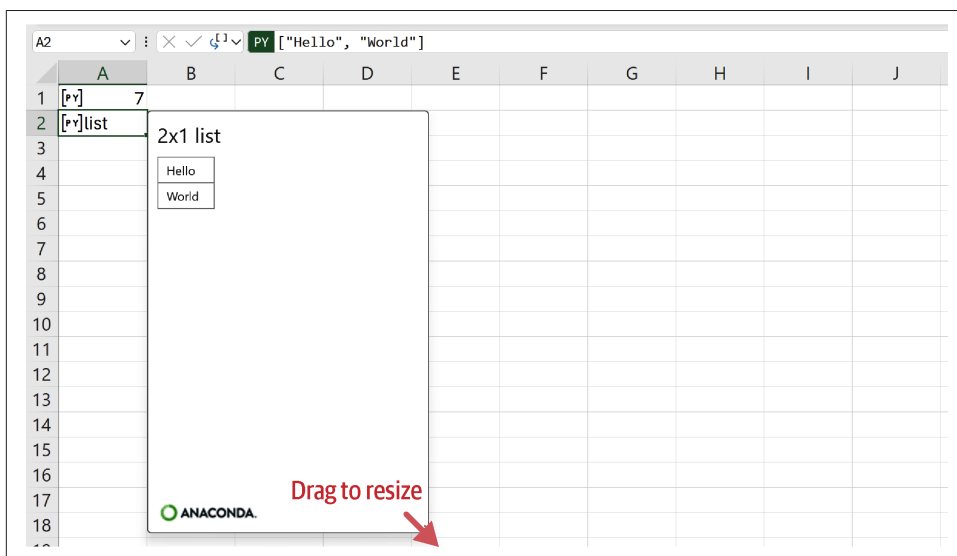


Figure 6-4. The preview card of a Python object

To make the preview card bigger, drag the lower-right corner, as indicated on [Figure 6-4](#). Keep in mind that—as the name says—it's just a preview, and you won't usually see all the data. To look at bigger datasets in detail, convert the Python object to Excel values, as shown in the next section.

Excel value

Once you're done with your Python calculations, you usually want to turn the results back into native Excel values to present them to the user. Here's how you change a cell's output type from Python object to Excel value:

1. Select the Python cell A2.
2. Click the dropdown on the left of the formula bar, as shown in [Figure 6-5](#).
3. Select "Excel Value" in the dropdown.

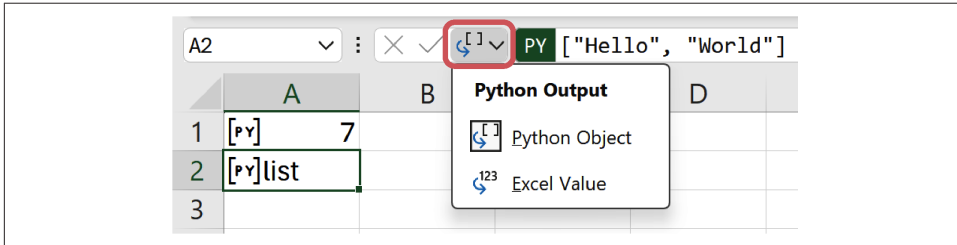


Figure 6-5. Python output type selection

[Figure 6-6](#) shows how the list in cell A2 looks after switching its output type from Python object to Excel value. With native Excel cell values, you can examine all the data, apply formatting, or add an Excel chart.

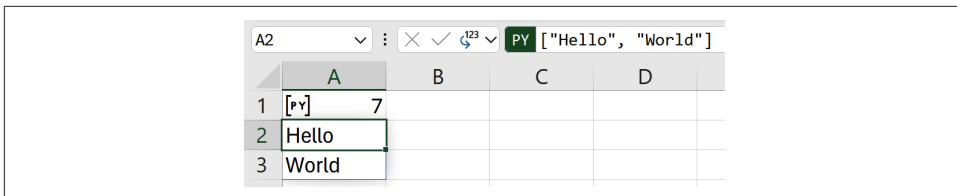


Figure 6-6. Cell A2 after switching its output type to Excel value

Dynamic Arrays

When you choose the Excel value output type, you will get a dynamic array, which means that values will be written out to multiple cells with a thin blue border around the cells. Microsoft calls this *spilling*, and accordingly, if it doesn't find enough empty cells to write the data to, it will show you a `#SPILL!` error. If you ever run into this error, you would need to delete nonempty Excel cells to make room for the dynamic array. Note that if you wanted to delete a dynamic array, you need to delete its top-left cell—you can't delete individual cells.

In addition to using the dropdown on the left of the formula bar, you can use the keyboard shortcut Ctrl+Alt+Shift+M (Windows) or Command-Option-Shift-M (macOS) to toggle between the output types Python object and Excel value.



Horizontal One-Dimensional Arrays

One-dimensional arrays such as lists, NumPy arrays, or pandas Series are displayed vertically when turned into Excel values. To change this to horizontal, wrap the respective object in a Python list. So to show ["Hello", "World"] from cell A2 horizontally, write `[["Hello", "World"]]`. This also works with one-dimensional NumPy arrays and pandas Series.

To understand how Python objects are converted to Excel values, have a look at [Table 6-1](#). The table assumes that you have imported NumPy as `np` and pandas as `pd`.

Table 6-1. Data type conversion from Python to Excel

Python	Excel	Comments
str	Text	
int	Number	Excel treats every number as floating-point number.
float	Number	Excel treats every number as floating-point number.
bool	Boolean	
datetime.datetime	Date-formatted number	
None	0	While the cell value is 0, it displays the text None.
np.nan	#NUM!	
np.inf	#NUM!	
pd.NaT	#NUM!	
pd.NA	#N/A	

Data structures like lists, NumPy arrays, or pandas DataFrames use one cell per element when turned from Python object to Excel values, while each element is converted according to [Table 6-1](#). We will work with pandas DataFrames in just a moment.

Now we know how to turn Python objects into Excel cell values, but how can we do the opposite, i.e., turn values from Excel cells into Python objects? This is where the `xl` function comes in.

The xl Function

To have your Python code read a value from a regular Excel cell, use the `xl` function. We'll repeat our first example of summing up two numbers, but instead of writing

them directly in the Python cell, we're reading them from Excel cells. Start by writing **3** in cell A5 and **4** in cell A6. Use regular Excel cells for doing this. Now, turn cell A7 into a Python cell by typing **=py** followed by the Tab key. While you're editing cell A7, do the following:

1. Click into cell A5.
2. Type the plus sign "+."
3. Click into cell A6.

Your Python code should now read `x1("A5")+x1("A6")`, as shown in [Figure 6-7](#).

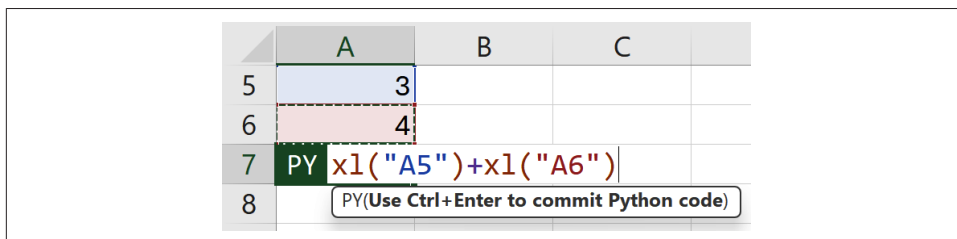


Figure 6-7. The `x1` function

Hit Ctrl+Enter (Windows) or Command+Enter (macOS) to run your code and you'll see [PY] 7. Now, when you change the value in cell A5 or A6, the Python cell in A7 will recalculate automatically. Actually, all of the Python cells will recalculate, but we'll come back to this in ["Row-Major Order of Cell Execution"](#) on page 144.



Absolute Versus Relative Cell References

The `x1` function works like regular Excel functions when it comes to absolute versus relative cell references. For example, if your Python cell in A1 uses `x1("A2")` and you copy that Python cell over from A1 to B1, the Python code will adjust to `x1("B2")` as A2 is a relative reference. Accordingly, if you write `x1("$A$2")` in cell A1 and copy it over to cell B1, it will still refer to the same range: `x1("$A$2")`. The dollar signs make the reference absolute, just like it works with regular Excel functions.

Typically, you don't write the `x1` function yourself; instead, you select the respective range in Excel as we just did. The `x1` function supports the data sources outlined in [Table 6-2](#).

Table 6-2. Data sources accepted by the xl function

Source	Explanation
Single cells	Single cells are converted to the corresponding data type in Python according to Table 6-3 .
Multiple cells	Two or more cells are converted to a pandas DataFrame. We'll get into the details of this in the next section.
Named ranges	If you select a named range, the xl function will automatically use the name as the reference, e.g., <code>xl("MyNamedRange")</code> .
Excel tables	Since Excel tables usually consist of more than one cell, they are converted to pandas DataFrames. We will dive into this in the next section.
Dynamic arrays	If you select a dynamic array from a Python cell, the xl function will refer to its top-left cell, followed by a #, e.g., <code>xl("A2#")</code> . This makes sure that the pandas DataFrame will update automatically whenever the dynamic array changes.
Power Query	The xl function also accepts Power Query connections—we will cover this topic in the next chapter.



The xl Function Doesn't Accept Variables

The xl function is a special function that requires a direct string reference of the source—it doesn't accept a variable:

```
data = xl("A1:C3") # This works
ref = "A1:C3"
data = xl(ref) # This doesn't work!
```

Now that we know what the xl function accepts as a source reference, let's see how it converts the referenced Excel values to Python objects by looking at [Table 6-3](#).

Table 6-3. Data type conversion from Excel to Python

Excel	Python	Comment
Text	str	
Number	int or float	Excel treats every number as floating-point number.
Boolean	bool	
Date-formatted number	datetime.datetime	
Empty cell	None	
Cell error (e.g., #N/A)	Cell error or np.nan	Single cells show the cell error, DataFrames show np.nan.



Don't Use the xl Function to Reference Other Python cells

While the xl function technically allows you to reference other Python cells with output type Python object, my advice is to stay away from it: it will lead to brittle code that breaks as soon as you append another line of code in the referenced Python cell or if you change its output type from Python object to Excel value. Instead, assign your Python object, e.g., a pandas DataFrame, to a variable, and use that variable in other Python cells.

For your reference, I have stored the end result of what we did so far in the Excel file *python_in_excel_hello_world.xlsx*, which you will find in the *xl* folder of the companion repository.

I've mentioned it a few times in this section: when you reference two or more cells in the *xl* function, Python in Excel returns a pandas DataFrame. Let's get into it!

Working with pandas DataFrames

pandas DataFrames are the foundation of Python in Excel. To get some practice, open the *fitness_tracker.xlsx* Excel file in the *xl* directory of the companion repository. It shows a few data points that are typically collected by wearable fitness trackers or the health app of your smartphone (see [Figure 6-8](#)).

	A	B	C	D	E	F
1	Date	Steps	Distance (km)	Calories	Sleep (hrs)	Comment
2	01/06/2025	8,200	6.1	320	7.2	
3	02/06/2025	10,500	7.9	390	6.5	Felt tired
4	03/06/2025	6,000	4.5	280		No sleep tracked
5	04/06/2025	12,300	9	450	6.8	Running group

Figure 6-8. The *fitness_tracker.xlsx* file

To analyze that mini dataset in Python, we'll turn our Excel table into a pandas DataFrame. Turn cell I1 into a Python cell, then write **df=** and select the range A1:F5. Selecting the cells will add the following at the position of your cursor:

```
xl("FitnessData[#All]", headers=True)
```

The whole cell therefore reads:

```
df = xl("FitnessData[#All]", headers=True)
```

As our data lives in an Excel table called "FitnessData," the *xl* function automatically refers to it as `FitnessData[#All]`. The syntax that Excel uses to refer to Excel tables is called *structured reference*, and `[#All]` is used to refer to the entire Excel table, including potential headers and totals. So if you type the reference yourself, make sure to include `[#All]`. If you want to learn more about the syntax of structured references with Excel tables, check out the [Microsoft documentation](#). The `headers=True` argument is added automatically when Excel figures out that the first row contains headers. This ensures that the DataFrame uses the first row of the data as its column headers. In our case, the first row corresponds to the header of the Excel table.

Now hit Enter to add a second line where we can set the index of the DataFrame. Setting the date column as the DataFrame's index is a common operation, as you will learn in [Chapter 7](#):

```
df = xl("FitnessData[#All]", headers=True)
df = df.set_index("Date")
```



True Versus TRUE

Coming from Excel, you might be used to typing TRUE in all caps, as that's how it's written in Excel formulas. However, in Python, it's important to use the correct notation, which only capitalizes the first letter: True.

Press Ctrl+Enter (Windows) or Command-Enter (macOS) to run the code, which should make the sheet look like [Figure 6-9](#).

	A	B	C	D	E	F	G	H	I
1	Date	Steps	Distance (km)	Calories	Sleep (hrs)	Comment			[*]DataFrame
2	01/06/2025	8,200	6.1	320	7.2				
3	02/06/2025	10,500	7.9	390	6.5	Felt tired			
4	03/06/2025	6,000	4.5	280		No sleep tracked			
5	04/06/2025	12,300	9	450	6.8	Running group			

Figure 6-9. Turning an Excel table into a pandas DataFrame



Expand the Formula Bar

If you want to read your Python code in the formula bar, you may want to drag the lower part of the formula bar down so that it spans over multiple lines, as shown in [Figure 6-9](#).

Go ahead and add a second Python cell in I2 by typing **=py** again, followed by the Tab key. To get some statistics about our data, add the following code that we have already met in [Chapter 5](#):

```
df.describe()
```

Run the cell by hitting Ctrl+Enter (Windows) or Command-Enter (macOS). Now change the output type of cell I2 to Excel values by selecting the cell and clicking the button to the left of the formula bar, as introduced earlier in [Figure 6-5](#). This will allow us to see the descriptive statistics returned by the `describe` method, as shown in [Figure 6-10](#).

	I	J	K	L	M
1	[r] DataFrame				
2		Steps	Distance (km)	Calories	Sleep (hrs)
3	count	4	4	4	3
4	mean	9250	6.875	360	6.83333333
5	std	2740.43792	1.983893478	75.2772653	0.35118846
6	min	6000	4.5	280	6.5
7	25%	7650	5.7	310	6.65
8	50%	9350	7	355	6.8
9	75%	10950	8.175	405	7
10	max	12300	9	450	7.2

Figure 6-10. Descriptive statistics

Before adding more Python cells to your workbook, let's bring some order into them!

Cell Organization

As introduced in [Chapter 2](#), in a classic Jupyter notebook, you can use either Markdown cells or Python comments to describe what you're doing in your code. Python in Excel, however, doesn't offer Markdown cells, and Python comments are hidden unless you click on the cell. Fortunately, we have two options to organize our Python cells:

Regular Excel cells

An easy way to add a comment to your Python cell is to simply write it in a regular Excel cell, usually on the left-hand side of the Python cell.

Python object

Alternatively, write a string on the last line of your Python cell while using the output type Python object. As we've seen earlier in this chapter, Python in Excel will display that string directly in the cell.

Figure 6-11 shows how these approaches compare to each other.

	H	I	J
PY	df = xl("FitnessData[#A11]", headers=True)		
	df = df.set_index("Date")		
	Create DataFrame	[r] DataFrame	

	I	J
PY	df = xl("FitnessData[#A11]", headers=True)	
	df = df.set_index("Date")	
	"Create DataFrame"	
	[r] Create DataFrame	

Figure 6-11. Using regular Excel cells (left) versus Python objects (right) for comments

For this book, I'll be using the regular cell approach as it works with both output types: Python objects, and Excel values.

Besides commenting your Python cells, Python in Excel offers another way to get a better grip on your Python cells: the Python editor.

Python Editor

The Python editor offers a more comfortable way of editing your Python code compared to Excel cells or the formula bar. To open it, go to the Formulas tab in the Excel ribbon and click on the Editor button in the Python group, as shown in [Figure 6-12](#).

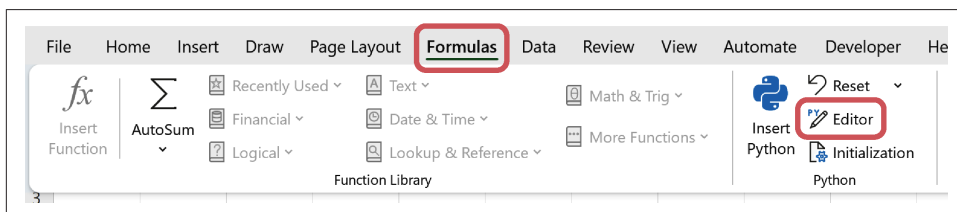


Figure 6-12. Starting the Python Editor from the Excel Ribbon

Alternatively, right-click on a Python cell and select Show Editor or use the keyboard shortcut `Ctrl+Alt+Shift+F2` (Windows) or `Command-Option-Shift-F2` (macOS). This opens the editor on the right-hand side of Excel. As [Figure 6-13](#) shows, the Python editor looks and feels similar to a Jupyter notebook.

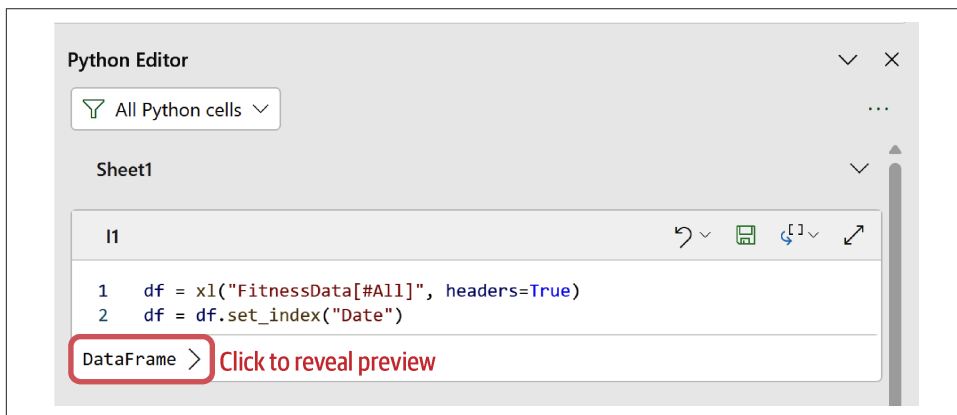


Figure 6-13. The Python editor showing the first Python cell I1

Here are a few advantages of using the editor in comparison to writing your code directly into cells or the formula bar:

Overview

If you have many Python cells scattered across worksheets and cells, the editor will show them all in order of execution from top to bottom. I'll say more about the order of execution later in this chapter under [“Row-Major Order of Cell Execution” on page 144](#).

IntelliSense

Code autocompletion, or *IntelliSense*, as Microsoft calls it, allows you to pick object attributes from a list instead of having to remember them all. The Python editor offers much better IntelliSense compared to writing your code directly in cells or the formula bar.

Output preview

The editor will show a similar output below the code cell like Jupyter notebooks. Note that many objects such as pandas DataFrames are collapsed by default, so you need to click on the link below the cell to show the output, as shown in [Figure 6-13](#).

Printed output and error messages

Printed output and error messages appear in the editor. It will automatically pop up when you use the `print` function or run into an error. If you don't like that behavior, you can change it via settings (as explained in [“Settings” on page 147](#)).

Standalone editor

If you want the Python editor to live in its own window, drag the editor out of Excel by grabbing it by the gray title bar. This enables you to expand it to full screen. But even if you leave it inside Excel, you can make the editor bigger by grabbing the left border and dragging it toward the center of Excel.

Keyboard shortcuts

The editor supports many keyboard shortcuts that you may be used to from VS Code, e.g., `Ctrl+/` (Windows) or `Command-/` (macOS) for commenting/uncommenting a line of code.



The xl Function in the Python Editor

At the time of this writing, the Python editor doesn't automatically insert the `xl` function when you select a range in Excel. You either have to type the `xl` function manually or—and this is my recommendation—you need to edit the Python cell directly in the Excel grid or in the formula bar whenever you want to add a cell reference via the `xl` function.

In the next section, we're going to use the editor to add a Python plot.

Plotting

Python in Excel supports a couple of plotting libraries, including Matplotlib and seaborn. We've already introduced Matplotlib in [Chapter 5](#), where we used it via the `plot` method of pandas DataFrames. We'll do the same here to understand how Python in Excel works with plots.

Click on cell I12, then open the Python editor (see previous section) and click the “+ Add Python cell” button at the bottom of the editor to turn I12 into a Python cell. To plot the number of steps over time, write the following code into that cell:

```
df["Steps"].plot.bar()
```

As usual, hit Ctrl+Enter (Windows) or Command+Enter (macOS). This will show [PY] Image in the Excel cell. To preview the plot, click on `PngImageFile` in the editor below the cell or click on the [PY] icon directly in cell I12 to pull up the preview card.

Most of the time, though, you probably want to show the plot similarly to Excel charts. To do that, right-click the Python cell with the image and select Display Plot over Cells. This adds the plot as an image to the sheet. Feel free to resize it and move it around to fit your worksheet. Alternatively, you could also click on the context menu button and select “Display Plot over Cells” from there, as shown in [Figure 6-14](#).

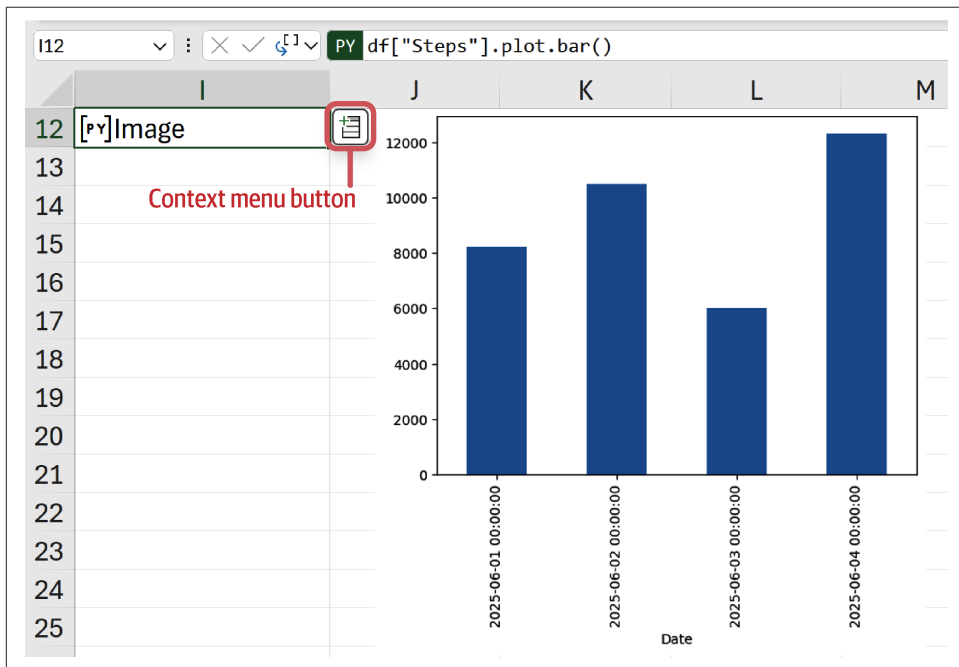


Figure 6-14. Displaying a Python plot over the Excel cells



Plotting Tips

- If you're playing around with your plot by trying out different settings and plot types, it may end up being distorted. To fix this, right-click your plot and select Reset Picture & Size.
- If you prefer to show your plot as an in-cell picture, select the respective Python cell showing [PY] Image and turn its output type from Python object to Excel value.
- In addition to Matplotlib and seaborn, Python in Excel supports the **Plotnine** library, which has a syntax similar to R's ggplot2 package.

Matplotlib does the job, but its defaults are often not ideal. For example, the date scale shows the time even if it's always 00:00:00. You could use various tricks to format this date axis, but it's a good excuse to try out the seaborn package instead. While seaborn uses Matplotlib under the hood, it has nicer defaults and offers complex statistical plots out of the box.

Start by turning cell I13 into a Python cell, then write the following code for a seaborn bar plot:

```
sns.barplot(df.reset_index(), x="Date", y="Steps")
```

Run the Python cell, then right-click and select “Display Plot over Cells:” you’ll see that the plot looks arguably nicer, as shown in **Figure 6-15**.

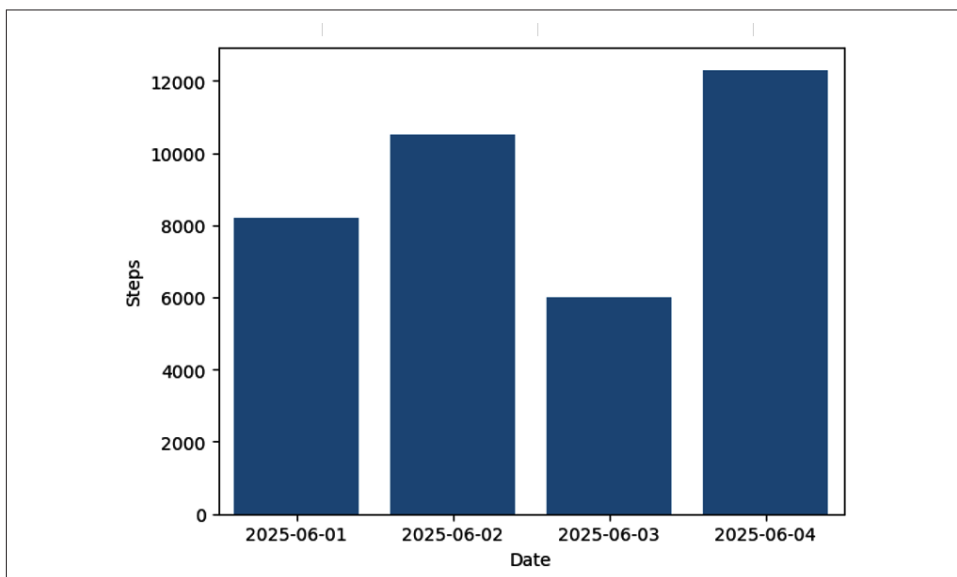


Figure 6-15. A seaborn plot

But wait, we didn't write `import seaborn as sns` anywhere in our code, and it still worked! Let's find out why!

Python in Excel Under the Hood

In this section, we're going a bit deeper to understand a few specific tools and concepts that Python in Excel uses. These include the initialization script, row-major order of cell execution, and the settings. Let's start with the initialization script!

Initialization Script

In the previous section, we wrote `sns.barplot` without having to write `import seaborn as sns`. This works because of the *initialization script*. It is a script that runs automatically when you start Python in Excel. Open it by clicking the “Initialization” button that you'll find in the Python menu on the Formulas tab of the Excel ribbon. It contains a couple of common imports:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels as sm
import seaborn as sns
```

All of these aliases can be used in any Python cell without having to import the package again. The content of the script may change over time, so make sure to check the initialization script yourself.



The Initialization Script Is Saved in the Workbook

Like the code that you write in Python cells in the Excel grid, the code from the initialization script is stored in the workbook itself.

The initialization script is also the right place to add your own imports or function definitions, but if you run on an older version of Python in Excel, the initialization script may be read-only. In this case, you could turn cell A1 of your very first sheet into a Python cell and write the imports there instead. Why in cell A1 of the very first sheet? Because Python cells run in a specific order, as we'll see next.

Row-Major Order of Cell Execution

In plain English, row-major order of cell execution translates to “from left to right and top to bottom.” Unlike regular Excel cells, where the calculation order is defined based on how the cells depend on each other, Python cells are simply executed by going from left to right and top to bottom on each sheet. This logic includes the sheets, which are processed from the leftmost to the rightmost.

The Python editor, as introduced earlier in this chapter, shows the execution order by presenting all cells from top to bottom, similar to a Jupyter notebook. And like with Jupyter notebooks, all Python cells run in the same global namespace, which means that the variables that you define in one cell are accessible in all other cells.

To see the effect of the cell execution order, continue editing our *fitness_tracker.xlsx* file: turn cell H1 into a Python cell and write `df` in it (this will overwrite the “Create DataFrame” comment from before). When you run the cell by hitting Ctrl+Enter (Windows) or Command+Enter (macOS), the Python editor opens and will show you the following error: `NameError: name 'df' is not defined`. As is easily visible in the editor, H1 is the very first Python cell and therefore runs before `df` is defined in cell I1.

Fix this by moving the cell from H1 to A7, just below our Excel table. As you see in the Python editor, A7 comes after cell I1, where we defined the variable `df`, so everything works now. Change the output type of A7 from Python object to Excel value via the dropdown on the left of the formula bar or by hitting the keyboard shortcut Ctrl+Alt+Shift+M (Windows) or Command+Option+Shift+M (macOS). If you compare the values with the original values in Excel, you will notice a few differences. Let’s examine them in the next section!

A DataFrame Roundtrip

Having the original Excel table and the values of our DataFrame sitting on top of each other in [Figure 6-16](#) puts us in a good position to understand what can happen when we turn Excel data into a pandas DataFrame and back again into Excel values.

	A	B	C	D	E	F
1	Date	Steps	Distance (km)	Calories	Sleep (hrs)	Comment
2	6/1/2025	8,200	6.1	320	7.2	
3	6/2/2025	10,500	7.9	390	6.5	Felt tired
4	6/3/2025	6,000	4.5	280		No sleep tracked
5	6/4/2025	12,300	9	450	6.8	Running group
6						
7		Steps	Distance (km)	Calories	Sleep (hrs)	Comment
8	Date					
9	6/1/2025	8200	6.1	320	7.2	None
10	6/2/2025	10500	7.9	390	6.5	Felt tired
11	6/3/2025	6000	4.5	280	#NUM!	No sleep tracked
12	6/4/2025	12300	9	450	6.8	Running group

Figure 6-16. A DataFrame roundtrip

In our case, this roundtrip leads to a few differences:

Empty row after title row

Python in Excel introduces an empty row between the header and the first row of data when the DataFrame has a nondefault index (we set the index to be the Date column in cell I1). Python in Excel tries to mimic how Jupyter notebooks print DataFrames, but if you would rather get rid of that empty row, reset the index:

```
df.reset_index()
```

As you may remember from [Chapter 5](#), resetting the index makes the DataFrame replace the current index with the default integer index. However, unlike our Date index from before, the integer index isn't shown. That's a bit of magic that Python in Excel applies here: it only shows nondefault indices. Resetting the index means that the current index is turned into a normal DataFrame column.

Missing formatting

By choosing output type Excel value, you get an unformatted dynamic array. While you can apply formatting like bold headers or background colors of cells, you can't turn the output into an Excel table, as dynamic arrays and Excel tables aren't compatible with each other.

Empty cells

As we saw earlier in [Table 6-3](#), empty cells are translated to None when you go from Excel to Python. Since the xl function immediately converts the data to a pandas DataFrame, None will be turned into np.nan. This impacts how these originally empty cells will be returned to Excel: np.nan is turned into a #NUM! error cell, while None is turned into 0 but displayed as None, as explained earlier in [Table 6-1](#).

If you wanted to keep the original empty cells for display purposes, you could replace missing values with empty quotes like so: `df.fillna("")`. Note, however, that this converts numeric columns with missing values into object type. So use this trick only for display purposes but not for further calculations in Python.

At this point, we've gone through the main features of Python in Excel. I have included a version of the Excel file with all the changes we did in this chapter. You'll find it under *fitness_tracker_end.xlsx* in the *xl* folder of the companion repository.

Sometimes, you may need to tweak the behavior of Python in Excel, so let me walk you through the settings that Python in Excel offers.

Settings

Go to File > Options > Advanced (Windows) or Excel > Preferences > Calculation (macOS) to find the Python in Excel settings. Under the header “When calculating this workbook,” you will find the following two options:

Automatically show Python editor

If this is activated (default), it opens the Python editor automatically if your Python cell runs into a Python error or uses the `print` function. Since the `info` method of pandas DataFrames uses the `print` function, running `df.info()` will also open the Python editor.

Python Formula Timeout

As you learned at the beginning of this chapter, Python in Excel runs your code in the cloud. By default, any formula that takes longer than 30 seconds will be cancelled and show a `#TIMEOUT!` error. To change this limit, adjust the value under “Python Formula Timeout (in seconds).”

If you want to clear all existing variables and recalculate everything from scratch, use the Reset button. It’s not under Excel Options but is directly accessible from the Python menu in the Formulas tab of the Excel ribbon. It offers the following actions via dropdown:

Reset

Reset deletes the global Python variables and recalculates the workbook. This is primarily useful if you use the partial calculation mode, which requires the Python in Excel add-on (see “[Python in Excel Add-On](#)” on page 150).

Reset Runtime

Reset Runtime restarts the Python runtime and recalculates the workbook. Use this if you run into `#BUSY!` or `#CONNECT!` errors.

With settings and the Reset button out of our way, let’s recap the most important keyboard shortcuts!

Keyboard Shortcuts

This section serves as a quick reference to all the keyboard shortcuts that we met in this chapter. If you are on macOS, replace Ctrl with Command and Alt with Option, as in [Table 6-4](#).

Table 6-4. Keyboard shortcuts

Keyboard shortcut	Explanation
Ctrl+Enter	Runs the Python cell
Ctrl+Alt+Shift+M	Toggles between the output types Python object and Excel value
Ctrl+Alt+Shift+F2	Opens the Python editor
Ctrl+Alt+Shift+C	Used with [PY] Image cells, shows the image over the Excel cells
Ctrl+Shift+F5	Opens the preview card (requires that you show your cell as Python object)

Let's wrap this chapter up by going through the limitations of Python in Excel!

Limitations

Python in Excel has a very narrow focus: deliver powerful analytics and visuals via the pandas DataFrame ecosystem. Therefore, it has quite a few limitations. However, keep in mind that Python in Excel has only been out for one year at the time of this writing, and some of these limitations may be lifted or changed over time.

Usage limitations

Python in Excel has both a daily and a monthly usage quota. While Microsoft doesn't specify how much the allowance is, you will get a #BLOCKED! error once you've exceeded your usage quota.

Runs in the cloud

Python in Excel runs in the cloud, which has privacy implications and prevents offline usage.

Fixed set of packages

Python in Excel comes with a fixed set of packages and doesn't allow you to install additional ones. To see which packages are installed, run the following code in a Python cell and change the output type to values:

```
import pkg_resources
sorted([[pkg.key, pkg.version] for pkg in pkg_resources.working_set])
```

Note, however, just because a package is installed doesn't necessarily mean it's also supported by Python in Excel!

No direct access to external files or systems

Python in Excel doesn't allow you to download data from the internet and can't connect to databases or external files. If you need this functionality, you need Power Query to do the heavy lifting, while Python in Excel allows you to reference a Power Query connection via the xl function. We will get into the details of how this works in the next chapter.

Hardwired to pandas DataFrames

As soon as the `xl` function references more than one cell, it returns a pandas DataFrame. There's no simple way to return a Python list or a NumPy array, etc. If you are working a lot with NumPy arrays and lists instead of DataFrames, add the following code at the end of your initialization script:

```
def custom_converter(x, headers=False, convert=None, **kwargs):
    # Leave default unchanged
    if convert is None or convert == pd.DataFrame:
        return excel.convert_to_dataframe(x, headers=headers, **kwargs)
    # Flatten 1-dimensional column and row arrays
    if isinstance(x[0], list) and len(x[0]) == 1:
        x = [row[0] for row in x]
    elif len(x) == 1 and isinstance(x[0], list):
        x = x[0]
    # Handle NumPy arrays and lists
    if convert in (np.array, np.ndarray):
        return np.array(x)
    elif convert == list:
        return x

    # Override the array conversion to use the custom converter
    excel.set_xl_array_conversion(custom_converter)
```

This keeps the default behavior of the `xl` function unchanged but adds an easy and efficient way to get a NumPy array or a (nested) list:

```
# Getting a pandas DataFrame works as before
xl("...", headers=True)
# This returns a 1- or 2-dimensional NumPy array
xl("...", convert=np.ndarray)
# This returns a list or a nested list
xl("...", convert=list)
```

Alternatively, you could use `df.to_numpy()` or `df.to_numpy().tolist()` to turn your DataFrame into a NumPy array or a nested list, respectively (this assumes that your DataFrame is called `df`). However, this is less efficient as the data is first converted from a list to a DataFrame and then back again and can leave you with undesired conversion effects, such as having empty cells represented as `#NUM!` instead of `None`.

No automation

Python in Excel doesn't have access to the Excel object model. This means that you can't, e.g., insert a new sheet or make text bold. It also means that instead of writing a loop to grab all the data from all the sheets and combine them into a single DataFrame, you'd have to apply the `xl` function to each individual worksheet.

Maximum dataset size

Python in Excel can work with datasets that use up to 100 MB of memory. Otherwise it will return a #CALC! error. To get an idea of how big the memory consumption of your pandas DataFrame is, run `df.info()` (this assumes that your DataFrame is called `df`): the Python editor will open and print the memory usage at the bottom of the output (click on DataFrame below the cell in the editor to show the output).

Cells can't be run in isolation

Unlike Jupyter notebooks, Python in Excel always recalculates all Python cells even if you just run a single cell. This has the advantage that you never run into cells that are out of sync. On the flip side, it means that you always have to wait for every cell to calculate, which can make things slow. It will also use up your daily and monthly quota much faster. There's a solution to that problem, but it requires you to buy the Python in Excel add-on (see [“Python in Excel Add-On” on page 150](#)). With the Python in Excel add-on, you can set calculation mode to partial or manual, allowing you to run single cells, like in a regular Jupyter notebook.

No way to reuse code

There's no way to reuse code from one workbook in another workbook without copy/pasting the respective Python cells.

Maximum cell length

Excel can store up to 32,767 characters in a cell, so if your Python code is longer, you need to split it up between multiple cells.

Python in Excel Add-On

While the basic version of Python in Excel is included in your Microsoft 365 subscription, Microsoft also offers a paid add-on for Python in Excel. It offers premium compute and the ability to use partial and manual calculation mode in addition to automatic calculation mode. However, I'd expect this offer to evolve over time, so make sure to check the [Python in Excel home page](#) for up-to-date information.

Conclusion

This chapter introduced you to the basics of how Python in Excel works. You've learned how to use the `xl` function to convert Excel data into pandas DataFrames. You've also seen how to do the opposite, turning a DataFrame back into native Excel values, by switching the output type from Python object to Excel value.

Python in Excel brings Excel's analytical capabilities to a whole new level by leveraging pandas DataFrames. It's also great that Excel subscription users can start using Python without any additional setup. For my taste, however, the section about limitations could be a bit shorter. For example, it would be nice to use Python in Excel with a permanently licensed version of Office. It would also be helpful to understand where you are in your consumption allowance so that you won't be suddenly blocked. However, since Python in Excel is still young, there's always a chance that these things will become possible in the future!

In the next chapter, we're going to use Python in Excel for one of its prime use cases: time series analysis with pandas. Time series analysis is one of the main reasons why pandas has been adopted by the financial industry. So let's see why this part of pandas has such an edge over Excel!

Time Series Analysis with pandas

Python in Excel or Jupyter Notebook: Your Choice!

To follow along with this chapter, you can either use Python in Excel or a traditional Jupyter notebook. The code samples are presented as written in the Excel workbook *ch07.xlsx*, which you will find in the *xl* folder of the companion repository.

If you don't have access to Python in Excel, simply run the Jupyter notebook *ch07.ipynb* instead, which you'll find in the root directory of the companion repository. There are small differences to the Python in Excel version, which are pointed out directly in the Jupyter notebook.

The output of the code samples is formatted like in the rest of the book, i.e., as shown by a traditional Jupyter notebook. It's very similar, though, to what Python in Excel shows as output in the Python editor—remember to expand the output section below the cells, as shown in the previous chapter, in [Figure 6-13](#).

A *time series* is a series of data points along a time-based axis that plays a central role in many different scenarios: while traders use historical stock prices to calculate risk measures, the weather forecast is based on time series generated by sensors measuring temperature, humidity, and air pressure. And the digital marketing department relies on time series generated by web pages, e.g., the source and number of page views per hour, and will use them to draw conclusions with regard to their marketing campaigns.

Time series analysis is one of the main driving forces behind why data scientists and analysts have started to work with pandas. The following points summarize some of the reasons behind this move:

Big datasets

Time series can quickly grow beyond Excel's limit of roughly one million rows per sheet. For example, if you work with intraday stock prices on a tick data level, you're often dealing with hundreds of thousands of records—per stock and day!

Date and time

As we have seen in [Chapter 3](#), Excel has various limitations when it comes to handling date and time, the backbone of time series. Missing support for time zones and a number format that is limited to milliseconds are some of them. pandas supports time zones and uses NumPy's `datetime64[us]` data type with microsecond resolution ("us"). However, if your data requires it, pandas will use nanosecond resolution via `datetime64[ns]`.

Missing functionality

Excel misses even basic tools to be able to work with time series data efficiently. For example, if you want to turn a daily time series into a monthly time series, there is no easy way of doing this despite it being a very common task.

DataFrames allow you to work with various time-based indices: `DatetimeIndex` is the most common one and represents an index with timestamps. Other index types, like `PeriodIndex`, are based on time intervals such as hours or months. In this chapter, however, we are only looking at `DatetimeIndex`, which I will introduce now in more detail.

DatetimeIndex

In this section, we'll learn the basics of working with a `DatetimeIndex`, including how to construct one, how to convert text to timestamps, and how to filter a `DatetimeIndex` to a specific time range.

Creating a DatetimeIndex

To construct a `DatetimeIndex` manually, pandas offers the `date_range` function. It accepts a start date, a frequency, and either the number of periods or the end date:

```
In [1]: # Start by importing the packages we use in this chapter
import numpy as np
import pandas as pd
import seaborn as sns

In [2]: # This creates a DatetimeIndex based on a start timestamp,
# number of periods, and frequency ("D" = daily).
daily_index = pd.date_range("2020-02-28", periods=4, freq="D")

Out[2]: DatetimeIndex(['2020-02-28', '2020-02-29', '2020-03-01', '2020-03-02'],
                        dtype='datetime64[us]', freq='D')
```

```

In [3]: # This creates a DatetimeIndex based on a start and end timestamp.
        # The frequency is set to "weekly on Sundays" ("W-SUN").
        weekly_index = pd.date_range("2020-01-01", "2020-01-31", freq="W-SUN")

Out[3]: DatetimeIndex(['2020-01-05', '2020-01-12', '2020-01-19', '2020-01-26'],
                        dtype='datetime64[us]', freq='W-SUN')

In [4]: # Create a DataFrame based on weekly_index. This could be
        # the visitor count of a museum that only opens on Sundays.
        pd.DataFrame({"visitors": [21, 15, 33, 34]}, index=weekly_index)

Out[4]:
            visitors
2020-01-05         21
2020-01-12         15
2020-01-19         33
2020-01-26         34

```

Remember that we played around with Microsoft’s stock history at the end of [Chapter 5](#)? Let’s load it again, but this time we’re adding a few additional stocks: Amazon (AMZN), Google (GOOGL), and Apple (AAPL). All of these time series are from the Quandl Wiki EOD Stock Prices dataset that was introduced in [Chapter 5](#). This time, however, we’re loading the data directly from *ch07.xlsx*, where each stock is stored in an Excel table on a separate sheet. For example, you’ll find the prices for Microsoft on a sheet called “MSFT” with the table name `TableMSFT`. We are using the `xl` function as introduced in the previous chapter to load these tables as pandas DataFrames:

```

In [5]: msft = xl("TableMSFT[#All]", headers=True).set_index("date")
        aapl = xl("TableAAPL[#All]", headers=True).set_index("date")
        amzn = xl("TableAMZN[#All]", headers=True).set_index("date")
        googl = xl("TableGOOGL[#All]", headers=True).set_index("date")

```

Note that we’re setting the date column as the index by chaining the `xl` function with `set_index("date")`. We’re doing this as we almost always want to turn the relevant timestamps into the index of the DataFrame; this will give us easy access to functionality commonly used with time series analysis, e.g., filtering the data by year, as we will see in a moment.

For now, let’s concentrate on the Microsoft stock. When you take a closer look at the index of the `msft` DataFrame, you will notice that it shows Index rather than `DatetimeIndex`, which means that pandas is currently interpreting the timestamps as strings:

```

In [6]: msft.info()

Out[6]: <class 'pandas.DataFrame'>
Index: 8076 entries, 1986-03-13 to 2018-03-27
Data columns (total 13 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   ticker      8076 non-null  str
1   open        8076 non-null  float64
...  ...

```

```

11 adj_close    8076 non-null    float64
12 adj_volume   8076 non-null    float64
dtypes: float64(12), str(1)
memory usage: 993.7 KB

```

This happens because the timestamp in the `date` column of the Excel tables is formatted as text (it has the “General” Excel cell format). There are two ways to fix this and turn it into a `DatetimeIndex`. The first one is to properly format the dates in Excel as date-formatted cells. This way, the `xl` function will convert them automatically to `datetime` objects in Python. The other option is to run the `to_datetime` function on the index. Make sure to assign the transformed index back to the original `DataFrame` if you want to change it at the source:

```

In [7]: # Convert the index of all DataFrames to a DatetimeIndex
        for df in [msft, aapl, amzn, googl]:
            df.index = pd.to_datetime(df.index)

```

```

In [8]: msft.info()

```

```

Out[8]: <class 'pandas.DataFrame'>
DatetimeIndex: 8076 entries, 1986-03-13 to 2018-03-27
Data columns (total 13 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   ticker          8076 non-null   str
1   open            8076 non-null   float64
...  ...
11  adj_close        8076 non-null   float64
12  adj_volume       8076 non-null   float64
dtypes: float64(12), str(1)
memory usage: 914.9 KB

```



Output in Python in Excel

With Python in Excel, to see the output of Out [8], you will need to look at the Python editor; you can’t see this in the Excel cell. Also, as long as Python in Excel uses a version of `pandas` < 3.0, you will see `object` instead of `str` as the data type.

As `info` reveals on its second line of output, you are now dealing with a proper `DatetimeIndex`. Compare this again with the output of the cell [6], where it showed `Index` instead.

If you want to change the data type of a column that isn’t time-based, things are a bit easier. Let’s say you wanted the `volume` column to be a `float` instead of an `int`; in this case, apply the `astype` method as follows:

```

In [9]: msft["volume"] = msft["volume"].astype("float")
        msft["volume"].dtype

```

```
Out[9]: dtype('float64')
```

Since time series data comes in a specific order, you should always make sure that the data is sorted chronologically before any further analysis:

```
In [10]: msft = msft.sort_index()
```

And finally, if you need to access only parts of a `DatetimeIndex`, like the date part without the time, access the `date` attribute like this:

```
In [11]: msft.index.date
```

```
Out[11]: array([datetime.date(1986, 3, 13), datetime.date(1986, 3, 14),
               datetime.date(1986, 3, 17), ..., datetime.date(2018, 3, 23),
               datetime.date(2018, 3, 26), datetime.date(2018, 3, 27)],
              shape=(8076,), dtype=object)
```

Instead of `date`, you can also use other attributes that refer to parts of a date like `year`, `month`, `day`, etc.



Accessing Datetime Attributes in Regular DataFrame Columns

To access datetime attributes of regular DataFrame columns with data type `datetime64[us]`, you will have to use the `dt` accessor, e.g., `df["column_name"].dt.date`. As we've just seen, the `dt` accessor isn't required with `DatetimeIndex`, where the syntax is simply `df.index.date`.

With a sorted `DatetimeIndex`, let's see how we can filter the DataFrame to certain time periods!

Filtering a DatetimeIndex

If your DataFrame has a `DatetimeIndex`, there is an easy way to select rows from a specific time period by using `loc` with a string in the format `YYYY-MM-DD HH:MM:SS`. pandas will turn this string into a slice so it covers the whole period. For example, to filter the adjusted close price to the year 2017, provide the year as a string (not a number):

```
In [12]: msft.loc["2017", "adj_close"]
```

```
Out[12]: date
2017-01-03    61.219142
2017-01-04    60.945231
2017-01-05    60.945231
2017-01-06    61.473488
2017-01-09    61.277837
...
2017-12-22    85.510000
2017-12-26    85.400000
```

```

2017-12-27    85.710000
2017-12-28    85.720000
2017-12-29    85.540000
Name: adj_close, Length: 250, dtype: float64

```

Let's take this a step further and plot the data between March 2000 and February 2001. This was the time when the tech bubble burst, with the NASDAQ index peaking on March 10, 2000 (see [Figure 7-1](#)):

```
In [13]: msft.loc["2000-03":"2001-02", "adj_close"].plot()
```

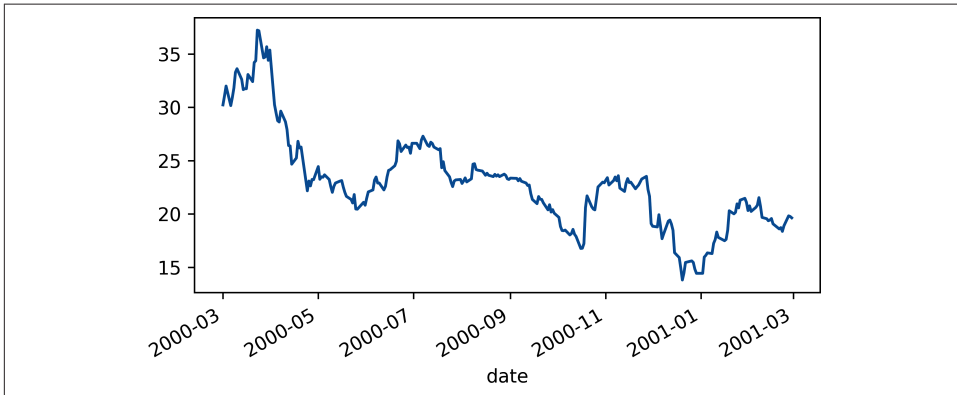


Figure 7-1. Adjusted close price for Microsoft (MSFT)

We'll use the adjusted close price in the next section to learn about common time series manipulations.

Common Time Series Manipulations

In this section, I'll show you how to perform common tasks such as calculating stock returns, comparing the performance of various stocks in a plot, and visualizing the correlation of their returns in a heatmap. We'll also see how to change the frequency of time series and how to calculate rolling statistics.

Shifting and Percentage Changes

In finance, the *log returns* (logarithmic returns) of stocks are often assumed to be normally distributed. Log returns are defined as the natural logarithm of the ratio of the current and previous price. To get a feeling for the distribution of the daily log returns, let's plot a histogram. First, however, we need to calculate the log returns. In Excel, it's typically done with a formula that involves cells from two rows, as shown in [Figure 7-2](#).

	A	B	C
1	Date	Adj Close	
2	3/13/1986	0.062205	
3	3/14/1986	0.064427	=LN(B3/B2)
4	3/17/1986	0.065537	0.017082

Figure 7-2. Calculating log returns in Excel



Logarithms in Excel and Python

Excel uses LN to denote the natural logarithm and LOG for the logarithm with base 10. Python's math module and NumPy, however, use log for the natural logarithm and log10 for the logarithm with base 10.

With pandas, rather than having a formula accessing two different rows, you use the `shift` method to shift the values down by one row. This allows you to operate on a single row so your calculations can make use of vectorization. `shift` accepts a positive or negative integer that shifts the time series down or up by the respective number of rows. Let's first see how `shift` works:

```
In [14]: msft_close = msft[["adj_close"]].copy()
         msft_close.head()
```

```
Out[14]:      adj_close
date
1986-03-13    0.064720
1986-03-14    0.067031
1986-03-17    0.068187
1986-03-18    0.066454
1986-03-19    0.065298
```

```
In [15]: msft_close.shift(1).head()
```

```
Out[15]:      adj_close
date
1986-03-13         NaN
1986-03-14    0.064720
1986-03-17    0.067031
1986-03-18    0.068187
1986-03-19    0.066454
```

You are now able to write a single vector-based formula that is easy to read and understand. To get the natural logarithm, use NumPy's `log` ufunc, which is applied to each element. Then we can plot a histogram (see [Figure 7-3](#)):

```
In [16]: returns = np.log(msft_close / msft_close.shift(1))
         returns = returns.rename(columns={"adj_close": "returns"})
         returns.head()
```

```
Out[16]:
```

	date	returns
	1986-03-13	NaN
	1986-03-14	0.035091
	1986-03-17	0.017094
	1986-03-18	-0.025752
	1986-03-19	-0.017544

```
In [17]: # Plot a histogram with the daily log returns.
# The argument "bins=90" sets the number of bars in the histogram.
returns.plot.hist(bins=90)
```

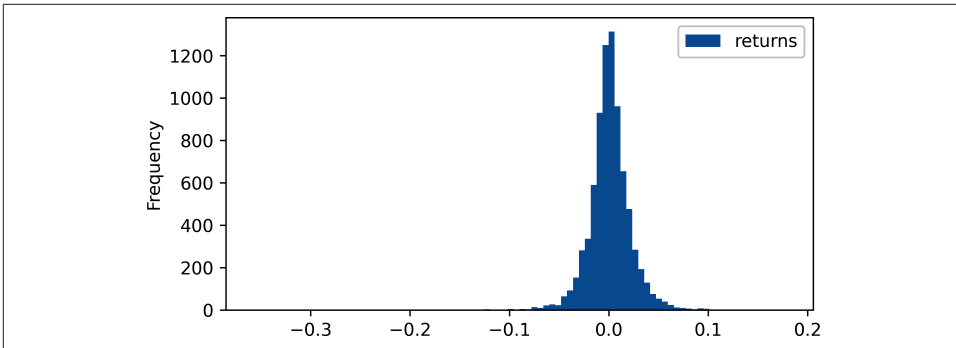


Figure 7-3. Histogram plot

To get *simple returns* instead, use pandas' built-in `pct_change` method. By default, it calculates the percentage change between consecutive values, which is also the definition of simple returns:

```
In [18]: simple_rets = msft_close.pct_change()
simple_rets = simple_rets.rename(columns={"adj_close": "simple_rets"})
simple_rets.head()
```

```
Out[18]:
```

	date	simple_rets
	1986-03-13	NaN
	1986-03-14	0.035714
	1986-03-17	0.017241
	1986-03-18	-0.025424
	1986-03-19	-0.017391

So far, we have only used the Microsoft stock. In the next section, we're going to work with multiple stocks to compare their performance, among other things.

Rebasing and Correlation

Things get slightly more interesting when we work with more than one time series. Let's combine the adjusted close prices for all of the stocks that we loaded in cell [5] into a single DataFrame:

```

In [19]: parts = [] # List to collect individual DataFrames
        for df in [msft, aapl, amzn, googl]:
            ticker = df["ticker"].iloc[0] # Get ticker name
            df = df[["adj_close"]] # Select adj_close column
            df = df.rename(columns={"adj_close": ticker}) # Rename column
            parts.append(df) # Append the DataFrame to the parts list

In [20]: # Combine the 4 DataFrames into a single DataFrame
        adj_close = pd.concat(parts, axis=1, sort=True)

Out[20]:
      date      MSFT      AAPL      AMZN      GOOGL
1980-12-12    NaN    0.422706      NaN      NaN
1980-12-15    NaN    0.400652      NaN      NaN
1980-12-16    NaN    0.371246      NaN      NaN
1980-12-17    NaN    0.380362      NaN      NaN
1980-12-18    NaN    0.391536      NaN      NaN
...
2018-03-21  92.48  171.270000  1581.86  1094.00
2018-03-22  89.79  168.845000  1544.10  1053.15
2018-03-23  87.18  164.940000  1495.56  1026.55
2018-03-26  93.78  172.770000  1555.86  1054.09
2018-03-27  89.47  168.340000  1497.05  1006.94

[9401 rows x 4 columns]

```

Did you see the power of concat? pandas has automatically aligned the individual time series along the dates. This is why you get NaN values for those stocks that don't go back as far as Apple. Aligning time series by date is a typical operation that is hard and error-prone when done with native Excel formulas. Dropping rows with missing values will make sure that all stocks have the same amount of data points; see the column Non-Null Count of the printed output:

```

In [21]: adj_close = adj_close.dropna()
        adj_close.info()

Out[21]: <class 'pandas.DataFrame'>
        DatetimeIndex: 3423 entries, 2004-08-19 to 2018-03-27
        Data columns (total 4 columns):
         #   Column  Non-Null Count  Dtype
        ---  ---
         0   MSFT    3423 non-null    float64
         1   AAPL    3423 non-null    float64
         2   AMZN    3423 non-null    float64
         3   GOOGL   3423 non-null    float64
        dtypes: float64(4)
        memory usage: 133.7 KB

```

Let's now rebase the prices so that all time series start at 100. This allows us to compare their relative performance in a chart (see [Figure 7-4](#)). To rebase a time series, divide every value by its starting value and multiply by 100, the new base. If you did this in Excel, you would typically write a formula with a combination of

absolute and relative cell references, then copy the formula for every row and every time series. In pandas, thanks to vectorization and broadcasting, you are dealing with a single formula:

```
In [22]: # Use a sample from March 2015 - February 2016
adj_close_sample = adj_close.loc["2015-03":"2016-02", :]
rebased_prices = adj_close_sample / adj_close_sample.iloc[0, :] * 100
rebased_prices.head(2)
```

```
Out[22]:
```

	MSFT	AAPL	AMZN	GOOGL
date				
2015-03-02	100.000000	100.000000	100.000000	100.000000
2015-03-03	98.632634	100.209156	99.729032	100.656499

```
In [23]: # "style" allows us to differentiate the lines in a black/white print
rebased_prices.plot(style=["-", "--", "-.", ":"])
```

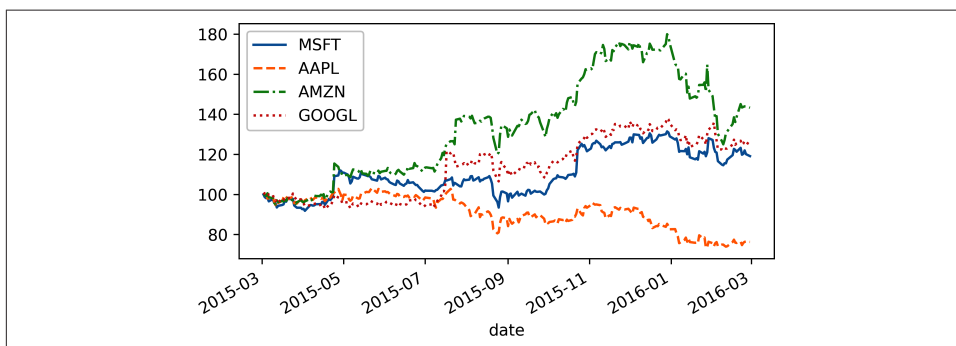


Figure 7-4. Rebased time series

To see how independent the returns of the different stocks are, have a look at their correlations by using the `corr` method. Unfortunately, pandas' default plotting library, Matplotlib, doesn't provide an easy way to visualize the correlation matrix as a heatmap, so we'll use seaborn instead (see Figure 7-5):

```
In [24]: # Correlation of daily log returns
returns = np.log(adj_close / adj_close.shift(1))
correlations = returns.corr()
```

```
Out[24]:
```

	MSFT	AAPL	AMZN	GOOGL
MSFT	1.000000	0.421173	0.448301	0.465176
AAPL	0.421173	1.000000	0.393542	0.460943
AMZN	0.448301	0.393542	1.000000	0.456864
GOOGL	0.465176	0.460943	0.456864	1.000000

```
In [25]: # Use seaborn to create a heatmap
ax = sns.heatmap(
    correlations, # Data
    cmap="coolwarm", # Colormap
    vmin=-1, # Min value to anchor the colormap
    vmax=1, # Max value to anchor the colormap
```

```

linewidths=0.5, # Width of white lines
xticklabels=True, # Show column names of DataFrame on x-axis
yticklabels=True, # Show column names of DataFrame on y-axis
annot=True, # Show the value in addition to the color
)
ax.tick_params(left=False, bottom=False) # Remove tick marks

```

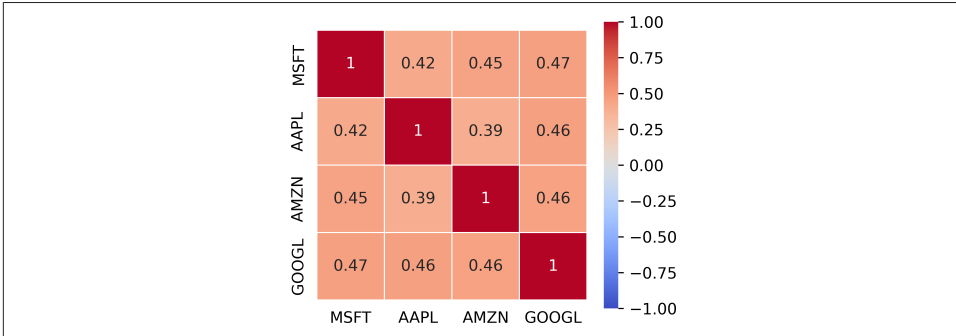


Figure 7-5. Correlation heatmap

A heatmap visualizes a table of numbers by mapping values to colors (the “coolwarm” colormap uses blue for smaller values and red for larger ones). Each cell shows the correlation between the stock in that row and the stock in that column—since our four tech stocks are all positively correlated, the heatmap shows mostly red tones. If you read the printed version of this book, make sure to run the accompanying notebook to see the heatmap in colors.

At this point, we have already learned quite a few things about time series, including how to combine and clean them and how to calculate returns and correlations. But what if you decide that daily returns are not a good base for your analysis and you want monthly returns? Changing the frequency of time series data is the topic of the next section.

Resampling

A regular task with time series is *up-* and *downsampling*. Upsampling means that the time series is converted into one with a higher frequency, while downsampling means that it is converted into one with a lower frequency. On financial factsheets, you often show monthly or quarterly performance, for example. To turn a daily time series into a monthly one, use the `resample` method that accepts a frequency string like `ME` for *calendar-month-end* or `BME` for *business-month-end*. You can find a list of all frequency strings in the [pandas docs](#). Similar to how `groupby` works, you then chain a method that defines *how* you are resampling. I am using `last` to always take the last observation of that month:

```
In [26]: end_of_month = adj_close.resample("ME").last()
end_of_month.head()

Out[26]:
```

	MSFT	AAPL	AMZN	GOOGL
date				
2004-08-31	18.386832	2.216223	38.14	51.343492
2004-09-30	18.622561	2.489958	40.86	65.000651
2004-10-31	18.838084	3.367065	34.13	95.615155
2004-11-30	20.087297	4.308430	39.68	91.271747
2004-12-31	20.019865	4.138149	44.29	96.693484

Instead of last, you can choose any other method that works on groupby, like sum or mean. There is also ohlc, which conveniently returns the open, high, low, and close values over that period. This may serve as the source to create the typical candlestick charts that are often used with stock prices.

If that end-of-month time series would be all you have and you need to produce a weekly time series out of it, you have to upsample your time series. By using `asfreq`, you are telling pandas not to apply any transformation, and hence you will see most of the values showing NaN. If you wanted to *forward-fill* the last known value instead, use the `ffill` method:

```
In [27]: # Upsample from monthly to daily without transformation
end_of_month.resample("D").asfreq().head()

Out[27]:
```

	MSFT	AAPL	AMZN	GOOGL
date				
2004-08-31	18.386832	2.216223	38.14	51.343492
2004-09-01	NaN	NaN	NaN	NaN
2004-09-02	NaN	NaN	NaN	NaN
2004-09-03	NaN	NaN	NaN	NaN
2004-09-04	NaN	NaN	NaN	NaN

```
In [28]: # Upsample from monthly to weekly using forward filling
end_of_month.resample("W-FRI").ffill().head()

Out[28]:
```

	MSFT	AAPL	AMZN	GOOGL
date				
2004-09-03	18.386832	2.216223	38.14	51.343492
2004-09-10	18.386832	2.216223	38.14	51.343492
2004-09-17	18.386832	2.216223	38.14	51.343492
2004-09-24	18.386832	2.216223	38.14	51.343492
2004-10-01	18.622561	2.489958	40.86	65.000651

Downsampling data is one way of smoothing a time series. Calculating statistics over a rolling window is another way, as we will see next.

Rolling Windows

When you calculate time series statistics, you often want a rolling statistic such as the *moving average*. The moving average looks at a subset of the time series (let's say 25 days) and takes the mean from this subset before moving the window forward by one period. This will result in a new time series that is smoother and less prone to outliers. If you are into algorithmic trading, you may be looking at the intersection of the moving average with the stock price and take this (or some variation of it) as a trading signal. DataFrames have a rolling method, which accepts the number of observations as argument. You then chain it with the statistical method that you want to use—to calculate the moving average, use the mean. By looking at [Figure 7-6](#), you are able to compare the original time series with the moving average:

```
In [29]: # Plot the moving average for MSFT with data from 2017
msft17 = msft.loc["2017", ["adj_close"]].copy()

# Add the 25 day moving average as a new column to the DataFrame
msft17["25day average"] = msft17["adj_close"].rolling(25).mean()
msft17.plot()
```

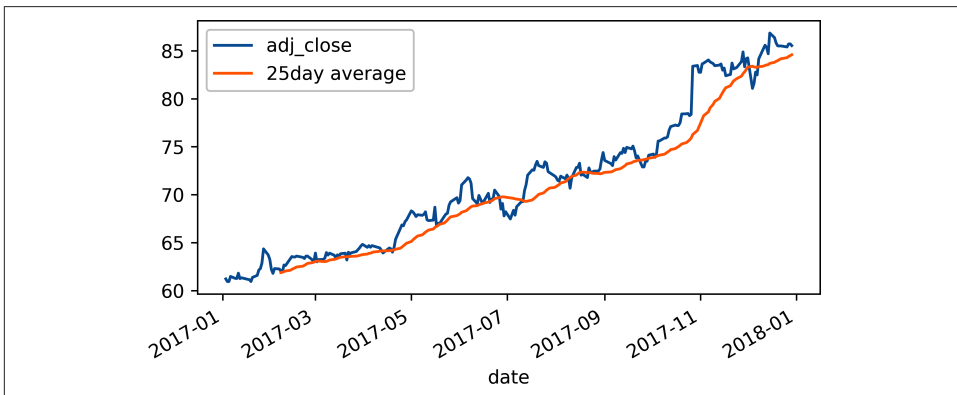


Figure 7-6. Moving average plot

Instead of mean, the rolling method can be chained with many other methods, including count, sum, median, min, max, std (standard deviation), or var (variance).

This chapter, together with [Chapter 5](#), introduced the most important functionality of pandas. What's left is to explore a more sophisticated data loading method that Python in Excel offers via Power Query.

Loading Data with Power Query

At the beginning of this chapter, in cell [5], we were loading the historical prices from Excel tables. This is fine, but often, your data source is outside of Excel, especially if it is bigger than what would fit into a worksheet. This is where Power Query comes in.

As you may remember from the previous chapter, Python in Excel can't directly load external files. Instead, you have to use Power Query to load the data, which you can then access from Python in Excel via the `xl` function. Importantly, you don't need to load the data into an Excel worksheet—it's enough to create a *connection-only* Power Query. So what is a connection-only Power Query? Going through the process of loading a file will make that clear!



Connection-Only Power Queries Can Only Be Created on Windows

At the time of this writing, only the Windows version of Excel allows you to create connection-only Power Queries. If you are on macOS or Excel on the web, you can still use workbooks with an existing connection-only Power Query, but you can't create new ones. I would expect the macOS and web version of Excel to catch up eventually, so make sure to double-check this on your end.

Loading a Single File

To understand how connection-only Power Queries work, we're going to load the historical prices of the Microsoft stock from the `msft.csv` file in the `csv` folder of the companion repository:

1. On the Data tab in the Excel ribbon, go to Get Data > From File > From Text/CSV (see [Figure 7-7](#)). With newer versions of Excel, Get Data will open a dialog where you can search for Text/CSV instead.
2. In the dialog, navigate to the folder with your companion repo and select `msft.csv` in the `csv` folder, then click on Import.
3. In the next screen, select the dropdown of the Load button and select "Load To" (see [Figure 7-8](#)).
4. In the Import Data dialog, select the radio button "Only Create Connection" and click OK (see [Figure 7-9](#)).

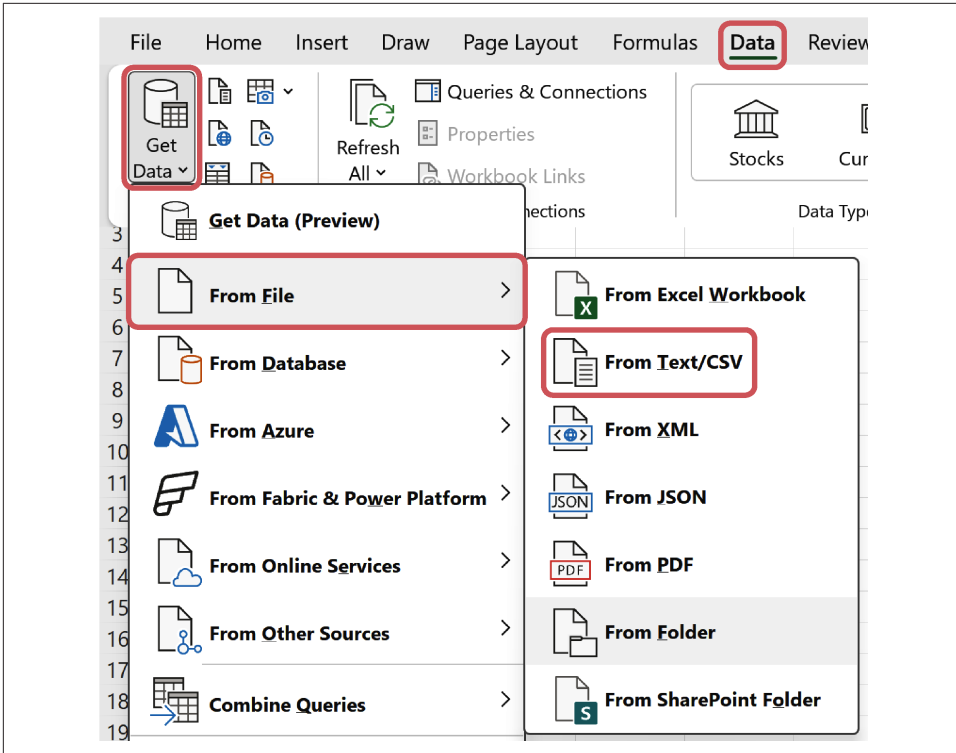


Figure 7-7. Get data

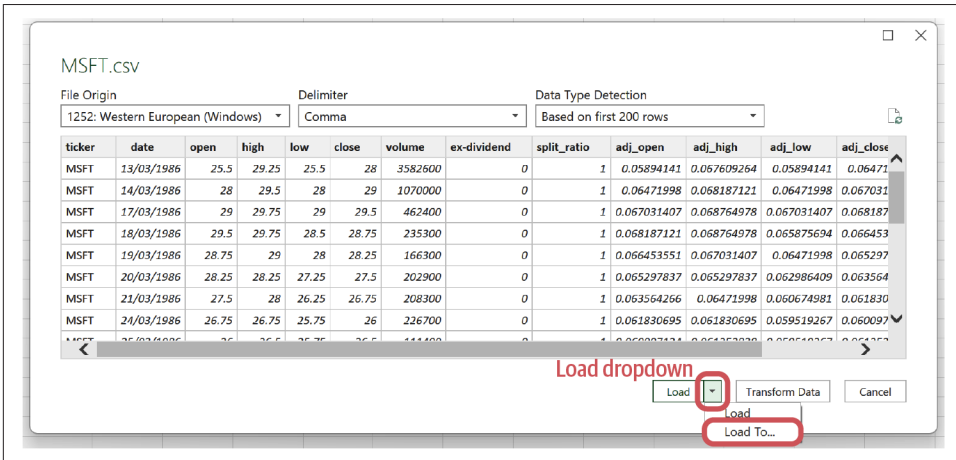


Figure 7-8. The "Load" dropdown in Power Query

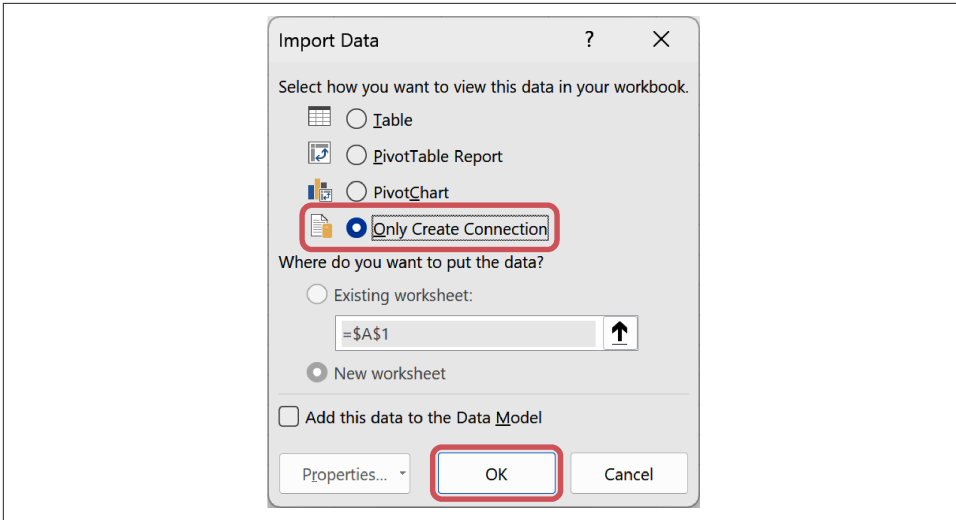


Figure 7-9. The “Only Create Connection” radio button in Power Query

The last action will open the Queries & Connections task pane, where you’ll see the name of your connection. By default, it should be “MSFT” and marked as “Connection only” (see Figure 7-10).

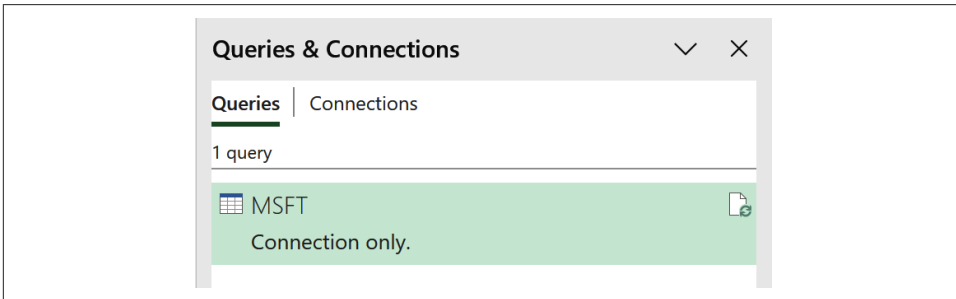


Figure 7-10. The Power Query Queries & Connections task pane



Reopen the Queries & Connections Task Pane

If you accidentally close the Queries & Connections task pane, you can reopen it by going to the Data tab on the ribbon, then clicking on the Queries & Connections button that you will find in the Queries & Connections group.

To use this Power Query connection instead of the TableMSFT Excel table, adjust the respective line in cell [5] as follows:

```
msft = xl("MSFT", headers=True).set_index("date")
```

You could now continue to do the same for the other stocks. There is, however, a more flexible approach: loading multiple files from a specific folder.

Loading Files from a Folder

To create a connection-only Power Query for multiple files, go through these steps:

1. On the Data tab in the Excel ribbon, go to Get Data > From File > From Folder. With newer versions of Excel, Get Data will open a dialog where you can search for “Folder” instead.
2. In the dialog, select the folder from where you want to import the files. In our case, this is the `csv` folder in the companion repository (see [Figure 7-11](#)).

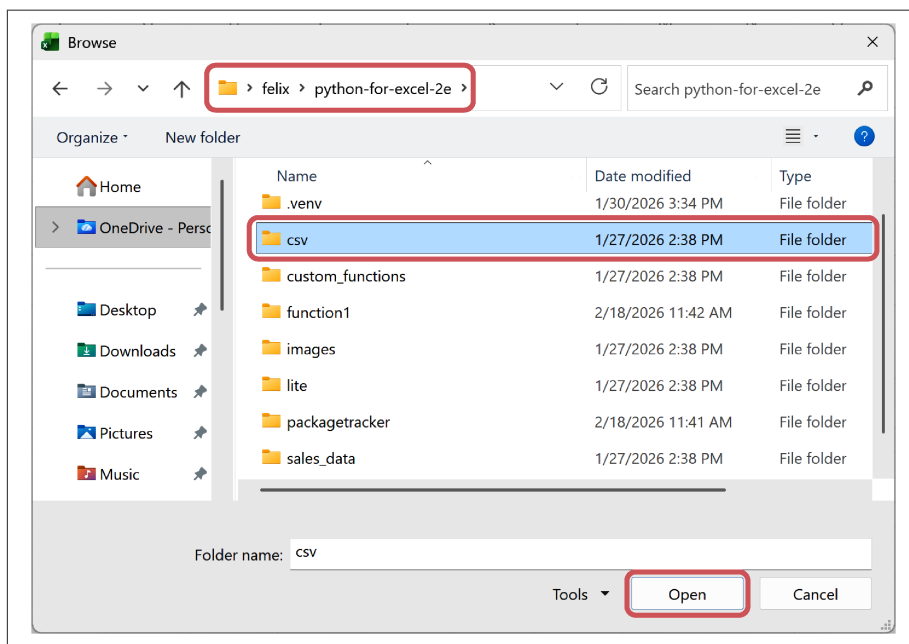


Figure 7-11. Selecting a folder in Power Query

3. The next screen will list all the files that are in that folder. At the bottom, click the dropdown of the Combine button and select “Combine & Load To.” Note that this requires all files to follow the same structure, i.e., they need to have the same column headers.
4. The Combine Files dialog will show a preview of a single file. Confirm by clicking on OK.

5. In the Import Data dialog, select the radio button “Only Create Connection” and click OK.

The last action will open the Queries & Connections task pane on the right-hand side of Excel, where you’ll see a bunch of helper queries together with the name of your connection. By default, it should be “csv” and marked as “Connection only.”

Now replace the code in cell [5] with the following:

```
# DataFrame containing data from all files (vertically stacked)
csv_data = xl("csv")
# Get the tickers by using the DataFrame's unique method
tickers = csv_data["ticker"].unique()
# Use a dict comprehension to create a dictionary in the form of
# {"MSFT": <DataFrame>, "AAPL": <DataFrame>, ...}
# Note that we are also setting the index here
data = {
    ticker: csv_data.loc[csv_data["ticker"] == ticker, :].set_index("date")
    for ticker in tickers
}
# Assign the DataFrames to their variables
msft = data["MSFT"]
aapl = data["AAPL"]
amzn = data["AMZN"]
googl = data["GOOGL"]
```

Note that Power Query added an additional column `Source.Name` to the DataFrame `csv_data` showing the name of the source file. To make everything more flexible, you could also remove the last four lines and replace `msft` with `data["MSFT"]`, `aapl` with `data["AAPL"]`, etc. across all cells.



Python in Excel Supports All Power Query Sources

While we have only used files as data sources in this section, you can use all the supported data sources that Power Query offers, including databases, certain Azure services, the web, etc.

Conclusion

Time series analysis is the area where I feel Excel has fallen behind the most, so after reading this chapter, you probably understand why pandas was received with open arms by the financial industry, an industry that heavily relies on time series. We've seen how easy it is to resample time series or produce correlation heatmaps, functionality that either isn't supported in Excel or requires cumbersome workarounds.

This was the last chapter about Python in Excel. As mentioned in [Chapter 6](#), Python in Excel has a very narrow focus: deliver powerful analytics and visuals primarily based on the pandas DataFrame ecosystem. Python, however, is a general-purpose programming language, so using it only for analytics means that you're missing out on all the other good parts! With Python, we can automate the whole Excel application instead of just analyzing data. This opens up a new range of functionality that we are going to explore in [Part IV](#). The next chapter starts this journey by showing you how to control Excel from Python remotely.

PART IV

xlwings

Excel Automation

This chapter kicks off **Part IV**, which is about `xlwings`. While Python in Excel turns Excel into a Jupyter notebook, `xlwings` aims to replace VBA with Python. Accordingly, `xlwings` allows you to automate the Excel application, run macros, and create custom functions. To do this, `xlwings` relies on the Python environment that we installed in **Chapter 2**. You can therefore work offline and use any third-party package that you want.

We'll start this chapter with the basics like reading and writing cell values. After learning how `xlwings` works with pandas DataFrames, charts, and pictures, we'll put these skills into practice with a reporting case study. The last section teaches you how to make your scripts performant as well as work around missing functionality. This chapter requires you to run the code samples on either Windows or macOS, as they depend on a local installation of Microsoft Excel.

Getting Started with `xlwings`

The main goal of `xlwings` is to serve as a drop-in replacement for VBA, allowing you to interact with Excel from Python. Since Excel's grid is the perfect layout to display data structures like lists, NumPy arrays, and pandas DataFrames, one of `xlwings`' core features is to make reading from and writing to Excel as easy as possible. I'll start this section by using Excel as a data viewer—this is useful when you are interacting with DataFrames in a Jupyter notebook. Next, I'll walk you through the Excel object model, and to wrap this section up, you'll learn how to call VBA code from Python.

Using Excel as Data Viewer

If a DataFrame has more than ten rows, Jupyter notebooks only show the top five and bottom five rows. One way to understand your data better is to plot it—this allows you to spot outliers or other irregularities. Sometimes, however, it's helpful to scroll through the full dataset in Excel. That's what the view function is here for:

```
In [1]: # First, let's import the packages that we'll use in this chapter
import datetime as dt
import numpy as np
import pandas as pd
import xlwings as xw

In [2]: # Let's create a DataFrame based on pseudorandom numbers.
# We'll make it large enough that only the head and tail are displayed.
rng = np.random.default_rng(seed=1234)
df = pd.DataFrame(
    data=rng.standard_normal((100, 5)),
    columns=[f"Sample {i}" for i in range(1, 6)],
)
df

Out[2]:
```

	Sample 1	Sample 2	Sample 3	Sample 4	Sample 5
0	-1.603837	0.064100	0.740891	0.152619	0.863744
1	2.913099	-1.478823	0.945473	-1.666135	0.343745
2	-0.512444	1.323759	-0.860280	0.519493	-1.265144
3	-2.159139	0.434734	1.733289	0.520134	-1.002166
4	0.268346	0.767175	1.191272	-1.157411	0.696279
..	...	...	...	...	...
95	0.127758	0.405665	-0.223037	0.614121	-0.304380
96	-1.645335	-2.086254	1.421543	-1.302643	-2.198949
97	0.259770	-0.537652	0.289122	1.588700	0.733091
98	-0.657177	1.668796	0.377436	1.082252	0.961028
99	1.657336	1.039326	1.124510	-0.606264	-1.181706

```
[100 rows x 5 columns]

In [3]: # View the DataFrame in Excel
xw.view(df)
```

The view function accepts all common Python objects, including lists, dictionaries, NumPy arrays, and pandas DataFrames. By default, it opens a new workbook and pastes the values into cell A1 of the first sheet—it also formats the data as an Excel table and adjusts the column widths. Instead of opening a new workbook every time, you can also reuse the same one by providing the view function an xlwings sheet object as the second argument: `xw.view(df, mysheet)`. How you get access to such a sheet object and how it fits into the Excel object model is what I'll explain next.



macOS: Permissions and Preferences

On macOS, I would recommend you to run Jupyter notebooks and VS Code from a Terminal, as shown in [Chapter 2](#). This ensures that you will be greeted by two pop-ups when you use xlwings for the first time: the first one is “Terminal wants access to control System Events” and the second one is “Terminal wants access to control Microsoft Excel.” You will need to confirm both pop-ups to allow Python to automate Excel. In theory, these pop-ups should be triggered by any application from which you run xlwings code, but in practice, that’s often not the case, so running them via the Terminal will keep you out of trouble. Also, you’ll need to open Excel’s Preferences and uncheck “Show Workbook Gallery when opening Excel” under the General category. This opens Excel directly on an empty workbook instead of opening the gallery first, which would get in your way when you open a new Excel instance via xlwings.

The Excel Object Model

When working with Excel programmatically, you interact with its components like a workbook or a sheet. These components are organized in the *Excel object model*, a hierarchical structure that represents Excel’s graphical user interface (see [Figure 8-1](#)). For a complete list of available xlwings objects, see the [API Reference](#).

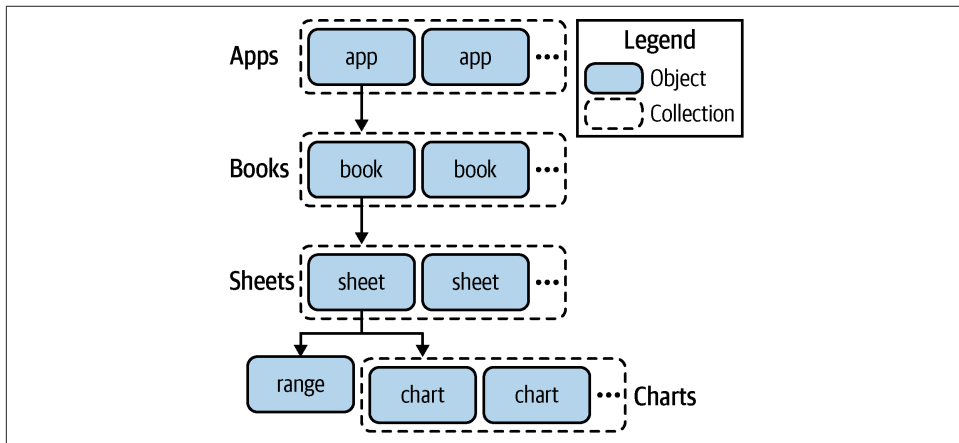


Figure 8-1. The Excel object model as implemented by xlwings (excerpt)

xlwings follows the VBA Excel object model very closely, only with a breath of fresh air: for example, xlwings uses the names `App` instead of `Application` and `Book` instead of `Workbook`:

- An app contains the books collection.
- A book contains the sheets collection.
- A sheet gives access to range objects and collections such as charts.
- A range contains one or more contiguous cells as its items.

The dashed boxes are *collections* and contain one or more objects of the same type. An app corresponds to an Excel instance, i.e., an Excel application that runs as a separate process. Power users sometimes use multiple Excel instances so they can run a process in one Excel window while working simultaneously in another Excel file. As we'll see in this chapter, the ability to work with multiple Excel instances is very useful. See the sidebar on how to do this manually.

Running Multiple Excel Instances

Here's how you open a second instance manually: start Excel and right-click on its icon in the Windows taskbar. In the appearing menu, left-click on the Excel entry while holding down the Alt key at the same time (make sure to keep the Alt key pressed until after you release your mouse button)—a pop-up will ask if you want to start a new instance of Excel. On macOS, there is no manual way of launching more than one instance of the same program, but you can launch multiple Excel instances programmatically via `xlwings`, as we will see later.



Language and Regional Settings

This book is based on the US-English version of Excel. I will occasionally refer to default names like “Book1” or “Sheet1,” which will be different if you use Excel in another language. For example, “Sheet1” is called “Feuil1” in French and “Hoja1” in Spanish. Also, the *list separator*, which is the separator that Excel uses in cell formulas, depends on your settings: I will be using the comma, but your version may require a semicolon or another character. For example, instead of writing `=SUM(A1, A2)`, you will need to write `=SUMME(A1; A2)` on a computer with German regional settings. Further traps to look out for are TRUE or FALSE and the decimal point: for example, with German regional settings, TRUE is called WAHR and the decimal point is a comma instead of a point.

The book object contains one or more sheets, and the sheet object gives you access to collections like charts, pictures, and names. To get a feel for the Excel object model, as usual, it's best to play around with it. Let's start with the Book class: it

allows you to create new workbooks and connect to existing ones; see [Table 8-1](#) for an overview.

Table 8-1. Working with Excel workbooks

Command	Description
<code>xw.Book()</code>	Returns a book object representing a new Excel workbook in the active Excel instance. If there is no active instance, Excel will start automatically.
<code>xw.Book("Book1")</code>	Returns a book object representing an unsaved workbook with the name <i>Book1</i> (name without file extension).
<code>xw.Book("Book1.xlsx")</code>	Returns a book object representing a previously saved workbook with the name <i>Book1.xlsx</i> (name with file extension). The file has to be either open or in the current working directory.
<code>xw.Book(r"C:\path\Book1.xlsx")</code>	Returns a book object of a previously saved workbook (full file path). The file can be open or closed. The leading <code>r</code> turns the string into a raw string so the backslashes (<code>\</code>) of the path are interpreted literally on Windows (I introduced raw strings in Chapter 5). On macOS, the <code>r</code> isn't required as file paths use forward slashes instead of backslashes.
<code>xw.books.active</code>	Returns a book object representing the active workbook in the active Excel instance. Note that this requires Excel to be running.

Let's see how we can walk through the object model hierarchy from the book object down to the range object:

```
In [4]: # Create a new empty workbook and print its name. This is the
        # book we will use to run most of the code samples in this chapter.
        book = xw.Book()
        book.name

Out[4]: 'Book2'

In [5]: # Accessing the sheets collection
        book.sheets

Out[5]: Sheets([<Sheet [Book2]Sheet1>])

In [6]: # Get a sheet object by index or name. You will need to adjust
        # "Sheet1" if your sheet is called differently. Remember: Python
        # uses zero-based indexing.
        sheet1 = book.sheets[0]
        sheet1 = book.sheets["Sheet1"]

In [7]: sheet1.range("A1")

Out[7]: <Range [Book2]Sheet1!$A$1>
```

With the range object, we have arrived at the bottom of the hierarchy. The string that gets printed in between angle brackets gives you useful information about that object, but to do something, you usually use the object with an attribute such as `value`, as the next samples show:

```

In [8]: # Most common tasks: write values...
        sheet1.range("A1").value = [[1, 2], [3, 4]]
        sheet1.range("A4").value = "Hello!"

In [9]: # ...and read values
        sheet1.range("A1:B2").value

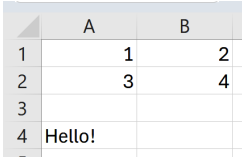
Out[9]: [[1.0, 2.0], [3.0, 4.0]]

In [10]: sheet1.range("A4").value

Out[10]: 'Hello!'

```

At this point, your workbook should look like **Figure 8-2**.



	A	B
1	1	2
2	3	4
3		
4	Hello!	

Figure 8-2. The workbook after running the first ten cells

As you can see, the `value` attribute accepts and returns a nested list for two-dimensional ranges and a scalar for a single cell. Everything we've used so far is almost identical to VBA: assuming that `mybook` is a VBA Workbook or an `xlwings` Book, respectively, this is how you read values with VBA and `xlwings`:

```

mybook.Sheets(1).Range("A1:B2").Value # VBA
mybook.sheets[0].range("A1:B2").value # xlwings

```

Here's how the Python version differs:

Attributes

Python uses lowercase letters, potentially with underscores.

Indexing

Python uses square brackets and zero-based indices to access an element in the `sheets` collection.

Table 8-2 gives you an overview of the strings that an `xlwings` range accepts.

Table 8-2. Strings to define a range in A1 notation

Reference	Description
"A1"	A Single Cell
"A1:B2"	Cells from A1 to B2
"A:A"	All cells in Column A
"A:B"	All cells in Columns A to B

Reference	Description
"1:1"	All cells in Row 1
"1:2"	All cells in Rows 1 to 2

You can index and slice range objects like NumPy arrays—watch the address in between angle brackets (the printed object representation) to see what cell range you end up with:

```
In [11]: # Indexing
         sheet1.range("A1:B2")[0, 0]

Out[11]: <Range [Book2]Sheet1!$A$1>

In [12]: # Slicing
         sheet1.range("A1:B2")[:, 1]

Out[12]: <Range [Book2]Sheet1!$B$1:$B$2>
```

Indexing corresponds to using the Cells property in VBA:

```
mybook.Sheets(1).Range("A1:B2").Cells(1, 1) # VBA
mybook.sheets[0].range("A1:B2")[0, 0] # xlwings
```

Instead of using range as an attribute of the sheet object, you can also index and slice the sheet object instead. Using this with A1 notation will allow you to type less, and using this with integer indices makes the Excel sheet feel like a NumPy array:

```
In [13]: # Single cell: A1 notation
         sheet1["A1"]

Out[13]: <Range [Book2]Sheet1!$A$1>

In [14]: # Multiple cells: A1 notation
         sheet1["A1:B2"]

Out[14]: <Range [Book2]Sheet1!$A$1:$B$2>

In [15]: # Single cell: indexing
         sheet1[0, 0]

Out[15]: <Range [Book2]Sheet1!$A$1>

In [16]: # Multiple cells: slicing
         sheet1[:, 2]

Out[16]: <Range [Book2]Sheet1!$A$1:$B$2>
```

Going forward, I will use this notation instead of using the explicit range object. However, there is one situation where using range explicitly makes more sense: if you want to stay closer to the VBA Cells notation that defines a range by referring to the top-left and bottom-right cell of a range with one-based indices. The following samples refer to the cell ranges D10 and D10:F11 and compare the 0-based index/slicing approach with the one-based range approach:

```

In [17]: # D10 via sheet indexing
         sheet1[9, 3]

Out[17]: <Range [Book2]Sheet1!$D$10>

In [18]: # D10 via range object
         sheet1.range((10, 4))

Out[18]: <Range [Book2]Sheet1!$D$10>

In [19]: # D10:F11 via sheet slicing
         sheet1[9:11, 3:6]

Out[19]: <Range [Book2]Sheet1!$D$10:$F$11>

In [20]: # D10:F11 via range object
         sheet1.range((10, 4), (11, 6))

Out[20]: <Range [Book2]Sheet1!$D$10:$F$11>

```

Using range with tuples is very similar to how the Cells property works in VBA, as the following comparison shows (this assumes again that mybook is either a VBA Workbook or an xlwings Book). Let's first look at the VBA version:

```

With mybook.Sheets(1)
    Set myrange = .Range(.Cells(10, 4), .Cells(11, 6))
End With

```

This is equivalent to the following xlwings expression:

```
myrange = mybook.sheets[0].range((10, 4), (11, 6))
```



Zero Versus One-Based Indices

As a Python package, xlwings consistently uses zero-based indexing whenever you access elements via Python's index or slice syntax, i.e., via square brackets. xlwings range objects, however, use Excel's one-based row and column indices. Having the same row/column indices as Excel's user interface may sometimes be beneficial. If you prefer to only ever use Python's zero-based indexing, simply use the `sheet[row_selection, column_selection]` syntax.

The following sample shows you how to get from the range object `sheet1["A1"]` all the way up again to the app object. Remember that the app object represents an Excel instance (the number in the output represents Excel's process ID and will therefore be different on your machine):

```

In [21]: sheet1["A1"].sheet.book.app

Out[21]: <App [excel] 12604>

```

Having arrived at the very top of the Excel object model, it's a good moment to see how you can work with multiple Excel instances. You will need to use the app

object explicitly if you want to open the same workbook in multiple Excel instances or if you specifically want to distribute your workbooks across different instances for performance reasons. Another common use case for working with an app object is to open your workbook in a hidden Excel instance: this allows you to run an xlwings script in the background:

```
In [22]: # Get one app object from the open workbook
        # and create an additional invisible app instance
        visible_app = sheet1.book.app
        invisible_app = xw.App(visible=False)

In [23]: # List the book names that are open in each instance
        # by using a list comprehension
        [book.name for book in visible_app.books]

Out[23]: ['Book1', 'Book2']

In [24]: [book.name for book in invisible_app.books]

Out[24]: ['Book3']

In [25]: # An app key represents the process ID (PID)
        xw.apps.keys()

Out[25]: [12492, 12604]

In [26]: # It can also be accessed via the pid attribute
        xw.apps.active.pid

Out[26]: 12492

In [27]: # Work with the book in the invisible Excel instance
        invisible_book = invisible_app.books[0]
        invisible_book.sheets[0]["A1"].value = "Created by an invisible app."

In [28]: # Save the Excel workbook in the xl directory
        invisible_book.save("xl/invisible.xlsx")

In [29]: # Quit the invisible Excel instance
        invisible_app.quit()
```



macOS: Accessing the File System Programmatically

If you run the save command on macOS, you will get a Grant File Access pop-up in Excel that you will need to confirm by clicking the Select button before clicking on Grant Access. On macOS, Excel is *sandboxed*, which means that your program can only access files and folders outside of the Excel app by confirming this prompt. Once confirmed, Excel will remember the locations and won't bug you again when you run the script the next time.

If you have the same workbook open in two instances of Excel or if you want to specify in which Excel instance you want to open a workbook, you can't use `xw.Book` anymore. Instead, you need to fully qualify the book by starting at the app object, as

laid out in [Table 8-3](#). Note that `myapp` stands for an `xlwings` app object. For example, to use the active Excel instance, you would use `xw.apps.active` in place of `myapp`. As an alternative to `xw.apps.active.books`, you can also use the shorthand notation `xw.books`.

Table 8-3. Working with the books collection

Command	Description
<code>myapp.books.add()</code>	Creates a new Excel workbook in the Excel instance that <code>myapp</code> refers to and returns the corresponding book object.
<code>myapp.books.open(r"C:\path\Book.xlsx")</code>	Returns the book if it's already open, otherwise opens it first in the Excel instance that <code>myapp</code> refers to. Remember that the leading <code>r</code> turns the file path into a raw string to interpret the backslashes literally.
<code>myapp.books["Book1.xlsx"]</code>	Returns the book object if it's open. This will raise a <code>KeyError</code> if it isn't open yet. Make sure to use the name and not the full path. Use this if you need to know if a workbook is already open in Excel.

Before we dive deeper into how `xlwings` can *replace* your VBA macros, let's see how `xlwings` can *call* your existing VBA code: this can be useful if you have a lot of existing code and don't want to migrate everything to Python or if you want to use Python to run existing VBA macros in an automated way, e.g., by using the Windows task scheduler.

Running VBA Code

If you have existing Excel projects with lots of VBA code, it may be too much work to migrate everything to Python. In this case, you can keep the VBA code that already works and use Python only where it adds real leverage, like data processing with `pandas` or creating visualizations with `Matplotlib`. This allows you to enhance your existing projects with minimal effort.

The following sample uses the `vba.xlsm` file that you will find in the `xl` folder of the companion repo and shows how Python can call your existing VBA macros.



Enable Macros

Make sure to properly enable macros when opening a macro-enabled workbook (*xls*m extension) such as *vba.xls*m:

- Click on Enable Content.
- When working with *xls*m files from the internet, you have to additionally click on Enable Editing.
- Before opening the workbook, though, you might need to unblock the file first by right-clicking on it in the Windows File Explorer and selecting Properties. On the General tab, check the box “Unblock,” then click OK to exit the properties window (macOS doesn’t require unblocking a file).

The file *vba.xls*m contains the following code in Module1:

```
Function MySum(x As Double, y As Double) As Double
    MySum = x + y
End Function

Sub ShowMsgBox(msg As String)
    MsgBox msg
End Sub
```

To call these functions via Python, you first need to instantiate an *xlwings* macro object that you subsequently call, making it feel as if it were a native Python function:

```
In [30]: vba_book = xw.Book("xl/vba.xls")

In [31]: # Instantiate a macro object with the VBA function
mysum = vba_book.macro("Module1.MySum")
# Call a VBA function
mysum(5, 4)

Out[31]: 9.0

In [32]: # It works the same with a VBA Sub procedure
show_msgbox = vba_book.macro("Module1.ShowMsgBox")
show_msgbox("Hello xlwings!")

In [33]: # Close the book again (make sure to close the MessageBox first)
vba_book.close()
```



Don't Store VBA Functions in Sheet and ThisWorkbook Modules

If you store the VBA function `MySum` in the workbook module `ThisWorkbook` or a sheet module (e.g., `Sheet1`), you have to refer to it as `ThisWorkbook.MySum` or `Sheet1.MySum`. However, you won't be able to access the function's return value from Python, so make sure to store VBA functions in a standard VBA code module that you insert by right-clicking on the `Modules` folder in the VBA editor.

Now that you know how to interact with existing VBA code, you'll next learn how to convert Excel values to pandas DataFrames and how to work with collections such as charts.

Converters, Options, and Collections

In the introductory code samples of this chapter, we were reading and writing a string and a nested list from and to Excel by using the `value` attribute of an `xlwings` range object. I'll start this section by showing you how this works with pandas DataFrames before having a closer look at the `options` method that allows us to influence how `xlwings` reads and writes values. To conclude, we'll work with collections, including charts, pictures, and names.

Working with DataFrames

Writing a DataFrame to Excel is no different from writing a string or a nested list: simply assign the DataFrame to the top-left cell of an Excel range:

```
In [34]: data = [
          ["Mark", 1966, "Italy", 4.5, "Europe"],
          ["John", 1988, "USA", 6.7, "America"],
        ]
          students = pd.DataFrame(
              data=data,
              columns=["name", "born", "country", "score", "continent"],
              index=[1001, 1000],
          )
          students.index.name = "user_id"
          students
```

```
Out[34]:
```

	name	born	country	score	continent
user_id					
1001	Mark	1966	Italy	4.5	Europe
1000	John	1988	USA	6.7	America

```
In [35]: sheet1["A6"].value = students
```

If, however, you would like to suppress the column headers and/or the index, use the options method like this:

```
In [36]: sheet1["B10"].options(header=False, index=False).value = students
```

Reading Excel ranges as DataFrames requires you to provide the DataFrame class as the convert parameter in the options method. By default, it expects that your data has both a header and index, but you can again use the index and header parameters to change this. Instead of using the converter, you could also read in the values first as a nested list and then manually construct your DataFrame, but using the converter makes it easier to handle the index and header.



The expand Method

In the following code sample, I introduce the expand method that makes it easy to read a contiguous block of Excel cells, delivering the same range as if you were doing Ctrl+Shift+Down-Arrow+Right-Arrow in Excel, except that expand jumps over an empty cell in the top-left corner:

```
In [37]: students2 = sheet1["A6"].expand().options(pd.DataFrame).value
students2
```

```
Out[37]:
```

	name	born	country	score	continent
user_id					
1001.0	Mark	1966.0	Italy	4.5	Europe
1000.0	John	1988.0	USA	6.7	America

```
In [38]: # If you want the index to be an integer index,
# you can change its data type via "astype"
students2.index = students2.index.astype(int)
students2
```

```
Out[38]:
```

	name	born	country	score	continent
user_id					
1001	Mark	1966.0	Italy	4.5	Europe
1000	John	1988.0	USA	6.7	America

```
In [39]: # By setting index=False, it will put all the values from Excel into
# the data part of the DataFrame and will use the default index
sheet1["A6"].expand().options(pd.DataFrame, index=False).value
```

```
Out[39]:
```

	user_id	name	born	country	score	continent
0	1001.0	Mark	1966.0	Italy	4.5	Europe
1	1000.0	John	1988.0	USA	6.7	America

Reading and writing DataFrames was a first example of how converters and options work. Next, let's see how converters and options are formally defined and how you can use them with other data structures.

Converters and Options

As we have just seen, the `options` method of the `xlwings` range object allows you to influence the way that values are read and written from and to Excel. That is, `options` are only evaluated when you call the `value` attribute on a range object. The syntax is as follows (myrange is an `xlwings` range object):

```
myrange.options(convert=None, option1=value1, option2=value2, ...).value
```

Table 8-4 shows the built-in converters, i.e., the values that the `convert` argument accepts. They are called *built-in* as `xlwings` offers a way to write your own converters, which could be useful if you have to repeatedly apply additional transformations before writing or after reading values—to see how it works, have a look at the [xlwings docs](#).

Table 8-4. Built-in converters

Converter	Description
dict	Simple dictionaries without nesting, i.e., in the form {key1: value1, key2: value2, ...}.
tuple	Tuples are helpful for caching functions, as we will see in Chapter 11 .
np.array	NumPy arrays, requires <code>import numpy as np</code> .
pd.Series	pandas Series, requires <code>import pandas as pd</code> .
pd.DataFrame	pandas DataFrame, requires <code>import pandas as pd</code> .
pl.DataFrame	polars DataFrame, requires <code>import polars as pl</code> .

We have already used the `index` and `header` options with the DataFrame example, but there are more options available, as shown in Table 8-5.

Table 8-5. Built-in options

Option	Description
empty	By default, empty cells are read as <code>None</code> . Change this by providing a value for <code>empty</code> .
date	Accepts a function that is applied to values from date-formatted cells.
numbers	Accepts a function that is applied to numbers.
ndim	<i>Number of dimensions</i> : when reading, use <code>ndim</code> to force the values of a range to arrive in a certain dimensionality. Must be either <code>None</code> , <code>1</code> , or <code>2</code> . Can be used when reading values as lists or NumPy arrays.
transpose	Transposes the values, i.e., turns the columns into rows, or vice versa.
index	To be used with pandas DataFrames and Series: when reading, use it to define whether the Excel range contains the index. Can be <code>True/False</code> or an integer. The integer defines how many columns from the left should be turned into the index. For example, <code>2</code> will use the two leftmost columns as the index. When writing, you can decide if you want to write out the index by setting <code>index</code> to <code>True</code> or <code>False</code> .
header	Works the same as <code>index</code> but is applied to the column headers.

Let's have a closer look at `ndim`: by default, when you read in a single cell from Excel, you will get a scalar (e.g., a float or a string); when you read in a column or row, you will get a simple list; and finally, when you read in a two-dimensional range, you will get a nested (i.e., two-dimensional) list. This is not only consistent in itself, but it is also equivalent to how slicing works with NumPy arrays, as seen in [Chapter 4](#). The one-dimensional case is a special one: sometimes, a column may just be an edge case of what is otherwise a two-dimensional range. In this case, force a range to always arrive as a two-dimensional list by using `ndim=2`:

```
In [40]: # Horizontal range (one-dimensional)
         sheet1["A1:B1"].value

Out[40]: [1.0, 2.0]

In [41]: # Vertical range (one-dimensional)
         sheet1["A1:A2"].value

Out[41]: [1.0, 3.0]

In [42]: # Horizontal range (two-dimensional)
         sheet1["A1:B1"].options(ndim=2).value

Out[42]: [[1.0, 2.0]]

In [43]: # Vertical range (two-dimensional)
         sheet1["A1:A2"].options(ndim=2).value

Out[43]: [[1.0], [3.0]]

In [44]: # Using the NumPy array converter behaves the same:
         # vertical range leads to a one-dimensional array
         sheet1["A1:A2"].options(np.array).value

Out[44]: array([1., 3.])

In [45]: # Preserving the column orientation
         sheet1["A1:A2"].options(np.array, ndim=2).value

Out[45]: array([[1.],
                 [3.]])

In [46]: # If you need to write out a list vertically,
         # the "transpose" option comes in handy
         sheet1["D1"].options(transpose=True).value = [100, 200]
```

In the same spirit, use `ndim=1` to force the value of a single cell to be read as a list instead of a scalar. You won't need `ndim` with pandas, as a DataFrame is always two-dimensional and a Series is always one-dimensional. Here is one more example showing how the empty, date, and numbers options work:

```
In [47]: # Write out some sample data
         sheet1["A13"].value = [dt.datetime(2020, 1, 1), None, 1.0]

In [48]: # Read it back using the default options
         sheet1["A13:C13"].value
```

```
Out[48]: [datetime.datetime(2020, 1, 1, 0, 0), None, 1.0]

In [49]: # Read it back using non-default options
         sheet1["A13:C13"].options(empty="NA", dates=dt.date, numbers=int).value

Out[49]: [datetime.date(2020, 1, 1), 'NA', 1]
```

So far, we have worked with the book, sheet, and range objects. Next, you'll learn how to deal with collections such as charts or pictures!

Charts, Pictures, and Defined Names

In this section, I'll show you how to work with three collections that you access via the sheet or book object: charts, pictures, and names. These are just a few examples—xlwings supports additional collections like shapes and tables. Let's start by first creating an Excel chart!

Excel charts

To add a new chart, use the add method of the charts collection, and set the chart type and source data:

```
In [50]: sheet1["A15"].value = [
         [None, "North", "South"],
         ["Last Year", 2, 5],
         ["This Year", 3, 6],
         ]

In [51]: chart = sheet1.charts.add(
         top=sheet1["A19"].top,
         left=sheet1["A19"].left,
         )
         chart.chart_type = "column_clustered"
         chart.set_source_data(sheet1["A15"].expand())
```

This will produce the chart shown on the left-hand side of [Figure 8-3](#). To look up the available chart types, have a look at the [xlwings docs](#).

xlwings only supports the most basic chart functionality natively, but since you can work with Excel templates, you may not even be missing much. And to compensate, xlwings allows you to use Matplotlib plots as pictures—let's see how!

Pictures: Matplotlib plots

When you use pandas' default plotting backend, you are creating a Matplotlib plot. To bring such a plot over to Excel, you need to get ahold of its figure object, which you provide as an argument to `pictures.add`—this will convert the plot into a picture and insert it into your workbook:


```

In [52]: # Read in the chart data as DataFrame
revenues = sheet1["A15"].expand().options(pd.DataFrame).value
revenues

Out[52]:
           North  South
Last Year    2.0    5.0
This Year    3.0    6.0

In [53]: # The pandas plot method returns a Matplotlib "Axes" object from
# where you can get the figure. "T" transposes the
# DataFrame to bring the plot into the desired orientation
ax = revenues.T.plot.bar()
fig = ax.get_figure()

In [54]: # Send the plot to Excel
plot = sheet1.pictures.add(fig, name="SalesPlot", anchor=sheet1["I19"])
# Let's scale the plot to 80%
plot.width, plot.height = plot.width * 0.8, plot.height * 0.8

```



Use AI to Get a Figure Object

Matplotlib has various interfaces and is used by libraries such as pandas and seaborn. Sometimes, this makes it hard to know how you get access to the underlying figure object of your specific plot. Fortunately, AI is very good at this, so simply ask your favorite AI assistant something along the following lines: “How do I get the Matplotlib figure object in the following code sample?” followed by a short code snippet that generates your plot.

Figure 8-3 shows how the workbook looks at this point.

To update the picture with a new plot, use the update method with another figure object—technically, this will replace the picture in Excel but will preserve all properties like the location, size, and name:

```

In [55]: ax = (revenues + 1).T.plot.bar()
plot = plot.update(ax.get_figure())

```

You access both the charts and pictures collections via the sheet object. To work with defined names, the collection we’re looking at in the next section, you can go through either the sheet or the book object. Let’s see what difference this makes!

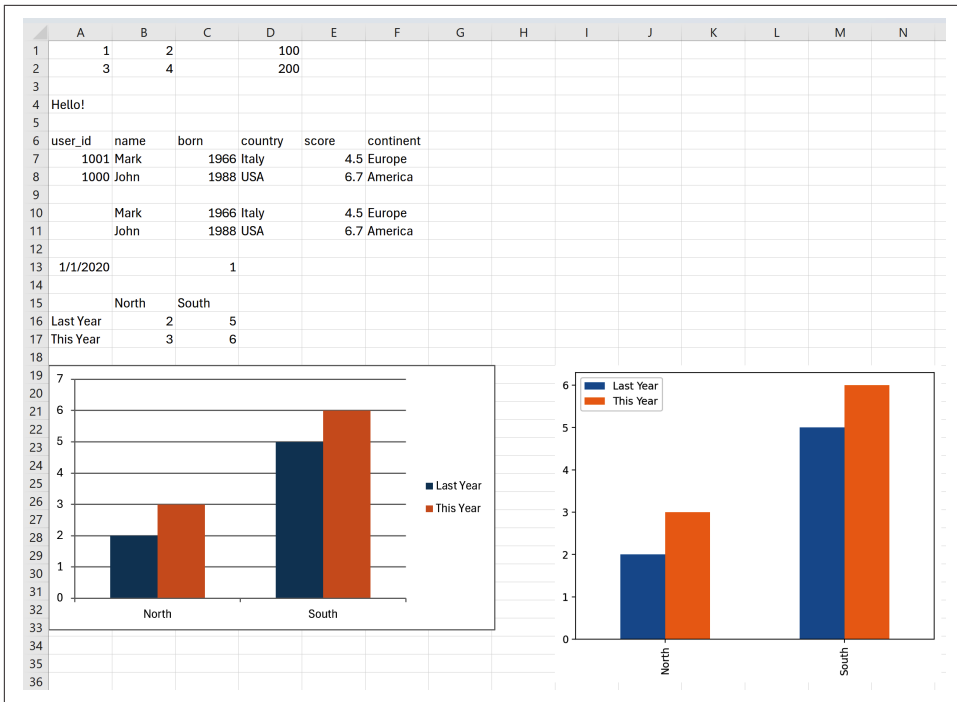


Figure 8-3. The workbook after running the first 54 cells

Defined names

In Excel, you create a *defined name* by assigning a name to a range, a formula, or a constant. Assigning a name to a range is probably the most common case and called a *named range*. With a named range, you can refer to the Excel range in formulas and code by using a descriptive name rather than an abstract address in the form of A1:B2. Using them with xlwings makes your code more flexible and solid: reading and writing values from and to named ranges gives you the flexibility to reorganize your workbook without having to adjust your Python code: a name sticks to the cell, even if you move it around by inserting a new row, for example. Defined names can be set on either book scope or sheet scope. The advantage of a name with sheet scope is that you could copy the sheet without running into conflicts with duplicate named ranges. In Excel, you add defined names manually by going to Formulas > Define Name or by selecting a range, then writing the desired name into the Name Box—this is the text box to the left of the formula bar, where you see the cell address by default. Here is how you manage defined names with xlwings:

```

In [56]: # The book scope is the default scope
         sheet1["A1:B2"].name = "matrix1"

In [57]: # For the sheet scope, prepend the sheet name with
         # an exclamation point. Make sure to replace "Sheet1"
         # with the name of your worksheet if it is called differently!
         sheet1["B10:E11"].name = "Sheet1!matrix2"

In [58]: # Now you can access the range by name
         sheet1["matrix1"]

Out[58]: <Range [Book2]Sheet1!$A$1:$B$2>

In [59]: # If you access the names collection via the "sheet1" object,
         # it contains only names with that sheet's scope
         sheet1.names

Out[59]: [<Name 'Sheet1!matrix2': =Sheet1!$B$10:$E$11>]

In [60]: # If you access the names collection via the "book" object,
         # it contains all names, including book and sheet scope
         book.names

Out[60]: [<Name 'matrix1': =Sheet1!$A$1:$B$2>, <Name 'Sheet1!matrix2':
         =Sheet1!$B$10:$E$11>]

In [61]: # Names have various methods and attributes.
         # You can, for example, get the respective range object.
         book.names["matrix1"].refers_to_range

Out[61]: <Range [Book2]Sheet1!$A$1:$B$2>

In [62]: # If you want to assign a name to a constant
         # or a formula, use the "add" method.
         # You may need to replace the decimal point with a comma
         # if you are using an international version of Excel.
         book.names.add("EURUSD", "=1.1151")

Out[62]: <Name 'EURUSD': =1.1151>

```

Have a look at the generated names in Excel by opening the Name Manager via Formulas > Name Manager (see [Figure 8-4](#)).

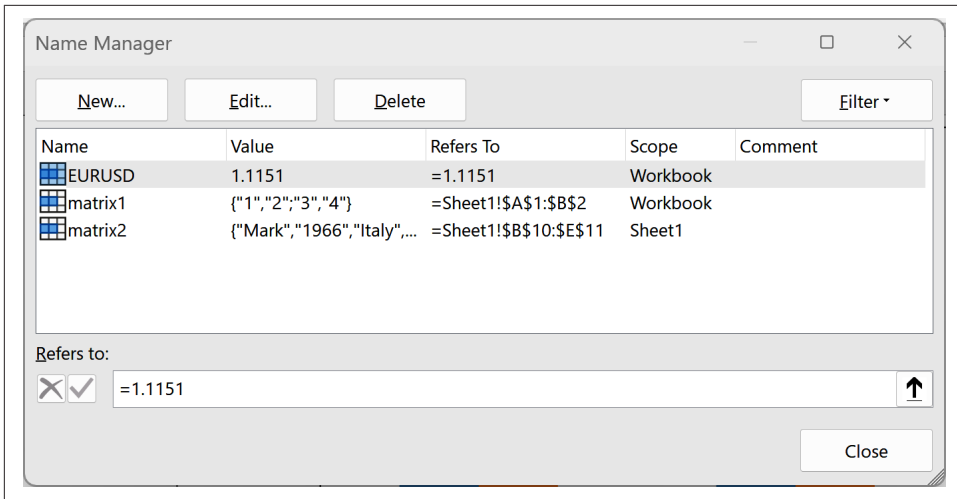


Figure 8-4. Excel's Name Manager after adding a few defined names via *xlwings*

At this point, you know how to work with the most commonly used components of an Excel workbook. This means that you are ready to create a fully automated Excel report!

Reporting Case Study (Take 1)

This case study tackles a common challenge: consolidating a directory of Excel files into a report. In the companion repository, in the *sales_data* directory, you will find Excel files with fictitious sales transactions for a telecommunication provider selling different plans (Bronze, Silver, Gold) in a few stores throughout the United States. For every month, there are two files, one in the *new* subfolder for new contracts and one in the *existing* subfolder for existing customers. As the reports are generated by different systems, they come in different formats: the new customers are delivered as *xlsx* files, while the existing customers arrive in the older *xls* format. Each file has up to 10,000 transactions, and our goal is to produce an Excel report that shows the total revenue per store and month.

We'll use *pandas* to read the Excel files from the *sales_data* directory and *xlwings* to write the report to a preformatted Excel template that you'll find under the name *sales_report_template.xlsx* in the *xl* directory of the companion repository. The `pd.read_excel` function that we already met in [Chapter 5](#) works great if you just need to read the data from an Excel file. For this job, it's faster than *xlwings*, as it reads the files directly from disk, without opening them in Excel. *xlwings*, on the other hand, allows you to edit an Excel workbook that is open in Excel and is therefore ideal to fill in templates. Being able to use a preformatted Excel template will save you a lot of work as you don't need to write any code to apply the right

font size or cell color. This, however, comes at a price: unlike `pd.read_excel`, `xlwings` requires an installation of Excel. We will come back to this case study in [Part VI](#), where we'll re-create this report without the Excel dependency. To get started, let's have a look at the *January.xlsx* file from the *new* subfolder in [Figure 8-5](#).

	A	B	C	D	E	F	G
1	transaction_id	store	status	transaction_date	plan	contract_type	amount
2	abfbdd6d	Chicago	ACTIVE	1/1/2019	Silver	NEW	14.25
3	136a9997	San Francisco	ACTIVE	1/1/2019	Gold	NEW	19.35
4	c6688f32	San Francisco	ACTIVE	1/1/2019	Bronze	NEW	12.2
5	6ef349c1	Chicago	ACTIVE	1/1/2019	Gold	NEW	19.35
6	22066f29	San Francisco	ACTIVE	1/1/2019	Silver	NEW	14.25

Figure 8-5. The first few rows of *January.xlsx*

The Excel files in the *existing* subfolder look practically the same, except that they are missing the `status` column and are stored in the legacy *xls* format. To get started, let's read the new transactions from January with pandas' `read_excel` function:

```
In [63]: jan = pd.read_excel("sales_data/new/January.xlsx")
         jan.info()

Out[63]: <class 'pandas.DataFrame'>
         RangeIndex: 9493 entries, 0 to 9492
         Data columns (total 7 columns):
         #   Column                Non-Null Count  Dtype
         ---  ---
         0   transaction_id          9493 non-null   str
         1   store                  9493 non-null   str
         2   status                 9493 non-null   str
         3   transaction_date       9493 non-null   datetime64[us]
         4   plan                   9493 non-null   str
         5   contract_type          9493 non-null   str
         6   amount                 9493 non-null   float64
         dtypes: datetime64[us](1), float64(1), str(5)
         memory usage: 816.5 KB
```

As you can see, pandas has properly recognized the data types of all columns, including the date format of `transaction_date`. This allows us to work with the data without further preparation. As this case study is deliberately simple, we can move on with creating a short script called *sales_report_xlwings.py*, as shown in [Example 8-1](#). This script will read in all Excel files from both directories, aggregate the data, and write the summary table into the Excel template. Use VS Code to write the script yourself, or open it from the root directory of the companion repository. For a refresher on how to create or open files in VS Code, have another look at [Chapter 2](#). If you create it yourself, make sure to place it next to the *sales_data* folder—this will allow you to run the script without having to adjust any file paths.

Example 8-1. sales_report_xlwings.py

```
from pathlib import Path

import pandas as pd
import xlwings as xw

# Directory of this file
this_dir = Path(__file__).resolve().parent ❶

# Read in all files
parts = []
for path in (this_dir / "sales_data").rglob("*.xls*"): ❷
    print(f"Reading {path.name}")
    part = pd.read_excel(path)
    parts.append(part)

# Combine the DataFrames from each file into a single DataFrame
df = pd.concat(parts)

# Pivot each store into a column and sum up all transactions per date
pivot = pd.pivot_table(
    df, index="transaction_date", columns="store", values="amount", aggfunc="sum"
)

# Resample to end of month and assign an index name
summary = pivot.resample("ME").sum()
summary.index.name = "Month"

# Sort columns by total revenue
summary = summary.loc[:, summary.sum().sort_values().index]

# Add row and column totals
summary["Total"] = summary.sum(axis=1)
total_row = pd.DataFrame([summary.sum(axis=0)], index=["Total"])
summary = pd.concat([summary, total_row])

#### Write summary report to Excel file ####

# Open the template, paste the data, autofit the columns, and
# adjust the chart source. Then save it under a different name.
template = xw.Book(this_dir / "xl" / "sales_report_template.xlsx")
sheet = template.sheets["Sheet1"]
sheet["B3"].value = summary
sheet["B3"].expand().columns.autofit()
sheet.charts["Chart 1"].set_source_data(sheet["B3"].expand()[:-1, :-1])
template.save(this_dir / "sales_report_xlwings.xlsx")
```

- ❶ Up until now, I was using strings to specify file paths. By using the Path class from the pathlib module instead, you get access to a powerful set of tools: path objects enable you to easily construct paths via forward slashes, as it's done four

lines below with `this_dir / "sales_data"`. These paths work across platforms and allow you to apply filters like `rglob`, as explained under the next point. `__file__` resolves to the path of the source code file when you run it—using its parent will therefore give you the name of the directory of this file. The `resolve` method turns the path into an absolute path. You would have to replace this line with `this_dir = Path(".").resolve()`, if you run this from a Jupyter notebook instead, with the dot representing the current directory. In most cases, functions and classes that accept a path in the form of a string also accept a `Path` object.

- 2 The easiest way to read in all Excel files recursively from within a certain directory is to use the `rglob` method of the `Path` object. `glob` is short for *globbing*, which refers to pathname expansion using wildcards. The `?` wildcard represents exactly one character, while `*` stands for any number of characters (including zero). The `r` in `rglob` means *recursive* globbing, i.e., it will look for matching files across all subdirectories—accordingly, `glob` would ignore subdirectories. Using `*.xls*` as the globbing expression makes sure that the old and new Excel files are found, as it matches both `.xls` and `.xlsx`. It's usually a good idea to slightly enhance the expression as follows: `[!~$]*.xls*`. This ignores temporary Excel files (their file name starts with `~$`). For more background on how to use globbing in Python, see the [Python docs](#).

Run the script, for example, by clicking the Run Python File button at the top right of VS Code or by running the following command in a Terminal:

Windows (Terminal)

```
cd %userprofile%\python-for-excel-2e
uv run sales_report_xlwings.py
```

macOS (Terminal)

```
cd ~/python-for-excel-2e
uv run sales_report_xlwings.py
```

At the end of the script, the code will open the Excel template in Excel and will save it in the same directory as the Python file under `sales_report_xlwings.xlsx`. The content of Sheet1 should look like [Figure 8-6](#). The red numbers use conditional formatting defined in the template to highlight sales below 20,000. That's quite an impressive result for only a few lines of code! When you run this script for the very first time on macOS, you will have to confirm a pop-up to grant access to the file system, something we've already come across earlier in this chapter.



xlwings Versus Power Query

Power Query is designed to transform data and load it into an Excel table. In contrast, xlwings allows for full Excel automation after the data transformation. It lets you open external templates and write data anywhere on the sheet—avoiding the rigid formatting constraints of Excel Tables. You can even go beyond that and send reports out via email—all with a single Python script.

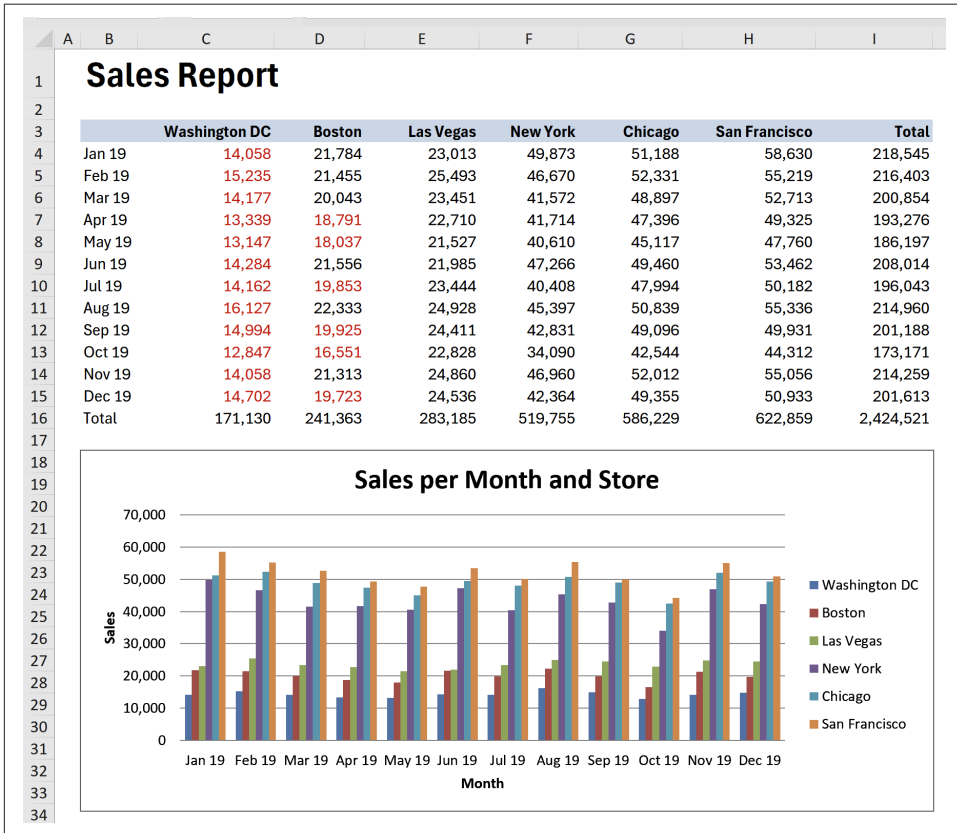


Figure 8-6. The sales report based on a preformatted template

When you start using xlwings for more than just filling in a couple of cells in a template, it's good to know what impacts performance and how you can work around missing features.

Advanced xlwings Topics

This section is about making your xlwings code performant and working around missing functionality. On the way, you'll learn what magic Jupyter notebooks commands are and how xlwings communicates with Excel.

Improving Performance

To keep your xlwings scripts performant, there is a golden rule: keep cross-application calls to an absolute minimum. Every cross-application call from Python to Excel is “expensive,” i.e., slow. Therefore, such calls should be reduced as much as possible. The easiest way to do this is by reading and writing entire Excel ranges instead of individual cells.

In the following example, we write 150 cells, first by looping through every cell and then by dealing with the entire range in one call. To measure the time, I am using a Jupyter notebook magic command (see sidebar).

Magic Commands

In the following samples, I am going to make use of the `%time` magic command. Magic commands are a set of simple commands that cause a Jupyter notebook cell to behave in a certain way or make cumbersome tasks easy. You write magic commands in cells like Python code, but they either start with `%%` or `%`. Commands that affect the whole cell start with `%%`, and commands that only affect a single line in a cell start with `%`. To list all currently available magic commands, run `%lsmagic`, and for a detailed description, run `%magic`.

`%time` is a cell magic that can be very useful for simple performance tuning as it makes it easy to compare the execution time of two cells with different code snippets. *Wall time* is the elapsed time from the start to the end of the program, i.e., the cell. *CPU times* measures the time spent on the CPU, which can be lower than the wall time (if the program has to wait for the CPU to become available) or higher (if the program is running on multiple CPU cores in parallel). To measure the time more accurately, use `%%timeit` instead of `%time`, which runs the cell multiple times and takes the average of all runs. `%time` and `%%timeit` are cell magics, i.e., they need to be on the first line of the cell and will measure the execution time of the whole cell. If, instead, you want to measure just a single line, start that line with `%time` or `%timeit`.

```
In [64]: # Add a new sheet and create a NumPy
         # array with 150 values.
         sheet2 = book.sheets.add()
         data = np.arange(150)
```

```

In [65]: %%time
          # This makes 150 cross-application calls
          for i, cell in enumerate(sheet2["A1:E30"]):
              cell.value = data[i]

Out[65]: CPU times: total: 328 ms
          Wall time: 793 ms

In [66]: %%time
          # This makes just one cross-application call
          sheet2["G1"].value = data.reshape(30, 5)

Out[66]: CPU times: total: 31.2 ms
          Wall time: 38.2 ms

```

These numbers are even more extreme on macOS, where the second option is about 45 times faster than the first one on my machine.

Now that you know how to keep performance under control, let's have a look at how to extend the functionality of xlwings.

Missing Functionality

To be able to work around missing functionality, you first need to understand how xlwings communicates with Excel:

Windows

On Windows, xlwings relies on the COM technology, short for *Component Object Model*. COM is a standard that allows two processes to communicate with each other—in our case Excel and Python. xlwings uses the Python package [pywin32](#) to handle COM calls.

macOS

On macOS, xlwings relies on *AppleScript*. AppleScript is Apple's scripting language to automate scriptable applications—fortunately, Excel is such a scriptable application. To run AppleScript commands, xlwings uses the Python package [appscript](#).

xlwings provides a user-friendly interface for the most popular Excel functionality. There are, however, many methods and attributes of the Excel object model that are not (yet) covered natively by xlwings—but all is not lost! xlwings gives you access to the underlying [pywin32](#) object on Windows and the [appscript](#) object on macOS by using the `api` attribute. This way, you have access to the whole Excel object model, but in turn, you lose cross-platform compatibility. For example, assume you wanted to clear the formatting of a cell—here is how you would go about this:

- Check if the method is available on the xlwings range object by searching the [xlwings API reference](#).

- If the method is missing, use the `api` attribute to get the underlying object: on Windows, `sheet["A1"].api` will give you a `pywin32` object, and on macOS, you will get an `appscript` object.
- Check Excel's object model in the [Excel VBA reference](#). To clear the format of a range, you would end up under `Range.ClearFormats`.
- On Windows, in most cases, you can use the VBA method or property directly with your `api` object. If it is a method, make sure to add parentheses in Python: `sheet["A1"].api.ClearFormats()`. If you are doing this on macOS, things are more complicated as `appscript` uses a syntax that can be difficult to guess. Your best approach is to look at the developer guide that is part of the [xlwings source code](#). Clearing the cell formatting, however, is easy enough: just apply Python's syntax rules on the method name by using lowercase characters with underscores: `sheet["A1"].api.clear_formats()`.

If you want `ClearFormats` to work across both platforms, you can do it as follows (darwin is the core of macOS and used as its name by `sys.platform`):

```
import sys

if sys.platform.startswith("darwin"):
    sheet["A10"].api.clear_formats()
elif sys.platform.startswith("win"):
    sheet["A10"].api.ClearFormats()
```

In any case, it's worth opening an issue on [xlwings' GitHub repository](#) to have the functionality included in a future version.

Conclusion

This chapter showed you how to automate Excel with `xlwings`, leveraging Python for tasks that you would traditionally do in VBA. You learned about the Excel object model and how `xlwings` allows you to interact with its components such as the sheet and range objects. With this knowledge, you filled in a preformatted report template.

While `xlwings` is designed to be a drop-in replacement for VBA, the real benefit is that you get access to all the advantages that I talked about in [Chapter 1](#)—most importantly, the ability to use the entire ecosystem of Python's third-party packages, such as `NumPy` and `pandas`.

Now that you know the `xlwings` basics, you are ready for the following chapter. There, we'll take the next step and call `xlwings` scripts from Excel itself, allowing you to build Python-powered Excel tools.

Python-Powered Excel Tools

In the last chapter, we learned how to write Python scripts to automate Microsoft Excel. While this is very powerful, the user must feel comfortable using either the Terminal, VS Code, or Jupyter notebooks to run the scripts. This is most likely not the case if you give your tools to business users. For them, you'll want to hide away the Python part so that the Excel tool feels like a normal macro-enabled workbook again. This chapter shows you how to accomplish that with xlwings. I'll start by showing you the shortest path to run Python code in Excel before looking at the challenges of deploying xlwings tools—this will also allow us to have a more detailed look at the configuration that xlwings offers. Like the last chapter, this chapter requires you to have an installation of Microsoft Excel on either Windows or macOS.

Using Excel as Frontend

The *frontend* is the part of an application that users see and interact with. Other common names for the frontend are *graphical user interface* (GUI) or just *user interface* (UI). When I ask xlwings users why they choose Excel over a modern web app, the answer is usually along the following lines: “Excel is the interface that our users are familiar with.” Relying on spreadsheet cells allows the users to provide inputs quickly and intuitively, making them often more productive than if they had to use a half-baked web interface. I'll start this section by introducing you to the xlwings add-in and the xlwings CLI (*command line interface*) before creating our first project via the `quickstart` command. I'll wrap this section up by showing you two ways of calling Python code from Excel: by clicking the Run main button in the add-in and by using the `RunPython` function in VBA. Let's get started by installing the xlwings add-in!

xlwings Add-in

If you want to call Python scripts from Excel, you need to either install the Excel add-in or set the workbook up in the standalone mode. I will introduce the standalone mode under “[Deployment](#)” on [page 212](#), so this section focuses on the add-in. To install it, run the following in a Terminal:

Windows (Terminal)

```
cd %userprofile%\python-for-excel-2e
uv run xlwings addin install
```

macOS (Terminal)

```
cd ~/python-for-excel-2e
uv run xlwings addin install
```



Reconfigure an Existing xlwings Add-in

If you already have the xlwings add-in present in Excel from a previous installation, additionally run the following command to point the add-in to the Python environment we created in [Chapter 2](#):

```
uv run xlwings config create --force
```

Update xlwings

You will need to keep the version of the Python package and the version of the add-in in sync whenever you update xlwings. Therefore, you should always run two commands when you update xlwings—one for the Python package and one for the Excel add-in:

Windows (Terminal)

```
cd %userprofile%\python-for-excel-2e
uv sync --upgrade
uv run xlwings addin install
```

macOS (Terminal)

```
cd ~/python-for-excel-2e
uv sync --upgrade
uv run xlwings addin install
```

If you want to only upgrade xlwings, use `uv sync --upgrade-package xlwings` instead of `uv sync --upgrade`, which upgrades all packages in your virtual environment.

When you type `uv run xlwings` in a Terminal, you are using the xlwings CLI. Apart from making the installation of the xlwings add-in easy, it offers a few more commands that I will introduce whenever we need them.

The actual installation of the add-in is done by copying *xlwings.xlam* from the directory of the Python package into Excel's *XLSTART* folder, which is a special folder: Excel will open all files that are in this folder whenever you start Excel. Running the following command will print the *XLSTART* directory and tell you whether or not the add-in is installed:

Windows (Terminal)

```
cd %userprofile%\python-for-excel-2e
uv run xlwings addin status
```

macOS (Terminal)

```
cd ~/python-for-excel-2e
uv run xlwings addin status
```



Antivirus Software

Unfortunately, the xlwings add-in is sometimes flagged as a malicious add-in by antivirus software, especially if you're using a brand-new release. If this happens on your machine, go to the settings of your antivirus software, where you should be able to mark xlwings as safe to run. Usually, it's also possible to report such false positives via the software's home page.

When you install the add-in for the very first time, it will also configure it to use the Python interpreter from where you are running the `install` command: as you see in [Figure 9-1](#), the value for Interpreter is filled in automatically by the xlwings CLI. These values are stored in a file called *xlwings.conf* in the *.xlwings* folder within your user directory, which is usually `C:\Users\<username>\.xlwings\xlwings.conf` (Windows) or `~/Library/Containers/com.microsoft.Excel/Data/xlwings.conf` (macOS). On macOS, folders and files with a leading dot are hidden by default, so use the keyboard shortcut Command-Shift-. to toggle their visibility in the Finder.

After running the installation command, restart Excel to see the xlwings tab in the ribbon, as shown in [Figure 9-1](#).

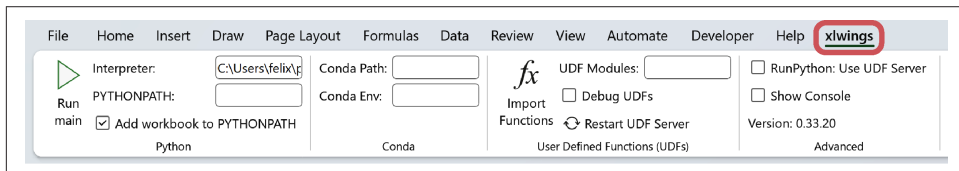


Figure 9-1. The xlwings ribbon add-in



The Ribbon Add-In on macOS

On macOS, the ribbon looks a bit different as it's missing the sections about custom functions (“UDFs”) and Conda: while custom functions are not supported on macOS, Conda environments don't require special treatment, i.e., are configured as Interpreter under the Python group.

Now that you have the xlwings add-in installed, we'll need a workbook and some Python code to test it out. The fastest way of getting there is by using the `quickstart` command.

Quickstart Command

To make the creation of your first xlwings tool as easy as possible, the xlwings CLI offers the `quickstart` command. In a Terminal, run the following to create a project with the name `project1`:

Windows (Terminal)

```
cd %userprofile%\python-for-excel-2e
uv run xlwings quickstart project1
```

macOS (Terminal)

```
cd ~/python-for-excel-2e
uv run xlwings quickstart project1
```

The project name has to be a valid Python module name: it can contain characters, numbers, and underscores, but no spaces or dashes, and it must not start with a number. I will show you under [“RunPython Function” on page 208](#) how you can change the name of the Excel file later into something that doesn't have to follow these rules. Running the `quickstart` command will create a folder called `project1` in the directory of the companion repository—once created, you can move the folder anywhere you like. When you open `project1` in the File Explorer (Windows) or in the Finder (macOS), you will see two files: `project1.xlsm` and `project1.py`. Open both files—the Excel file in Excel and the Python file in VS Code. The easiest way to run the Python code from Excel is by using the Run main button in the add-in—let's see how it works!

Run Main

Before looking at `project1.py` in more detail, go ahead and click the Run main button on the very left of the xlwings add-in while `project1.xlsm` is your active file; it will write “Hello xlwings!” into cell A1 of the first sheet. Click the button again and it will change to “Bye xlwings!” Congratulations, you have just run your first Python

function from Excel! After all, that wasn't much harder than writing a VBA macro, was it? Let's now have a look at *project1.py* in [Example 9-1](#).

Example 9-1. project1.py

```
import xlwings as xw
from xlwings import func

def main():
    wb = xw.Book.caller() ❶
    sheet = wb.sheets[0]
    if sheet["A1"].value == "Hello xlwings!":
        sheet["A1"].value = "Bye xlwings!"
    else:
        sheet["A1"].value = "Hello xlwings!"

@func ❷
def hello(name):
    return f"Hello {name}!"

if __name__ == "__main__": ❸
    xw.Book("project1.xlsm").set_mock_caller()
    main()
```

- ❶ `xw.Book.caller()` is an `xlwings` `Book` object that refers to the Excel workbook that is active when you click the Run main button. In our case, it corresponds to `xw.Book("project1.xlsm")`. Using `xw.Book.caller()` allows you to rename and move your Excel file around on the file system without breaking the reference. It also makes sure that you are manipulating the correct workbook if you have it open in multiple Excel instances.
- ❷ In this chapter, we will ignore the function `hello` as this will be the topic of [Chapter 11](#). If you run the `quickstart` command on macOS, you won't see the `hello` function anyway, as custom functions are only supported on Windows.
- ❸ I will explain the last three lines when we look at debugging in the next chapter. For the purpose of this chapter, you can ignore or even delete everything below the `main` function.

The Run main button in the Excel add-in is a convenience feature: it allows you to call a function with the name `main` in a Python module that has the same name as the Excel file without having to add a button first to your workbook. It will even work if you save your workbook in the macro-free *xlsx* format. If, however, you need more flexibility, you have to use the `RunPython` function instead. The next section has the details!

RunPython Function

If you need more control over how you call your Python code, use the VBA function RunPython. Consequently, RunPython requires your workbook to be saved as a macro-enabled workbook. The first part of this section explains how the RunPython command works, while the second part covers what to do to make RunPython work with a workbook that wasn't created via the quickstart command.



Enable Macros

Make sure to properly enable macros when opening a macro-enabled workbook (*xlsm* extension) such as the one that is generated by the quickstart command:

- Click on Enable Content.
- When working with *xlsm* files from the internet, you have to additionally click on Enable Editing.
- Before opening the workbook, though, you might need to unblock the file first by right-clicking on it in the Windows File Explorer and selecting Properties. On the General tab, check the box “Unblock,” then click OK to exit the properties window (macOS doesn't require unblocking a file).

Getting started with RunPython

RunPython accepts a string with Python code: most commonly, you import a Python module and run one of its functions. When you open the VBA editor via Alt+F11 (Windows) or Option-F11 (macOS), you will see that the quickstart command adds a macro called SampleCall in a VBA module with the name “Module1” (see [Figure 9-2](#)). If you don't see the SampleCall, double-click Module1 in the VBA project tree on the left-hand side.

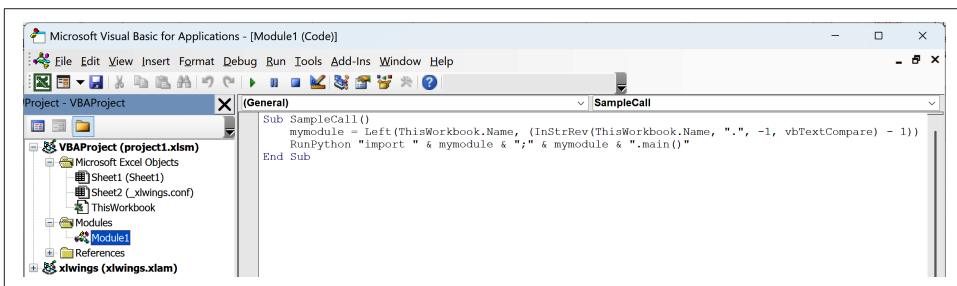


Figure 9-2. The VBA editor showing Module1

The code looks a bit convoluted, but this is only to make it work dynamically no matter what project name you choose when running the quickstart command. As

our Python module is called `project1`, you could replace the code with the following easy-to-understand equivalent:

```
Sub SampleCall()  
    RunPython "import project1; project1.main()"  
End Sub
```

Since it's no fun to write multiline strings in VBA, we use a semicolon that Python accepts instead of a line break. There are a couple of ways you can run this code: for example, while you are in the VBA editor, place your cursor on any line of the `SampleCall` macro and hit F5. Usually, however, you will be running the code from an Excel sheet and not from the VBA editor. Therefore, close the VBA editor and switch back to the workbook. Typing Alt+F8 (Windows) or Option-F8 (macOS) will bring up the macro menu: select `SampleCall` and click on the Run button. Or, to make it more user-friendly, add a button to your Excel workbook and connect it with the `SampleCall`: first, make sure that the Developer tab in the ribbon is shown. If it isn't, go to File > Options > Customize Ribbon and activate the checkbox next to Developer (on macOS, you'll find it under Excel > Preferences > Ribbon & Toolbar instead). To insert a button, go to the Developer tab and in the Controls group, click on Insert > Button (under Form Controls). On macOS, you'll be presented with the button without having to go to Insert first. When you click the button icon, your cursor turns into a small cross: use it to draw a button on your sheet by holding your left mouse button down while drawing a rectangular form. Once you let go of your mouse button, you'll be presented with the Assign Macro menu—select `SampleCall` and click OK. Click the button that you've just created (in my case it's called "Button 1"), and it will run our `main` function again, as in [Figure 9-3](#).

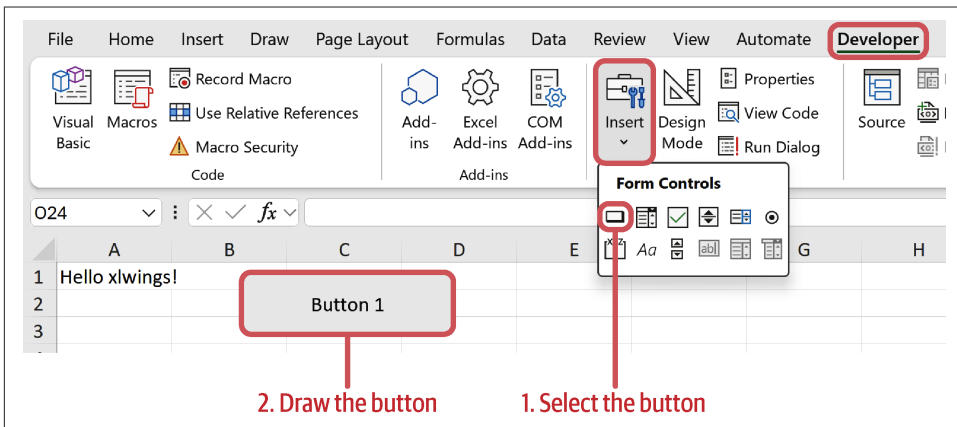


Figure 9-3. Drawing a button on a sheet

If you think that the button looks a bit old-school, don't worry: I'll show you a more modern way to create buttons in the next chapter. Now let's take a look at how we can

change the default names that were assigned by the quickstart command: go back to your Python file and rename it from *project1.py* to *hello.py*. Also, rename your main function to `hello_world`. Make sure to save the file, then open the VBA editor again via Alt+F11 (Windows) or Option-F11 (macOS) and edit `SampleCall` as follows to reflect the changes:

```
Sub SampleCall()
    RunPython "import hello; hello.hello_world()"
End Sub
```

Back on the sheet, click the “Button 1” to verify that everything still works. Finally, you may also want to keep the Python script and the Excel file in two different directories. To understand the implications of this, I’ll first need to say a word about Python’s *module search path*: if you import a module in your code, Python searches for it in various directories. In simplified terms, Python checks if there is a built-in module with this name, and if it doesn’t find one, moves on to look in the current working directory and in the directories provided by the PYTHONPATH environment variable. xlwings automatically adds the directory of the workbook to the PYTHONPATH (unless you deselect the Add workbook to PYTHONPATH checkbox) and allows you to add additional paths via the add-in. To try this out, create a directory called *xlwings_scripts* that you create in your user directory. Then move the Python script, which is now called *hello.py*, into the *xlwings_scripts* folder: the full path would be something like `C:\Users\<username>\xlwings_scripts` on Windows or `/Users/<username>/xlwings_scripts` on macOS. When you click the button again, you will get the following error in a pop-up:

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'hello'
```

To fix this, simply add the path of the *xlwings_scripts* directory to the PYTHONPATH setting in your xlwings ribbon, as in [Figure 9-4](#). When you now click the button one more time, it will work again.

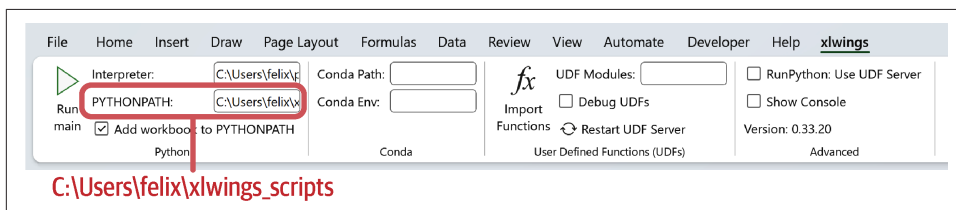


Figure 9-4. The PYTHONPATH setting

What I haven’t touched on yet is how to change the name of the Excel workbook: once your RunPython function call uses an explicit module name like `project1`

instead of the code that was added by quickstart, you are free to rename your Excel workbook anything you want.

Relying on the quickstart command is the easiest way if you start a new xlwings project. If you have an existing workbook, however, you may prefer to set it up manually. Let's see how it's done!

RunPython without quickstart command

If you want to use the RunPython function with an existing workbook that wasn't created by the quickstart command, you'll need to configure it manually. Note that the following steps are only required for the RunPython call but not when you want to use the Run main button:

1. First of all, make sure to save your workbook as a macro-enabled workbook with either the *xlsm* or *xlsb* extension.
2. Add a VBA module; to do so, open the VBA editor via Alt+F11 (Windows) or Option-F11 (macOS) and select the VBAProject of your workbook in the tree view on the left-hand side, then right-click on it and choose Insert > Module, as in [Figure 9-5](#). This will insert an empty VBA module where you can write your VBA macro with the RunPython call.

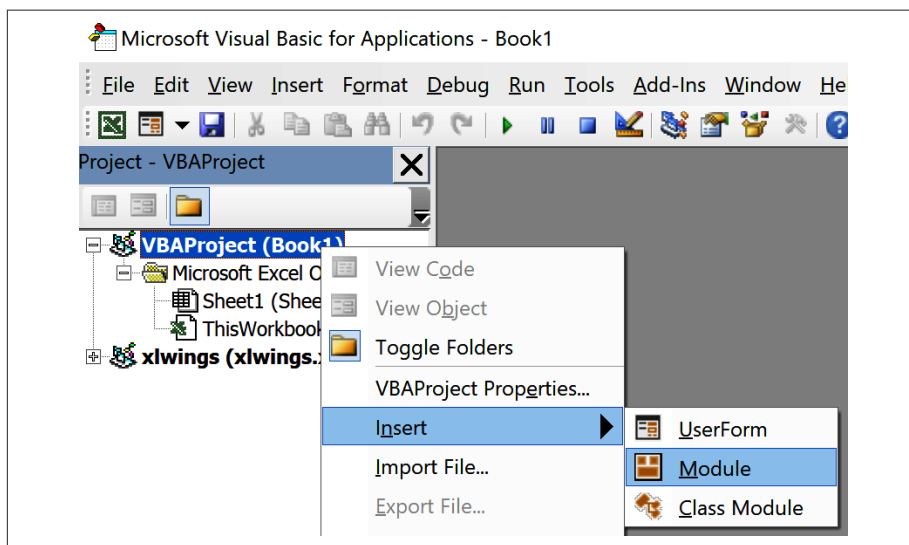


Figure 9-5. Adding a VBA module

3. Add a reference to xlwings: RunPython is a function that is part of the xlwings add-in. To use it, you need to set a reference to xlwings in your VBAProject. Again, start by selecting the correct workbook in the tree view on the left-hand

side of the VBA editor, then go to Tools > References and activate the checkbox for xlwings, as seen in [Figure 9-6](#).

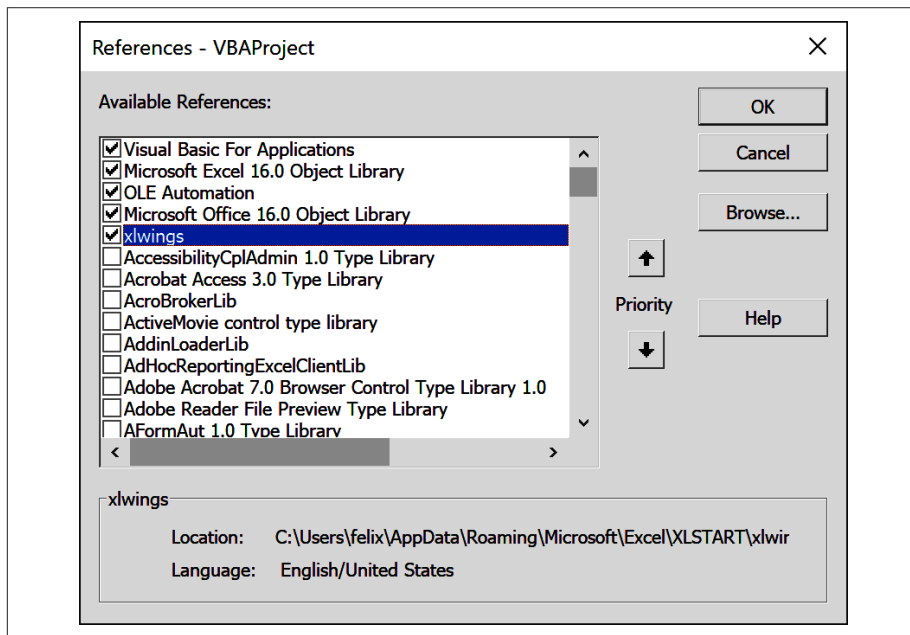


Figure 9-6. RunPython requires a reference to xlwings

Your workbook is now ready to be used with the RunPython call again. Once everything works on your machine, you may want to share it with your colleagues—let's go through a couple of options that make this part easier!

Deployment

In software development, *deployment* refers to distributing and installing software so that end users are able to use it. To deploy xlwings tools, you need to also deploy Python, which is the starting point of this section. We'll then learn how you can get rid of the xlwings add-in by setting your workbooks up in the standalone mode. Finally, we'll have a closer look at how configuration works with xlwings.

Python Dependency

To be able to run xlwings tools, your end users must have an installation of Python. Here are a couple of options to make this part easier:

Anaconda

Instruct your users to download and install a specific version of the [Anaconda distribution](#) as it comes with xlwings preinstalled. This is a good option if you only use packages that are part of Anaconda. Note, however, that Anaconda isn't free for larger organizations.

Shared drive

If you have access to a reasonably fast shared drive, you may be able to install Python directly on there, which will allow everyone to use the tools without a local Python installation.

Frozen executable

On Windows, xlwings allows you to work with *frozen executables*, which are files with the `.exe` extension that contain Python and all the dependencies. A popular package to produce frozen executables is [PyInstaller](#). Frozen executables have the advantage that they are only packaging up what your program needs and can produce a single file, which can make distribution easier. For more details on how to work with frozen executables, have a look at the [xlwings docs](#). Note that frozen executables will not work with custom functions, the functionality that I will introduce in [Chapter 11](#).

While Python is a hard requirement, the installation of the xlwings add-in is not, as I will explain next.

Standalone Workbooks

In this chapter, we have always relied on the xlwings add-in to call Python code either by clicking the Run main button or by using the RunPython function. Even if the xlwings CLI makes it easy to install the add-in, it may still be a hassle for less technical users who don't feel comfortable using a Terminal. Also, since the xlwings add-in and the xlwings Python package need to have the same version, you may run into a conflict where your recipients already have the xlwings add-in installed, but with a different version than your tool requires. There is a simple solution, though: xlwings doesn't require the Excel add-in and can be set up as a *standalone workbook* instead. In this case, the VBA code of the add-in is stored directly in your workbook. As usual, the easiest way to get everything set up is by using the `quickstart` command, this time with the `--standalone` flag:

Windows (Terminal)

```
cd %userprofile%\python-for-excel-2e
uv run xlwings quickstart project2 --standalone
```

macOS (Terminal)

```
cd ~/python-for-excel-2e
uv run xlwings quickstart project2 --standalone
```

When you open the generated *project2.xlsm* workbook in Excel and press Alt+F11 (Windows) or Option-F11 (macOS), you will see the *xlwings* module and the *Dictionary* class module that are used instead of the add-in. Most importantly, a standalone project must not have a reference to *xlwings* anymore. While this is configured automatically when using the `--standalone` flag, it is important that you remove the reference in case you want to convert an existing workbook: in your VBA editor, go to Tools > References and clear the checkbox for *xlwings*.



Building a Custom Add-In

This section showed you how to get rid of the *xlwings* add-in. Sometimes, however, you may want to build your own add-in instead. This makes sense if you want to use the same macros and custom functions with many different workbooks. You will find instructions on how to build custom add-ins in the [xlwings docs](#).

Having touched upon Python and standalone workbooks, let's now have a more in-depth look at how the *xlwings* configuration works.

xlwings Configuration

As mentioned at the beginning of this chapter, the ribbon stores its configuration in the *xlwings.conf* file. The *configuration* consists of individual *settings*, like the `PYTHONPATH` that we already saw at the beginning of this chapter. The add-in settings can be overridden on the directory and workbook level—*xlwings* looks for settings in the following locations and order:

Workbook configuration

First, *xlwings* looks for a sheet called *xlwings.conf*. This is the recommended way to configure your workbook for deployment as you don't have to handle an additional config file. When you run the `quickstart` command, it will create a sample configuration on a sheet called “`_xlwings.conf`”: remove the leading underscore in the name to activate it. If you don't want to use it, feel free to delete the sheet.

Directory configuration

Next, *xlwings* looks for a file called *xlwings.conf* in the same directory as your Excel workbook.

User configuration (add-in)

Finally, *xlwings* looks for a file called *xlwings.conf* in the *.xlwings* folder in the user directory, e.g., under `C:\Users\<username>\.xlwings\xlwings.conf`. Normally, you don't edit this file directly—instead, it is created and edited by the add-in

whenever you change a setting. If you want to edit the file directly, look up the exact format and available settings by going to the [xlwings docs](#).

For deployment, here are the most critical settings:

Python interpreter

xlwings relies on a locally installed Python installation. This, however, doesn't necessarily mean that the recipients of your xlwings tool need to mess around with the configuration before they can use the tool. As mentioned earlier in this section, you could tell them to install the Anaconda distribution with the default settings, which will install it in the user directory. If you use *environment variables* in your configuration, xlwings will find the correct path to the Python interpreter. An environment variable is a variable set on the user's computer that allows programs to query information specific to this environment, like the name of the user directory. As an example, on Windows, set the CONDA_PATH to %USERPROFILE%\anaconda3 and on macOS, set INTERPRETER_MAC to \$HOME/opt/anaconda3/bin/python. These paths will then dynamically resolve to Anaconda's default installation path.

PYTHONPATH

By default, xlwings looks for the Python source file in the same directory as the Excel file. This may not be ideal when you give your tool to users who aren't familiar with Python as they could forget to keep the two files together when moving the Excel file around. Instead, you can put your Python source files in a dedicated folder (this could be on a shared drive) and add this folder to the PYTHONPATH setting (we did this with the *xlwings_scripts* folder in the last section). Alternatively, you could distribute your source code as a Python package—installing it will place it in Python's *site-packages* directory, where Python will find your code. For more information on how to build a Python package, see the [Python Packaging User Guide](#).

RunPython: Use UDF Server (Windows only)

You may have noticed that a RunPython call can be rather slow. This is because xlwings starts a Python interpreter, runs the Python code, and finally shuts the interpreter down again. This may not be so bad during development, as it makes sure that all modules are loaded from scratch with all changes every time you call the RunPython command. Once your code is stable, though, you might want to activate the checkbox “RunPython: Use UDF Server” that is only available on Windows. This will use the same Python server as the custom functions use (the topic of [Chapter 11](#)) and keep the Python session running in between calls, which will be much faster. Note, however, that you need to click the Restart UDF Server button in the ribbon after code changes.

Conclusion

This chapter showed you how easy it is to run Python code from Excel: you only need to run `uv run xlwings addin install` followed by `uv run xlwings quickstart myproject`, and you are ready to click the Run main button in the xlwings add-in or use the RunPython VBA function.

The beauty of using Excel as a frontend and Python as a backend is that users get access to all the possibilities Python offers while feeling right at home thanks to the familiar user interface they know and love.

In this chapter, we were merely using the Hello World example to learn how everything works. The next chapter takes these foundations to build the Python Package Tracker, a full-fledged business application.

The Python Package Tracker

In this chapter, we will create a typical business application that downloads data from the internet and stores it in a database before visualizing it in Excel. This will help you understand what role *xlwings* plays in such an application and allows you to see how easy it is to connect to external systems with Python. In an attempt to build a project that is close to a real-world application yet relatively simple to follow, I have come up with the *Python Package Tracker*, an Excel tool that shows the number of releases per year for a given Python package. Despite being a case study, you might actually find the tool useful to understand if a certain Python package is being actively developed or not.

After getting acquainted with the application, we'll look at a few topics that we need to understand to be able to follow its code: we'll see how we can download data from the internet and how we can interact with databases before we learn about exception handling in Python, an important concept when it comes to application development. Once we're done with these preliminaries, we'll go through the components of the Python Package Tracker to see how everything fits together. To wrap this chapter up, we'll look into how debugging *xlwings* code works. Like the last two chapters, this chapter requires you to have an installation of Microsoft Excel on either Windows or macOS. Let's get started by taking the Python Package Tracker for a test drive!

Taking It for a Spin

To understand how the Package Tracker works from a user's perspective, there's no better way than taking it for a spin. Head over to the companion repository, where you will find the *packagetracker* folder. There are a couple of files in that folder, but for now just open the Excel file *packagetracker.xlsm*.



Enable Macros

Make sure to properly enable macros when opening a macro-enabled workbook (*xls*m extension), such as *packagetracker.xls*m:

- Click on Enable Content.
- When working with *xls*m files from the internet, you have to additionally click on Enable Editing.
- Before opening the workbook, though, you might need to unblock the file first by right-clicking on it in the Windows File Explorer and selecting Properties. On the General tab, check the box “Unblock,” then click OK to exit the properties window (macOS doesn’t require unblocking a file).

Now, head over to the Database sheet: we first need to get some data into the database to have something to work with. As shown in **Figure 10-1**, type in a package name, for example **xlwings**, then click on Add Package. You can choose any package name that exists on the **Python Package Index** (PyPI).



macOS: Confirm Access to Folder

When you add the very first package on macOS, you will have to confirm a pop-up (“Microsoft Excel.app would like to access data from other apps”) so that the application can access the *package-tracker* folder.

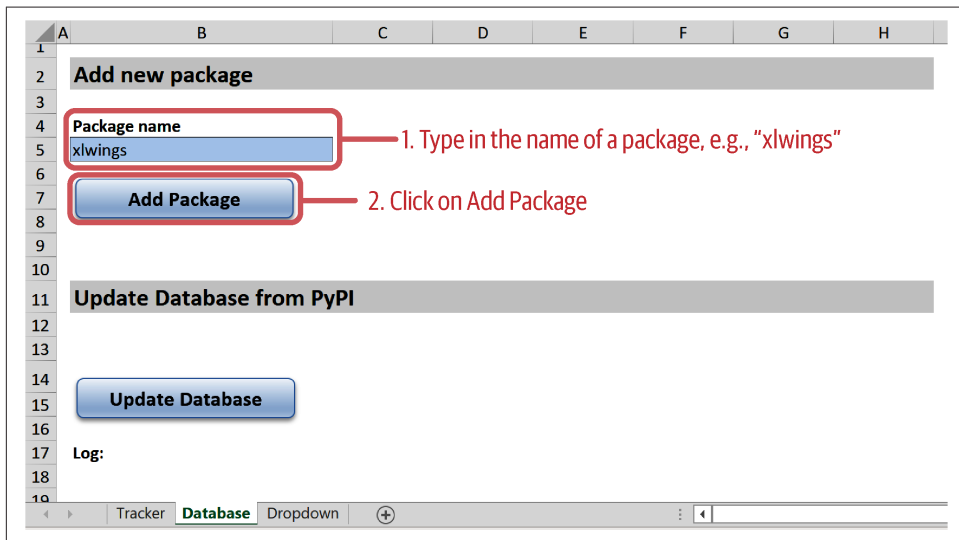


Figure 10-1. The Python Package Tracker (database sheet)

If everything works according to plan, you will see the message “Added xlwings successfully” to the right of where you typed in the package name. Also, you will see a Last updated timestamp under the Update Database section as well as an updated Log section where it says that it downloaded xlwings successfully and stored it to the database. Let’s do this one more time and add the pandas package so we have some more data to play around with. Now, switch to the Tracker sheet and select xlwings from the dropdown in cell B5 before clicking on Show History. Your screen should now look similar to **Figure 10-2**, showing the latest release of the package as well as a chart with the number of releases over the years.

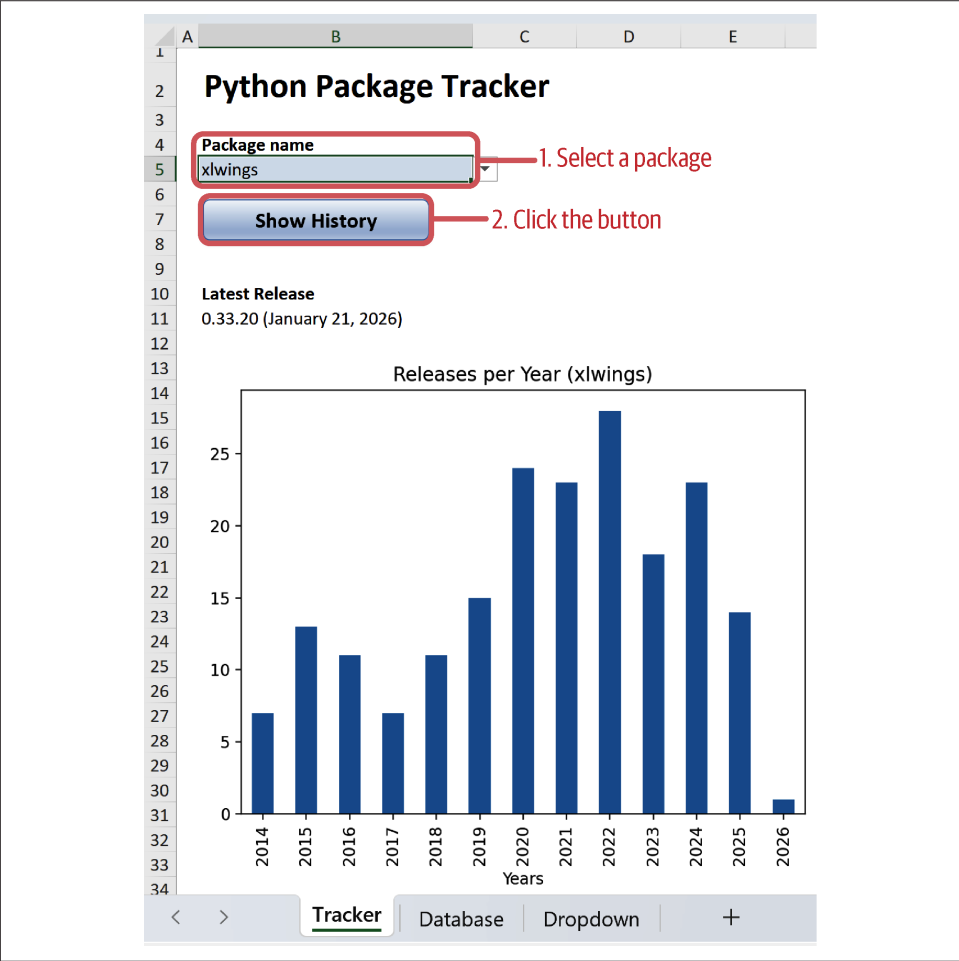


Figure 10-2. The Python Package Tracker (tracker sheet)

Feel free to go back to the Database sheet and add additional packages. Whenever you want to update the database, click on the Update Database button: this will synchronize your database with the latest data from PyPI.

Now that you've seen the Python Package Tracker in action, let's go through a few fundamental concepts that the Package Tracker relies upon.

Core Functionality

In this section, I will introduce you to the core functionality of the Python Package Tracker: how to fetch data via web APIs and how to query databases. I'll also show you how to handle exceptions, a topic that will inevitably arise when writing application code. Let's get started with web APIs!

Web APIs

Web APIs, sometimes called REST APIs, are one of the most popular ways for an application to fetch data from the internet: API stands for *application programming interface* and defines how you interact with an application programmatically. A web API, therefore, is an API that is accessed over a network, usually the internet. To understand how web APIs work, let's take a step back and see what happens (in simplified terms) when you open a web page in your browser: after entering a URL into the address bar, your browser sends a *GET request* to the server, asking for the web page you want. A GET request is a method of the HTTP protocol that your browser uses to communicate with the server. When the server receives the request, it responds by sending back the requested HTML document, which your browser displays: *voilà*, your web page has loaded. The HTTP protocol has various other methods; the most common one—apart from the GET request—is the *POST request*, which is used to send data to the server (for example, when you fill in a contact form on a web page).

While it makes sense for servers to send back a nicely styled HTML page to interact with humans, applications don't care about design and are only interested in the data. Therefore, a GET request to a web API works like requesting a web page, but instead of an HTML page, you usually get a *JSON* document. JSON stands for *JavaScript Object Notation* and is a text format for exchanging data. It is human-readable and supported by virtually every programming language, making it ideal for connecting different systems. Although the notation is based on JavaScript syntax, it's very close to how you use (nested) dictionaries and lists in Python. The differences are the following:

- JSON only accepts double quotes for strings.
- JSON uses `null` where Python uses `None`.

- JSON uses lowercase `true` and `false`, while they are capitalized in Python.
- JSON only accepts strings as keys, whereas Python dictionaries accept a wide range of objects as keys.

The `json` module from the standard library allows you to convert a Python data structure to a JSON string, and vice versa:

```
In [1]: import json

In [2]: # A Python dictionary...
user_dict = {
    "name": "Jane Doe",
    "age": 23,
    "married": False,
    "company": None,
    "hobbies": ["hiking", "reading"]
}

In [3]: # ...converted to a JSON string via json.dumps ("dump string").
# "indent" is optional and prettifies the output when printing.
user_json = json.dumps(user_dict, indent=4)
print(user_json)

Out[3]: {
    "name": "Jane Doe",
    "age": 23,
    "married": false,
    "company": null,
    "hobbies": [
        "hiking",
        "reading"
    ]
}

In [4]: # Convert the JSON string back to a native Python data structure.
# "loads" stands for load string.
json.loads(user_json)

Out[4]: {'name': 'Jane Doe',
'age': 23,
'married': False,
'company': None,
'hobbies': ['hiking', 'reading']}
```

Consuming web APIs is usually simple, and almost every service offers one. For example, you can download your Spotify playlist or get information about your latest Uber trips by querying their web APIs. To use these APIs, though, you need to be authenticated, which usually means that you require an account and a token that you can send along with your requests. For the Python Package Tracker, we'll need to fetch data from PyPI to get information about the releases of a specific package. Fortunately, PyPI's web API doesn't require any authentication, so we have one less thing to worry about. When you look at the [PyPI JSON API docs](#), you will see that

there are only two *endpoints*, i.e., URL fragments that are appended to the common *base URL*, <https://pypi.org/pypi:>

```
GET /project/json
GET /project/version/json
```

The second endpoint gives you the same information as the first one, but for a specific version only. For the Python Package Tracker, the first endpoint is all we need to get the details about the past releases of a package, so let's see how this works. In Python, the Requests package offers a simple way to interact with a web API. Run the following commands to fetch PyPI data about pandas:

```
In [5]: import requests

In [6]: response = requests.get("https://pypi.org/pypi/pandas/json")
        response.status_code

Out[6]: 200
```



HTTPX

HTTPX is a modern alternative to Requests. It is largely compatible with the Requests API but additionally supports asynchronous code. Since we're sticking mostly to synchronous programming in this book, we'll use Requests in this chapter.

Every response comes with an HTTP status code: for example, 200 means *OK* and 404 means *Not Found*. You can look up the full list of HTTP response status codes in the [Mozilla web docs](#). You may be familiar with the status code 404 as your browser sometimes displays it when you click on a dead link, i.e., ask for a page that doesn't exist. Similarly, you will also get a 404 status code if you run a GET request with a package name that doesn't exist on PyPI. To look at the content of the response, it's easiest to call the `json` method of the response object, which will transform the JSON string into a native Python data structure:

```
In [7]: response.json()
```

The response is very long, so I am printing a short subset here to allow you to understand the structure:

```
Out[7]: {
    'info': {
        'name': 'pandas',
        ...
    },
    'releases': {
        '0.1': [
            {
                'filename': 'pandas-0.1.tar.gz',
                'upload_time': '2009-12-25T23:58:31',
```



```

        },
        {
            'filename': 'pandas-0.1.win32-py2.5.exe',
            'upload_time': '2009-12-26T17:14:35',
            ...
        },
        ...
    ],
    ...
},
...
}

```

To get a list with all releases and their dates, something we need for the Python Package Tracker, run the following code to loop through the releases dictionary:

```

In [8]: releases = []
        for version, files in response.json()["releases"].items():
            releases.append(f"{version}: {files[0]['upload_time']}")
        releases[:3] # Show the first 3 elements of the list

Out[8]: ['0.1: 2009-12-25T23:58:31',
         '0.10.0: 2012-12-17T16:52:06',
         '0.10.1: 2013-01-22T05:22:09']

```

Note that we are arbitrarily picking the release timestamp from the package that appears first in the list by doing `files[0]`. A specific release often has multiple packages to account for different versions of Python and operating systems. To wrap up this topic, you may remember from [Chapter 5](#) that pandas has a `read_json` method to return a DataFrame directly from a JSON string. This, however, wouldn't help us here as the response from PyPI isn't in a structure that can be directly transformed into a DataFrame.

This was a short introduction to web APIs to understand their use in the code base of the Python Package Tracker. Let's now see how we can communicate with databases, the other external system that we make use of in our application.

Databases

To be able to use the data from PyPI even when you're offline, you need to store it after downloading. While you could store your JSON responses as text files on disk, a far more comfortable solution is to use a database: this allows you to query your data in an easy way. The Python Package Tracker is using [SQLite](#), a *relational database*. Relational database systems get their name from *relation*, which refers to the database table itself (and not to the relation between tables, which is a common misconception): their highest goal is data integrity, which they achieve by splitting the data up into different tables (a process called *normalization*) and by applying constraints to avoid inconsistent and redundant data. Relational databases use SQL

(Structured Query Language) to perform database queries, and among the most popular server-based relational database systems are SQL Server, Oracle Database, PostgreSQL, and MySQL. As an Excel user, you may also be familiar with Microsoft Access. SQLite is similar to Access in that it's a file-based database, but unlike Access, SQLite works on all platforms where Python runs. On the other hand, SQLite doesn't allow you to build a user interface like Microsoft Access does, but Excel comes in handy for this part.

Let's now have a look at the structure of the Package Tracker's database before finding out how we can use Python to connect to databases and make SQL queries. Then, to conclude this introduction about databases, we'll have a look at SQL injections, a popular vulnerability of database-driven applications.

The Package Tracker database

The database of the Python Package Tracker couldn't be any simpler. It only has two tables—the table `packages` stores the package name and the table `package_versions` stores the version strings and the date of the upload. The two tables can be joined on the `package_id`: rather than storing the `package_name` with every row in the `package_versions` table, it has been *normalized* into the `packages` table. This gets rid of redundant data—name changes, for example, only have to be done in a single field across the entire database. To get a better idea about how the database looks with the `xlwings` and `pandas` packages loaded, have a look at Tables 10-1 and 10-2.

Table 10-1. The `packages` table

package_id	package_name
1	xlwings
2	pandas

Table 10-2. The `package_versions` table (first three rows of each `package_id`)

package_id	version	uploaded_at
1	0.1.0	2014-03-19 18:18:49.000000
1	0.1.1	2014-06-27 16:26:36.000000
1	0.2.0	2014-07-29 17:14:22.000000
...	...	...
2	0.1	2009-12-25 23:58:31.000000
2	0.2	2010-05-18 15:14:26.000000
2	0.3.0	2011-02-20 02:00:14.000000
...	...	...

Figure 10-3 is a database diagram that shows the two tables again schematically. You can read off the table and column names and get information about the primary and foreign keys.

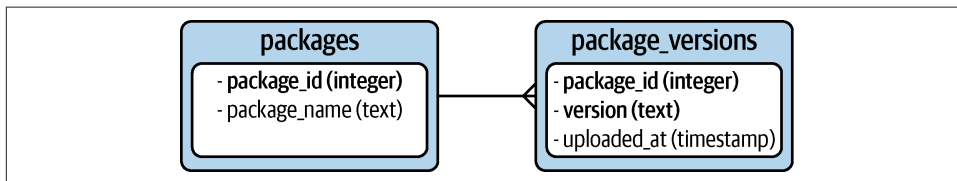


Figure 10-3. Database diagram (primary keys are bold)

Primary key

Relational databases require every table to have a *primary key*. A primary key consists of one or more columns that uniquely identify a row (a row is also called a *record*). In the case of the **packages** table, the primary key is **package_id**, and in the case of the **package_versions** table, the primary key is a so-called *composite key*, i.e., a combination of **package_id** and **version**.

Foreign key

The column **package_id** in the **package_versions** table is a *foreign key* to the same column in the **packages** table, symbolized by the line that connects the tables: a foreign key is a constraint that, in our case, ensures that every **package_id** in the **package_versions** table exists in the **packages** table—this guarantees data integrity. The branches at the right end of the connection line symbolize the nature of the relationship: one package can have many **package_versions**, which is called a *one-to-many* relationship.

To have a look at the content of the database tables and run SQL queries, you could install a VS Code extension called **SQLite** or use a dedicated SQLite management software, of which there are plenty. We, however, will be using Python to run SQL queries. Before anything else, let's see how we can connect to a database!

Database connections

To connect to a database from Python, you need a *driver*, i.e., a Python package that knows how to communicate with the database you are using. Each database requires a different driver and each driver uses a different syntax, but luckily, there is a powerful package called **SQLAlchemy** that abstracts away most of the differences between the various databases and drivers. SQLAlchemy is primarily used as an *object relational mapper* (ORM) that translates your database records into Python objects, a concept that many developers—albeit not all—find more natural to work with. To keep things simple, we're ignoring the ORM functionality, and we're only using SQLAlchemy to make it easier to run raw SQL queries. SQLAlchemy is also used by

pandas to read and write database tables if you aren't using one of the more recent ADBC drivers (see sidebar).

ADBC Database Drivers

If you have large datasets, you can try the relatively new *Arrow Database Connectivity* (ADBC) drivers instead of using SQLAlchemy with a traditional database driver. ADBC drivers are designed for analytical workloads with column-oriented datasets and provide better performance and data type detection. See the [pandas ADBC documentation](#) to learn how you use them.

Running a database query from pandas via SQLAlchemy involves three layers, as shown in [Figure 10-4](#):

- pandas
- SQLAlchemy
- database driver

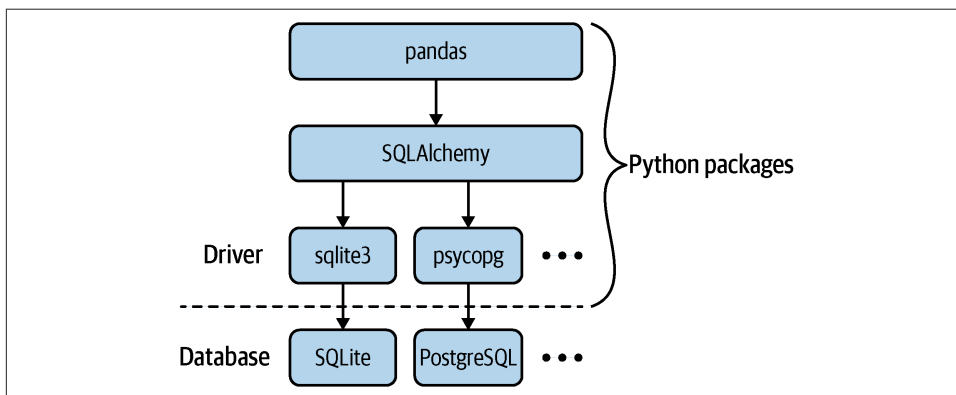


Figure 10-4. Accessing databases from Python

You can run database queries from each of these three layers—let's see how!

SQL queries

Even if you are new to SQL, you should have no trouble understanding the few SQL queries that I will use in the following samples and in the Python Package Tracker. SQL is a *declarative language*, which means that you tell the database *what you want* instead of *what to do*. Simple queries almost read like plain English:

```
SELECT * FROM packages
```

This tells the database that you want to *select all columns from the packages table*. In production code, you wouldn't want to use the wildcard *, which means *all columns*, but rather specify each column explicitly as this makes your query less error-prone:

```
SELECT package_id, package_name FROM packages
```



Database Queries Versus pandas DataFrames

SQL is a *set-based* language, which means that you operate on a set of rows rather than looping through individual rows. Therefore, working with SQL and pandas DataFrames follows a very similar approach. The SQL query:

```
SELECT package_id, package_name FROM packages
```

corresponds to the following pandas code (assuming that packages is a DataFrame):

```
packages[["package_id", "package_name"]]
```

The following code samples use the *packagetracker.db* file that you will find in the *packagetracker* folder of the companion repo.



Database State

The examples expect that you have added xlwings and pandas to the database as described at the beginning of this chapter—otherwise you would get only empty results.

Following [Figure 10-4](#) from bottom to top, we will first make our SQL query from the driver directly, then use SQLAlchemy and finally pandas. The code makes use of the `with` statement, which I explain in the following sidebar.

Context Managers and the with Statement

The `with` statement in Python doesn't have anything to do with the `With` statement in VBA: in VBA, it is used to run a series of statements on the same object, while in Python, it is used to manage resources like files or database connections. If you want to load any kind of data, you may have to open a file or establish a connection to a database. After you're done reading the data, it's best practice to close the file or connection as soon as possible again. Otherwise, you may run into situations where you can't open another file or can't establish another connection to the database—file handles and database connections are limited resources. Opening and closing a text file manually works like this (w stands for opening the file in write mode, which replaces the file if it already exists):

```
f = open("output.txt", "w")
f.write("Some text")
f.close()
```

This code will create a file called *output.txt* and write “Some text” to it. To *read* a file, you would use *r* instead of *w*, and to *append* to the end of the file, use *a*. Since files can also be manipulated from outside of your program, such an operation could fail. You could handle this by using the try/except mechanism that I will introduce in the next section. However, since this is such a common operation, Python provides the *with* statement to make things easier:

```
with open("output.txt", "w") as f:
    f.write("Some text")
```

When code execution leaves the body of the *with* statement, the file is automatically closed—even if there’s an exception happening. This guarantees that the resources are properly cleaned up. Objects that support the *with* statement are called *context managers*; this includes the database connection objects in the following code samples.

```
In [9]: # Let's start with the imports
import sqlite3 # sqlite3 is part of the standard library
import pandas as pd
from sqlalchemy import create_engine, text

In [10]: # Our SQL query: "select all columns from the packages table"
sql = "SELECT * FROM packages"

In [11]: # Option 1: Database driver
# Using the connection as context manager automatically commits
# the transaction or rolls it back in case of an error.
with sqlite3.connect("packagetracker/packagetracker.db") as con:
    cursor = con.cursor() # We need a cursor to run SQL queries
    result = cursor.execute(sql).fetchall() # Return all records
result

Out[11]: [(1, 'xlwings'), (2, 'pandas')]

In [12]: # Option 2: SQLAlchemy
# "create_engine" expects the connection string of your database.
# Here, we can execute a query as a method of the connection object.
engine = create_engine("sqlite:///packagetracker/packagetracker.db")
with engine.begin() as con:
    result = con.execute(text(sql)).all()
result

Out[12]: [(1, 'xlwings'), (2, 'pandas')]

In [13]: # Option 3: pandas
# Load an entire table via "read_sql_table".
# If you don't use an ADBC driver (see sidebar earlier in this
# chapter), pandas requires an SQLAlchemy engine that we reuse from
# the previous example.
```

```

packages = pd.read_sql_table("packages", engine)
packages

Out[13]:
  package_id package_name
0           1      xlwings
1           2      pandas

In [14]: # Run an SQL query via "read_sql_query"
pd.read_sql_query(sql, engine)

Out[14]:
  package_id package_name
0           1      xlwings
1           2      pandas

In [15]: # The DataFrame method "to_sql" writes a DataFrame to a database
# table. "if_exists" has to be either "fail", "append" or "replace"
# and defines what happens if the table already exists. It returns
# the number of affected rows.
packages.to_sql("packages2", con=engine, if_exists="append")

Out[15]: 2

In [16]: # The previous command created a new table "packages2" and
# inserted the records from the DataFrame packages as we can
# verify by reading it back
pd.read_sql_table("packages2", engine)

Out[16]:
  index  package_id package_name
0      0           1      xlwings
1      1           2      pandas

In [17]: # Let's get rid of the table again by running the
# "drop table" command via SQLAlchemy
with engine.begin() as con:
    con.execute(text("DROP TABLE packages2"))

```

Whether you should use SQLAlchemy, pandas, or the database driver directly to run your queries largely depends on your preferences: I personally like the fine-grained control you get by using SQLAlchemy, and enjoy that I can use the same syntax with different databases. On the other hand, pandas' `read_sql_query` is convenient to get the result of a query in the form of a DataFrame.

Now that you know how to run simple SQL queries, let's wrap this section up by looking at SQL injections, which can pose a security risk to your application.

SQL injection

If you don't protect your SQL queries properly, a malicious user can run arbitrary SQL code by injecting SQL statements into data input fields: for example, instead of selecting a package name like `xlwings` in the dropdown of the Python Package Tracker, they could send an SQL statement that changes your intended query. This can expose sensitive information or perform destructive actions like deleting a table. Consider the following example:

```
SELECT * FROM package_versions WHERE package_id = 1
```

This query only returns those rows where the `package_id` is 1. To use a variable value for `package_id`, you could be tempted to use f-strings (we first met them in [Chapter 3](#)):

```
# Don't do this!
f"SELECT * FROM package_versions WHERE package_id = {package_id}"
```

While this would technically work, you must never do this as it opens up the door for SQL injection: for example, somebody could send `'1 OR TRUE'` instead of an integer representing the `package_id`. The resulting query would return the rows of the whole table instead of just those where the `package_id` is 1. Therefore, always use the syntax that SQLAlchemy offers you for placeholders. SQLAlchemy usually refers to these placeholders as *bound parameters*, and they start with a colon:

```
In [18]: # ":package_id" is the placeholder
        sql = "SELECT * FROM package_versions WHERE package_id = :package_id"

In [19]: # Via SQLAlchemy
        with engine.begin() as con:
            result = con.execute(text(sql), {"package_id": 1}).all()
            result[:3] # Print the first 3 records

Out[19]: [(1, '0.1.0', '2014-03-19 18:18:49.000000'),
          (1, '0.1.1', '2014-06-27 16:26:36.000000'),
          (1, '0.10.0', '2016-09-20 09:40:19.000000')]

In [20]: # Via pandas
        pd.read_sql_query(
            text(sql),
            engine,
            parse_dates=["uploaded_at"],
            params={"package_id": 1},
        ).head(3)

Out[20]:   package_id version      uploaded_at
         0         1   0.1.0 2014-03-19 18:18:49
         1         1   0.1.1 2014-06-27 16:26:36
         2         1   0.10.0 2016-09-20 09:40:19
```

Using SQLAlchemy has the advantage that you can use the same syntax for placeholders across different databases. If you use the driver directly, the placeholders are different: as an example, `sqlite3` uses `?` and `psycopg` uses `%s`.

You may argue that SQL injection isn't much of an issue when your users have direct access to Python and could run arbitrary code on the database anyway. But if you take your `xlwings` prototype and transform it into a web application one day, it will become a critical vulnerability, so it's better to do it properly from the beginning.

Besides web APIs and databases, there is another topic that we have jumped over so far that is indispensable for solid application development: exception handling. Let's see how it works!

Exceptions

I mentioned exception handling in [Chapter 1](#) as an example of where VBA with its *GoTo* mechanism has fallen behind. In this section, you learn how the *try/except* mechanism works and how to raise your own exceptions.

Handling exceptions

Whenever something is outside of your control, errors will happen. For example, the web API may be down when you try to read the package information from PyPI, or a file may be missing that your program expects—in the case of the Python Package Tracker, this could be the database file. As an introductory example, let's pick up the reciprocal example again from [Chapter 1](#). If the following function is called with a zero, you will get a `ZeroDivisionError`:

```
In [21]: def print_reciprocal(number):
         result = 1 / number
         print(f"The reciprocal is: {result}")

In [22]: print_reciprocal(0) # This will raise an error

-----
ZeroDivisionError                                Traceback (most recent call last)
Cell In[23], line 1
----> 1 print_reciprocal(0) # This will raise an error

Cell In[22], line 2, in print_reciprocal(number)
      1 def print_reciprocal(number):
----> 2     result = 1 / number
      3     print(f"The reciprocal is: {result}")

ZeroDivisionError: division by zero
```

To let your program react gracefully to such errors, use the *try/except* statements:

```
In [23]: def print_reciprocal(number):
         try:
             result = 1 / number
         except Exception as e:
             # "as e" makes the Exception object available as variable "e".
             # "repr" stands for "printable representation" of an object
             # and gives you back a string with the error message.
             print(f"There was an error: {repr(e)}")
             result = "N/A"
         else:
             print("There was no error!")
```

```

finally:
    print(f"The reciprocal is: {result}")

```

Whenever an error occurs in the try block, code execution moves on to the except block where you can handle the error: this allows you to give the user helpful feedback or write the error to a log file. The else clause only runs if there is no error raised during the try block, and the finally block runs always, whether or not an error was raised. Most of the time, you will get away with just the try and except blocks. Let's see the output of the function given different inputs:

```

In [24]: print_reciprocal(10)
Out[24]: There was no error!
         The reciprocal is: 0.1

In [25]: print_reciprocal("a")
Out[25]: There was an error: TypeError("unsupported operand type(s) for /: 'int'
         and 'str'")
         The reciprocal is: N/A

In [26]: print_reciprocal(0)
Out[26]: There was an error: ZeroDivisionError('division by zero')
         The reciprocal is: N/A

```

The way that I have used the except statement means that any exception that happens in the try block will cause the code execution to continue in the except block. Usually, that is not what you want. Instead, you want to check for an error as specific as possible and handle only those you expect. Your program may otherwise fail for something completely unexpected, which makes it hard to debug. To fix this, rewrite the function as follows, checking explicitly for the two errors that we expect (I am removing the else and finally statements):

```

In [27]: def print_reciprocal(number):
         try:
             result = 1 / number
             print(f"The reciprocal is: {result}")
         except (TypeError, ZeroDivisionError):
             print("Please type in any non-zero number.")

```

Let's run the code again:

```

In [28]: print_reciprocal("a")
Out[28]: Please type in any non-zero number.

```

If you want to treat an error differently depending on the exception, handle them separately:

```

In [29]: def print_reciprocal(number):
         try:
             result = 1 / number
             print(f"The reciprocal is: {result}")

```

```

except TypeError:
    print("Please type in a number.")
except ZeroDivisionError:
    print("The reciprocal of 0 is not defined.")

In [30]: print_reciprocal("a")

Out[30]: Please type in a number.

In [31]: print_reciprocal(0)

Out[31]: The reciprocal of 0 is not defined.

```

Now that you know how to handle exceptions, let's learn how you can raise one yourself!

Raising exceptions

In the previous section, you saw how exceptions can help you determine the cause of an error: if you get a `ZeroDivisionError`, you know that the issue is caused by an illegal division. When writing your own functions, you should help the users in a similar way by raising an appropriate exception. For example, if they pass an argument with the wrong value when calling your function, you should raise a `ValueError` as we did in [Chapter 3](#), where we were using the following function:

```

In [32]: def convert_to_celsius(degrees, source="fahrenheit"):
        if source.lower() == "fahrenheit":
            return (degrees - 32) * (5 / 9)
        elif source.lower() == "kelvin":
            return degrees - 273.15
        else:
            raise ValueError(f"Don't know how to convert from {source}")

```

Accordingly, when calling the function with an unsupported temperature scale, it will raise a `ValueError`:

```

In [33]: convert_to_celsius(100, "Rankine") # This will raise an error
-----
ValueError                                Traceback (most recent call last)
Cell In[34], line 1
----> 1 convert_to_celsius(100, "Rankine") # This will raise an error

Cell In[33], line 7, in convert_to_celsius(degrees, source)
      5     return degrees - 273.15
      6 else:
----> 7     raise ValueError(f"Don't know how to convert from {source}")

ValueError: Don't know how to convert from Rankine

```

For a comprehensive list of all built-in exceptions, see the Python documentation about [Built-in Exceptions](#).

Now that you are familiar with error handling, web APIs, and databases, you are ready to move on to the next section, where we'll go through each component of the Python Package Tracker.

Application Walkthrough

In this section, we'll look behind the scenes of the Python Package Tracker to understand how everything fits together. First, we'll walk through the application's frontend, i.e., the Excel file, before looking at its backend, i.e., the Python code. To wrap this section up, we'll see how debugging an xlwings project works, a useful skill with projects of the size and complexity of the Package Tracker.

In the *packagetracker* directory in the companion repo, you'll find four files. They all have a specific purpose, as shown in [Table 10-3](#):

Table 10-3. The components of the Package Tracker

Purpose	File	Description
User interface	<i>packagetracker.xlsm</i>	This is the frontend and as such the only file the end-user interacts with.
Business logic	<i>packagetracker.py</i>	This module handles the data download via web API and does the number crunching with pandas.
Data access	<i>database.py</i>	This module handles all database queries.
Database	<i>packagetracker.db</i>	This is an SQLite database file.

With each file's responsibility defined, let's take a closer look at how the Excel frontend works.

Frontend

When you build a web application, you differentiate between the *frontend*, which is the part of the application that runs in your browser, and the *backend*, which is the code that runs on the server. We can apply the same terminology with xlwings tools: the frontend is the Excel file and the backend is the Python code that you call via RunPython. If you want to build the frontend from scratch, begin by running the following command on a Terminal. Before running these commands, however, rename the existing *packagetracker* directory *packagetracker_original*:

Windows (Terminal)

```
cd %userprofile%\python-for-excel-2e
uv run xlwings quickstart packagetracker
```

macOS (Terminal)

```
cd ~/python-for-excel-2e
uv run xlwings quickstart packagetracker
```

Navigate to the *packagetracker* directory and open *packagetracker.xlsm* in Excel. Start by adding the three tabs, Tracker, Database and Dropdown, as shown in [Figure 10-5](#).

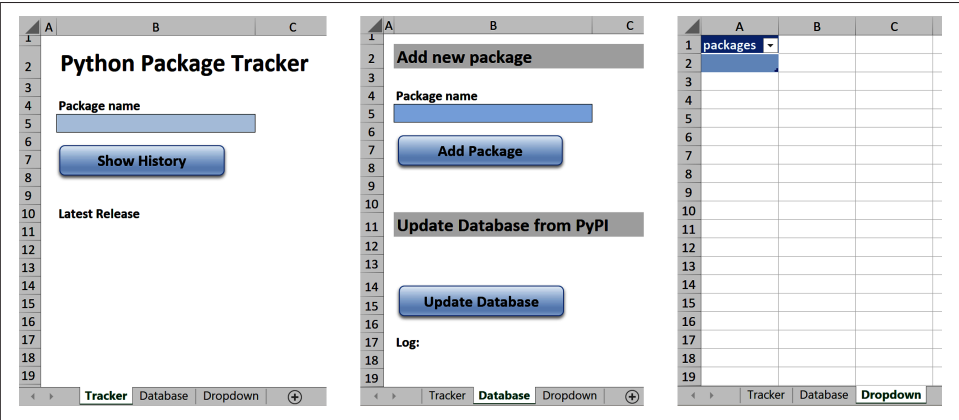


Figure 10-5. Building the user interface

While you should be able to take over the text and formatting from [Figure 10-5](#), I need to give you a few more details about the things that aren't visible:

Buttons

To make the tool look a bit less like Windows 3.1, I didn't use the standard macro buttons that we used in the previous chapter. Instead, I went to Insert > Shapes (on Windows, this is in the Illustrations group) and inserted a rounded rectangle. If you want to use the standard button from Form Controls, that's also fine, but at this point, don't assign a macro just yet.

Named ranges

To make the tool a little easier to maintain, we will use named ranges rather than cell addresses in the Python code. Therefore, add the named ranges as shown in [Table 10-4](#).

Table 10-4. Named ranges

Sheet	Cell	Name
Tracker	B5	package_selection
Tracker	B11	latest_release
Database	B5	new_package
Database	B13	updated_at
Database	B18	log

One way to add named ranges is to select the cell, then write the name into the Name box, and finally confirm by hitting Enter, as in [Figure 10-6](#).

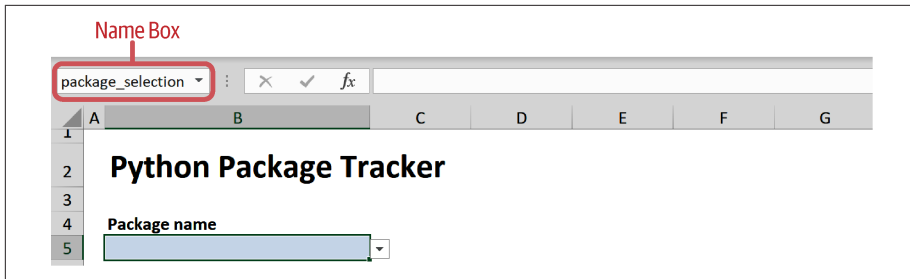


Figure 10-6. The Name box

Tables

On the Dropdown sheet, type **packages** into cell A1, then select A1 and go to Insert > Table. Make sure to activate the checkbox next to “My table has headers.” To finalize, select the table, go to the ribbon tab Table Design (Windows) or Table (macOS), and rename the table from Table1 to dropdown_content, as shown in Figure 10-7.

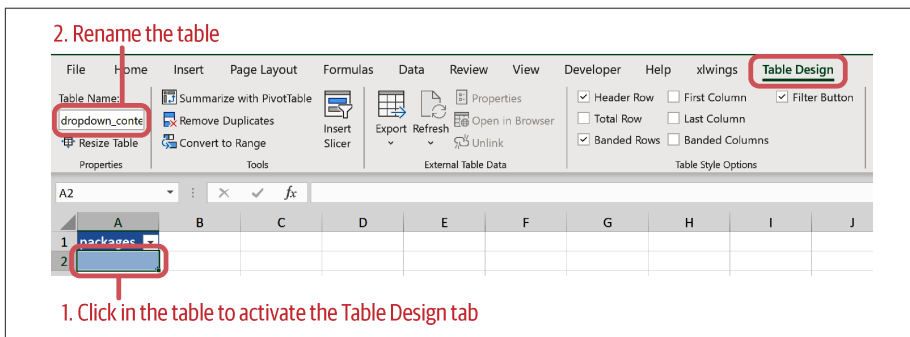


Figure 10-7. Renaming an Excel table

Data validation

We use an Excel feature called *data validation* to provide the dropdown in cell B5 on the Tracker sheet. To add it, select cell B5, then go to Data > Data Validation, and under Allow, select List. Under Source, write the following formula:

```
=INDIRECT("dropdown_content[packages]")
```

Then, confirm with OK. This is just a reference to the body of the table, but since Excel doesn’t accept a table reference directly, we have to wrap it in an INDIRECT formula, which resolves the table to its address. With this formula, Excel will automatically resize the range that defines the dropdown values when we add more packages.

Conditional formatting

When you add a package, there can be a few errors that we'd like to show to the user: the field could be empty, the package may already exist in the database, or it may be missing on PyPI. To show the error in red and other messages in black, we'll use a simple trick based on conditional formatting: we want a red font whenever the message contains the word **error**. On the Database sheet, select cell C5, which is where we'll write out the message. Then go to Home > Conditional Formatting > Highlight Cells Rules > Text That Contains. Enter the value **error** and select Red Text in the dropdown as shown in Figure 10-8, then click on OK. Apply the same conditional format to cell C5 on the Tracker sheet.

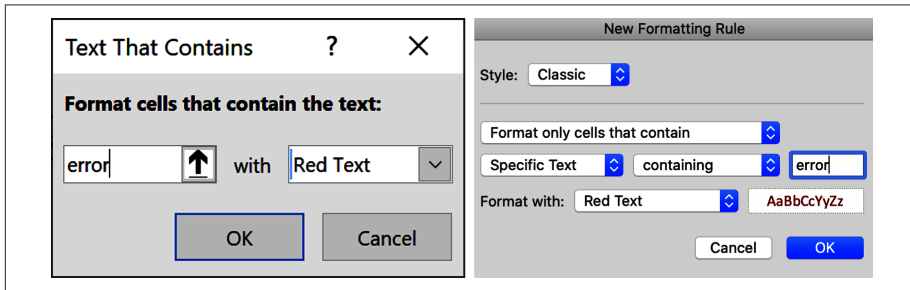


Figure 10-8. Conditional formatting on Windows (left) and macOS (right)

Gridlines

On the Tracker and Database sheets, the gridlines have been hidden by unchecking the View checkbox under Page Layout > Gridlines.

At this point, the user interface is complete and should look like Figure 10-5. We now need to add the RunPython calls in the VBA editor and connect them with the buttons. Press Alt+F11 (Windows) or Option-F11 (macOS) to open the VBA editor, then, under the VBAProject of *packagetracker.xlsm*, double-click Module1 on the left-hand side under Modules to open it. Delete the existing SampleCall code and replace it with the following macros:

```
Sub AddPackage()  
    RunPython "import packagetracker; packagetracker.add_package()"  
End Sub  
  
Sub ShowHistory()  
    RunPython "import packagetracker; packagetracker.show_history()"  
End Sub  
  
Sub UpdateDatabase()  
    RunPython "import packagetracker; packagetracker.update_database()"  
End Sub
```

Next, right-click the buttons on the Tracker and Database sheets, click Assign Macro, and select the macro that corresponds to the button. Confirm by clicking OK. **Figure 10-9** shows the Show History button, but it works the same for the Add Package and Update Database buttons.

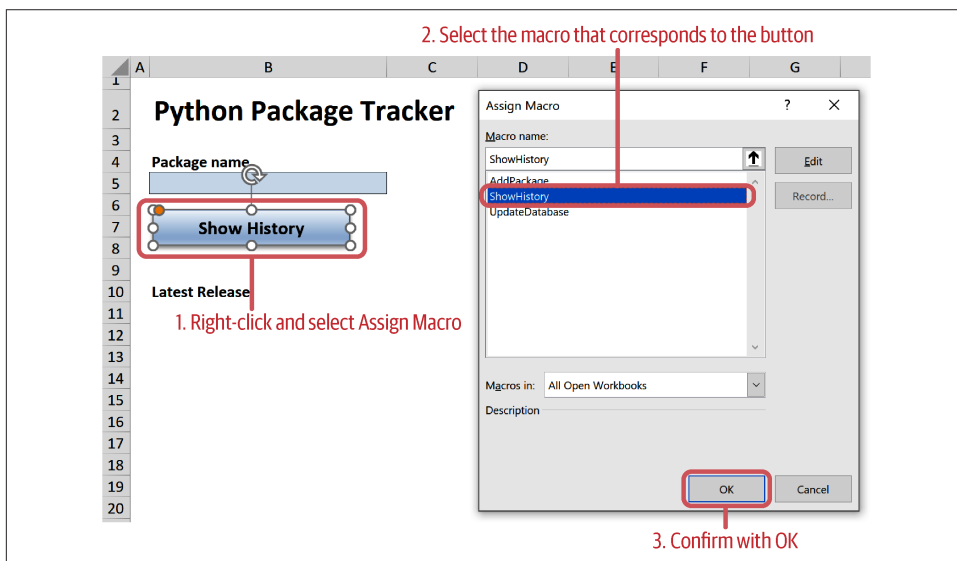


Figure 10-9. Assigning the ShowHistory macro to the Show History button

The frontend is now done and we can move on with the Python backend.

Backend

The code of the two Python files *packagetracker.py* and *database.py* is too long to be shown here, so you will need to open them from the companion repository in VS Code. I will, however, refer to a couple of code snippets in this section to explain a few key concepts.



Type Hints

Both modules, *packagetracker.py* and *database.py*, make use of type hints in all function definitions—this should give you a much better feel for how they work in practice compared to the brief introduction we had in **Chapter 3**.

Let's understand what happens when you click the Add Package button on the Database sheet. The button has the following VBA macro assigned:


```

Sub AddPackage()
    RunPython "import packagetracker; packagetracker.add_package()"
End Sub

```

As you see, the RunPython function calls the add_package Python function in the packagetracker module, as shown in [Example 10-1](#) (for brevity, the code is shown without comments here—please look at *packagetracker.py* directly for a commented version of the function).

Example 10-1. The add_package function in packagetracker.py (without comments)

```

def add_package() -> None:
    db_sheet = xw.Book.caller().sheets["Database"]
    feedback_cell = db_sheet["new_package"].offset(column_offset=1)

    feedback_cell.clear_contents()

    package_name = db_sheet["new_package"].value
    if not package_name:
        feedback_cell.value = "ERROR: Please provide a name!" ❶
        return

    package_name = package_name.strip()

    if database.package_exists(package_name):
        feedback_cell.value = f"ERROR: {package_name} already exists!"
        return

    try:
        fetch_package_data(package_name) ❷
    except requests.HTTPError as e:
        if e.response.status_code == 404:
            feedback_cell.value = f"ERROR: '{package_name}' doesn't exist on PyPI"
        else:
            feedback_cell.value = f"ERROR: Couldn't access '{package_name}' on PyPI"
        return
    except requests.RequestException as e:
        feedback_cell.value = f"ERROR: Couldn't contact PyPI! Details: {repr(e)}"
        return

    try:
        database.store_package(package_name)
    except Exception as e:
        feedback_cell.value = f"ERROR: {repr(e)}"
        return

    db_sheet["new_package"].clear_contents()
    feedback_cell.value = f"Added {package_name} successfully."
    update_database() ❸
    refresh_dropdown() ❹

```

- ❶ The “error” in the feedback message will trigger the red font in Excel via conditional formatting.
- ❷ This is a bit of a heavy call (it downloads all the details for all the releases) just to check if the name exists, but it helps us keep things simple.
- ❸ Again, to keep things simple, the whole database is updated. In a production environment, you would only add the records of the new package.
- ❹ This will update the table on the Dropdown sheet with the content of the packages table. Together with the data validation that we have set up in Excel, this makes sure that all packages appear in the dropdown on the Tracker sheet.



No Production Code

The application is kept as simple as possible to make it easy to follow—it doesn’t check for every possible thing that can go wrong. In a production environment, you’d want to make it more robust: for example, you would show a user-friendly error if it can’t find the database file.

You should be able to follow the other functions in the *packagetracker.py* file with the help of the comments in the code. Let’s now turn our attention to the *database.py* file. The first few lines are shown in [Example 10-2](#).

Example 10-2. database.py (excerpt)

```
from pathlib import Path

import pandas as pd
import sqlalchemy
from sqlalchemy import text

# We want the database file to sit next to this file.
# Here, we are turning the path into an absolute path.
this_dir = Path(__file__).resolve().parent ❶
db_path = this_dir / "packagetracker.db"

# Database engine
engine = sqlalchemy.create_engine(f"sqlite:///db_path")
```

- ❶ If you need a refresher of what this line does, have a look at [Chapter 8](#), where I explain it in the code of the sales report case study.

While this snippet is concerned with putting together the path of the database file, it also shows you how to get around a common error when you work with any sort of file, whether that's a picture, a CSV file, or, like in this case, a database file. When you put together a quick Python script, you may just use a relative path as I have done in most of the Jupyter notebook samples:

```
engine = sqlalchemy.create_engine("sqlite:///packagetracker.db")
```

This works as long as your file is in your working directory. However, when you run this code from Excel via RunPython, the working directory can be different, which will cause Python to look for the file in the wrong folder; you will get a `File not found` error. You can solve this issue by providing an absolute path or—for greater flexibility—by creating a path the way we do in [Example 10-2](#). This makes sure that Python is looking for the file in the same directory as the source file no matter what your current working directory is.

If you want to create the Python Package Tracker from scratch and have renamed the original *packagetracker* directory to *packagetracker_original*, you will need to copy *packagetracker.py* and *database.py* from *packagetracker_original* to *packagetracker*, then create the database: run the *database.py* file as a script, for example, by clicking the Run Python File button in VS Code or by running the following in a Terminal:

Windows (Terminal)

```
cd %userprofile%\python-for-excel-2e
uv run packagetracker\database.py
```

macOS (Terminal)

```
cd ~/python-for-excel-2e
uv run packagetracker/database.py
```

This will create the database file *packagetracker.db* with the two tables. The code that creates the database is found at the very bottom of *database.py*:

```
if __name__ == "__main__":
    create_db()
```

While the last line calls the `create_db` function, the meaning of the preceding `if` statement is explained in the following sidebar.

if `__name__` == "`__main__`"

You will see this `if` statement at the bottom of many Python files. It makes sure that this code only runs when you run the file *as a script*, for example, from a Terminal by running `uv run database.py` or by clicking the Run Python File button in VS Code. It, however, will not be triggered when you run the file *by importing it as a module*, i.e., by doing `import database` in your code. The reason for this is that Python assigns the name `__main__` to the file if you run it directly as script, whereas it will

be called by its module name (`database`) when you run it via the `import` statement. Since Python tracks the file name in a variable called `__name__`, the `if` statement will evaluate to `True` only when you run it as script; it will not be triggered when you import it into the *packagetracker.py* file.

The rest of the `database` module runs SQL statements both via `SQLAlchemy` and `pandas` so you get a feel for both approaches.

When you create tools of the complexity of the Python Package Tracker, you probably run into a few issues along the way: for example, you might have renamed a named range in Excel and forgot to adjust the Python code accordingly. This is a good moment to look into debugging!

Debugging

To easily debug your *xlwings* scripts, run your functions directly from VS Code, instead of clicking a button in Excel. The following lines at the very bottom of the *packagetracker.py* file will help you with debugging the `add_package` function (this is the same code that you will also find at the bottom of a quickstart project):

```
if __name__ == "__main__": ❶
    xw.Book("packagetracker.xlsm").set_mock_caller() ❷
    add_package()
```

- ❶ We have just seen how this `if` statement works when we were looking at the *database.py* code; see the previous sidebar.
- ❷ As this code is only executed when you run the file directly from Python as a script, the `set_mock_caller()` method is only meant for debugging purposes: when you run the file in VS Code or from a Terminal, it sets `xw.Book.caller()` to `xw.Book("packagetracker.xlsm")`. This allows you to run your script from both sides, Python and Excel, without having to switch the book object within the `add_package` function back and forth between `xw.Book("packagetracker.xlsm")` (when you call it from VS Code) and `xw.Book.caller()` (when you call it from Excel).

Open *packagetracker.py* in VS Code and set a breakpoint on any line within the `add_package` function by clicking to the left of the line numbers or by pressing the `F9` key, which will toggle breakpoints. Then click on the dropdown of the play button at the top right and select “Python Debugger: Debug Python File” to start the debugger and to make your code stop at the breakpoint.

If you're not familiar with how the debugger in VS Code works, have a look at [Chapter 2](#), where I explain all the relevant functionality and buttons. If you want to debug a different function, stop the current debug session, then adjust the function name at the bottom of your file. For example, to debug the `show_history` function, change the last line in *packagetracker.py* as follows before starting the debugger again:

```
if __name__ == "__main__":  
    xw.Book("packagetracker.xlsx").set_mock_caller()  
    show_history()
```

On Windows, you can also activate the Show Console checkbox in the xlwings add-in, which will show a Terminal while the RunPython call is running (this option is not yet available on macOS). This allows you to print additional information to help you debug the issue. For example, you could print the value of a variable to inspect it on the Terminal. After the code has been run, however, the Terminal will be closed. If you need to keep it open for a little longer, there is an easy trick: add `input()` as the last line in your function. This causes Python to wait for user input instead of closing the Terminal right away. When you're done with inspecting the output, hit Enter in the Terminal to close it—just make sure to remove the `input()` line again before unchecking the Show Console option!

Conclusion

This chapter showed you that it's possible to build reasonably complex applications with a minimum of effort. Being able to leverage powerful Python packages like Requests or SQLAlchemy makes all the difference to me when I compare this with VBA, where talking to external systems is so much harder. If you have similar use cases, I would highly recommend you look more closely into both Requests and SQLAlchemy—being able to efficiently deal with external data sources will allow you to make copy/paste a thing of the past.

Instead of clicking buttons, many Excel users prefer to create their tools by using cell formulas. The next chapter shows you how to write custom functions in Python, allowing you to reuse most of the xlwings concepts we've learned so far.

Custom Functions

The previous three chapters showed you how to automate Excel with a Python script and how to run such a script from Excel at the click of a button. This chapter introduces custom functions to call Python code from Excel.



User-Defined Functions (UDFs)

Microsoft previously called custom functions *user-defined functions* or *UDFs*. While the terms are interchangeable, this book consistently uses *custom functions*, which is also the terminology used by Google Sheets.

Custom functions are Python functions that you use in Excel cells in the same way that you use built-in functions like SUM or AVERAGE. As in the previous chapter, we will start with the `quickstart` command that allows us to try out a first custom function in no time. Then, we'll learn how to work with pandas DataFrames and plots, as well as how to debug custom functions. To conclude this chapter, we'll dig into performance optimization. Custom functions with the local version of xlwings—as covered in this chapter—only work on Windows. However, xlwings Lite supports custom functions on all platforms, as we will see in [Chapter 13](#).

Getting Started with Custom Functions

In this section, you'll build and run your first custom function. Before you begin, let's ensure you have the necessary setup in place.

Prerequisites

To follow along with the examples in this chapter, you'll need the xlwings add-in installed and have the Excel option “Trust access to the VBA project object model” enabled:

Add-in

I assume you have the xlwings add-in installed, as explained in [Chapter 9](#). This is not a hard requirement, though: while it makes development easy, especially to click the Import Functions button, it is not required for deployment and can be replaced by setting the workbook up in the standalone mode—for the details, see [Chapter 9](#).

Trust access to the VBA project object model

To be able to import custom functions, you need to change a setting in Excel: go to File > Options > Trust Center > Trust Center Settings > Macro Settings and activate the checkbox to “Trust access to the VBA project object model,” as in [Figure 11-1](#). This enables xlwings to automatically insert a VBA module into your workbook when you click the Import Functions button in the add-in. Since you only rely on this setting during the import process, this is a developer setting that end users don't need to be concerned about.

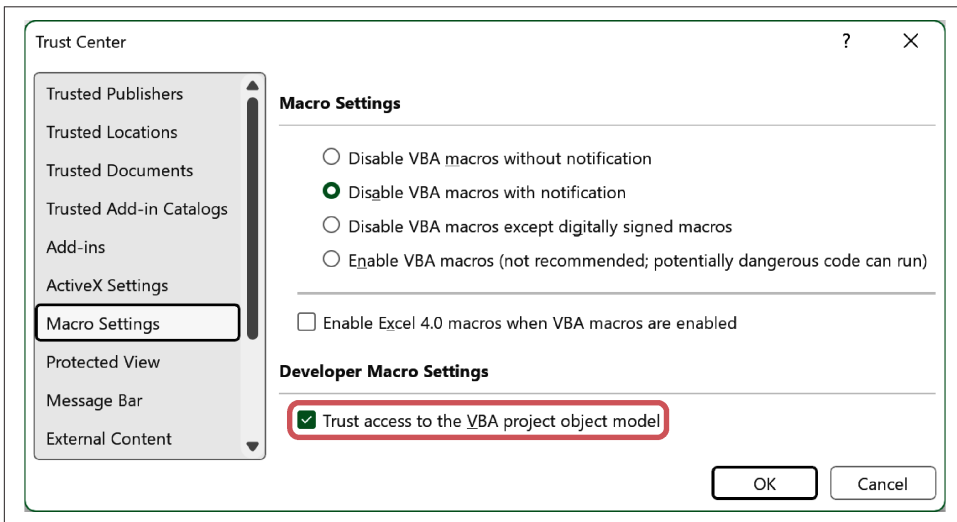


Figure 11-1. Trust access to the VBA project object model

With these two prerequisites in place, you're ready to write your first custom function!

Hello World

As usual, the easiest way to get off the ground is to use the `quickstart` command in the Terminal. Let's use it to create the classic "Hello World!" function:

Windows (Terminal)

```
cd %userprofile%\python-for-excel-2e
uv run xlwings quickstart function1
```

macOS (Terminal)

```
cd ~/python-for-excel-2e
uv run xlwings quickstart function1
```

Running the `quickstart` command will create a folder called *function1* in the directory of the companion repository—once created, you can move the folder anywhere you like. In the *function1* directory, open *function1.xlsm* in Excel and *function1.py* in VS Code. Then, in the xlwings ribbon add-in, click the Import Functions button. By default, this is a silent action, i.e., you will only see something in case of an error. However, if you activate the checkbox Show Console in the Excel add-in and click the Import Functions button again, a Terminal opens and prints the following:

```
xlwings server running [...]
Imported functions from the following modules: function1
```

The first line prints a few more details that we can ignore—the important part is that once this line is printed, Python is up and running. The second line confirms that it imported the functions from the `function1` module correctly. Now type `=hello("xlwings")` into cell A1 of the active sheet in *function1.xlsm*, and after hitting Enter, you will see the formula evaluated, as shown in [Figure 11-2](#).

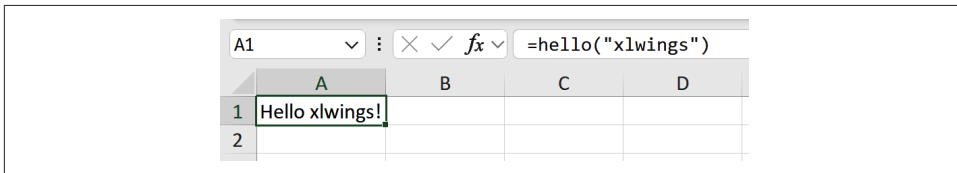


Figure 11-2. *function1.xlsm*

Let's break this down to see how everything works: start by looking at the `hello` function in *function1.py* ([Example 11-1](#)), which is the part of the `quickstart` code that we've ignored so far.

Example 11-1. The hello function in function1.py

```
import xlwings as xw
from xlwings import func
```

```
@func
def hello(name):
    return f"Hello {name}!"
```



Custom Functions Versus Python in Excel

Custom functions are native Excel functions that you call directly in a cell by writing `=hello()`, just like a built-in function such as `=SUM()`. They follow Excel's calculation chain and recalculate only when their dependencies change. In contrast, Python in Excel cells execute sequentially from left to right and top to bottom, and if you change one Python in Excel cell, all Python in Excel cells will be recalculated.

Every function that you mark with `@func` will get imported into Excel when you click on Import Functions in the xlwings add-in. Importing a function makes it available in Excel so you can use it in your cell formulas—we'll get to the technical details in a moment. `@func` is a *decorator*, which means that you have to place it directly on top of the function definition. If you want to know a bit more about how decorators work, have a look at the sidebar.

Function Decorators

A *decorator* is a function name that you put on top of a function definition, starting with the `@` sign. It's a simple way to change the behavior of a function and is used by xlwings to recognize which functions you want to expose in Excel. To help you understand how a decorator works, the following example shows the definition of a decorator called `verbose` that will print some text before and after the function `print_hello` is run. Technically, the decorator takes the function (`print_hello`) and provides it as argument `func` to the `verbose` function. The inner function called `wrapper` can then execute additional operations; in this case, it prints a value before and after calling the `print_hello` function. The name of the inner function doesn't matter, and to keep it simple, we're not handling any function arguments:

```
In [1]: # This is the definition of the function decorator
def verbose(func):
    def wrapper():
        print("Before calling the function.")
        result = func()
        print("After calling the function.")
        return result
    return wrapper

In [2]: # Using a function decorator
@verbose
def print_hello():
    print("hello!")
```

```
In [3]: # Effect of calling the decorated function
        print_hello()

Out[3]: Before calling the function.
        hello!
        After calling the function.
```

If the function arguments are Excel ranges, xlwings delivers the cell values, which is equivalent to how it works in VBA. For example, you could write **xlwings** into cell A2, then change the formula in A1 into the following:

```
=hello(A2)
```

The result will be the same as in [Figure 11-2](#). For your reference, the equivalent VBA function would look like this:

```
Function hello(name As String) As String
    hello = "Hello " & name & "!"
End Function
```

When you click the Import Functions button in the add-in, xlwings inserts a VBA module called `xlwings_udfs` into your Excel workbook. It contains a VBA function for each Python function you import: these wrapper VBA functions take care of running the respective function in Python. While nobody stops you from looking at the `xlwings_udfs` VBA module, you can ignore it as the code is autogenerated and any changes would get lost when you click the Import Functions button again. Let's now play around with our hello function in *function1.py* and replace Hello in the return value with Bye:

```
@func
def hello(name):
    return f"Bye {name}!"
```

To recalculate the function in Excel, either double-click the cell A1 to edit the formula (or select the cell and press the F2 function key to activate the edit mode), then hit Enter. Alternatively, type the keyboard shortcut Ctrl+Alt+F9: this will *force* the recalculation of all worksheets in all open workbooks including the hello formula. Note that F9 (recalculate all worksheets in all open workbooks) or Shift+F9 (recalculate the active worksheet) will not recalculate the custom function as Excel only triggers a recalculation of custom functions if a dependent cell changed. To change this behavior, you could make the function *volatile* by adding the respective argument to the func decorator:

```
@func(volatile=True)
def hello(name):
    return f"Bye {name}!"
```

Volatile functions are evaluated every time Excel performs a recalculation—whether or not the function's dependencies have changed. A few of Excel's built-in functions

are volatile like `=RAND()` or `=NOW()`, and using lots of them will make your workbook slow, so don't overdo it. When you change a function's name or arguments or the func decorator, as we just did, you will need to reimport your function by clicking the Import Functions button again: this will restart the Python interpreter before importing the updated function. When you now change the function back from Bye to Hello, it is enough to use the keyboard shortcuts Shift+F9 or F9 to recalculate the cell as the function is now volatile.



Save the Python File After Changing It

A common gotcha is forgetting to save the Python source file after making changes. Therefore, always double-check that the Python file is saved before hitting the Import Functions button or recalculating the custom functions in Excel.

By default, xlwings imports functions from a Python file in the same directory with the same name as the Excel file. Renaming and moving your Python source file requires changes similar to what we did in [Chapter 9](#) in connection with RunPython calls: go ahead and rename the file from `function1.py` to `hello.py`. Now that your module has a different name than your workbook, you need to add the module name, i.e., `hello` (without the `.py` extension!) to UDF Modules in the xlwings add-in, as shown in [Figure 11-3](#).

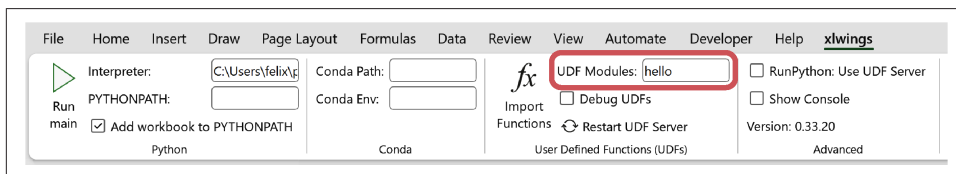


Figure 11-3. The UDF Modules setting

Click the Import Functions button to reimport the function. Then recalculate the formula in Excel to be sure everything still works.



Import Functions from Multiple Python Modules

If you want to import functions from multiple modules, use a semicolon between their names in the UDF Modules setting, e.g., `hello;another_module`.

Now go ahead and move `hello.py` to your desktop: this requires you to add the path of your desktop to the PYTHONPATH in the xlwings add-in. As seen in [Chapter 9](#), you could use environment variables to achieve this, i.e., you could set the PYTHONPATH setting in the add-in to `%USERPROFILE%\Desktop`. If you still have the path to the

xlwings_scripts folder in there from [Chapter 9](#), either overwrite it or leave it in there, separating the paths with a semicolon. After these changes, click the Import Functions button again, then recalculate the function in Excel to verify that everything still works.

Configuration and Deployment

In this chapter, I am always referring to changing a setting in the add-in; however, you can apply everything from [Chapter 9](#) with regard to configuration and deployment to this chapter too. This means that you can also change a setting in the *xlwings.conf* sheet or a config file that is in the same directory as the Excel file. And instead of using the *xlwings* add-in, you could use a workbook that has been set up in the standalone mode. With custom functions, you may also want to build your own custom add-in—this allows you to share your custom functions across files so you don't have to import them into each workbook. For more information about building your own custom add-in, see the [xlwings docs](#).

If you change the Python code of your custom function, *xlwings* automatically picks up the changes whenever you save the Python file. As mentioned, you only need to reimport your custom functions if you change something in the function's name, arguments, or decorators. If, however, your source file imports code from other modules, and you change something in these modules, the easiest way to let Excel pick up all of the changes is to click on Restart UDF Server in the *xlwings* add-in.

At this point, you know how to write a simple custom function in Python and how to use it in Excel. The next section will introduce you to more realistic custom functions that make use of pandas DataFrames.

Working with DataFrames

In this section, you'll learn how to use pandas DataFrames with custom functions. We'll start with an introduction to converters and options before downloading data from the web and generating Matplotlib plots. Finally, we'll cover how to debug custom functions.

Converters and Options

To turn Excel cell values into pandas DataFrames, we'll use the same converters and options that I introduced in [Chapter 8](#). Navigate to *custom functions\describe* in the companion repository and open *describe.xlsxm* in Excel and *describe.py* in VS Code. The Excel file contains data from *csv/MSFT.csv*, and in the Python file, you'll find a simple function to start with, as shown in [Example 11-2](#).



Enable Macros

Make sure to properly enable macros when opening a macro-enabled workbook (*xlsm* extension) such as *describe.xlsm*:

- Click on Enable Content.
- When working with *xlsm* files from the internet, you have to additionally click on Enable Editing.
- Before opening the workbook, though, you might need to unblock the file first by right-clicking on it in the Windows File Explorer and selecting Properties. On the General tab, check the box “Unblock,” then click OK to exit the properties window (macOS doesn’t require unblocking a file).

Example 11-2. describe.py

```
import pandas as pd
from xlwings import arg, func

@func
@arg("df", pd.DataFrame, index=True, header=True)
def describe(df):
    return df.describe()
```

Compared to the `hello` function from the quickstart project, you’ll notice a second decorator: `@arg` is short for *argument* and allows you to apply the same converters and options that I introduced in [Chapter 8](#). In other words, the `@arg` decorator offers the same functionality for custom functions as the `options` method for xlwings range objects. Formally, this is the syntax of the `arg` decorator:

```
@arg("argument_name", convert=None, option1=value1, option2=value2, ...)
```



Using Type Hints Instead of Decorators

Instead of using the `@arg` decorator, custom functions allow you to use type hints instead. I will show you how this works in [Chapter 13](#) where we’ll use custom functions in connection with xlwings Lite.

To help you make the connection back to [Chapter 8](#), the equivalent of the `describe` function in the form of a script looks like this (this assumes that *describe.xlsm* is open in Excel and that the function is applied to the range A2:F8078):

```
import pandas as pd
import xlwings as xw

data_range = xw.Book("describe.xlsm").sheets[0]["A2:F8078"]
```

```
df = data_range.options(pd.DataFrame, index=True, header=True).value
df.describe()
```

With *describe.xlsm* as your active workbook, click the Import Functions button, then type **=describe(A2:F8078)** in a free cell, in H2, for example. What happens when you hit Enter depends on your version of Excel—more specifically if your version of Excel is recent enough to support *dynamic arrays*. If you are using Microsoft 365 or Office 2021 or later, dynamic arrays are supported, and you will see the situation as shown in **Figure 11-4**, i.e., the output of the *describe* function in cells H2:M10 is surrounded by a thin blue border. You will only be able to see the blue border if your cursor is within the array, and it is so subtle that you probably won't see it in **Figure 11-4**, so check it out in Excel for yourself. We have already met dynamic arrays in **Chapter 6**, but I present them again with a little more detail in the sidebar “Dynamic Arrays” on page 256.

	A	B	C	D	E	F	G	H	I	J	K	L	M
1													
2	date	open	high	low	close	volume							
3	13/03/1986	25.5	29.25	25.5	28	3582600		count	open	high	low	close	volume
4	14/03/1986	28	29.5	28	29	1070000		mean	59.8321545	60.666713	59.0252605	59.8748209	31258675.6
5	17/03/1986	29	29.75	29	29.5	462400		std	32.5269352	33.0336952	32.0175521	32.5629264	32041253.9
6	18/03/1986	29.5	29.75	28.5	28.75	235300		min	15.2	15.62	14.87	15.15	8000
7	19/03/1986	28.75	29	28	28.25	166300		25%	28.94	29.25	28.66	28.94	2370594.5
8	20/03/1986	28.25	28.25	27.25	27.5	202900		50%	54.25	55.005	53.5	54.25	26179200
9	21/03/1986	27.5	28	26.25	26.75	208300		75%	83.75	84.955	82.7025	83.87	49910175
10	24/03/1986	26.75	26.75	25.75	26	226700		max	178.94	180.38	175.75	179.94	591052200
11	25/03/1986	26	26.5	25.75	26.5	111400							

Figure 11-4. The *describe* function with dynamic arrays

Legacy CSE Arrays

If you are using a version of Excel that doesn't support dynamic arrays (Office 2019 or older), it will look as if nothing is happening when you hit Enter after typing in the formula **=describe(A2:F8078)** in H2: by default, the formula will only return the top-left cell in H2, which is empty. To fix this, use what Microsoft nowadays calls *legacy CSE arrays*. CSE arrays need to be confirmed by typing the Ctrl+Shift+Enter key combination instead of hitting just Enter—hence their name. Let's see how they work in detail:

- Make sure that H2 is an empty cell by selecting it and hitting the Delete key.
- Select the output range by starting in cell H2, and then select all cells on the way to M10.
- With the range H2:M10 selected, type in the formula **=describe(A2:F8078)**, and then confirm by hitting Ctrl+Shift+Enter.

You should now see almost the same picture as in [Figure 11-4](#) with these differences:

- There is no blue border around the range H2:M10.
- The formula shows curly braces around it to mark it as a CSE array: `{=describe(A2:F8078)}`.
- While you delete dynamic arrays by going to the top-left cell and hitting the Delete key, with CSE arrays, you always have to select the whole array first to be able to delete it.

Let's now make our function slightly more useful by introducing an optional parameter called `selection` that will allow us to specify which columns we want to include in our output. Change the function as follows:

```
@func
@arg("df", pd.DataFrame) ❶
def describe(df, selection=None): ❷
    if selection is not None:
        return df.loc[:, selection].describe() ❸
    else:
        return df.describe()
```

- ❶ I left off the `index` and `header` arguments, as they are using the defaults, but feel free to leave them in.
- ❷ Add the parameter `selection` and make it optional by assigning `None` as its default value.
- ❸ If `selection` is provided, filter the `DataFrame`'s columns based on it.



Reimport the Functions

Once you have changed the function, make sure to save *describe.py*, and then hit the Import Functions button in the *xlwings* add-in—this is required since we have added a new parameter.

Write **Selection** into cell A1 and **TRUE** into cells B1:F1 (note that you have to use your local version of **TRUE** if you are using an international version of Excel). Finally, adjust your formula in cell H2 depending on whether you have dynamic arrays or not:

With dynamic arrays

Select H2, then change the formula to `=describe(A2:F8078, B1:F1)` and hit Enter.

Without dynamic arrays

Starting at cell H2, select H2:M10, then hit the F2 function key to activate the edit mode of cell H2 and change the formula to **=describe(A2:F8078, B1:F1)**. To finalize, hit Ctrl+Shift+Enter.

To try out the enhanced function, let's change cell E1 from TRUE to FALSE and see what happens: with dynamic arrays, you will see the table magically shrink by one column. With legacy CSE arrays, however, you will end up with a column full of #N/A values, as shown in **Figure 11-5**.

H2

fx

=describe(A2:F8078,B1:F1)

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	Selection	TRUE	TRUE	TRUE	FALSE	TRUE							
2	date	open	high	low	close	volume			open	high	low	volume	
3	13/03/1986	25.5	29.25	25.5	28	3582600		count	8076	8076	8076	8076	
4	14/03/1986	28	29.5	28	29	1070000		mean	59.8321545	60.666713	59.0252605	31258675.6	
5	17/03/1986	29	29.75	29	29.5	462400		std	32.5269352	33.0336952	32.0175521	32041253.9	
6	18/03/1986	29.5	29.75	28.5	28.75	235300		min	15.2	15.62	14.87	8000	
7	19/03/1986	28.75	29	28	28.25	166300		25%	28.94	29.25	28.66	2370594.5	
8	20/03/1986	28.25	28.25	27.25	27.5	202900		50%	54.25	55.005	53.5	26179200	
9	21/03/1986	27.5	28	26.25	26.75	208300		75%	83.75	84.955	82.7025	49910175	
10	24/03/1986	26.75	26.75	25.75	26	226700		max	178.94	180.38	175.75	591052200	

H2

fx

{=describe(A2:F8078,B1:F1)}

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	Selection	TRUE	TRUE	TRUE	FALSE	TRUE							
2	date	open	high	low	close	volume			open	high	low	volume	#N/A
3	13/03/1986	25.5	29.25	25.5	28	3582600		count	8076	8076	8076	8076	#N/A
4	14/03/1986	28	29.5	28	29	1070000		mean	59.8321545	60.666713	59.0252605	31258675.6	#N/A
5	17/03/1986	29	29.75	29	29.5	462400		std	32.5269352	33.0336952	32.0175521	32041253.9	#N/A
6	18/03/1986	29.5	29.75	28.5	28.75	235300		min	15.2	15.62	14.87	8000	#N/A
7	19/03/1986	28.75	29	28	28.25	166300		25%	28.94	29.25	28.66	2370594.5	#N/A
8	20/03/1986	28.25	28.25	27.25	27.5	202900		50%	54.25	55.005	53.5	26179200	#N/A
9	21/03/1986	27.5	28	26.25	26.75	208300		75%	83.75	84.955	82.7025	49910175	#N/A
10	24/03/1986	26.75	26.75	25.75	26	226700		max	178.94	180.38	175.75	591052200	#N/A

Figure 11-5. Dynamic arrays (top) versus CSE arrays (bottom) after excluding a column

To work around this issue, xlwings can resize legacy CSE arrays by making use of the return decorator `@ret`. Add it by changing your function like this:

```
import pandas as pd
from xlwings import arg, func, ret ❶

@func
@arg("df", pd.DataFrame)
@ret(expand="table") ❷
def describe(df, selection=None):
    if selection is not None:
        return df.loc[:, selection].describe()
    else:
        return df.describe()
```

- ❶ Add `ret` to your imports.
- ❷ By adding the return decorator with the option `expand="table"`, xlwings will resize the CSE array to match the dimensions of the returned DataFrame—only

use this if you are using an older version of Excel that doesn't support dynamic arrays (Excel 2019 or older)!

After adding the return decorator, save the Python source file, switch over to Excel, and hit Ctrl+Alt+F9 to recalculate: this will resize the CSE array and remove the #N/A column. Since this is a workaround, I highly recommend you do whatever is in your power to get your hands on a version of Excel that supports dynamic arrays (Microsoft 365 and Excel 2021 or newer).



Order of Function Decorators

Make sure to place the `@func` decorator on top of the `@arg` and `@ret` decorators; the order of `@arg` and `@ret` doesn't matter. The technical reason behind this is that decorators are applied from bottom to top, so placing `@func` on top ensures it's applied last, after `@arg` and `@ret` have done their work.

The *return decorator* works conceptually the same way as the argument decorator, with the only difference that you don't have to specify the name of an argument. Formally, its syntax looks like this:

```
@ret(convert=None, option1=value1, option2=value2, ...)
```

You usually don't have to provide an explicit `convert` argument as `xlwings` recognizes the type of the return value automatically—that's the same behavior we saw in [Chapter 8](#) with the `options` method when writing values to Excel.

As an example, if you want to suppress the index of the `DataFrame` you return, use this decorator:

```
@ret(index=False)
```

Dynamic Arrays

Dynamic arrays require at least Office 2021 or Microsoft 365. To be sure that your version of Excel supports dynamic arrays, check for the existence of the `UNIQUE` function: start typing `=UNIQUE` in a cell, and if Excel suggests the function name, dynamic arrays are supported. Here are a few technical notes about the behavior of dynamic arrays:

- If dynamic arrays overwrite a cell with a value, you will get a `#SPILL!` error. After making room for the dynamic array by deleting or moving the cell that is in the way, the array will be written out. Note that the legacy `xlwings` return decorator with `expand="table"` is less smart and will overwrite existing cell values without warning!

- You can refer to the range of a dynamic array by using the top-left cell followed by a # sign. For example, if your dynamic array is in the range A1:B2 and you wanted to sum up all cells, write `=SUM(A1#)`.
- If you ever want your arrays to behave like the legacy CSE arrays again, start your formula with an @ sign, e.g., to have a matrix multiplication return a legacy CSE array, use `=@MMULT()`.

Copy/pasting the values from a CSV file into an Excel file worked fine for this introductory DataFrame example, but copy/paste is such an error-prone process that you'll want to get rid of it whenever you can. Therefore, the next section shows you how to download data directly from the web into Excel!

Fetching Data from the Web

The previous examples were all very simple, pretty much just wrapping a single pandas function. To get our hands on a more real-world case, let's create a custom function that downloads data from the web, as shown in [Example 11-3](#). You'll find the source code under `custom_functions/stocks/stocks.py` in the companion repository.

Example 11-3. The `get_stock_history` function in `stocks.py`

```
import pandas as pd
from xlwings import arg, func

@func(call_in_wizard=False) ❶
@arg("tickers", ndim=1, doc="A range of tickers.") ❷
@arg("start_date", doc="A date-formatted cell") ❸
@arg("end_date", doc="A date-formatted cell")
@arg("rebase", doc="If TRUE, starts all time series at 100")
@arg("column", doc="Defines the column name. Default: adj_close")
def get_stock_history(
    tickers, start_date=None, end_date=None, rebase=False, column="adj_close"
):
    """Fetch historical stock data for given tickers and combine them into a
    single DataFrame.
    """ ❹
    base_url = "https://raw.githubusercontent.com/fzumstein/python-for-excel/2e/csv"
    parts = []

    # Fetch data for each ticker
    for ticker in tickers:
        # Download the data from the online GitHub repository
        url = f"{base_url}/{ticker}.csv"
        df = pd.read_csv(url, parse_dates=["date"], index_col="date")
        df.index.name = "Date"
        parts.append(df[[column]].rename(columns={column: ticker}))
```

```

# Combine all DataFrames
result = pd.concat(parts, axis=1)

# Filter by date range
if start_date is not None or end_date is not None:
    result = result.loc[start_date:end_date, :]

# Rebase
if rebase:
    result = result / result.iloc[0] * 100

return result

```

- ❶ By default, Excel calls the function when you open it in the Function Wizard. As this can make it slow, especially with API requests involved, we're switching this off. To get an overview of which arguments the `@func` decorator accepts, have a look at the [decorator docs](#).
- ❷ The argument `ndim=1` makes sure that `tickers` is always a list—even if you only select a single cell.
- ❸ Optionally, `doc` allows you to document the function argument, which will be shown in the Function Wizard when you edit the respective argument, as in [Figure 11-7](#).
- ❹ The function's docstring is displayed in the Function Wizard, as in [Figure 11-7](#).

Now open `custom_functions/stocks/stocks.xlsm` from the companion repository, click on Import Functions in the xlwings add-in, then write the following function into cell C1:

```
=get_stock_history(Stocks[Tickers], A2, A4, TRUE)
```

You don't have to write out the argument `Stocks[Tickers]` manually—simply select the body of the Excel table, i.e., range A7:A8. After hitting Enter, your workbook should look like [Figure 11-6](#).

To get help with regard to the function arguments, click the Insert Function button to the left of the formula bar while cell C1 is selected: this will open the Function Wizard where you will find your custom functions under the xlwings category. After selecting `get_stock_history`, you'll see the name of the function arguments as well as the docstring as function description (restricted to the first 256 characters): see [Figure 11-7](#). Alternatively, start typing `=get_stock_history(` into cell C1 (including the opening parenthesis) before hitting the Insert Function button—this will take you directly to the view shown in [Figure 11-7](#).

Insert Function

	A	B	C	D	E	F	G
1	Start date		Date	AAPL	MSFT		
2	01/01/2015		02/01/2015	100	100		
3	End date		05/01/2015	97.1828409	99.0697177		
4	01/01/2018		06/01/2015	97.1919876	97.6261762		
5			07/01/2015	98.554834	98.8665526		
6	Tickers		08/01/2015	102.341535	101.775021		
7	AAPL		09/01/2015	102.451294	100.919589		
8	MSFT		12/01/2015	99.926827	99.6578272		

Formula bar: `=get_stock_history(Stocks[Tickers], A2, A4, TRUE)`

Figure 11-6. *stocks.xlsm*

Function Arguments
?
×

get_stock_history
Place the cursor in this field

Tickers

↑
=

Start_date

↑
=

End_date

↑
=

Rebase

↑
=

Column

↑
=

=

Fetch historical stock data for given tickers and combine them into a single DataFrame.

Tickers
A range of tickers.

The function's docstring

The "doc" argument from the @arg decorator

Formula result =

[Help on this function](#)

OK

Cancel

Figure 11-7. *The Function Wizard*



Date Formatting

Custom functions return dates unformatted. To fix this, right-click the column with the dates, select Format Cells from the context menu (Ctrl+1 keyboard shortcut), and choose your preferred format under the Date category. This won't be required when using custom functions with xlwings Lite, as we'll see in [Chapter 13](#).

The `get_stock_history` function relies on dynamic arrays. If your version of Excel doesn't support dynamic arrays, use the workaround by adding the following return decorator to the `get_stock_history` function below the existing decorators:

```
@ret(expand="table") # requires: from xlwings import ret
```

Now that you understand DataFrames, let's see how we can visualize them!

Plotting

As you might remember from [Chapter 5](#), calling a DataFrame's `plot` method returns a Matplotlib plot by default. In [Chapters 8](#) and [10](#), we've already seen how you add such a plot as a picture to Excel. To see how plotting works with custom functions, have a look at the second function in `stocks.py`, shown in [Example 11-4](#).

Example 11-4. The `plot` function in `stocks.py`

```
import pandas as pd
from xlwings import arg, func

@func
@arg("df", pd.DataFrame)
def plot(df, name, caller): ❶
    if not df.empty: ❷
        caller.sheet.pictures.add(
            df.plot().get_figure(), ❸
            anchor=caller.offset(row_offset=1), ❹
            name=name,
            update=True, ❺
        )
    return f"<Plot: {name}>" ❻
```

- ❶ The `caller` argument is a special argument that is reserved by xlwings: this argument will not be exposed when you call the function from an Excel cell. Instead, `caller` will be provided by xlwings behind the scenes and corresponds to the cell from which you are calling the function (in the form of an xlwings range object). Having the range object of the calling cell makes it easy to place the plot by using the `anchor` argument of `pictures.add`. The `name` argument will define the name of the picture in Excel.

- ❷ Only call the `plot` method if the `DataFrame` isn't empty. Calling the `plot` method on an empty `DataFrame` would raise an error.
- ❸ The pandas `plot` method returns a Matplotlib `Axes` object, from which you can get the figure object, which is what `pictures.add` expects.
- ❹ The `anchor` argument is only used when you insert the plot for the first time. It *anchors* the plot at the given `xlwings Range` object. The provided value `caller.offset(row_offset=1)` will place the plot in a convenient place—one cell below the one from where you call this function.
- ❺ The `update=True` argument makes sure that repeated function calls will update the existing picture with the provided name in Excel, without changing its position or size. Without this argument, `xlwings` would complain that there is already a picture with that name in Excel.
- ❻ While you don't technically need to return anything, returning a string allows you to recognize where in your sheet your plotting function sits.

In *stocks.xlsm*, in cell H1, call the `plot` function like so:

```
=plot(C1#, "History")
```

Figure 11-8 shows the final result. Note that I have added the `AMZN` and `GOOGL` tickers in cells A9 and A10, respectively. When you add the additional tickers on your end, you'll notice that the Excel table expands, causing the time series and plots to update automatically. If your version of Excel doesn't support dynamic arrays, use `C1:G755` instead of `C1#`.

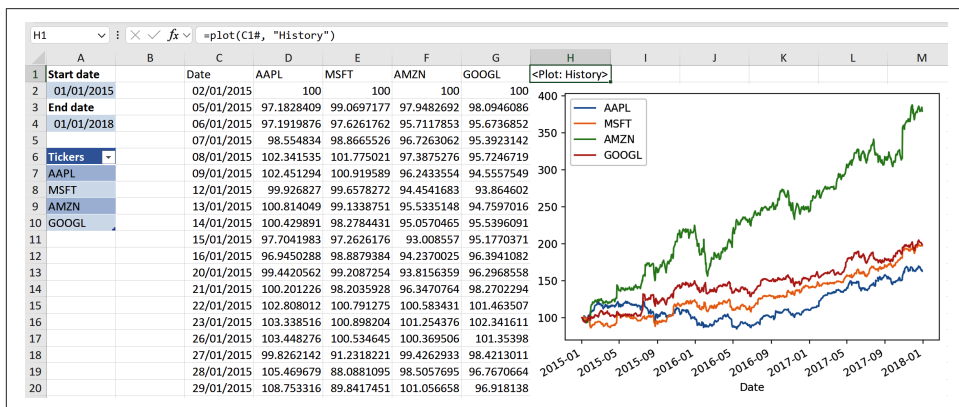


Figure 11-8. The `plot` function in action

If there's something you don't understand about the `get_stock_history` function, you could ask your favorite AI tool to explain the code. There is, however, another option: you can step through the code with a debugger!

Debugging

A simple way to debug a custom function is to use the `print` function. If you have the Show Console setting enabled in the xlwings add-in, you will be able to print the value of a variable in the Terminal that shows up when you call your custom function. A slightly more comfortable option is to use a debugger, which will allow you to pause at breakpoints and step through the code line by line. To use the VS Code debugger (or the debugger of any other IDE), you'll need to do two things:

1. In the Excel add-in, activate the checkbox Debug UDFs. This prevents Excel from automatically starting Python, which means you have to do it manually, as explained under the next point.
2. The easiest way to start the Python server manually is by adding the following lines at the end of the file you're trying to debug. I have already done this for the *stocks.py* file:

```
if __name__ == "__main__":  
    xw.serve() # Requires: import xlwings as xw
```

As you may remember from [Chapter 10](#), this `if` statement makes sure that the code only runs when you run the file as a script—it doesn't run when you import the code as a module.

Now run *stocks.py* in debug mode; in VS Code, set a breakpoint on line 27 by clicking to the left of the line number or by pressing F9. Now click on the dropdown of the play button at the top right of the Python file and select “Python Debugger: Debug Python File.” If you are not familiar with using the debugger of VS Code, go back to [Chapter 2](#) where I introduced it in more detail. When you now recalculate cell C1, your function call will stop at the breakpoint and you can inspect the variables. What's always helpful during debugging is to run `df.info()`. Activate the Debug Console tab, write `df.info()` in the prompt at the bottom, and confirm by hitting Enter, as shown in [Figure 11-9](#).

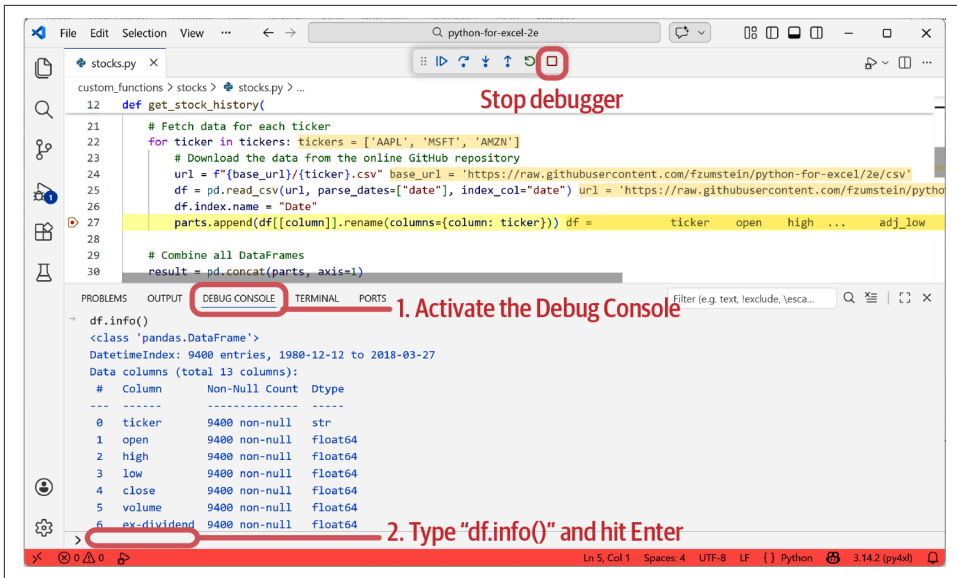


Figure 11-9. Using the Debug Console while the code is paused at a breakpoint



Excel Is Waiting for Another Application

If you keep your program paused for more than ninety seconds on a breakpoint, Excel will show you a pop-up saying that “Microsoft Excel is waiting for another application to complete an OLE action.” This shouldn’t have an impact on your debugging experience other than having to confirm the pop-up to make it disappear once you’re done with debugging. Note that if you click it away while continuing with your debugging session, it’ll come back later.

To finish this debugging session, click on the Stop button in VS Code (see [Figure 11-9](#)) and make sure to uncheck the Debug UDFs setting again in the xlwings ribbon add-in. If you forget to uncheck the Debug UDFs setting, your functions will return an error the next time you recalculate them.

At this point, I’ve shown you the most commonly used features of custom functions. The next section will touch on a few ways to make your custom functions performant.

Performance Optimization

If you use many custom functions in your workbook, performance may suffer. This section starts by showing you how to minimize cross-application calls, the easiest way to improve performance. We then look at caching, an additional performance optimization technique that we can use with custom functions. At the end of this section, I will introduce you to the `script` decorator that you can use as an alternative to the `RunPython` call if you are exclusively working on Windows.

Minimize Cross-Application Calls

As you probably recall from [Chapter 8](#), cross-application calls, i.e., calls between Excel and Python, are relatively slow, so the fewer custom functions you have, the better. You should therefore work with dynamic arrays whenever you can. When you are working with pandas DataFrames, you're automatically returning dynamic arrays. However, since dynamic arrays are relatively new, you may still gravitate toward using Excel in its traditional single-cell mode. Consider the example in [Figure 11-10](#) that calculates total revenues as the sum of a given Base Fee plus a variable fee determined by Users times Price.

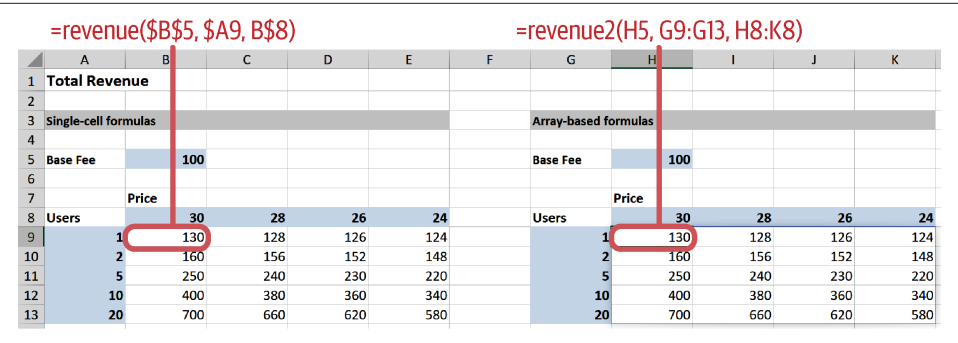


Figure 11-10. Single-cell formulas (left) versus array-based formulas (right)

Single-cell formulas

The left table in [Figure 11-10](#) uses the formula `=revenue($B$5, $A9, B$8)` in cell B9. This formula is then applied to the whole range B9:E13. This means that you have 20 single-cell formulas that call the `revenue` function.

Array-based formulas

The right table in [Figure 11-10](#) uses the formula `=revenue2(H5, G9:G13, H8:K8)`. If you don't have dynamic arrays in your version of Excel, you would need to add the decorator `@ret(expand="table")` to the `revenue2` function. Unlike the single-cell formula, this version only calls the `revenue2` function once.

You can see the Python code for the two custom functions in [Example 11-5](#), and you'll find the source file under *custom_functions/revenues/revenues.py* in the companion repository.

Example 11-5. revenues.py

```
import numpy as np
from xlwings import arg, func

@func
def revenue(base_fee, users, price):
    return base_fee + users * price

@func
@arg("users", np.array, ndim=2)
@arg("price", np.array)
def revenue2(base_fee, users, price):
    return base_fee + users * price
```

When you change the base fee in cell B5 or H5, respectively, you will see that the right example will be much faster than the left one. The differences in the Python functions are minimal and only differ in the argument decorators: the array-based version reads in users and prices as a NumPy array—the only caveat here is to read in users as a two-dimensional column vector by setting `ndim=2` in the argument decorator. You probably remember that NumPy arrays are similar to DataFrames but without an index or header and with only one data type, but if you want a more detailed refresher, have another look at [Chapter 4](#).

When working with custom functions, you have another easy option to improve performance: prevent repeated calculations of slow functions by caching their results.

Caching

When you call a *deterministic* function, i.e., a function that given the same inputs, always returns the same output, you can store the result in a *cache*: repeated calls of the function don't have to wait for the slow calculation anymore but can take the result from the cache where it's already precalculated. This is best explained with a short example. You can program a very basic caching mechanism with a dictionary:

```
In [4]: import time

In [5]: cache = {}

def slow_sum(a, b):
    key = (a, b)
    if key in cache:
```

```

        print(f"Returning cached result for {key}")
        return cache[key]
    else:
        print(f"Calculating result for {key}")
        time.sleep(2) # sleep for 2 seconds
        result = a + b
        cache[key] = result
        return result

```

When you call this function for the first time, the cache is empty. The code will therefore execute the `else` clause with the two seconds pause that mimics a slow calculation. After performing the calculation, it will add the result to the cache dictionary before returning the result. When you now call this function a second time with the same arguments and during the same Python session, it will find it in the cache and return it right away, without having to perform the slow calculation again. Caching a result based on its arguments is also called *memoization*. Accordingly, you will see the time difference when you call the function for the first and second time:

```

In [6]: %%time
        slow_sum(1, 2)

Out[6]: Calculating result for (1, 2)
        CPU times: user 1.05 ms, sys: 988 µs, total: 2.04 ms
        Wall time: 2 s

Out[6]: 3

In [7]: %%time
        slow_sum(1, 2)

Out[7]: Returning cached result for (1, 2)
        CPU times: user 43 µs, sys: 6 µs, total: 49 µs
        Wall time: 47.2 µs

Out[7]: 3

```

To make your life easier, Python offers a built-in decorator called `cache` that you import from the standard library's `functools` module. We can use this decorator with our `get_stock_history` function from the last section. As long as we're only querying historical values, we can safely cache the result. Here are the first few lines of the `get_stock_history` function with the required changes to apply caching:

```

from functools import cache ❶

import pandas as pd
import xlwings as xw
from xlwings import arg, func

@cache ❷
@func(call_in_wizard=False)
@arg("tickers", tuple, ndim=1, doc="A range of tickers.") ❸
@arg("start_date", doc="A date-formatted cell")
@arg("end_date", doc="A date-formatted cell")

```

```

@arg("rebase", doc="If TRUE, starts all time series at 100")
@arg("column", doc="Defines the column name. Default: adj_close")
def get_stock_history(
    tickers, start_date=None, end_date=None, rebase=False, column="adj_close"
):
    ...

```

- ❶ Import the cache decorator.
- ❷ Use the decorator. The decorator has to be on top of the @func decorator.
- ❸ By default, tickers is a list. This creates a problem in this case as functions with lists as arguments can't be cached. The underlying issue is that lists are mutable objects that can't be used as keys in dictionaries; see [Appendix C](#) for more information about mutable versus immutable objects. Using the tuple converter allows us to retrieve tickers as tuple rather than as list, which solves our problem.

You'll find the complete source code under *custom_functions/stocks/stocks_cache.py* in the companion repository.

Now that you know how to tweak the performance of custom functions, let's wrap this section up by introducing the script decorator.

The Script Decorator

This section isn't about custom functions, but it uses the same technology and workflow as custom functions, which is why I put it here. In [Chapter 9](#), I showed you how to speed up the RunPython call by activating the Use UDF Server setting. If you are living in a Windows-only world, there is an alternative to the RunPython/Use UDF Server combination in the form of the script decorator. This will allow you to import your Python functions as Sub procedures into Excel, without having to manually write any RunPython calls. In Excel, you will need a sub procedure to be able to attach it to a button—a VBA function, as you get it when using the @func decorator, won't work. To try this out, create a new quickstart project called `import_script`:

Windows (Terminal)

```

cd %userprofile%\python-for-excel-2e
uv run xlwings quickstart import_script

```

macOS (Terminal)

```

cd ~/python-for-excel-2e
uv run xlwings quickstart import_script

```

Once created, feel free to move the *import_script* directory anywhere you like. Navigate to the *import_script* folder and open *import_script.xlsm* in Excel and *import_script.py* in VS Code, then decorate the *main* function with `@script`, as shown in [Example 11-6](#).

Example 11-6. import_script.py (excerpt)

```
import xlwings as xw
from xlwings import script ❶

@script ❷
def main():
    wb = xw.Book.caller()
    sheet = wb.sheets[0]
    if sheet["A1"].value == "Hello xlwings!":
        sheet["A1"].value = "Bye xlwings!"
    else:
        sheet["A1"].value = "Hello xlwings!"
```

- ❶ Import `script` from `xlwings`.
- ❷ Add the `@script` decorator to the *main* function.

In the *xlwings* add-in, click on *Import Functions* before hitting `Alt+F8` to see the available macros: in addition to the *SampleCall* that uses *RunPython*, you'll now also see a macro called *main*. Select it and click the *Run* button—you'll see the familiar greeting in cell A1. You could now go ahead and assign the *main* macro to a button, as we did in [Chapter 9](#). With `script`, we have met all *xlwings* decorators—I have summarized them again in [Table 11-1](#).

Table 11-1. xlwings decorators

Decorator	Description
<code>@func</code>	Put this decorator on top of all functions that you want to import into Excel as custom functions so you can call them from Excel cells.
<code>@script</code>	Put this decorator on top of all functions that you want to import into Excel in order to bind them to buttons.
<code>@arg</code>	Apply converters and options to arguments, e.g., add a docstring via the <code>doc</code> argument, or you can have a range arrive as <code>DataFrame</code> by providing <code>pd.DataFrame</code> as the first argument (this assumes that you have imported <code>pandas</code> as <code>pd</code>).
<code>@ret</code>	Apply converters and options to return values, e.g., suppress a <code>DataFrame</code> 's index by providing <code>index=False</code> .

For more details on these decorators, have a look at the [xlwings documentation](#).

Alternatives to xlwings

There are other solutions with similar functionality to xlwings that you may want to check out:

- **PyXLL** is a commercial offering and one of the oldest solutions available. It uses Excel's C API, which makes their implementation of custom functions very performant.
- **xlOil** is similar to PyXLL in scope and speed but is open source and free.
- **xlSlim** offers functionality similar to PyXLL and xlOil, as well as features comparable to Python in Excel. It is available in both free and commercial versions.

Conclusion

This chapter was about writing Python functions and importing them into Excel as custom functions. You learned how to influence the behavior of the function arguments and return values by using the `@arg` and `@ret` decorators, respectively. The last part showed you a few performance tricks and introduced the `@script` decorator, which you can use as a `RunPython` replacement.

The nice thing about writing custom functions in Python is that it allows you to replace long and complex cell formulas with Python code that will be easier to understand and maintain. My preferred way to work with custom functions is to use DataFrames together with dynamic arrays, a combination that makes it easy to work with dynamic source data.

Having reached the end of **Part IV**, we've met a wide range of xlwings features and use cases. However, there's one big challenge: xlwings requires a local Python installation, and that can be a major hassle for nontechnical end users. The next chapter, which starts **Part V**, introduces a solution to this problem: xlwings Lite.

PART V

xlwings Lite

Scripts with xlwings Lite

Distributing xlwings tools like those we built in [Part IV](#) is a big challenge, as they require Python and the necessary packages to be installed on each end user's machine. xlwings Lite has solved this challenge by running Python inside a modern Excel add-in instead of depending on a local installation. Users only need to install xlwings Lite from Excel's add-in store and they're all set.

The design goal for xlwings Lite is to make it behave as closely as possible to the VBA experience, for developers and end users alike. Like VBA, xlwings Lite stores the Python code directly in the workbook, making the installation of the xlwings Lite add-in the only requirement for end users. And you, as a developer, should feel right at home: the code editor feels similar to the VBA editor—you can even use the F5 key to run code.

To run xlwings Lite, you need at least Office 2021 (permanent license) or the Excel installation that comes with your Microsoft 365 subscription. However, even if you are using an older version of Excel, you can still work through this chapter's examples by using Excel on the web. Even better, it also works with the free version of Excel—simply [sign up](#) for a free Microsoft account and you're ready to install xlwings Lite.

I'll start this chapter by explaining how xlwings Lite works before walking you through its installation. After a quick tour of the add-in, we'll analyze the NYC taxi dataset. On the way, we'll meet a few exciting modern technologies, including DuckDB, Parquet files, and Python's Asyncio library. We'll wrap this chapter up by looking at the particularities and limitations that come with xlwings Lite.

Getting Started with xlwings Lite

In this section, I'll give you a brief overview of how xlwings Lite works before walking you through the installation of the add-in. You'll then run your first script in xlwings Lite—from the xlwings Lite user interface as well as from a button on a sheet.

How xlwings Lite Works

xlwings Lite is a modern Office.js add-in that is powered by **Pyodide**, a Python distribution based on WebAssembly (Wasm). That was quite a mouthful, so let's see what all this means!

Over the years, Microsoft has introduced a variety of technologies to build Excel add-ins: there are VBA (Visual Basic for Applications), VSTO (Visual Studio Tools for Office), COM (Component Object Model), and XLL (Excel-specific Dynamic Link Library) add-ins. While their technology differs, they all have one thing in common: none of them supports Excel on the web or on mobile devices, so Microsoft had to come up with yet another technology called *Office.js* that—in its current form—was released with Office 2016. Office.js is a JavaScript library that enables the creation of cross-platform add-ins running in a browser. This means that the add-in's source files have to be hosted on a server or in the cloud. The browser is a sandbox and doesn't have access to anything on your computer by default, making Office.js add-ins much more secure than VBA. The only major security concern is that malicious add-ins can manipulate your active workbook and send out its content. Like apps for your smartphone, Office.js add-ins can be published in the Office add-in store from which you can install them with a single click.

Wasm is a technology that browsers started to support in 2017. It allows a variety of programming languages, including Python, to run in the browser at near-native speed. A year later, in 2018, Pyodide was born. Pyodide is a Python distribution that runs on Wasm and that supports most of the scientific packages, including NumPy, pandas, scikit-learn, and Matplotlib.

xlwings Lite now simply connects the two sides: it's an Office.js add-in that talks to Pyodide so you can use Python to automate Excel and write custom functions. Unlike Microsoft's cloud-based Python in Excel solution, xlwings Lite runs Python directly on your own computer, keeping your data private. An internet connection is only required to initially load the add-in, as shown in **Figure 12-1**.

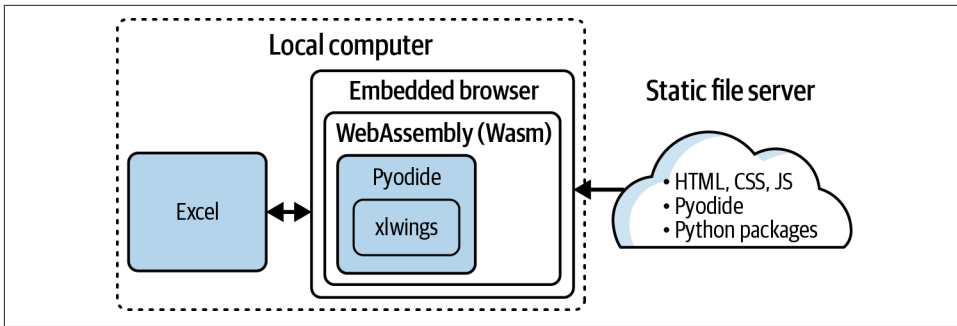


Figure 12-1. How xlwings Lite works

Now that you know how xlwings Lite works, let's go ahead and install it!

Installation

This section introduces you to the Excel add-in store, from which you can install xlwings Lite with a single click. To get started, follow the steps shown in Figure 12-2.

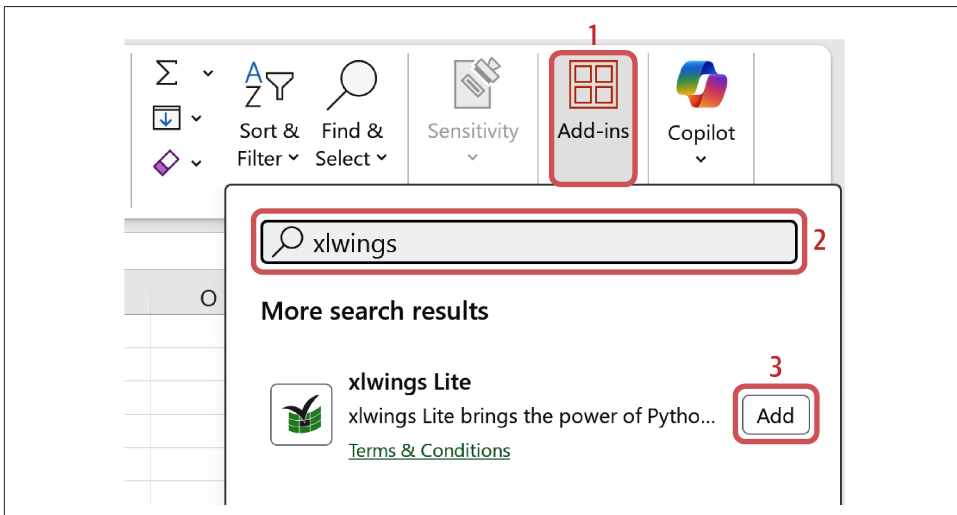


Figure 12-2. The Home tab of the ribbon with the Add-ins menu

1. On the Home tab of your Excel ribbon, click the Add-ins button. On older versions of Excel, you may instead find it under File > Get Add-ins or on the Insert tab of the ribbon.
2. Search for xlwings.
3. On the xlwings Lite entry, click on Add to install the add-in.

In general, Microsoft changes the location and design of the Add-ins menu regularly, so you might need a bit of flexibility with these instructions. If everything fails, you can also find xlwings Lite on the [Microsoft Marketplace](#), from which you can install it by clicking on Get it now.

If you wanted to uninstall xlwings Lite later, start again by clicking on the Add-ins button, then click on More Add-ins at the bottom of the appearing menu. Under Manage Your Apps, you'll find xlwings Lite. Click on the three dots to the right and select Remove. On older versions of Excel, you have to go to My Add-ins instead of Manage Your Apps, then hover over xlwings Lite, which makes the three dots appear.



Installing xlwings Lite in a Corporate Environment

Most companies have the Excel add-in store blocked for the simple reason that add-ins have access to your workbook's data and could send them out if they are programmed to do so or if they get hacked. Therefore, to get access to xlwings Lite in your company, you have to talk to your IT desk or Microsoft Office administrator: they can install xlwings Lite in the Microsoft 365 admin center.

And that's it—there's no need to install Python or to go through any of the other steps from [Chapter 2](#). Once the installation completes, you'll find the xlwings Lite button on the Home tab of your ribbon (see [Figure 12-3](#)).

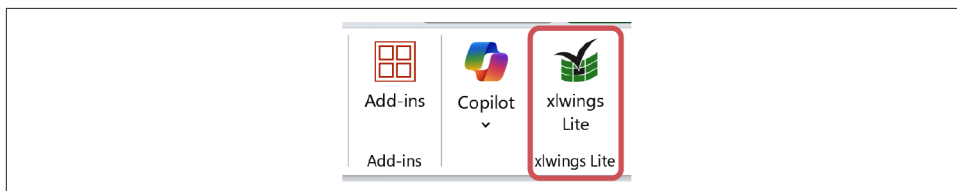


Figure 12-3. The xlwings Lite button in the Home tab of the Excel ribbon

You can click the xlwings button anytime to show the xlwings user interface—let me walk you through it in the next section!

A User Interface Tour

Clicking the xlwings Lite button on the Home tab of the Excel ribbon opens the xlwings Lite user interface, also called *task pane*, on the right-hand side of Excel, as shown in [Figure 12-4](#).

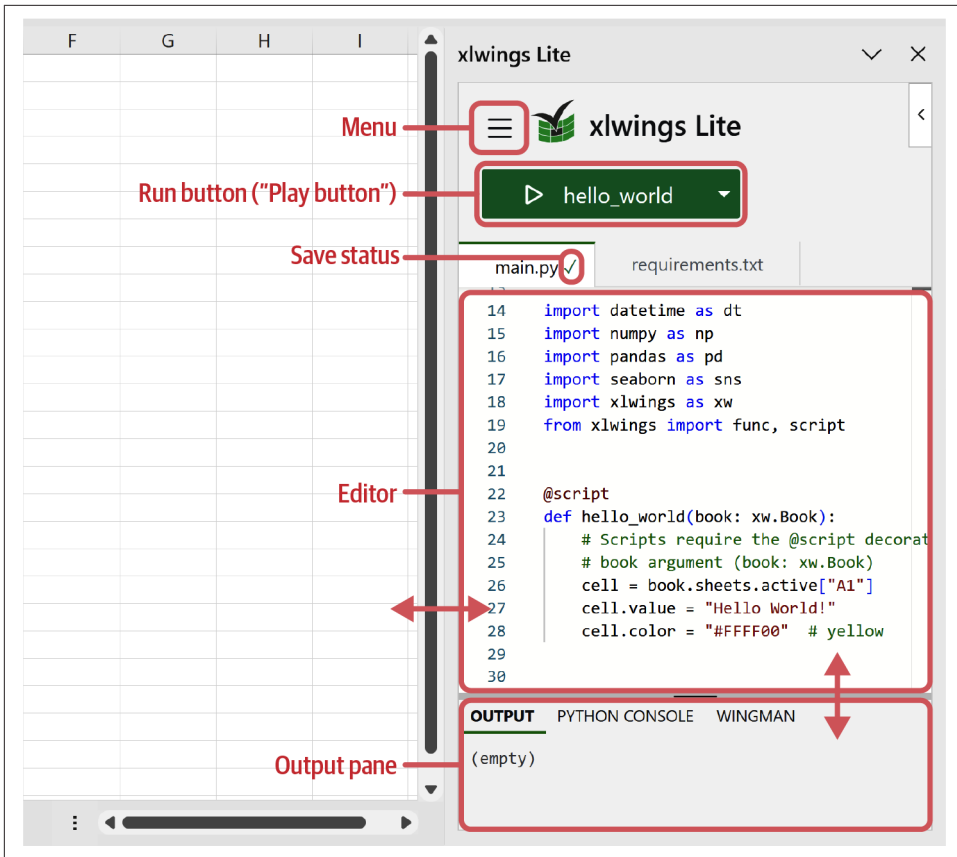


Figure 12-4. The xlwings Lite user interface (“task pane”)

Let’s go through the main parts from top to bottom:

Menu

Click the hamburger menu icon to manage environment variables, change settings, restart xlwings Lite, etc.

xlwings logo

Click the logo to return to the start screen from any other screen.

Run button

The run button, also called the *play button*, allows you to run functions that have the `@script` decorator. If you have multiple scripts, you can select one from the dropdown before clicking the button. While the xlwings Lite user interface is the active window, you can also use the keyboard shortcut F5 to run the selected script.

Editor

The editor is the same editor that VS Code uses. Press the F1 key to search for commands such as “Replace” or “Transform to Uppercase.” You can also use many of the same keyboard shortcuts. For example, use Alt+↑ to move a line up or Ctrl+Shift+K to delete an entire line of code. Or, to comment out one or various lines of code, use Ctrl+/. As usual, on macOS, use Option instead of Alt, and Command instead of Ctrl in these keyboard shortcuts. Whenever you edit the code, it is automatically saved to the workbook. The green checkmark in the editor tab means that everything was successfully saved.

Output pane

The output pane displays anything you print in your code. This is also the place where the output of `df.info()` shows up, assuming `df` is a pandas DataFrame. You can make the output pane bigger by clicking on its upper border and dragging it up.

You can drag the left border of the xlwings task pane toward the middle of your Excel workbook to make it bigger. You can even drag the xlwings Lite add-in into a separate window by grabbing it in the gray title bar and pulling it out. Now that you are familiar with the xlwings Lite add-in, let’s use it to run your first script!

Running Your First Script

Create a new workbook and open the xlwings Lite add-in by clicking the button on the Home tab of the Excel ribbon. The editor comes with a few sample functions in the *main.py* file to make it easy to try everything out. We, however, start from scratch, so click into the code and select everything via Ctrl+A (Windows) or Command-A (macOS), then delete it. Now write the following code into the *main.py* file (the name of the file is shown in the tab of the editor):

```
import xlwings as xw
from xlwings import script

@script
def hello_world(book: xw.Book):
    book.sheets.active["A1"].value = "Hello xlwings!"
```

The code looks almost the same as what we wrote in [Chapter 9](#), but it’s slightly more modern. Here are the two important bits:

The script decorator

Whenever you decorate a function with `@script`, it will show up in the run button on top of the editor (see [Figure 12-4](#)). If you have more than one function with the `@script` decorator, you can select the script that you want to run by clicking the dropdown on the right-hand side of the run button. The `@script`

decorator will also allow you to bind the function to a button on the sheet, as we'll see in the next section.

The book parameter

You get access to the active Book object via the mandatory function parameter `book`. The type hint `xw.Book` is required—if you need a refresher on type hints, go back to [Chapter 3](#). While the type hint is required, you can call the parameter anything you want. For example, if you prefer `wb` instead of `book`, write `wb: xw.Book`.

Now click the play button to run the `hello_world` function. As expected, this writes “Hello xlwings!” into cell A1 of your active worksheet. But what if you want to run the function from a button on a sheet? The next section shows you how this works.

Sheet Buttons

Since Office.js add-ins don't offer a native way to trigger a function from a button on a sheet, xlwings Lite has to use a bit of trickery. Here's how it works:

1. On the Excel ribbon, go to Insert > Shapes and select, e.g., a rounded rectangle. Then draw the shape on the sheet. Adjust colors and text to your liking. For this example, make sure that the button covers cells B4 and B5, for reasons we'll see in a minute.
2. Select the shape. In the Name box (on the left of the formula bar), give it a specific name, then hit Enter, as shown in [Figure 12-5](#). For this sample, let's use the name `hello_button`.

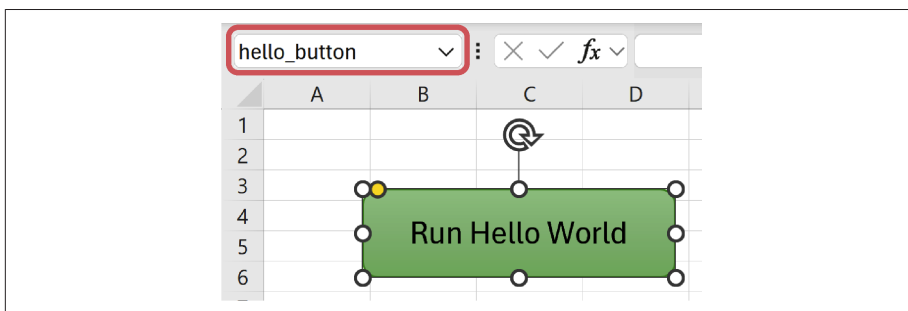


Figure 12-5. Select the shape and assign the name “hello_button”

3. Right-click on the shape and select Link (Windows) or Hyperlink (macOS). On the tab Place in This Document (Windows) or This Document (macOS), in the field with the title “Type the cell reference,” write the name of a cell that is covered by the shape, e.g., B4. Confirm by clicking OK.
4. Provide the following arguments in the `@script` decorator:

```
@script(button="[hello_button]Sheet1!B4", show_taskpane=True)
def hello_world(book: xw.Book):
    book.sheets.active["A1"].value = "Hello xlwings!"
```

The argument `show_taskpane=True` is optional; it opens the task pane if you closed it previously. This is useful because the task pane displays a banner while the script is running so the user knows something is happening.

Now click on the button to run the script again. If you encounter an issue, double-check that the button in Excel is called the same as in the `button` argument of the `@script` decorator, make sure B4 isn't already selected when you click the button, and restart xlwings Lite by going to Menu > Restart.

How does it work? When xlwings Lite saves the Python code, it registers an event handler that runs the script when the referenced cell (B4) is selected. This happens when you click the button as we have set up a hyperlink. Immediately after B4 is selected, the event handler selects the cell below it (B5) to be ready for the next call. This is also the reason why I asked you to place the button over cells B4 and B5, so that all this is hidden away from the user. If you want to change the cell reference, make sure to change it both in the hyperlink of the button and the `button` argument of the `@script` decorator.

Since xlwings Lite uses the same xlwings package that we worked with in [Part IV](#), I don't need to explain any more basics, and you are ready to tackle a real-world project!

Analyzing the NYC Taxi Dataset

In this section, we're going to analyze data from 3.5 million taxi trips directly in Excel. Along the way we'll meet a few exciting modern technologies, including Parquet files, DuckDB, and Python's Asyncio library. Let's start by exploring the dataset and then move on to generating a report with hourly statistics.

Exploring the Dataset with DuckDB

The New York City Taxi and Limousine Commission (TLC) publishes a dataset on [NYC.gov](#) called *Trip Record Data* that I will refer to as the *NYC taxi dataset* from here on. It includes every single taxi trip with details such as distance and fees and comes without usage restrictions other than the website's [Terms of Use](#). The data is published on a monthly basis and split into various files, e.g., one for yellow taxis and one for green taxis. In this analysis, we're going to work with the yellow taxi dataset from January 2025, which contains around 3.5 million taxi rides. The files are published as Parquet files (see the sidebar).

Apache Parquet File Format

Apache Parquet is a columnar storage file format offering efficient analytical queries and small file sizes. Another advantage is that it preserves data types unlike, e.g., CSV files, which store everything as text. It has therefore become the de facto standard for storing large datasets. It's open source and supported by most programming languages.

Let's begin our analysis by opening the file *taxi.xlsx* that you'll find in the *lite* directory of the companion repository. Then start the xlwings Lite add-in via the button on the Home tab of the Excel ribbon.



Do You Trust the Authors of This Workbook?

When you open an xlwings Lite workbook for the first time, you will have to click Trust and Continue in order to use it. This is similar to the warning that VS Code shows when you open a folder for the first time.

Finally, run the `explore` script by clicking the Play button. If the Play button doesn't show `explore`, click on the dropdown first to select it. This will download the Parquet file with the NYC taxi dataset from January 2025, which is roughly 60 MB, so depending on your internet connection, this will take a moment when you run this the first time. Once the file has been downloaded, it will print a couple of tables in the output pane. But before looking at the output, let's go through the code of the `explore` script that you'll find in the *main.py* file (in the xlwings Lite editor):

```
import duckdb

@script
async def explore(book: xw.Book):
    # Load the Parquet file into a DuckDB relation
    taxi = await load_parquet(TAXI_DATA_URL)

    # Run queries
    duckdb.sql("SELECT count(*) FROM taxi").show()
    duckdb.sql("DESCRIBE SELECT * FROM taxi").show()
    print(duckdb.sql("SELECT * FROM taxi LIMIT 1").df().squeeze())
```

As you can see, we're using DuckDB here! I briefly introduced DuckDB at the end of [Chapter 5](#) as a modern alternative to pandas, so you may remember that DuckDB is a file-based database similar to SQLite but optimized for analytical workloads. The queries we're using in this chapter are deliberately simple so you can understand them even without being proficient in SQL. We could, of course, also use pandas to analyze our dataset, but DuckDB is much more efficient with datasets of this size and

works well within the limitations of Pyodide—I’ll discuss the limitations at the end of this chapter.

There is one DuckDB concept that requires a special introduction: in the `explore` function, we are calling another function called `load_parquet`. Please ignore the `async/await` keywords for now—I will introduce you to Python’s Asyncio library a little later in this chapter. The `load_parquet` function returns a DuckDB *relation*. A relation is a symbolic representation of the query but does not yet run the query. This only happens when a result is requested, in our case, by calling the `show` or `df` methods. While `show` is a method of a DuckDB relation to print a summary of the data, `df` converts the result into a pandas DataFrame. A key characteristic of a relation is that you can use it directly in the SQL of subsequent queries. This is the reason why we can write `SELECT count(*) FROM taxi` in the `explore` function as `taxi` from the previous code line is a DuckDB relation.



DuckDB Relational API

Instead of writing SQL, DuckDB also offers a **relational API**, which allows you to write object-oriented queries. To give you an idea how this works, here is a simple query using the SQL interface as we used it in our `explore` function:

```
taxi = duckdb.read_parquet("../")
duckdb.sql("SELECT count(*) FROM taxi").show()
```

The same query using the relational API looks like this:

```
taxi = duckdb.read_parquet("../")
taxi.count("*").show()
```

In this book, I am sticking to the SQL interface as it’s more universal and easier to get started.

Let’s now get back to the output pane and have a closer look at the tables that the `explore` function printed. The first table shows that there were 3,475,226 yellow cab trips in January 2025:

count_star()
int64
3475226

The second table gives an overview of the database columns with some meta information:

column_name varchar	column_type varchar	null varchar	key varchar	default varchar	extra varchar
VendorID	INTEGER	YES	NULL	NULL	NULL
tpep_pickup_datetime	TIMESTAMP	YES	NULL	NULL	NULL
tpep_dropoff_datetime	TIMESTAMP	YES	NULL	NULL	NULL
passenger_count	BIGINT	YES	NULL	NULL	NULL
trip_distance	DOUBLE	YES	NULL	NULL	NULL
RatecodeID	BIGINT	YES	NULL	NULL	NULL
store_and_fwd_flag	VARCHAR	YES	NULL	NULL	NULL
PULocationID	INTEGER	YES	NULL	NULL	NULL
DOLocationID	INTEGER	YES	NULL	NULL	NULL
payment_type	BIGINT	YES	NULL	NULL	NULL
fare_amount	DOUBLE	YES	NULL	NULL	NULL
extra	DOUBLE	YES	NULL	NULL	NULL
mta_tax	DOUBLE	YES	NULL	NULL	NULL
tip_amount	DOUBLE	YES	NULL	NULL	NULL
tolls_amount	DOUBLE	YES	NULL	NULL	NULL
improvement_surcharge	DOUBLE	YES	NULL	NULL	NULL
total_amount	DOUBLE	YES	NULL	NULL	NULL
congestion_surcharge	DOUBLE	YES	NULL	NULL	NULL
Airport_fee	DOUBLE	YES	NULL	NULL	NULL
cbd_congestion_fee	DOUBLE	YES	NULL	NULL	NULL
20 rows					6 columns

For our purposes, we can focus on `column_name` and `column_type` and ignore the other meta information like `null`, `key`, `default`, and `extra`. For the report that we're creating in the next section, we're going to work with the following two fields (the description of these fields is taken directly from the [Yellow Trips Data Dictionary](#)):

- `tpep_pickup_datetime`: the date and time when the meter was engaged
- `trip_distance`: the elapsed trip distance in miles reported by the taximeter

And finally, the third table shows a sample record so you get a better feel for what the data looks like:

VendorID	1
tpep_pickup_datetime	2025-01-01 00:18:38
tpep_dropoff_datetime	2025-01-01 00:26:59
passenger_count	1
trip_distance	1.6
RatecodeID	1
store_and_fwd_flag	N
PULocationID	229
DOLocationID	237

payment_type	1
fare_amount	10.0
extra	3.5
mta_tax	0.5
tip_amount	3.0
tolls_amount	0.0
improvement_surcharge	1.0
total_amount	18.0
congestion_surcharge	2.5
Airport_fee	0.0
cbd_congestion_fee	0.0

Name: 0, dtype: object

To print this table, I didn't use DuckDB's `show` method as this would print a very wide table. Instead, I am using the `df` method to turn the result into a pandas DataFrame. Calling the `squeeze` method on a DataFrame with a single row turns it into a Series, which enables us to print a full record.

Now that you have a better idea of the NYC taxi dataset, let's use DuckDB to analyze how trips and distances vary throughout the day and produce an Excel report with our findings!

Analyzing Trips and Distance by Hour

Our *taxi.xlsx* workbook offers a second script called `report`. Select `report` from the dropdown of the play button, then click the button to run it. Here is what the script does:

- It inserts a new sheet.
- It inserts an Excel table into the new sheet showing the Number of Trips and Average Distance (miles) aggregated by Hour.
- It also inserts a Matplotlib plot with two subplots to visualize the numbers from the table.

The final report is shown in [Figure 12-6](#). You can read a few interesting details: for example, during rush hour at 6 p.m., trips are short, while the number of trips reaches the maximum. At 5 in the morning, on the other hand, only a few people are taking a cab, but those trips cover longer distances on average—at this hour, a relatively high percentage of trips are to or from the airport.

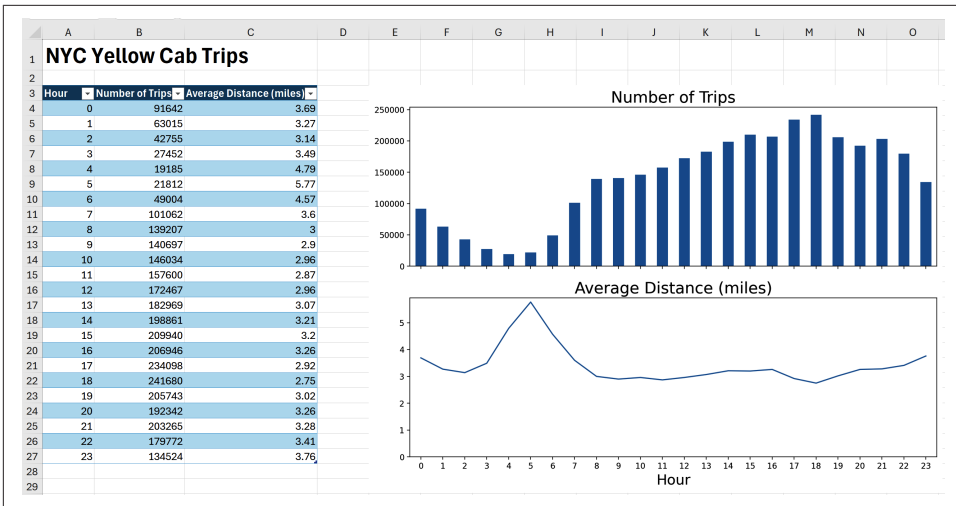


Figure 12-6. The Excel report

After running the report function, the xlwings Lite output pane shows something along the following lines:

```
Using existing file yellow_tripdata_2025-01.parquet
Processed 3,362,072 records in 0.30 seconds
```

This is a very impressive performance on its own and much more so for anything running in Excel! To put this into context: doing the same analysis in Power Query with a CSV file (Power Query doesn't support Parquet files) takes around 50 seconds on my machine. Let's now have a look at the core of the report function (for the full function, see the *main.py* file in the xlwings Lite editor):

```
# SQL query: Group trips by hour and calculate statistics
query = """
SELECT
    hour(tpcp_pickup_datetime) AS "Hour",
    count(*) AS "Number of Trips",
    round(avg(trip_distance), 2) AS "Average Distance (miles)"
FROM taxi
WHERE trip_distance BETWEEN 0.1 AND 50
GROUP BY "Hour"
ORDER BY "Hour"
"""

# Execute query and convert to pandas DataFrame
df = duckdb.sql(query).df()
```

This SQL query is straightforward, and I'd argue that it almost reads like plain English. One thing I need to explain, though, is the WHERE clause. Since the taxi dataset is raw data, it comes with data errors caused by, e.g., incorrect GPS signals

or taxi drivers who forgot to switch off their taximeters. Therefore, we're filtering outliers to keep only reasonable taxi trip lengths. Feel free to remove the `WHERE` clause and see how it distorts the average distance plot. After the SQL query, we're turning the result into a pandas DataFrame by using the `df` method, as this allows us to send it to Excel easily and use pandas' plot interface in the `create_plot` function:

```
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.figure import Figure

def create_plot(df: pd.DataFrame) -> Figure:
    # Share the x-axis between both subplots via sharex=True
    fig, axes = plt.subplots(nrows=2, ncols=1, figsize=(12, 8), sharex=True)

    # Plotting
    df = df.set_index("Hour")
    df["Number of Trips"].plot.bar(ax=axes[0])
    df["Average Distance (miles)"].plot.line(ax=axes[1])

    # Set titles, font size, and make sure y-axis starts at 0
    axes[0].set_title("Number of Trips", fontsize=20)
    axes[1].set_title("Average Distance (miles)", fontsize=20)
    axes[1].set_xlabel("Hour", fontsize=18)
    axes[1].set_ylim(bottom=0)

    return fig
```

At this point, you know how it feels to use an `xlwings` Lite tool and how efficiently you can query a Parquet file with DuckDB. What I haven't discussed yet are the challenges of querying web APIs (to download the Parquet file) and installing Python packages (to install DuckDB). Both topics have a few quirks when working with `xlwings` Lite, which we'll explore in the next section.

Platform Specifics

In this section, we'll explore two key aspects of working with `xlwings` Lite that come with a few caveats: making web API requests and installing Python packages. Because `xlwings` Lite runs on Pyodide, it inherits several of its limitations and behaviors. The biggest ones are that web API requests require the server to set Cross-Origin Resource Sharing (CORS) headers and that Python packages need to be either pure Python packages or specifically compiled for Pyodide. We'll also cover how `xlwings` Lite handles the installation of dependencies.

Making Web API Requests

As mentioned earlier, Pyodide, the Python distribution that powers `xlwings` Lite, runs in an embedded browser on your own computer. As a result, it also inherits

the browser's restrictions, some of which become apparent when you work with web APIs. Let's have a look at the `load_parquet` function that you'll find in the `main.py` file in `taxi.xlsx`:

```
from pathlib import Path
from pyodide.http import pyfetch

async def load_parquet(url: str) -> duckdb.DuckDBPyRelation:
    """Read a Parquet file into a DuckDB Relation"""
    # Extract filename from the end of the URL
    filename = url.split("/")[-1]

    # Download only if the file doesn't already exist
    if not Path(filename).exists():
        print(f"Downloading {filename}...")
        response = await pyfetch(url)
        data = await response.bytes()
        with open(filename, "wb") as f:
            f.write(data)
    else:
        print(f"Using existing file {filename}")

    # Return a DuckDB Relation
    return duckdb.read_parquet(filename)
```

There are a couple of concepts I need to explain: how Asyncio works, why we're using `pyfetch` instead of `Requests`, and what CORS headers are. Let's go through these topics one after another!

Asyncio

Asyncio is part of Python's standard library and stands for Asynchronous I/O, with I/O meaning input/output. Asyncio can be a complex topic, but for the purpose of `xlwings Lite`, you only need to know as much as I will explain in this section. Traditional Python code runs synchronously, executing one line after another. That's not efficient: when you make a network request, e.g., to a web API or a database, your CPU has nothing to do while waiting for a response from the web API server. While waiting, Python could do other tasks concurrently. This is exactly what asynchronous code allows you to do: perform other tasks while *awaiting* slow operations. To write asynchronous code, Python offers the Asyncio library, which roughly works like this:

- To mark a function as asynchronous, define it using `async def` instead of `def`.
- To call an async function, you need to put `await` in front of the function call, e.g., `await pyfetch(url)`.
- You can only use `await` inside a function, and that function must be an `async def` function defined as `async def`.

Here is how this translates to our case: `pyfetch` is an `async` function, so we need to `await` it. To be able to `await` it, we need to mark the `load_parquet` function as an `async` function. Now, to call `load_parquet` from our `explore` or `report` functions, we need to `await` it, and hence we need to turn `explore` and `report` into asynchronous functions by using `async def`.

pyfetch

You may be wondering, Why are we using `pyfetch` to download the Parquet file instead of `Requests` or `HTTPX` that I introduced in [Chapter 10](#)? Or pushing this even further, why aren't we just providing the URL directly to `duckdb.read_parquet(url)` since this works in a standard Python environment? The reason is that traditional HTTP clients like `Requests` or the one used by `read_parquet` use network features that aren't supported in a browser. To fix this, the most reliable solution—and therefore your first choice—is to use `pyfetch`, which is provided by `Pyodide`. If you need to write code that works on both traditional Python installations and `Pyodide`, you can use `Requests` or `HTTPX` as `Pyodide` provides patched versions for these.

Cross-Origin Resource Sharing (CORS)

For security reasons, browsers only allow you to make requests to the same origin, i.e., domain, where your application is hosted. If you want to access a resource, e.g., our Parquet file, from a different domain, it requires the server to add so-called `CORS` headers to their response. `CORS` stands for Cross-Origin Resource Sharing. Specifically, the server needs to set the header `Access-Control-Allow-Origin: https://addin.xlwings.org`, as this is the domain of the `xlwings` Lite add-in. Alternatively, to allow all pages to make requests, the server can set `Access-Control-Allow-Origin: *`. Since the original URL of the Parquet file doesn't provide these headers, the `xlwings` Lite add-in blocks the download. If a web API doesn't provide the correct `CORS` headers and you don't control the server, you can't access it from `xlwings` Lite except if you work with a `CORS` proxy—that's a server, which makes the request on your behalf and adds the required headers to the response. For our case, I have solved the issue by hosting the file on my own server where I can set the `CORS` headers.



Use Local Files

Instead of downloading the Parquet file programmatically, you could also download it first to your local machine, then go to `xlwings` Lite Menu > Files to import it. This will allow you to work with it by using a normal file path. If you're using the desktop version of `Excel`, you can even mount a local directory to access local files without having to import them first.

Besides web APIs, managing dependencies, i.e., Python packages, also requires a closer look.

Installing Python Packages

The code in *taxi.xlsx* depends on a few packages, most importantly DuckDB, pandas, and Matplotlib. So how does xlwings Lite know that it has to install them? xlwings Lite checks the *requirements.txt* file for a list of packages that are required and installs them when you open the xlwings Lite add-in. To look at *requirements.txt*, click on the respective tab in the xlwings Lite editor. Importantly, only the *requirements.txt* file is stored in the workbook but not the packages themselves. To avoid downloading them from the internet every time you open xlwings Lite, the browser caches the Python packages. To give you a better idea of how this works, go ahead and create a new Excel workbook, open the xlwings Lite add-in, then delete the default code and write `import duckdb` in the *main.py* file. You will immediately get an error in the Problems pane:

```
Cannot resolve imported module 'duckdb'
```

To fix this, go to the *requirements.txt* file by clicking the respective tab and add `duckdb` at the bottom. Wait a moment and xlwings Lite will automatically install it (see [Figure 12-7](#)). Once installed, make sure to restart xlwings Lite by going to the xlwings Lite Menu > Restart.

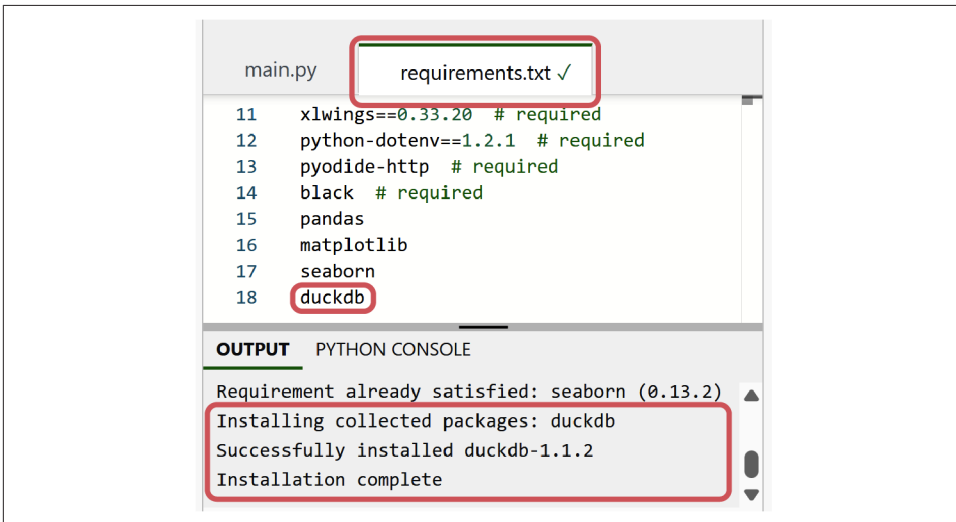


Figure 12-7. Adding an additional package (your version of duckdb may be different)

When you now switch back to the *main.py* file, the error is gone. So what's the matter with *requirements.txt*? To explain this, let's quickly remember what we did in [Chapter 2](#): back then, we were using `uv` to install Python and all of the third-party

packages. To do this, uv was relying on the file *uv.lock* to know which packages we needed in which version. While uv is still very new, Python also has a built-in package manager called *pip*. Instead of *uv.lock*, pip uses a text file that is—by convention—called *requirements.txt*. The file allows comments starting with a # symbol and lets you specify the exact version of a package like this:

```
# This is a comment
xlwings==0.33.16
```

Instead of using `==`, you can also use `>=`/`>` for a minimum version or `<=`/`<` for a maximum version. Pyodide uses its own package manager *micropip* that is able to install two types of Python packages:

- Pure Python wheels
- Wheels specifically built for Pyodide

First, micropip checks PyPI for a pure Python wheel (wheels are Python packages with the `.whl` extension) and installs it if one is available. If it doesn't find one, it checks Pyodide's own repository hosted on [jsDelivr](#). If it can't find a compatible Python package, as in the case for, e.g., `pytorch`, you'll see the following error message in the xlwings Lite output pane:

```
[...]
ValueError: Can't find a pure Python 3 wheel for: 'pytorch'
```

You will also get this error if the package itself is a pure Python package but one of its dependencies is not. If you deal with a package from Pyodide's own repository, as in the case of DuckDB, it's often only available in a single version. In this case, it's easier to add the package—at least initially—without a version specifier to *requirements.txt* as micropip will pick the correct one automatically. To know whether a package is provided by Pyodide, have a look at the list [Packages built in Pyodide](#).

While Pyodide makes a great effort to support more and more packages, most likely, it will never be able to support all of them. This isn't the only limitation when using Pyodide though—see the next section for an overview of what to expect.

Limitations

xlwings Lite is called “Lite” for a reason. While WebAssembly makes things possible in a browser that wouldn't have been imaginable just a few years ago, it still operates within the browser's security sandbox. This leads to the following limitations:

- Web API requests require the server to set the correct CORS headers (see the previous section).

- Some Python packages like PyTorch may never be supported by Pyodide because of their complexity (see the previous section).
- Python's multiprocessing and threading modules aren't supported. Pyodide often gets around this issue by building packages to run with a single thread and process—this, however, comes with a performance penalty.
- TCP connections aren't supported, which means that you can't directly connect to databases like PostgreSQL. You can, however, expose a database via a web API to solve this issue—some vendors like [Supabase](#) offer this out of the box. As we've seen, file-based databases like SQLite or DuckDB cause no issues.
- Pyodide has a memory limit of 2 GB or 4 GB depending on the Pyodide version and browser, but this may increase in the future.
- Pyodide currently doesn't offer a debugger, so you have to either use `print` functions for debugging or run your code locally first with the classic version of `xlwings`.

If you get bitten by any of these limitations, you may want to have a look at `xlwings Server`, which uses a full Python installation on a server (see the sidebar).

xlwings Server

`xlwings Server` is an enterprise solution that allows you to create modern Excel add-ins with a full Python installation running on your self-hosted server. Key features include:

- No need for a Python installation on the end user's machine
- Company-internal deployment of the Excel add-in via Microsoft 365 admin center
- Secure storage of your Python code on the server, inaccessible to end users
- Single sign-on (SSO) authentication and role-based access control (RBAC) for users via Microsoft Entra ID
- Secure handling of sensitive credentials by storing them on the server rather than on the end user's computer or in the Excel file
- Advanced custom functions, including streaming functions and object handles to, e.g., write an entire DataFrame to a single cell
- Support for task panes built with web technology

Conclusion

xlwings Lite makes it very easy to share Python-powered workbooks as users can install the add-in with a single click. That ease of use comes with certain limitations, though: some packages aren't available, you can't directly connect to databases, and you can only interact with web APIs that provide the proper CORS headers.

Despite these limitations, it still amazes me to see what's possible today with Python running in a browser. The NYC taxi dataset analysis was an impressive demonstration of how powerful things can become when you combine a modern analysis engine (DuckDB) with a modern file format (Parquet).

This chapter was a general introduction to xlwings Lite and showed you how to automate Excel by running scripts at the click of a button. The next chapter expands on this knowledge by teaching you how to write custom functions with xlwings Lite.

Custom Functions with xlwings Lite

Custom functions are Python functions that you use in Excel cells in the same way you use built-in functions like SUM or AVERAGE. If that sounds familiar, it's because I already introduced custom functions in [Chapter 11](#). The difference is that those were only compatible with Windows, whereas custom functions with xlwings Lite work across Windows, macOS, and Excel on the web.

We'll start this chapter with a simple Hello World function, and then I'll introduce a few modern twists, including how to use type hints for data conversion and namespaces for organizing custom functions. After that, we'll work through three projects:

- Building an AI function, which will show you how to handle sensitive credentials with environment variables
- Using a machine learning model from Hugging Face, which will demonstrate how far you can get with Pyodide, the restricted Python environment that powers xlwings Lite (see previous chapter)
- Working with weather data from Open-Meteo, which will teach you how to handle dates and plots

Most of what we learned in [Chapter 11](#) also applies to custom functions with xlwings Lite, but xlwings Lite makes use of modern Excel functionality such as returning plots as in-cell images. But let's not get ahead of ourselves—let's start with the basics!

Modern Custom Functions

In this section, I will show you how to use type hints instead of decorators to turn Excel values into DataFrames and other Python data types. We'll also look at how to deal with errors and how to organize functions into namespaces. But first things first: let's start with the familiar Hello World function.

Hello World

To write our Hello World function, create a new workbook and start xlwings Lite by clicking the button on the Home tab of the Excel ribbon. Like in the last chapter, select the default sample code in the editor via Ctrl+A (Windows) or Command-A (macOS) and delete it. Then write the following code:

```
from xlwings import arg, func

@func
def hello(name):
    return f"Hello {name}!"
```

Wait for the green checkmark to appear in the tab of the *main.py* file, which indicates that the file has been saved. Then, to test the function, select cell A1, type the formula **=HELLO("xlwings")**, and hit Enter. As expected, the formula will evaluate to Hello xlwings!.



Troubleshooting Custom Functions

If you get a #NAME? error when trying to call the HELLO function, your function hasn't been registered correctly. This happens occasionally, and you can usually resolve it by clearing your Office cache, as explained in the [xlwings Lite troubleshooting docs](#). If that doesn't help and if you have other add-ins from the add-in store installed, uninstall all add-ins first before reinstalling them again.

If you compare this version of the Hello World function with the one from [Chapter 11](#), you will notice a few differences:

- You don't need to change any Excel settings.
- You don't need to import the functions manually—functions are ready when *main.py* is saved.
- When you type **=HELLO(**, you get a tooltip showing the name of the function parameters.
- The lowercase Python function is automatically transformed into an uppercase name in Excel to follow the syntax conventions on each platform.

Now that you've seen the most basic custom function in action, you're going to learn how to use type hints to convert Excel values into specific Python data types such as NumPy arrays or pandas DataFrames.

Using Type Hints as Converters

In this section, I'll show you how you can use type hints to convert Excel cell values into specific Python data types. Type hints are a modern alternative to using decorators, as we did in [Chapter 11](#). To have something to work with, let's add our temperature conversion function from [Chapter 3](#) to the *main.py* file, below the hello function, as shown in [Example 13-1](#):

Example 13-1. Temperature conversion (version 1)

```
from xlwings import arg, func

@func
def convert_to_celsius(degrees, source="fahrenheit"):
    if source.lower() == "fahrenheit":
        return (degrees - 32) * (5 / 9)
    elif source.lower() == "kelvin":
        return degrees - 273.15
    else:
        raise ValueError(f"Don't know how to convert from {source}")
```



Type or Copy the Example Code

You can either type the `convert_to_celsius` function manually or copy/paste it from the *temperature_version1.py* file that you'll find in the *lite* directory of the companion repository. There, you'll also find all the other versions that we are going through here.

Now replicate what I have done in [Figure 13-1](#): write **Fahrenheit** into cell A3 and **Celsius** into cell B3, and add three values in the Fahrenheit column that you'd like to convert to Celsius, e.g., 0, 32, and 70. Then type **=CONVERT_TO_CELSIUS(A4,\$A\$3)** into cell B4 and hit Enter. To finish, drag the formula down to cells B5 and B6.

	B4	:	✕	✓	<i>fx</i>	=CONVERT_TO_CELSIUS(A4,\$A\$3)
	A	B	C	D	E	F
1	Hello xlwings!					
2						
3	Fahrenheit	Celsius				
4	0	-17.77777778				
5	32	0				
6	70	21.11111111				

Figure 13-1. Converting from Fahrenheit to Celsius

This worked and was straightforward! However, you may remember from [Chapter 11](#) that repeating the same formula in each cell is the old-school way from when Excel

didn't have dynamic arrays. So let's make our spreadsheet less error-prone and faster by using a single custom function instead of three individual ones.



Performance Optimization

All performance optimization tips from “[Performance Optimization](#)” on page 264 are also applicable with xlwings Lite.

One way to achieve this is by implementing a loop and returning a list, as shown in [Example 13-2](#):

Example 13-2. Temperature conversion (version 2)

```
from xlwings import arg, func

@func
def convert_to_celsius(degrees, source="fahrenheit"):
    if not isinstance(degrees, list):
        # Make sure the function also works with single cells.
        # We'll see how to handle this with the "ndim" option below.
        degrees = [degrees]

    results = []
    for degree in degrees:
        if source.lower() == "fahrenheit":
            results.append((degree - 32) * (5 / 9))
        elif source.lower() == "kelvin":
            results.append(degree - 273.15)
        else:
            raise ValueError(f"Don't know how to convert from {source}")

    return results
```

Delete the content of cells B4 to B6 and write the following formula into cell B4: **=CONVERT_TO_CELSIUS(A4:A6,A3)**. When you hit Enter, this will return a dynamic array with the result, but unfortunately, it's horizontal. Wrap the formula in cell B4 with Excel's TRANSPOSE function: **=TRANSPOSE(CONVERT_TO_CELSIUS(A4:A6,A3))** to solve this. However, it would be nice if the formula works for horizontal and vertical Excel ranges, without manually using the TRANSPOSE function. You may remember from [Chapter 4](#) that working with NumPy arrays is usually more efficient and makes the function easier to read. Using type hints, we can rewrite the function like this instead of writing a loop. Note that the type hint for NumPy arrays is `np.ndarray` (see [Example 13-3](#)).

Example 13-3. Temperature conversion (version 3)

```
import numpy as np
from xlwings import arg, func

@func
def convert_to_celsius(degrees: np.ndarray, source: str = "fahrenheit") -> np.ndarray:
    if source.lower() == "fahrenheit":
        return (degrees - 32) * (5 / 9)
    elif source.lower() == "kelvin":
        return degrees - 273.15
    else:
        raise ValueError(f"Don't know how to convert from {source}")
```



Return Type Hint

Since xlwings doesn't require a return type hint (`-> np.ndarray`) and to keep things simple, I'll omit it in the following examples.

If you recalculate the sheet by hitting Ctrl+Alt+F9 (Windows) or Ctrl-Option-F9 (macOS), you will see the same result. However, that function still depends on TRANSPOSE, so let's see how we can get rid of it and make the formula work automatically whether it's used in a horizontal or vertical way. We can do this by using the `ndim` option that we met for the first time in [Chapter 8](#) (see [Example 13-4](#)).

Example 13-4. Temperature Conversion (version 4)

```
import numpy as np
from xlwings import arg, func

@func
@arg("degrees", ndim=2) # Apply the option ndim=2 to the argument "degrees"
def convert_to_celsius(degrees: np.ndarray, source: str = "fahrenheit"):
    if source.lower() == "fahrenheit":
        return (degrees - 32) * (5 / 9)
    elif source.lower() == "kelvin":
        return degrees - 273.15
    else:
        raise ValueError(f"Don't know how to convert from {source}")
```

The option `ndim=2` forces the NumPy arrays to arrive as two-dimensional arrays, even if the source in Excel is a one-dimensional range, i.e., a row or column. This allows you to remove the TRANSPOSE function again from cell B4 so that the formula reads `=CONVERT_TO_CELSIUS(A4:A6,A3)`. If you have lots of functions where you need to enforce a two-dimensional NumPy array, it may become a bit tedious to always put that `@arg` decorator on top of the function to set `ndim=2`. Instead, you can use

Annotated from the typing module to create a reusable type alias. Annotated accepts the type hint as the first argument and a dictionary with the desired options as the second argument, as shown in [Example 13-5](#).

Example 13-5. Temperature conversion (version 5)

```
from typing import Annotated
import numpy as np
from xlwings import arg, func

Array2d = Annotated[np.ndarray, {"ndim": 2}]

@func
def convert_to_celsius(degrees: Array2d, source: str = "fahrenheit"):
    if source.lower() == "fahrenheit":
        return (degrees - 32) * (5 / 9)
    elif source.lower() == "kelvin":
        return degrees - 273.15
    else:
        raise ValueError(f"Don't know how to convert from {source}")
```

This allows us to get rid of the @arg decorator again, and you can reuse the Array2d type annotation with other functions.



Type Hints in Classic xlwings

You can also use type hints with custom functions of the classic version of xlwings ([Chapter 11](#)).

Now that you know type hints can be used to convert Excel cell values to the desired Python data types, let's see how xlwings Lite handles errors when things go wrong.

Error Handling

When a custom function raises an exception, Excel shows a #VALUE! error. Try it out by changing Fahrenheit in cell A3 into Rankine, a temperature scale that our function doesn't support. This will show #VALUE! in cell B4. To get more details, select cell B4, then hover (don't click) over the exclamation mark and wait for the tooltip to appear—it'll show the error string Error: Don't know how to convert from Rankine (see [Figure 13-2](#)).



Newer Versions of Excel Don't Show the Exclamation Mark

In newer versions of Excel, the exclamation mark isn't shown anymore; in this case, simply hover over the green triangle in the top-left corner of the cell.

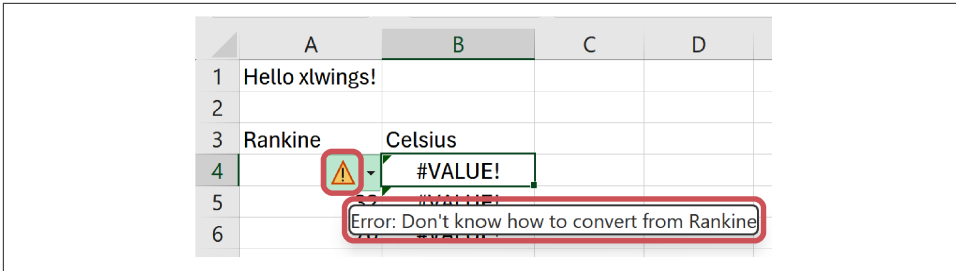


Figure 13-2. Waiting for the error tooltip to appear

If this is a little too subtle, you can handle the exception and return a string instead, as in [Example 13-6](#).

Example 13-6. Temperature conversion (version 6)

```
from typing import Annotated
import numpy as np
from xlwings import arg, func

Array2d = Annotated[np.ndarray, {"ndim": 2}]

def _convert_to_celsius(degrees: Array2d, source: str = "fahrenheit"):
    if source.lower() == "fahrenheit":
        return (degrees - 32) * (5 / 9)
    elif source.lower() == "kelvin":
        return degrees - 273.15
    else:
        raise ValueError(f"Don't know how to convert from {source}")

@func
def convert_to_celsius(degrees: Array2d, source: str = "fahrenheit"):
    try:
        return _convert_to_celsius(degrees, source)
    except Exception as e:
        return f"ERROR: {e}"
```

With error handling covered, let's see how namespaces can help users find the functions they need.

Organizing Functions with Namespaces

This section shows you how to make custom functions easier to access for end-users. We'll start with the Function Wizard before introducing namespaces.

All your custom functions will turn up in the Function Wizard under the category "xlwings Lite." You can start the Function Wizard (see [Figure 13-3](#)) by clicking on the Insert Function button on the left of the formula bar (as introduced previously in

Figure 11-6). Note that macOS doesn't have a proper function wizard—therefore, this only works on Windows and with Excel on the web.

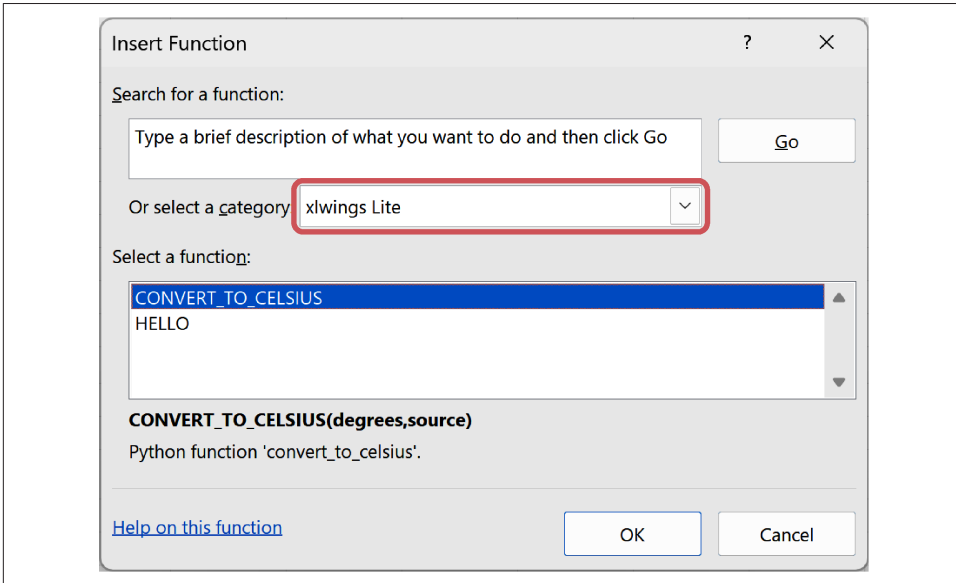


Figure 13-3. Custom functions in the Function Wizard



Use Doc Strings

To provide users with better information about the function and arguments in the Function Wizard, you can write doc strings. For details, see [Chapter 11](#).

To further organize your functions, you can use a *namespace*. A namespace is a way to group related functions together using function name prefixes. Microsoft uses this with a few built-in functions; for example, to work with normal distributions, Excel offers the following functions among others:

- `NORM.DIST`: Returns the normal distribution
- `NORM.S.DIST`: Returns the standard normal distribution

`NORM` is the namespace, and `S` in the middle of `NORM.S.DIST` is called a *subnamespace*. This makes it easy to get all related functions to show up by simply typing `=NORM` into a cell. To use namespaces with your own custom functions, you can't just rename your Python function into something like `basics.hello` as Python doesn't accept a dot in the name of a function. Instead, provide the namespace as an argument to the `@func` decorator:

```
@func(namespace="basics")
def hello(name):
    return f"Hello {name}!"
```

If you'd like to use a subnamespace, add it with a dot:

```
@func(namespace="basics.greetings")
def hello(name):
    return f"Hello {name}!"
```

The function will turn up in Excel as `BASICS.HELLO` or `BASICS.GREETINGS.HELLO`, respectively.

At this point, we've covered the foundations of custom functions and are ready to create something useful in the form of an AI function.

Building an AI Function with OpenAI

In this section, we'll build a custom function called `AI` powered by one of OpenAI's models (OpenAI is the company behind ChatGPT). We'll design it as an alternative to Excel's native `COPILOT` function. The AI function is using OpenAI's API and therefore offers great flexibility: you can select the AI model and adjust the function to your specific needs. Unlike the `COPILOT` function, which requires a paid Copilot subscription, the AI function is cost per use, which makes it much more economical if you only use it occasionally. Since accessing the OpenAI API requires an API key, this section will also teach you how to work with environment variables in `xlwings` Lite to securely handle credentials.

Managing Credentials with Environment Variables

To work with the OpenAI API, you'll need to use an API key. If you store it directly in your code, you'll have an issue when you give your workbook to somebody else; since the Python code is stored in the workbook itself, you will give them access to your OpenAI credits. To prevent that, you want to store the API key outside of the workbook in an environment variable. Before I explain what an environment variable is, let's generate an OpenAI API key.

OpenAI API key

To be able to talk to the OpenAI API, you'll need an API key. Let me walk you through the process of how to get one:

1. Log in to the [OpenAI API Platform](#). The Log in and Sign up buttons are on the top right of the page. You can use a third-party account like a Google or Microsoft account or create a new account with OpenAI.
2. Once you're logged in, click on the cog on the top right to go to Settings.

3. On the menu bar on the left, under Billing, click on Add payment details to add at least USD 5 by credit card.
4. Again on the menu bar on the left, click on the API keys, then click the Create new secret key button. Under Project, select Default project, and under Permission, select Read only. Then click on Create secret key. The next screen will show your API key. Go ahead and copy it to the clipboard.



Instructions May Change

These instructions are likely to change over time. It should, however, be easy to get up-to-date information on how to get an OpenAI API key by asking your favorite search engine or AI assistant.

For the next steps, switch over to Excel, where we will store the API key as an environment variable.

Environment Variables

An *environment variable* is a variable that is stored outside of your code. In the case of xlwings Lite, you can store them in the workbook or on your local machine:

Workbook

You store environment variables in the workbook if you want them to travel with the workbook, so they shouldn't include anything sensitive. You could, for example, create an environment variable called `OPENAI_MODEL` to define the model you want to use without having to do any code changes.

Local

Environment variables that you store locally are the right choice for sensitive credentials like the OpenAI API key. Local environment variables can be accessed by all workbooks.

Let's now go through the steps of storing our OpenAI API key as a local environment variable:

1. Open the file *ai.xlsx*, which you'll find in the *lite* directory of the companion repository.
2. With *ai.xlsx* as the active workbook, open xlwings Lite. Click on the xlwings Lite menu (hamburger icon) and select Environment Variables.
3. Switch to the Local tab.
4. Click on the + sign at the top right of the table to add a new environment variable. Under Name, type in **OPENAI_API_KEY**, and under Value, paste the API key

that we copied previously (see [Figure 13-4](#)). Then click Save and, importantly, restart xlwings Lite via xlwings Lite Menu > Restart.

xlwings Lite

← Environment Variables

Workbook **Local**

Name
OPENAI_API_KEY

Value
sk-proj-foBQyCGa6N9.....

Save Cancel

Figure 13-4. Environment variables in xlwings Lite



The openai Package

If you have a look at the *requirements.txt* file in the xlwings Lite editor, you will see that the code depends on the package `openai`.

Now, we're ready to interact with the OpenAI API. Let's have a look at the first function in *main.py*:

```
import os

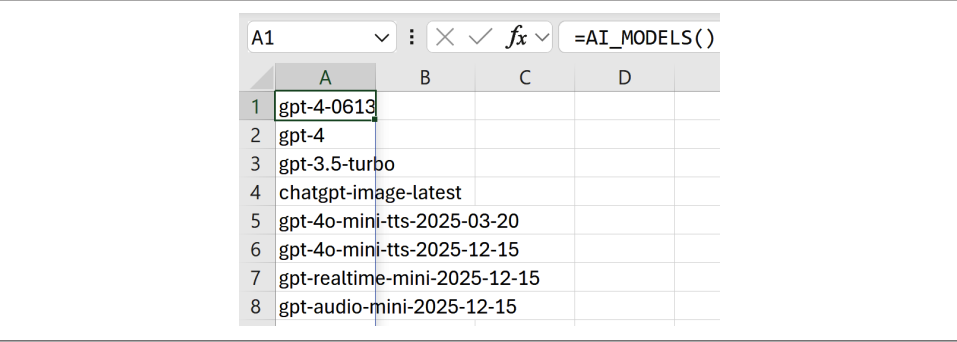
from openai import AsyncOpenAI
from xlwings import arg, func, ret

MODEL = "gpt-5-chat-latest"

client = AsyncOpenAI(
    api_key=os.environ["OPENAI_API_KEY"], # Read our environment variable
    timeout=30, # Some prompts take longer than the default timeout
)

@func
async def ai_models():
    models = await client.models.list()
    return [[m.id] for m in models.data]
```

As you can see, I am using an asynchronous function by making use of `async/await`, which I introduced in the last chapter. On Sheet1, in cell A1, type `=AI_MODELS()`. After hitting Enter, you'll see a list of currently supported models by OpenAI, as shown in Figure 13-5.



	A	B	C	D
1	gpt-4-0613			
2	gpt-4			
3	gpt-3.5-turbo			
4	chatgpt-image-latest			
5	gpt-4o-mini-tts-2025-03-20			
6	gpt-4o-mini-tts-2025-12-15			
7	gpt-realtime-mini-2025-12-15			
8	gpt-audio-mini-2025-12-15			

Figure 13-5. The available models via OpenAI's API (first few rows)

Not all of them are suitable models to use, as there are also models for audio and image generation, not just for text. I will try to keep the Excel file in the companion repository updated with a suitable model so you don't have to figure out one by yourself. When choosing a model, you usually need to decide between models that are fast, cheap, and less capable and those that are slow, expensive, and more capable. Which ones you should use depends on the nature of your task, as the more expensive models can normally solve much more complex tasks.

Knowing how to look up the available OpenAI models, we're ready to dive into the AI function next.

The AI Function

Now that we know which models are available, let's talk to one with the AI function that you will find in the second half of the *main.py* file:

```
MODEL = "gpt-5-chat-latest"

@func
@ret("json") ❶
@arg("prompt", "json", ndim="natural") ❷
async def ai(*prompt): ❸
    prompts = "\n\n".join(prompt)
    instructions = (
        f"You are an Excel assistant. {prompts}\n\n"
        "If required by the data, return a valid JSON 2D array without objects."
        "Do not include explanations, notes, or markdown formatting."
    )
```

```
# Use the Responses API
try:
    response = await client.responses.create(
        model=MODEL,
        input=instructions,
    )
except Exception as e:
    return f"ERROR: {e}"

return response.output_text ④
```

- ❶ We're using the json converter for both reading and writing the range, as this allows us to efficiently turn Excel cells into a string that AI models understand. I introduced JSON in [Chapter 10](#), which turns the list of lists that xlwings delivers by default, e.g., `[[1, "text"], [None, True]]`, into the following string: `'[[1, "text"], [null, true]]'`.
- ❷ By default, xlwings delivers vertical ranges, i.e., columns, as simple lists (e.g., `[1, 2, 3]`). By using `ndim="natural"`, columns are delivered as nested lists (e.g., `[[1], [2], [3]]`), while rows are kept as simple lists. This is important so that the AI model understands the orientation of our data.
- ❸ In Python, to allow a variable number of positional arguments, you start a function parameter with an asterisk. In the body of the function, `prompt` arrives as a tuple holding all arguments of the function call. This will allow us to call the function with one, two, or more arguments, which will make it easy to switch between text prompts and spreadsheet data, as we will see in a moment.
- ❹ If AI delivers what we asked for, i.e., a JSON 2D array, the `@ret("json")` converter will convert it back to a two-dimensional list so that the AI function returns a dynamic array in Excel.



Wingman

The AI function is a simplified version of the WINGMAN function that xlwings Lite provides out of the box. WINGMAN works like the COPILOT function but doesn't require a Microsoft 365 subscription and allows you to connect with multiple AI models, including self-hosted ones.

Let's now see the power of our AI function by starting with a simple question, then gradually increasing the complexity of the prompt. On Sheet1, in cell F1, add this simple text-based prompt:

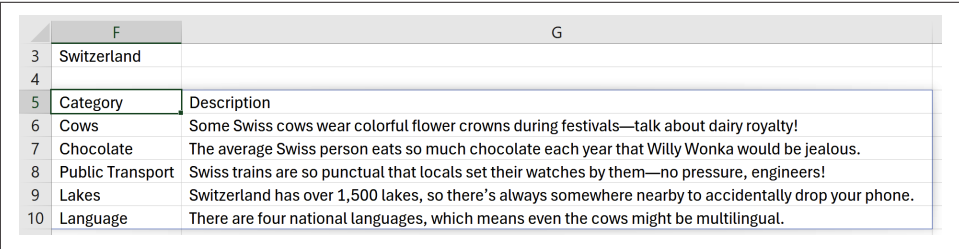
```
=AI("Describe xlwings in one sentence")
```

After hitting Enter, the cell shows “xlwings is a Python library that allows you to automate and interact with Excel directly from Python.”

For the next sample, we're going to use a single cell reference and give it specific instructions that we want a table as output. Here, I mean *table* in its traditional sense, not an Excel table—dynamic arrays and Excel tables are incompatible. Go ahead and—still on Sheet1—write the name of a country into cell F3, e.g., **Switzerland**. Then, in cell F5, write the following prompt (the line break is only to make it fit in this book; you can write everything on a single line):

```
=AI("List 5 funny facts about the following country and return a table  
with the columns Category and Description (include the table headers)", F3)
```

This will look something like **Figure 13-6**.



F		G	
3	Switzerland		
4			
5	Category	Description	
6	Cows	Some Swiss cows wear colorful flower crowns during festivals—talk about dairy royalty!	
7	Chocolate	The average Swiss person eats so much chocolate each year that Willy Wonka would be jealous.	
8	Public Transport	Swiss trains are so punctual that locals set their watches by them—no pressure, engineers!	
9	Lakes	Switzerland has over 1,500 lakes, so there's always somewhere nearby to accidentally drop your phone.	
10	Language	There are four national languages, which means even the cows might be multilingual.	

Figure 13-6. Five funny facts about Switzerland

For these two prompts, you don't need a very advanced model. However, for the next example, you will get better results if you choose a model that can cope with complex prompts, which is why I have set `MODEL = "gpt-5-chat-latest"` in the code. For this example, switch over to the Companies sheet and ask the AI function to fill in the table data by writing the following custom function in cell B2 (again, the line break is only for display purposes; you can omit it):

```
=AI("Return the body of the table with these columns", B1:E1,  
"for these companies (no column headers)", A2:A51)
```

If you're lucky, the table will be filled as shown in **Figure 13-7**. You may, however, also get wrong or empty results, e.g., the data may not correctly spill across the cells or columns may be misaligned. In this case, recalculate the formula again—usually, you'll be lucky on the second or third attempt!

B2

:

✕

✓

fx

=AI("Return the body of the table with these columns", B1:E1, "for these companies (no column headers)", A2:A51)

	A	B	C	D	E
1	Company	Industry	Country	Employees (integer)	Avg Annual Revenue (billion USD)
2	3M	Conglomerate	United States	92,000	33
3	AbbVie	Pharmaceuticals	United States	50,000	54
4	Allianz	Financial Services	Germany	158,000	135
5	Alphabet	Technology	United States	182,000	307
6	Amazon	E-commerce	United States	1,540,000	575
7	Amgen	Biotechnology	United States	26,000	28
8	Anheuser-Busch InBev	Beverages	Belgium	167,000	59
9	Apple	Technology	United States	161,000	383
10	BHP	Mining	Australia	49,000	54

Figure 13-7. The companies table with data filled in by AI (first few rows)



Convert AI Results to Static Values

If you use the AI function to fill in information, as we did in this example, I would recommend copying the output and pasting it as values again. Otherwise, you would have to double-check the data every time the function recalculates. That same recommendation applies to the COPILOT function.

Creating an alternative for the official COPILOT function with just a few lines of code was a fun exercise that showed us how to work with environment variables and how to make remote calls to powerful AI models. However, you may not always need the power of the latest AI model—in the next section, we'll perform sentiment analysis with a machine learning model that runs locally on your computer.

Performing Sentiment Analysis with Hugging Face

Sentiment analysis is a technique that evaluates the tone of a text to classify it—for example, as negative, neutral, or positive. There are various approaches:

- You could use the latest AI model, e.g., by using the AI function from the previous section or Excel's built-in COPILOT function.
- There are rule-based tools such as VADER (Valence Aware Dictionary and sEntiment Reasoner) that is offered by the NLTK (Natural Language Toolkit) library.
- And there are pretrained machine learning models, which fall somewhere in-between the previous two solutions: they are more sophisticated than rule-based tools, but unlike a full AI system, they can run locally—even in the constrained environment of xlwings Lite.

In this section, we will use a pretrained model to perform sentiment analysis on a set of user reviews about a cocktail bar. To do this, we'll use a transformer-based model from Hugging Face. To begin with, let me explain what Hugging Face, pretrained models, and transformer-based models are, as well as which model we're going to use.

Machine Learning Terminology

There are a few technical terms from the world of machine learning that you need to understand before looking at any code. Let's go through them!

Hugging Face

Hugging Face is both a company and a platform for the *machine learning* (ML) community (ML is a subset of AI). In simplified terms, ML creates models that are trained on data so they can learn patterns. This allows them to make predictions on new data without you having to write explicit rules for every possible scenario. The Hugging Face platform allows users to upload pretrained ML models (see the item that follows), datasets, and create applications.

Pretrained model

A pretrained model is an ML model that has been trained on a large dataset. Training a model is usually a resource-intensive activity, so being able to download a pretrained model saves us a lot of time and resources.

Transformer

Transformer refers to a specific type of ML model that is very good at processing language. Hugging Face created the Transformers Python package, which provides an easy way to work with pretrained transformer models.

DistilBERT Base Uncased Finetuned SST-2 English

We are going to use the *DistilBERT base uncased finetuned SST-2 English* model, which is a pretrained transformer model for sentiment analysis. This model is a distilled version of the BERT base model (Bidirectional Encoder Representations from Transformers), which was originally created by Google. A *distilled* version is a smaller and more efficient model generated by a process called *knowledge distillation*. The model is *uncased*, meaning that capitalization of words is ignored. The base model has then been *fine-tuned* using the *Stanford Sentiment Treebank v2* (SST2) dataset to classify text as positive or negative. And finally, the training material was in *English*, which makes this model suitable for only the English language.

Now that we know what's behind the model name, let's see how we can use it in a custom function!

The CLASSIFY Function

In this section, we're building the CLASSIFY custom function to tag the cocktail bar reviews with a positive or negative label. The good news is that you don't need a major in machine learning to use a pretrained ML model via Hugging Face's Transformer library. In fact, the CLASSIFY function is no more complicated than the AI function from the previous section. To follow along, open the *reviews_ml.xlsx* file from the *lite* directory in the companion repo. After starting the xlwings Lite add-in, you will see the following code in the *main.py* file:

```
from transformers_js_py import import_transformers_js ❶
from xlwings import func

@func
async def classify(texts): ❷
    transformers = await import_transformers_js()
    try:
        sentiment_pipeline = await transformers.pipeline(
            "text-classification", # task ❸
            "Xenova/distilbert-base-uncased-finetuned-sst-2-english", # model ❹
        )
        results = await sentiment_pipeline(texts)
    except Exception as e:
        return f"ERROR: {e}"
    return [[result["label"]] for result in results] ❺
```

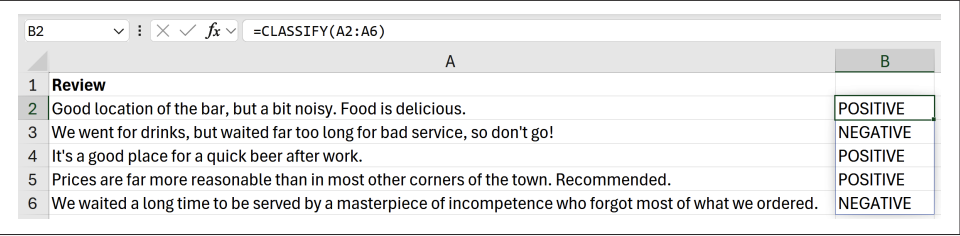
- ❶ If you look at the *requirements.txt* file, you will see that it lists the *transformers_js_py* package instead of the original Python package *transformers*. You probably remember from the last chapter that xlwings Lite runs Python in the browser via Pyodide. The original Python package uses features that aren't compatible with Pyodide. However, Hugging Face also provides *Transformers.js*, which is the Transformers library for JavaScript, allowing you to run it inside a browser. The *transformers_js_py* library wraps the JavaScript version of the library into a Python package, which explains its name.
- ❷ The pipeline function is an async function, so we need to define the *classify* function as *async*. Have a look at “[Asyncio](#)” on page 287 for a short introduction to Python's Asyncio library with the *async* / *await* keywords.
- ❸ Since *transformers_js_py* is a wrapper around *Transformers.js*, it's easiest to refer to the [Transformers.js documentation](#), e.g., to learn about the available tasks.
- ❹ You'll find the original model under [distilbert/distilbert-base-uncased](#). However, that one isn't compatible with *Transformers.js*. That's why we're using the version created by user Xenova, which is compatible with *Transformers.js*.

- 5 The results variable holds a list in this format: [{'label': 'POSITIVE', 'score': 0.9995325207710266}, ...]. We're using `[[result["label"]] for result in results]` to get a vertical list with just the label.

Now go ahead and write the following formula into cell B2:

=CLASSIFY(A2:A6)

You will see the following output, as shown in **Figure 13-8**.



	A	B
1	Review	
2	Good location of the bar, but a bit noisy. Food is delicious.	POSITIVE
3	We went for drinks, but waited far too long for bad service, so don't go!	NEGATIVE
4	It's a good place for a quick beer after work.	POSITIVE
5	Prices are far more reasonable than in most other corners of the town. Recommended.	POSITIVE
6	We waited a long time to be served by a masterpiece of incompetence who forgot most of what we ordered.	NEGATIVE

Figure 13-8. Sentiment analysis with Hugging Face Transformers



First-Time Usage

The first time you run the CLASSIFY function in Excel, the pre-trained model will be downloaded from Hugging Face, which can take a moment as the model's size is roughly 70 MB. Since the model will be cached by the browser, subsequent calls will be much faster.

Note that the model `distilbert-base-uncased-finetuned-sst-2-english` can only classify a review as either positive or negative. If you want a model that is able to classify into positive, negative, and neutral, you have to use a more advanced model, such as `Xenova/twitter-roberta-base-sentiment-latest`.

You could also use the COPILOT function or our AI function from the previous section to perform sentiment analysis. But using the Transformers library via our CLASSIFY function allows you to run everything locally. An internet connection is only required to initially load the xlwings Lite add-in and to download the model. Once loaded, it runs offline, which is privacy-friendly, fast, and doesn't require a credit card.

We've now seen how to set up custom functions that rely on AI or a specialized ML model. For our final project, we'll work with historical weather data. This will introduce you to how xlwings Lite handles dates and plots.

Plotting Weather Data with Open-Meteo

In this section, we'll use the **Open-Meteo** weather API to download historical temperature data and visualize it with a Matplotlib plot to build a mini dashboard. Open-Meteo is an open source project that offers a free weather API for noncommercial use. It provides both forecast and historical data with global coverage. We'll begin by exploring how xlwings Lite handles dates, then move on to returning plots from custom functions. Both topics work slightly differently compared to the classic version of xlwings from **Chapter 11**.

Working with Dates

xlwings Lite needs a bit of help to read date-formatted cells. On the other hand, it can format cells as dates automatically when writing datetime objects to Excel. To see how this works in detail, open *weather.xlsx*, which you will find in the *lite* directory of the companion repository. With *weather.xlsx* as the active workbook, open the xlwings Lite add-in, then type the following formula into cell A8 on Sheet1:

```
=GET_TEMPERATURE_HISTORY(B3, B4, B5, B6)
```

Once you hit Enter, the historical temperatures will be downloaded according to the inputs in cells B3 to B6 (see **Figure 13-9**).

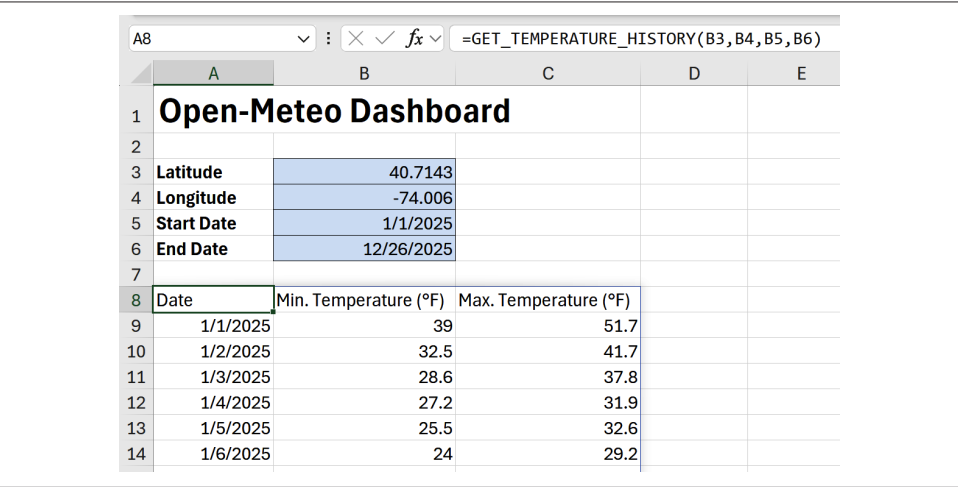


Figure 13-9. The `GET_TEMPERATURE_HISTORY` function in action

The workbook has prefilled the latitude and longitude from New York City, but you can change them with the values for another location. To display the latitude and longitude for your location, go to **Google Maps** and right-click on the desired location on the map. The context menu that appears will show the latitude and longitude

at the very top. Let's now have a look at the code of the GET_TEMPERATURE_HISTORY function:

```
import datetime as dt

import matplotlib.pyplot as plt
import pandas as pd
import requests ❶
from xlwings import arg, func, ret

@func
@ret(index=False)
def get_temperature_history(
    latitude: float,
    longitude: float,
    start_date: dt.date, ❷
    end_date: dt.date,
):
    # Open-Meteo API for historical data
    url = "https://archive-api.open-meteo.com/v1/archive"

    # Making the web API request
    params = {
        "latitude": latitude,
        "longitude": longitude,
        "daily": "temperature_2m_min,temperature_2m_max",
        "start_date": start_date.isoformat(),
        "end_date": end_date.isoformat(),
        "temperature_unit": "fahrenheit",
        "timezone": "auto",
    }
    response = requests.get(url, params=params)
    response.raise_for_status() ❸
    data = response.json()

    # Turn the data into a DataFrame to easily return it to Excel
    df = pd.DataFrame(
        {
            "Date": pd.to_datetime(data["daily"]["time"]), ❹
            "Min. Temperature (°F)": data["daily"]["temperature_2m_min"],
            "Max. Temperature (°F)": data["daily"]["temperature_2m_max"],
        }
    )

    return df
```

- ❶ To download the data, we're using the Requests package, as you will see when you switch to the *requirements.txt* file. This allows us to run the code unchanged in both classic xlwings and xlwings Lite.

- ❷ When reading `start_date` and `end_date`, the `dt.date` or `dt.datetime` type hints are mandatory, as you would otherwise get an integer (the date serial number that Excel uses behind the scenes). In the next section, we'll learn how to define an entire column as `datetime` when reading in a pandas `DataFrame`.
- ❸ The method `raise_for_status` raises a Python exception if the response isn't successful, i.e., returns an HTTP status code of 4xx or 5xx. Successful requests return the status code 2xx instead.
- ❹ Since the data comes from a JSON response, i.e., in text form, we need to use `pd.to_datetime` so that pandas uses the correct data type `datetime64[us]`.



Use a Fake Response

Open-Meteo is a free service for noncommercial use that doesn't require any authentication. If this changes in the future or the service shuts down, change the `url` to this one:

```
url = (  
    "https://raw.githubusercontent.com/fzumstein/"  
    "python-for-excel/refs/heads/2e/lite/temp.json"  
)
```

This will return a response with fake data in the same format as Open-Meteo and will allow you to follow the example—it will, however, always return the same data no matter what you provide as function arguments.

Note that returning the `DataFrame` with the `Date` column automatically formats the dates according to your default date format in Excel, as shown in [Figure 13-9](#). The next section shows you how to turn the data into a Matplotlib plot.

Plotting Temperature over Time

With `xlwings` Lite, custom functions allow you to return a Matplotlib `Figure` object. `PLOT_TEMPERATURE`, the second function in the `main.py` file of the `weather.xlsx` workbook, shows this as well as how to read in more than one date-formatted cell. We'll start by using the `PLOT_TEMPERATURE` function in Excel before going through the code in detail. In cell E1, write the following formula:

```
=PLOT_TEMPERATURE(A8#)
```

After hitting Enter, the cell will first show an error message as you need to enable this functionality in the `xlwings` Lite settings menu. That's because returning images requires an internet connection (see the warning that follows).



Returning Images Requires an Internet Connection

Because of technical limitations, custom functions that return a Matplotlib Figure object will send the image of the plot to the cloud for the duration of the function call. The image is still generated locally, and no other code or data is uploaded. Once the plot shows up in the Excel cell, it has already been deleted in the cloud—at this point, the picture is embedded in the Excel workbook. When you reopen Excel files that use custom functions with plots, you will see the following security warning: “Linked Data Types have been disabled.” Click on Enable Content to use the workbook properly.

To allow custom functions to return Matplotlib plots as images, follow the instructions in the error message: click on the xlwings Lite menu (hamburger icon) > Settings > Local. Then enable the setting *Allow custom functions to return images*. When you now recalculate cell E1, it will show an in-cell picture with the plot. Since we refer to the source data as A8#, the plot automatically adjusts to match the size of the returned dataset. This works because GET_TEMPERATURE_HISTORY returns a dynamic array, which we can reference by its upper-left corner, i.e., cell A8, together with the # symbol. To get a plot similar to how native Excel charts behave, right-click cell E1, then click on Picture in Cell > Create Reference. It is important to *not* choose Place over Cells, as this would disconnect the picture from its source data. And with this, our mini dashboard is finished, as shown in Figure 13-10.

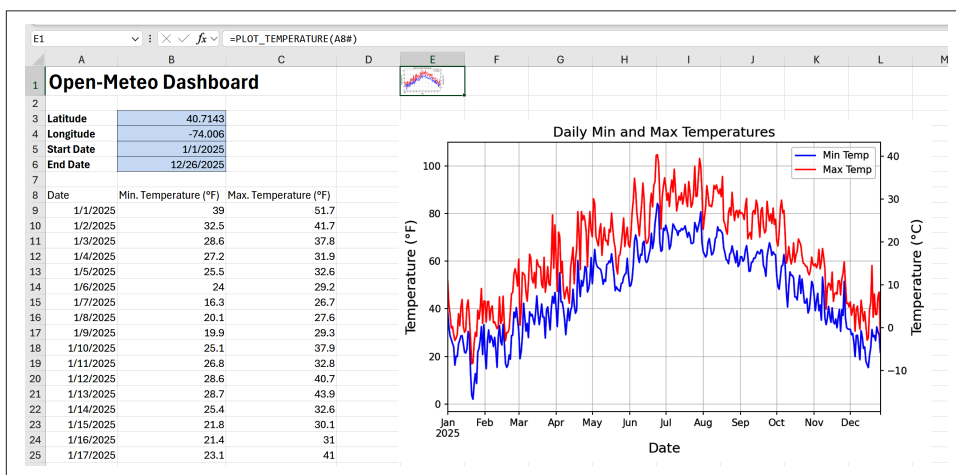


Figure 13-10. Returning Matplotlib figure objects from custom functions

When you change the parameters in cells B3 to B6, both the data and plot will update automatically. Let's now have a look at the source code:

```

@func
@arg("df", parse_dates=["Date"]) ❶
def plot_temperature(df: pd.DataFrame):
    # Create a figure and axes object to be able to create a secondary axis
    fig, ax = plt.subplots(figsize=(8, 5))

    # Plotting min. and max. temperature
    df["Min. Temperature (°F)"].plot(ax=ax, color="blue", label="Min Temp")
    df["Max. Temperature (°F)"].plot(ax=ax, color="red", label="Max Temp")

    # Secondary Y-axis for Celsius
    def f_to_c(x):
        return (x - 32) * 5 / 9

    def c_to_f(x):
        return x * 9 / 5 + 32

    secax = ax.secondary_yaxis("right", functions=(f_to_c, c_to_f)) ❷

    # Formatting
    ax.set_title("Daily Min and Max Temperatures", fontsize=14)
    ax.set_xlabel("Date", fontsize=14)
    ax.set_ylabel("Temperature (°F)", fontsize=14)
    secax.set_ylabel("Temperature (°C)", fontsize=14)
    ax.minorticks_off() # Hide the minor ticks on the x- and y-axis
    ax.grid(alpha=0.5) # The alpha argument makes the grid transparent
    ax.legend() # Show the legend

    return fig ❸

```

- ❶ When reading an Excel range with dates into a DataFrame, the easiest way to convert certain columns to the `datetime64[us]` data type is by using the `parse_dates` option. It works the same as with `pandas.read_csv`, i.e., you can also use the column position instead of its name, i.e., `parse_dates=[0]`.
- ❷ Matplotlib allows you to create a secondary axis based on a tuple of functions: the transform function and its inverse. Here, we're providing the function to transform from Fahrenheit to Celsius (`f_to_c`) and from Celsius to Fahrenheit (`c_to_f`).
- ❸ Returning a Matplotlib Figure object is all that's needed to get an in-cell picture in Excel. While I am setting up the `fig` and `ax` objects here explicitly to create a secondary y-axis, you may not need to do this with simpler cases. Remember, AI tools are very good at explaining how to get a Matplotlib Figure object from your pandas plot.



Reset Picture and Size

If your plot looks distorted, right-click on it and select Reset Picture & Size.

And with this, we have a fully functioning, albeit simple, weather dashboard. Instead of using buttons like we did in the last chapter, this dashboard is fully driven by custom functions. Being able to reference dynamic arrays in our plotting function saves us from having to manually adjust any cell references when the data source grows or shrinks, which was a major pain point in the predynamic array time of Excel.

Conclusion

In this chapter, we learned how to use custom functions with `xlwings` Lite. We worked with the OpenAI API and with a pretrained machine learning model from Hugging Face, and we plotted historical temperature data provided by Open-Meteo. These three projects helped us to understand the particularities of `xlwings` Lite, e.g., how to work with environment variables, dates, and plots.

We're at the beginning of the AI age, so a lot of what we do today will change substantially over the next few years. In that light, it's good to know that you can directly connect to the AI provider and model of your choice and build AI-powered functions with little effort—our AI function was just 20 lines of code! On the other hand, you don't need to use a full AI chat model for everything, as the sentiment analysis with a pretrained machine learning model showed. This was a very impressive demonstration of what is possible today with Pyodide, the restricted Python environment that `xlwings` Lite uses.

This was the last chapter in which we used Excel as the interactive user interface and Python as the calculation engine. **Part VI** is about reading and writing Excel files in ways that bypass the Excel application entirely. This is very helpful as it means you can manipulate Excel files with Python on every operating system that Python supports, including Linux. While we have used the `pd.read_excel` function a few times already in this book, the next chapter will finally introduce it properly along with its counterpart, the `df.to_excel` method.

Reading and Writing Excel Files Without Excel

Excel File Manipulation with pandas

So far, we've always worked with Python and Excel in an interactive way. We ran Python scripts to automate Excel and used Excel workbooks as a user interface to call Python functions—either via *xlwings* or Python in Excel. In **Part VI**, we're leaving the Excel application out of the equation: we'll work with Excel files directly without opening them in Excel. In this chapter, you'll learn how to read and write Excel workbooks with *pandas*. We'll start by revisiting the reporting case study from **Chapter 8**. I'll then give you a more in-depth introduction to the tools that *pandas* offers to work with Excel files: the `read_excel` function and the `ExcelFile` class for reading, and the `to_excel` method and the `ExcelWriter` class for writing Excel files. Since we're working with files rather than the Excel application, all code samples in this chapter run everywhere Python runs, including Linux.

Reporting Case Study (Take 2)

In the case study from **Chapter 8**, we used the *pandas* function `read_excel` to read in all source files from the *sales_data* directory in the companion repository. We'll do the same in this version of the case study. This time, however, instead of using *xlwings* to write out the report to a preformatted template, we're going to stick to *pandas* and write the `DataFrame` to a new Excel file. Since *pandas* on its own can't create Excel charts, this version of the report will only have a table. The Python script *sales_report_pandas.py*, as shown in **Example 14-1**, looks almost the same as *sales_report_xlwings.py*. The difference is that the report is produced via the `to_excel` `DataFrame` method on the last line of the script.

Example 14-1. sales_report_pandas.py

```
from pathlib import Path

import pandas as pd

# Directory of this file
this_dir = Path(__file__).resolve().parent

# Read in all files
parts = []
for path in (this_dir / "sales_data").rglob("*.xls*"):
    print(f"Reading {path.name}")
    part = pd.read_excel(path, engine="calamine")
    parts.append(part)

# Combine the DataFrames from each file into a single DataFrame
df = pd.concat(parts)

# Pivot each store into a column and sum up all transactions per date
pivot = pd.pivot_table(
    df, index="transaction_date", columns="store", values="amount", aggfunc="sum"
)

# Resample to end of month and assign an index name
summary = pivot.resample("ME").sum()
summary.index.name = "Month"

# Sort columns by total revenue
summary = summary.loc[:, summary.sum().sort_values().index]

# Add row and column totals
summary["Total"] = summary.sum(axis=1)
total_row = pd.DataFrame([summary.sum(axis=0)], index=["Total"])
summary = pd.concat([summary, total_row])

#### Write summary report to Excel file ####
summary.to_excel(this_dir / "sales_report_pandas.xlsx")
```

Run the script by clicking the Run Python File button at the top right of VS Code or by running the following command in a Terminal:

Windows (Terminal)

```
cd %userprofile%\python-for-excel-2e
uv run sales_report_pandas.py
```

macOS (Terminal)

```
cd ~/python-for-excel-2e
uv run sales_report_pandas.py
```

The script will take a moment to complete and once it's done, the Excel workbook *sales_report_pandas.xlsx* will show up in the same directory as the script. The content of Sheet1 will look like [Figure 14-1](#). That's quite an impressive result for just a few lines of code—even if you will need to adjust the width of the Excel columns manually!

	A	B	C	D	E	F	G	H
1		Washington DC	Boston	Las Vegas	New York	Chicago	San Francisco	Total
2	2019-01-31 00:00:00	14057.6	21784.1	23012.75	49872.85	51187.7	58629.85	218544.85
3	2019-02-28 00:00:00	15235.4	21454.9	25493.1	46669.85	52330.85	55218.65	216402.75
4	2019-03-31 00:00:00	14177.05	20043	23451.1	41572.25	48897.25	52712.95	200853.6
5	2019-04-30 00:00:00	13339.15	18791.05	22710.15	41714.3	47396.35	49324.65	193275.65
6	2019-05-31 00:00:00	13147.1	18036.75	21526.55	40610.4	45117.05	47759.6	186197.45
7	2019-06-30 00:00:00	14284.3	21556.25	21985.05	47265.65	49460.45	53462.4	208014.1
8	2019-07-31 00:00:00	14161.5	19853	23444.3	40408.3	47993.8	50181.6	196042.5
9	2019-08-31 00:00:00	16127.05	22332.9	24927.65	45396.85	50838.9	55336.35	214959.7
10	2019-09-30 00:00:00	14994.4	19924.5	24410.7	42830.6	49096.25	49931.45	201187.9
11	2019-10-31 00:00:00	12846.7	16550.95	22827.5	34090.05	42543.8	44311.65	173170.65
12	2019-11-30 00:00:00	14057.6	21312.9	24860.25	46959.85	52011.6	55056.45	214258.65
13	2019-12-31 00:00:00	14702.15	19722.6	24535.75	42364.35	49355.1	50933.45	201613.4
14	Total	171130	241362.9	283184.85	519755.3	586229.1	622859.05	2424521.2

Figure 14-1. *sales_report_pandas.xlsx* after adjusting the column widths manually

For simple cases like this one, pandas offers an easy solution to work with Excel files. However, you might remember that in [Chapter 8](#), we used conditional formatting to color revenues below 20,000 in red. While pandas doesn't support conditional formatting, you can use a Styler object to change how a DataFrame is displayed. Styler objects are primarily designed for use with Jupyter notebooks and other HTML-based output, but they also work with Excel to some extent. To highlight revenues below 20,000 in red, you can use the Styler object as shown in *sales_report_pandas_styler.py* (see the relevant code in [Example 14-2](#)).

Example 14-2. *sales_report_pandas_styler.py* (excerpt)

```
#### Write summary report to Excel file ####

# Create a Styler object and apply formatting
styler = (
    summary.style.highlight_between(left=0, right=20000, props="color: red")
)

# Write to Excel
styler.to_excel(this_dir / "sales_report_pandas.xlsx")
```

Since Styler objects are primarily intended to work with HTML output, they use a notation close to CSS (cascading style sheets), the language for styling HTML. For further details, see the pandas documentation about [Table Visualization](#).

Calamine: A Faster Reader Engine

Depending on the Excel file extension, pandas delegates the process of reading the Excel file to a specific *engine*, i.e., one of the Python packages that we'll look at in the next chapter. [Table 14-1](#) shows you the default engines that pandas uses.

Table 14-1. Default Excel Reader engines

Extensions	Default engine
xls	xlrd
xlsb	pyxlsb
xlsx, xlsxm	openpyxl

Instead of using the default engines, you should normally use the *calamine* engine. In most cases, using the calamine engine is a lot faster than the default engines as it uses *python-calamine*, a package that is implemented in Rust, a low-level programming language. The exception is when you just need to read a few rows from a huge sheet—in this case, OpenPyXL is faster and more memory-efficient (unlike calamine, OpenPyXL doesn't load the entire sheet into memory). In addition to being faster, the calamine engine has another advantage when reading *xlsb* files: unlike the *pyxlsb* engine, it recognizes date-formatted cells properly and converts them into `datetime64[us]` data types. To use the calamine engine, you need to specify the engine explicitly, as we did in [Example 14-1](#).

- `pd.read_excel(..., engine="calamine")`
- `pd.ExcelFile(..., engine="calamine")`

Note that whether you are using a default engine or calamine, you need to have the respective package installed. Otherwise, pandas will raise an exception and complain about the missing package. We're fine since we installed all possible engines during our initial setup in [Chapter 2](#).

To create a professional report, formatting the values of a `DataFrame` might not be enough, though—after all, a title and a chart wouldn't hurt. That's exactly what we will cover in the next chapter by working directly with the writer libraries that pandas uses under the hood. Before we get there, however, let's have a more detailed look at how we can read and write Excel files with pandas alone.

Reading and Writing Excel Files

Apart from setting the engine, the case study used `read_excel` and `to_excel` with their default arguments to keep things simple. In this section, I will show you the

most commonly used arguments and options when reading and writing Excel files with pandas and how to read multiple sheets in parallel. I will also introduce the `ExcelFile` and `ExcelWriter` classes and explain when you should use them.

The `read_excel` Function and `ExcelFile` Class

The case study used Excel workbooks where the data was conveniently in cell A1 of the first sheet. In reality, your Excel files are probably not so well organized. In this case, pandas offers parameters to fine-tune the reading process. For the next few samples, we're going to use the *stores.xlsx* file that you will find in the *xl* folder of the companion repository. The first sheet is shown in [Figure 14-2](#).

	A	B	C	D	E	F
1						
2		Store	Employees	Manager	Since	Flagship
3		New York	10	Sarah	7/20/2018	FALSE
4		San Francisco	12	Neriah	11/2/2019	MISSING
5		Chicago	4	Katelin	1/31/2020	TRUE
6		Boston	5	Georgiana	4/1/2017	TRUE
7		Washington DC	3	Evan		FALSE
8		Las Vegas	11	Paul	1/6/2020	FALSE
9						

Figure 14-2. The first sheet of *stores.xlsx*

By using the parameters `sheet_name`, `skiprows`, and `usecols`, we can tell pandas about the cell range that we want to read in. As usual, it's a good idea to have a look at the data types of the returned DataFrame by running the `info` method:

```
In [1]: import pandas as pd

In [2]: df = pd.read_excel(
        "xl/stores.xlsx",
        sheet_name="2019",
        skiprows=1,
        usecols="B:F",
        engine="calamine",
    )
df

Out[2]:
```

	Store	Employees	Manager	Since	Flagship
0	New York	10	Sarah	2018-07-20	False
1	San Francisco	12	Neriah	2019-11-02	MISSING
2	Chicago	4	Katelin	2020-01-31	NaN
3	Boston	5	Georgiana	2017-04-01	True
4	Washington DC	3	Evan	NaT	False
5	Las Vegas	11	Paul	2020-01-06	False

```
In [3]: df.info()
```

```

Out[3]: <class 'pandas.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Store        6 non-null      str
1   Employees    6 non-null      int64
2   Manager       6 non-null      str
3   Since        5 non-null      datetime64[us]
4   Flagship     5 non-null      object
dtypes: datetime64[us](1), int64(1), object(1), str(2)
memory usage: 463.0+ bytes

```

Everything looks good except for the Flagship column—its data type should be `bool` rather than `object`. To fix this, we can provide a converter function that deals with the offending cells in that column:

```

In [4]: def fix_missing(x):
        return False if x in ["", "MISSING"] else x

In [5]: df = pd.read_excel(
        "xl/stores.xlsx",
        sheet_name="2019",
        skiprows=1,
        usecols="B:F",
        converters={"Flagship": fix_missing},
        engine="calamine",
    )
df

```

```

Out[5]:

```

	Store	Employees	Manager	Since	Flagship
0	New York	10	Sarah	2018-07-20	False
1	San Francisco	12	Neriah	2019-11-02	False
2	Chicago	4	Katelin	2020-01-31	False
3	Boston	5	Georgiana	2017-04-01	True
4	Washington DC	3	Evan	NaT	False
5	Las Vegas	11	Paul	2020-01-06	False

```

In [6]: # The Flagship column now has Dtype "bool"
df.info()

Out[6]: <class 'pandas.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Store        6 non-null      str
1   Employees    6 non-null      int64
2   Manager       6 non-null      str
3   Since        5 non-null      datetime64[us]
4   Flagship     6 non-null      bool
dtypes: bool(1), datetime64[us](1), int64(1), str(2)
memory usage: 421.0 bytes

```

The `read_excel` function also accepts a list of sheet names. In this case, it returns a dictionary with the DataFrame as value and the name of the sheet as key. Also, note the slight variation in how I am using `usecols` by providing the column names of the table:

```
In [7]: sheets = pd.read_excel(
        "xl/stores.xlsx",
        sheet_name=["2019", "2020"],
        skiprows=1,
        usecols=["Store", "Employees"],
        engine="calamine",
    )
    sheets["2019"].head(2)
```

```
Out[7]:
```

	Store	Employees
0	New York	10
1	San Francisco	12



Read All Sheets

To read all sheets, provide the argument `sheet_name=None`.

If the source file doesn't have column headers, set `header=None` and provide them via names. Note that `sheet_name` also accepts sheet indices:

```
In [8]: df = pd.read_excel(
        "xl/stores.xlsx",
        sheet_name=0,
        skiprows=2,
        skipfooter=3,
        usecols="B:C,F",
        header=None,
        names=["Branch", "Employee_Count", "Is_Flagship"],
        engine="calamine",
    )
    df
```

```
Out[8]:
```

	Branch	Employee_Count	Is_Flagship
0	New York	10	False
1	San Francisco	12	MISSING
2	Chicago	4	NaN

To handle NaN values, use a combination of `na_values` and `keep_default_na`. The next sample tells pandas to only interpret cells with the word `MISSING` as NaN and nothing else:

```
In [9]: df = pd.read_excel(
        "xl/stores.xlsx",
        sheet_name="2019",
```

```

        skiprows=1,
        usecols="B,C,F",
        skipfooter=2,
        na_values="MISSING",
        keep_default_na=False,
        engine="calamine",
    )
df

```

Out[9]:

	Store	Employees	Flagship
0	New York	10	False
1	San Francisco	12	NaN
2	Chicago	4	
3	Boston	5	True

pandas offers an alternative way to read Excel files by using the `ExcelFile` class. This mostly makes a difference if you want to read in multiple sheets from a file in the legacy *xls* format with the default *xlrd* engine: in this case, using `ExcelFile` will be faster as it prevents pandas from reading in the whole file multiple times. `ExcelFile` can be used as a context manager (see “[Context Managers and the with Statement](#)” on [page 227](#)) so the file is properly closed again. Here is how you use `ExcelFile`:

```

In [10]: with pd.ExcelFile("xl/stores.xls") as f:
        df1 = pd.read_excel(f, "2019", skiprows=1, usecols="B:F", nrows=2)
        df2 = pd.read_excel(f, "2020", skiprows=1, usecols="B:F", nrows=2)

```

df1

Out[10]:

	Store	Employees	Manager	Since	Flagship
0	New York	10	Sarah	2018-07-20	False
1	San Francisco	12	Neriah	2019-11-02	MISSING

`ExcelFile` also gives you access to the names of all sheets:

```

In [11]: stores = pd.ExcelFile("xl/stores.xlsx", engine="calamine")
        stores.sheet_names

```

Out[11]: ['2019', '2020', '2019-2020']

Finally, pandas allows you to read Excel files from a URL, similar to what we did with CSV files in [Chapter 5](#). Let’s read it directly from the companion repo:

```

In [12]: url = (
        "https://raw.githubusercontent.com/fzumstein/"
        "python-for-excel/2e/xl/stores.xlsx"
    )
pd.read_excel(
    url,
    skiprows=1,
    usecols="B:E",
    nrows=2,
    engine="calamine",
)

```



```
Out[12]:
```

	Store	Employees	Manager	Since
0	New York	10	Sarah	2018-07-20
1	San Francisco	12	Neriah	2019-11-02

To summarize, [Table 14-2](#) shows you the most commonly used `read_excel` parameters. You will find the complete list in the [official docs](#).

Table 14-2. Selected parameters for `read_excel`

Parameter	Description
<code>sheet_name</code>	Instead of providing a sheet name, you could also provide the index of the sheet (zero-based), e.g., <code>sheet_name=0</code> . If you set <code>sheet_name=None</code> , pandas will read the whole workbook and return a dictionary in the form of <code>{"sheetname": df}</code> . To read a selection of sheets, provide a list with sheet names or indices.
<code>skiprows</code>	This allows you to skip over the indicated number of rows.
<code>usecols</code>	If the Excel file includes the names of the column headers, provide them in a list to select the columns, e.g., <code>["Store", "Employees"]</code> . Alternatively, it can also be a list of column indices, e.g., <code>[1, 2]</code> , or a string (not a list!) of Excel column names, including ranges, e.g., <code>"B:D,G"</code> . You can also provide a function: as an example, to only include the columns that start with <code>Manager</code> , use: <code>usecols=lambda x: x.startswith("Manager")</code> .
<code>nrows</code>	Number of rows you want to read.
<code>index_col</code>	Indicates which column should be the index; accepts a column name or an index, e.g., <code>index_col=0</code> . If you provide a list with multiple columns, a hierarchical index will be created.
<code>header</code>	If you set <code>header=None</code> , the default integer headers are assigned unless you provide the desired names via the <code>names</code> parameter.
<code>names</code>	Provide the desired names of your columns as a list.
<code>na_values</code>	Pandas interprets the following cell values as NaN by default (I introduced NaN in Chapter 5): empty cells, <code>#NA</code> , <code>NA</code> , <code>null</code> , <code>#N/A</code> , <code>N/A</code> , <code>NaN</code> , <code>n/a</code> , <code>-NaN</code> , <code>1.#IND</code> , <code>nan</code> , <code>#N/A</code> , <code>N/A</code> , <code>-1.#QNAN</code> , <code>-nan</code> , <code>NULL</code> , <code>-1.#IND</code> , <code><NA></code> , <code>1.#QNAN</code> . If you'd like to add one or more values to that list, provide them via <code>na_values</code> .
<code>keep_default_na</code>	If you'd like to ignore the default values that pandas interprets as NaN, set <code>keep_default_na=False</code> .
<code>converters</code>	Allows you to provide a function per column to convert its values. For example, to make the text in a certain column uppercase, use the following: <code>converters={"column_name": lambda x: x.upper()}</code> .
<code>engine</code>	Specify <code>engine="calamine"</code> to use a faster reader engine.

Now that you know how to read Excel files, let me show you how to speed up the reading process with big files.

Reading Sheets in Parallel

When you use pandas' `read_excel` function to read in multiple sheets of a big workbook, it can take a while—even with `engine="calamine"`. The reason is that pandas reads sheets sequentially, i.e., one after another. To speed things up, you

can read the sheets in parallel. While there is no easy way to parallelize the writing of workbooks due to the complexity of the internal file structure, reading multiple sheets in parallel is simple enough.

Python's standard library offers the multiprocessing module to take advantage of multiple CPU cores. The multiprocessing module spawns multiple Python interpreters (usually one per CPU core), which work on a task in parallel. Instead of processing one sheet after another, you have one Python interpreter working on the first sheet, while at the same time a second Python interpreter is processing the second sheet, etc. However, every additional Python interpreter takes some time to start up and uses additional memory, so if you have small files, parallelizing the reading process will most likely run slower instead of faster. In the case of a workbook with multiple big sheets, multiprocessing can speed up the process substantially, though—always assuming that your system has the required memory to handle the workload. The following function allows you to read sheets in parallel:

```
In [13]: import multiprocessing
         from functools import partial

         def read_excel_parallel(filename, sheet_name=0, engine=None):
             if sheet_name is None:
                 # Read all sheets
                 with pd.ExcelFile(filename, engine=engine) as xfile:
                     sheet_names = xfile.sheet_names

                 # Create a partial function with filename and engine prefilled.
                 # A partial function sets certain arguments of a function,
                 # creating a new function (read_func) with fewer parameters.
                 read_func = partial(pd.read_excel, filename, engine=engine)

                 with multiprocessing.Pool() as pool:
                     # map calls read_func for each sheet in parallel
                     dfs = pool.map(read_func, sheet_names)

                 # zip pairs sheet names with DataFrames into tuples
                 return dict(zip(sheet_names, dfs))
```



The Multiprocessing Module

If you'd like to understand this code and the underlying multiprocessing module better, have a look at the [multiprocessing docs](#).

The `read_excel_parallel` function behaves exactly the same as `pd.read_excel` and therefore returns a dictionary in the following form: `{"sheetname": df}`, i.e., keys are the sheet names and the values are the DataFrames. Let's see how much faster the parallelized version reads all sheets of the *big.xlsx* file that you will find in the companion repo's *xl* folder:

```

In [14]: %%time
         data = pd.read_excel("xl/big.xlsx", sheet_name=None, engine="calamine")

Out[14]: CPU times: user 3.04 s, sys: 104 ms, total: 3.15 s
         Wall time: 3.16 s

In [15]: data["Sheet1"].info()

Out[15]: <class 'pandas.DataFrame'>
         RangeIndex: 4999 entries, 0 to 4998
         Columns: 200 entries, col1 to col200
         dtypes: int64(200)
         memory usage: 7.6 MB

In [16]: %%time
         data = read_excel_parallel(
             "xl/big.xlsx",
             sheet_name=None,
             engine="calamine",
         )

Out[16]: CPU times: user 30.6 ms, sys: 90 ms, total: 121 ms
         Wall time: 1.31 s

In [17]: data["Sheet1"].info()

Out[17]: <class 'pandas.DataFrame'>
         RangeIndex: 4999 entries, 0 to 4998
         Columns: 200 entries, col1 to col200
         dtypes: int64(200)
         memory usage: 7.6 MB

```

Looking at the wall time of both samples, you'll see that the parallelized version was more than twice as fast as the nonparallelized `pd.read_excel` with this particular workbook on my machine with 8 CPU cores.

So much for reading Excel files with pandas—let's now switch sides and learn about writing Excel files!

The `to_excel` Method and `ExcelWriter` Class

The easiest way to write an Excel file with pandas is to use the `to_excel` `DataFrame` method. It allows you to specify which cell of which sheet you want to write the `DataFrame` to. You can also decide whether or not to include the column headers and the index of the `DataFrame` and how to treat data types like `np.nan` and `np.inf` that don't have an equivalent representation in Excel. Let's start by creating a `DataFrame` with different data types and use its `to_excel` method:

```

In [18]: import numpy as np
         import datetime as dt

In [19]: data = {
         "Dates": [
             dt.datetime(2020, 1, 1, 10, 13),

```

```

        dt.datetime(2020, 1, 2),
        dt.datetime(2020, 1, 2),
    ],
    "Floats": [2.222, np.nan, np.inf],
    "Integers": [1, 2, 3],
    "Booleans": [True, False, True],
}
df = pd.DataFrame(data)
df.index.name = "index"
df

Out[19]:
           Dates  Floats  Integers  Booleans
index
0    2020-01-01 10:13:00    2.222         1      True
1    2020-01-02 00:00:00     NaN         2     False
2    2020-01-02 00:00:00     inf         3      True

In [20]: df.to_excel(
    "written_with_pandas.xlsx",
    sheet_name="Output",
    startrow=1,
    startcol=1,
    index=True,
    header=True,
    na_rep="NA()", # Defaults to empty cell
    inf_rep="NA()", # Defaults to "inf"
)

```

Running the `to_excel` command will create the Excel file shown in [Figure 14-3](#) (you will need to make column C wider to see the dates properly):

	A	B	C	D	E	F
1						
2		index	Dates	Floats	Integers	Booleans
3		0	2020-01-01 10:13:00	2.222	1	TRUE
4		1	2020-01-02 00:00:00	#N/A	2	FALSE
5		2	2020-01-02 00:00:00	#N/A	3	TRUE

Figure 14-3. *written_with_pandas.xlsx*

If you want to write out multiple DataFrames, you will need to use the `ExcelWriter` class. The following sample writes the same DataFrame to two different locations on Sheet1 and one more time to Sheet2:

```

In [21]: with pd.ExcelWriter("written_with_pandas2.xlsx") as writer:
    df.to_excel(writer, sheet_name="Sheet1", startrow=1, startcol=1)
    df.to_excel(writer, sheet_name="Sheet1", startrow=10, startcol=1)
    df.to_excel(writer, sheet_name="Sheet2")

```

Since we're using the `ExcelWriter` class as a context manager, the file is automatically saved, i.e., written to disk, when the indentation stops. If you don't use a context manager, you will have to call `writer.save()` explicitly. For a summary of the most commonly used parameters that `to_excel` accepts, have a look at [Table 14-3](#). You will find the full list of parameters in the [official docs](#).

Table 14-3. Selected parameters for `to_excel`

Parameter	Description
<code>sheet_name</code>	Name of the sheet to write to.
<code>startrow</code> and <code>startcol</code>	<code>startrow</code> is the first row where the <code>DataFrame</code> will be written to, and <code>startcol</code> is the first column. This uses zero-based indexing, so if you want to write your <code>DataFrame</code> into cell B3, use <code>startrow=2</code> and <code>startcol=1</code> .
<code>index</code> and <code>header</code>	If you want to hide the index and/or header, set them to <code>index=False</code> and <code>header=False</code> , respectively.
<code>na_rep</code> and <code>inf_rep</code>	By default, <code>np.nan</code> will be converted to an empty cell, while <code>np.inf</code> , NumPy's representation of infinity, will be converted to the string <code>inf</code> . Providing values allows you to change this behavior. Using <code>=NA()</code> will create a proper #N/A error in Excel.
<code>freeze_panes</code>	Freeze the first couple of rows and columns by supplying a tuple: for example, <code>(2, 1)</code> will freeze the first two rows and the first column.

As you can see, reading and writing simple Excel files with pandas works well. There are limitations, though—let's see what they are!

Limitations

Using the pandas interface to read and write Excel files works great for simple cases, but there are limits:

- When writing `DataFrames` to files, you can't include a title or a chart.
- There is no way to change the default format of the header and index in Excel.
- When reading files, pandas automatically transforms cells with errors like `#REF!` or `#NUM!` into `NaN`, making it impossible to search for specific errors in your spreadsheets.
- Working with big Excel files may require extra settings that are easier to control by using the reader and writer packages directly, as we will see in the next chapter.

Conclusion

The nice thing about pandas is that it offers a consistent interface to work with all supported Excel file formats, whether that's *xls*, *xlsx*, *xlsm*, or *xlsb*. This made it easy for us to read a directory of Excel files, aggregate the data, and dump the summary into an Excel report—in only a few lines of code.

pandas, however, doesn't do the heavy lifting itself: under the hood, it selects a reader or writer package (the “engine”) to do the job. In the next chapter, I will show you which reader and writer packages pandas uses and how to use them directly or in combination with pandas. This will allow us to work around the limitations I mentioned in the previous section.

Excel File Manipulation with Reader and Writer Packages

This chapter introduces you to the reader and writer packages. They can read and write Excel files directly without the need of an Excel installation. pandas uses them under the hood when you call the `read_excel` or `to_excel` functions. Working with the reader and writer packages directly allows you to create more complex Excel reports. Also, if you work on a project where you need to read or write Excel files without the need for the rest of the pandas functionality, installing the full pandas stack would probably be overkill. We start this chapter with an overview of the available packages before learning how they work. After that, we'll look into styling DataFrames when writing them to Excel. To conclude, we will pick up the reporting case study one more time. Like the last chapter, this chapter does not require an installation of Excel, which means that all code samples run on Windows, macOS, and Linux.

Reading and Writing Excel Files

There are many Python packages that can read and write Excel workbooks. In this section, I'll give you an overview of those packages that we're going to cover in this chapter. To make things a little less overwhelming though, let me start with simple recommendations based on whether you want to read, write, or edit an Excel workbook:

Reading

Use either `python-calamine` or `fastexcel`. Both packages are built on top of `calamine`, a library programmed in Rust. They are best used via pandas or Polars (see [“Modern Alternatives” on page 125](#)) by setting `engine="calamine"`, but if you don't need a DataFrame, you can also use `python-calamine` on its own (`fastexcel`

has a standalone mode planned for their 1.0 release). Polars uses the calamine engine by default, but with pandas, you have to set it explicitly, as we've done throughout this book:

```
pd.read_excel("filename.xlsx", engine="calamine")
```

While pandas uses python-calamine as its calamine engine, Polars uses fastexcel. I'll introduce both packages in the next section.

Writing

Use XlsxWriter unless you hit a limitation—in this case, see if OpenPyXL or Excelize will solve your issue. The reasons I recommend XlsxWriter are:

- It's the fastest writer. While XlsxWriter is written in Python, there is now also a Rust-based XlsxWriter project from the same author. In the future, it will most likely be wrapped by a Python package that will be multiple times faster than the current pure Python implementation.
- It's very well documented.
- It's actively maintained.
- It creates fully compatible Excel workbooks.



Writing Engine with pandas

pandas uses either XlsxWriter or OpenPyXL as its writer engine, depending on which package you have installed. If you have both installed, XlsxWriter is the default. If you want to choose OpenPyXL instead, specify the engine parameter explicitly:

```
df.to_excel("filename.xlsx", engine="openpyxl")
```

Note that Polars uses XlsxWriter as its only writer engine.

Editing

Use OpenPyXL, as it is the only solution for modern Excel files. However, editing is a complex task as an Excel workbook consists of multiple files with interdependencies. So instead of literally editing the file, OpenPyXL reads the file, then re-creates it from scratch with only those features it supports. In other words, you're limited to working with simple files without complex charts, etc.

Apart from these packages, there are a few other packages that may occasionally be useful: Excelize can write PivotTables, and xlrd and xlwt can read and write the legacy *xls* workbook format. **Table 15-1** summarizes the technical capabilities of the various packages that we're looking into in this chapter.

Table 15-1. Packages to read, write, and edit Excel files

Excel file format	Read	Write	Edit
xlsx	fastexcel, python-calamine, OpenPyXL, Excelize	XlsxWriter, OpenPyXL, Excelize	OpenPyXL
xlsm	fastexcel, python-calamine, OpenPyXL, Excelize	XlsxWriter, OpenPyXL, Excelize	OpenPyXL
xltm, xlt	fastexcel, python-calamine, OpenPyXL, Excelize	OpenPyXL, Excelize	OpenPyXL
xlsb	fastexcel, python-calamine, pyxlsb	-	-
xls, xlt	fastexcel, python-calamine, xlrd	xlwt	xlutils

Before trying out the reader and writer packages in the next sections, we need to define a helper function that transforms a cell address from “A1” notation to a tuple of indices:

```
In [1]: # Import the regular expression module from the standard library
import re

In [2]: def address_to_indices(address):
        """Convert A1 notation to a zero-indexed tuple (row, col)."""
        # Use regex to split, e.g., "B10" into "B" and "10"
        match = re.match(r"([A-Z]+)(\d+)", address.upper())
        col_letters, row_str = match.groups()
        # Convert column letters to a number (A=1, ..., Z=26, AA=27, etc.)
        col_num = 0
        for char in col_letters:
            col_num = col_num * 26 + (ord(char) - ord("A") + 1)
        # Return as zero-indexed tuple
        return int(row_str) - 1, col_num - 1
```

With this helper function in place, let’s check out the reader and writer packages in detail, starting with those built on top of calamine.

Calamine-Based Excel Readers

The calamine library is written in Rust, a low-level programming language. It is much faster than the default engines used by pandas and is capable of reading all main Excel file types (*xlsx/xlsm*, *xlsb*, and *xls*). The only situation in which calamine will be slower than other reader packages is when you need just a few values from the top of a large sheet—calamine currently always reads the full sheet. *python-calamine* and *fastexcel* are both Python bindings for the Rust library. Since one is used by pandas and the other one by Polars, I am presenting them both in this section.

python-calamine

When you set `engine="calamine"`, pandas uses *python-calamine*, as we did in the last chapter:

```
pd.read_excel("myfile.xlsx", engine="calamine")
```

You can use python-calamine on its own to get a nested list:

```
In [3]: from python_calamine import CalamineWorkbook

In [4]: values = (
        CalamineWorkbook.from_path("xl/stores.xlsx")
        .get_sheet_by_name("2019")
        .to_python(skip_empty_area=True)
        )
values[:3]

Out[4]: [['Store', 'Employees', 'Manager', 'Since', 'Flagship'],
         ['New York', 10.0, 'Sarah', datetime.date(2018, 7, 20), False],
         ['San Francisco', 12.0, 'Neriah', datetime.date(2019, 11, 2), 'MISSING']]
```

`skip_empty_area=True` (True is the default) conveniently strips off the first empty row and column that we have on the Excel sheet “2019.” If you don’t want that, set it to False instead. While python-calamine is very fast, fastexcel offers an even faster way to create a pandas DataFrame from Excel data.

fastexcel

fastexcel is built on top of the same calamine library that python-calamine uses. Still, for large files, it can load an Excel sheet significantly faster into a pandas DataFrame than python-calamine while being much more memory efficient. This comes at a price, though: with pandas, fastexcel requires the PyArrow package, which is a heavy dependency. PyArrow is the Python library for Apache Arrow, which provides a memory format for efficient data access and interchange. With fastexcel, you can read in a pandas DataFrame as follows:

```
In [5]: import fastexcel

In [6]: # Requires PyArrow
reader = fastexcel.read_excel("xl/stores.xlsx")
sheet = reader.load_sheet("2019")
df = sheet.to_pandas()
df.head(2)

Out[6]:
```

	Store	Employees	Manager	Since	Flagship
0	New York	10.0	Sarah	2018-07-20	false
1	San Francisco	12.0	Neriah	2019-11-02	MISSING

Leading empty rows and columns are skipped automatically. In addition to reading values, fastexcel can also load data from an Excel table:

```
In [7]: reader = fastexcel.read_excel("xl/stores.xlsx")
reader.table_names()

Out[7]: ['Table2019', 'Table2020']

In [8]: # Requires PyArrow
table2019 = reader.load_table("Table2020").to_pandas()
table2019.head(2)
```

```
Out[8]:
```

	Store	Employees	Manager	Since	Flagship
0	New York	11.0	Sarah	2018-07-20	false
1	San Francisco	10.0	Neriah	2019-11-02	true



Using fastexcel with Polars DataFrames

fastexcel is the default Excel reading engine in Polars (see “[Modern Alternatives](#)” on page 125). Since Polars is built on Apache Arrow from the ground up, it doesn’t require the installation of PyArrow.

With the fastest reader libraries covered, let’s move on to the fastest writer library!

XlsxWriter

pandas and Polars both use **XlsxWriter** as their default engine to export DataFrames to Excel, and for good reason: it is the fastest package and produces fully compatible Excel files in the *xlsx* and *xlsm* formats. Let’s start with the basics before learning how to write large Excel workbooks.

Getting Started with XlsxWriter

As the name suggests, XlsxWriter can write Excel files but not read or edit them. The following code produces the report shown in [Figure 15-1](#). Note that XlsxWriter uses zero-based cell indices:

```
In [9]: import datetime as dt
import xlsxwriter

In [10]: # Helper function to write a nested list with XlsxWriter
def write_xlsxwriter(sheet, values, target_cell="A1"):
    target_cell = address_to_indices(target_cell)

    for ix, row_data in enumerate(values):
        sheet.write_row(target_cell[0] + ix, target_cell[1], row_data)

In [11]: # Instantiate a workbook
book1 = xlsxwriter.Workbook("xlsxwriter.xlsx")

# Add a sheet and give it a name
sheet = book1.add_worksheet("Sheet1")

# Writing individual cells using A1 notation
# and cell indices (0-based)
sheet.write("A1", "Hello 1")
sheet.write(1, 0, "Hello 2")

# Formatting: fill color, alignment, border and font
formatting = book1.add_format(
    {
        "font_color": "#FF0000",
```

```

        "bg_color": "#FFFF00",
        "bold": True,
        "align": "center",
        "border": 1,
        "border_color": "#FF0000",
    }
)
sheet.write("A3", "Hello 3", formatting)

# Number formatting (using Excel's formatting strings)
number_format = book1.add_format({"num_format": "0.00"})
sheet.write("A4", 3.3333, number_format)

# Date formatting (using Excel's formatting strings)
date_format = book1.add_format({"num_format": "mm/dd/yy"})
sheet.write("A5", dt.date(2016, 10, 13), date_format)

# Formula: you must use the English name of the formula
# with commas as delimiters
sheet.write("A6", "=SUM(A4, 2)")

# Image
sheet.insert_image(0, 2, "images/python.png")

# Two-dimensional list
data = [
    [None, "North", "South"],
    ["Last Year", 2, 5],
    ["This Year", 3, 6],
]
write_xlsxwriter(sheet=sheet, values=data, target_cell="A10")

# Chart: see the file "sales_report_xlsxwriter.py" in the
# companion repo to see how you can work with indices
# instead of cell addresses
chart = book1.add_chart({"type": "column"})
chart.set_title({"name": "Sales per Region"})
chart.add_series(
    {
        "name": "=Sheet1!A11",
        "categories": "=Sheet1!B10:C10",
        "values": "=Sheet1!B11:C11",
        "fill": {"color": "blue"},
    }
)
chart.add_series(
    {
        "name": "=Sheet1!A12",
        "categories": "=Sheet1!B10:C10",
        "values": "=Sheet1!B12:C12",
        "fill": {"color": "orange"},
    }
)

```

```

)
chart.set_x_axis({"name": "Regions"})
chart.set_y_axis({"name": "Sales"})
sheet.insert_chart("A15", chart)

# Closing the workbook creates the file on disk
book1.close()

```

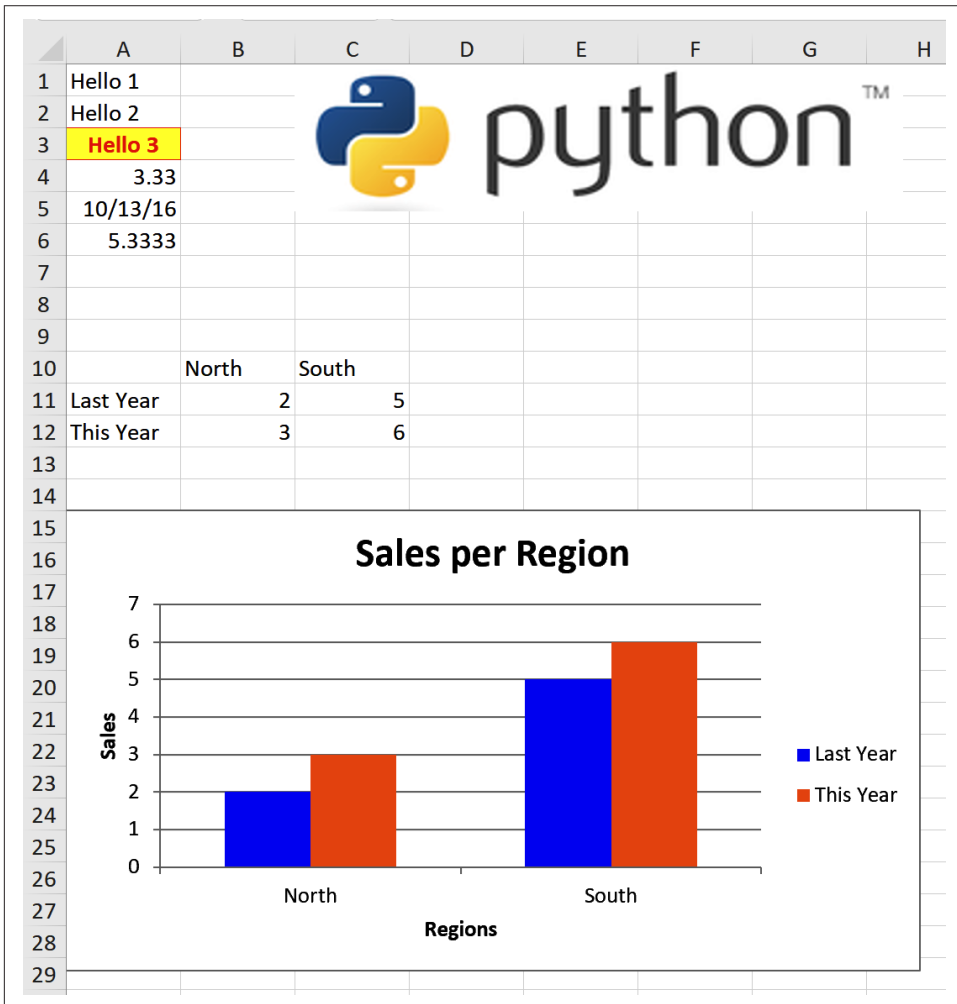


Figure 15-1. The Excel report produced by XlsxWriter (xlsxwriter.xlsx)

As you can see in the code, XlsxWriter is setting colors by providing a string like #FF0000. While the # symbol indicates that what follows is a hexadecimal color code, the actual value is made up of three hex values (FF, 00, and 00) that correspond to the red/green/blue values of the desired color. Hex stands for *hexadecimal* and

represents numbers using a base of sixteen instead of a base of ten, which is used by our standard decimal system.



Finding the Hex Value of a Color

To find the desired hex value of a color in Excel, click on the paint dropdown that you would use to change the fill color of a cell, then select More Colors. Now select your color and read off its hex value from the menu.

If you want to write *xlsm* files, you need to use the Terminal to extract the macro code from an existing Excel file (the example uses the *macro.xlsm* file, which you'll find in the *xl* folder of the companion repo):

Windows (Terminal)

```
cd %userprofile%\python-for-excel-2e
uv run .venv\Scripts\vba_extract.py xl\macro.xlsm
```

macOS (Terminal)

```
cd ~/python-for-excel-2e
uv run vba_extract.py xl/macro.xlsm
```

This will save the file *vbaProject.bin* in the root directory of the companion repository. I have, however, also included it in the *xl* folder of the companion repo so we can use it in the following sample to write a workbook with a macro button:

```
In [12]: book2 = xlsxwriter.Workbook("macro_xlsxwriter.xlsm")
        sheet = book2.add_worksheet("Sheet1")
        sheet.write("A1", "Click the button!")
        book2.add_vba_project("xl/vbaProject.bin")
        sheet.insert_button(
            "A3",
            {
                "macro": "Hello",
                "caption": "Button 1",
                "width": 130,
                "height": 35,
            },
        )
        book2.close()
```

At this point, we have covered the basic functionality of XlsxWriter. In the next section, you'll learn how XlsxWriter can write large datasets to an Excel workbook.

Writing Large Files with XlsxWriter

To keep memory usage under control, XlsxWriter has an option called `constant_memory`. It forces you to write sequential rows, i.e., you need to write

your data row by row—column by column wouldn't work. You enable the option by providing an options dictionary like this:

```
In [13]: import random

book3 = xlswriter.Workbook(
    "xlswriter_optimized.xlsx", options={"constant_memory": True}
)
sheet = book3.add_worksheet()
# This will produce a sheet with 1000 x 200 cells.
# With constant_memory, you must write data row by row, i.e.,
# you first need to loop through rows and then through columns.
for row in range(1000):
    for col in range(200):
        sheet.write(row, col, random.random())
book3.close()
```

XlsxWriter covers a lot more functionality than what I've shown you here; it is therefore a good idea to have a look at the [official docs](#). We will also see more functionality at the end of this chapter when we pick up the reporting case study one more time.

Now that you know how to work with XlsxWriter, let's have a look at OpenPyXL, which is both a reader and writer.

OpenPyXL

OpenPyXL can both read and write Excel files. You can even use it to edit Excel files—albeit only simple ones. OpenPyXL is the right choice if you need a pure Python package to read modern Excel files or if you prefer to have a single package to read and write files. Let's start by looking at how reading works!

Reading with OpenPyXL

The following sample code shows you how to perform common tasks when reading Excel files. To get the cell values, you need to open the workbook with `data_only=True`. The default is on `False`, which would return the formulas of the cells instead:

```
In [14]: import datetime as dt
import openpyxl
import pandas as pd

In [15]: # Open the workbook to read cell values.
# The file is automatically closed again after loading the data.
book4 = openpyxl.load_workbook("xl/stores.xlsx", data_only=True)

In [16]: # Get a worksheet object by name or index (0-based)
sheet = book4["2019"]
sheet = book4.worksheets[0]
```

```

In [17]: # Get a list with all sheet names
book4.sheetnames

Out[17]: ['2019', '2020', '2019-2020']

In [18]: # Loop through the sheet objects.
# Instead of "name", openpyxl uses "title".
for row_ix in book4.worksheets:
    print(row_ix.title)

Out[18]: 2019
2020
2019-2020

In [19]: # Getting the dimensions,
# i.e., the used range of the sheet
sheet.max_row, sheet.max_column

Out[19]: (8, 6)

In [20]: # Read the value of a single cell
# using "A1" notation and using cell indices (1-based)
sheet["B6"].value
sheet.cell(row=6, column=2).value

Out[20]: 'Boston'

In [21]: # Helper function to read an Excel range with OpenPyXL
# first_cell is the top-left cell, last_cell is the bottom-right cell
def read_openpyxl(sheet, first_cell="A1", last_cell=None):
    first_cell = openpyxl.utils.cell.coordinate_to_tuple(first_cell)
    if last_cell is None:
        last_cell = (sheet.max_row, sheet.max_column)
    else:
        last_cell = openpyxl.utils.cell.coordinate_to_tuple(last_cell)

    data = []
    for row in sheet.iter_rows(
        min_row=first_cell[0],
        min_col=first_cell[1],
        max_row=last_cell[0],
        max_col=last_cell[1],
        values_only=True,
    ):
        data.append(list(row))
    return data

In [22]: # Read in a range of cell values
data = read_openpyxl(
    sheet=book4["2019"],
    first_cell="B2",
    last_cell="F8",
)
data[:2] # Print the first two rows

```



```
Out[22]: [['Store', 'Employees', 'Manager', 'Since', 'Flagship'],
          ['New York', 10, 'Sarah', datetime.datetime(2018, 7, 20, 0, 0), False]]
```

Now that we've run through the most common read commands, let's see what you need to do when your files are big.

Reading Large Files with OpenPyXL

To keep memory under control when reading large Excel files with OpenPyXL, you should load the workbook with `read_only=True`. However, when using `read_only=True`, OpenPyXL doesn't close the file automatically anymore, so you need to make sure to call the `close` method when you're done (OpenPyXL doesn't support the `with` statement). If your file contains links to external workbooks, you may additionally want to use `keep_links=False` to make it faster. `keep_links` makes sure that the references to external workbooks are kept, which may unnecessarily slow down the process if you are only interested in reading the values of a workbook:

```
In [23]: book5 = openpyxl.load_workbook(
          "xl/big.xlsx", data_only=True, read_only=True, keep_links=False
        )
          # Perform the desired read operations here
          book5.close() # Required with read_only=True
```

Now that you know how to read Excel workbooks with OpenPyXL, let's move on by exploring its writing capabilities!

Writing with OpenPyXL

OpenPyXL builds the Excel file in memory and writes out the file once you call the `save` method. The following code produces the same report as we created with `XlsxWriter` (see [Figure 15-1](#)):

```
In [24]: import openpyxl
          from openpyxl.chart import BarChart, Reference
          from openpyxl.drawing.image import Image
          from openpyxl.styles import Font
          from openpyxl.styles.alignment import Alignment
          from openpyxl.styles.borders import Border, Side
          from openpyxl.styles.fills import PatternFill

In [25]: # Unfortunately, the latest version of OpenPyXL has an issue with
          # chart formatting. This patch sets the Application property to fix it.
          from openpyxl.packaging.extended import ExtendedProperties

          _original_init = ExtendedProperties.__init__

          def patched_init(self, *args, **kwargs):
              _original_init(self, *args, **kwargs)
              self.Application = "Microsoft Excel"
```

```

ExtendedProperties.__init__ = patched_init

In [26]: # Helper function to write a nested list with OpenPyXL
def write_openpyxl(
    sheet,
    values,
    target_cell="A1",
    date_format="mm/dd/yy",
):
    target_cell = openpyxl.utils.coordinate_to_tuple(target_cell)

    for row_ix, row in enumerate(values):
        for col_ix, value in enumerate(row):
            cell = sheet.cell(
                row=target_cell[0] + row_ix,
                column=target_cell[1] + col_ix,
            )
            cell.value = value
            # Format dates
            if isinstance(value, (dt.datetime, dt.date)):
                cell.number_format = date_format

In [27]: # Instantiate a workbook
book6 = openpyxl.Workbook()

# Get the first sheet and give it a name
sheet = book6.active
sheet.title = "Sheet1"

# Writing individual cells using A1 notation
# and cell indices (1-based)
sheet["A1"].value = "Hello 1"
sheet.cell(row=2, column=1, value="Hello 2")

# Formatting: fill color, alignment, border, and font
font_format = Font(color="FF0000", bold=True)
thin = Side(border_style="thin", color="FF0000")
sheet["A3"].value = "Hello 3"
sheet["A3"].font = font_format
sheet["A3"].border = Border(
    top=thin,
    left=thin,
    right=thin,
    bottom=thin,
)
sheet["A3"].alignment = Alignment(horizontal="center")
sheet["A3"].fill = PatternFill(fgColor="FFFF00", fill_type="solid")

# Number formatting (using Excel's formatting strings)
sheet["A4"].value = 3.3333
sheet["A4"].number_format = "0.00"

```

```

# Date formatting (using Excel's formatting strings)
sheet["A5"].value = dt.date(2016, 10, 13)
sheet["A5"].number_format = "mm/dd/yy"

# Formula: you must use the English name of the formula
# with commas as delimiters
sheet["A6"].value = "=SUM(A4, 2)"

# Image
sheet.add_image(Image("images/python.png"), "C1")

# Two-dimensional list
data = [
    [None, "North", "South"],
    ["Last Year", 2, 5],
    ["This Year", 3, 6],
]
write_openpyxl(sheet=sheet, values=data, target_cell="A10")

# Chart
chart = BarChart()
chart.type = "col"
chart.title = "Sales Per Region"
chart.x_axis.title = "Regions"
chart.y_axis.title = "Sales"
chart_data = Reference(
    sheet,
    min_row=11,
    min_col=1,
    max_row=12,
    max_col=3,
)
chart_categories = Reference(
    sheet,
    min_row=10,
    min_col=2,
    max_row=10,
    max_col=3,
)
# from_rows interprets the data in the same way
# as if you were adding a chart manually in Excel
chart.add_data(chart_data, titles_from_data=True, from_rows=True)
chart.set_categories(chart_categories)
sheet.add_chart(chart, "A15")

# Saving the workbook creates the file on disk
book6.save("openpyxl.xlsx")

```

Just as with reading large files, OpenPyXL offers several options that make writing large files more efficient.

Writing Large Files with OpenPyXL

When writing large files with OpenPyXL, make sure the `lxml` package is installed, as this makes the writing process faster. You can install `lxml` by running the following in a Terminal:

Windows (Terminal)

```
cd %userprofile%\python-for-excel-2e
uv add lxml
```

macOS (Terminal)

```
cd ~/python-for-excel-2e
uv add lxml
```

The critical option, though, is the `write_only=True` flag, which ensures that the memory consumption remains low. This flag, however, forces you to write row by row by using the `append` method, and won't allow you to write single cells anymore:

```
In [28]: book7 = openpyxl.Workbook(write_only=True)
        # With write_only=True, book.active doesn't work
        sheet = book7.create_sheet()
        # This will produce a sheet with 1000 x 200 cells
        for row in range(1000):
            sheet.append(list(range(200)))
        book7.save("openpyxl_optimized.xlsx")
```

With OpenPyXL's basic writing capabilities covered, let's now see how you can edit existing Excel files.

Editing with OpenPyXL

There is no Python package that can truly edit Excel files: in reality, OpenPyXL reads the file with everything it understands, then writes the file again from scratch—including any changes you make in between. This can be very powerful for simple Excel files containing mainly formatted cells, but it's limited when you have charts and other more advanced content in your spreadsheet, as OpenPyXL might change them or drop them altogether. For example, OpenPyXL might rename charts and drop their titles. Here is a simple editing example:

```
In [29]: # Read the stores.xlsx file, change a cell
        # and store it under a new location/name.
        book8 = openpyxl.load_workbook("xl/stores.xlsx")
        book8["2019"]["A1"].value = "modified"
        book8.save("stores_edited.xlsx")
```

If you want to write an *xlsm* file, OpenPyXL has to work off an existing file that you need to load with the `keep_vba` parameter set to `True`. The button in the example file calls a macro that shows a message box:

```
In [30]: book9 = openpyxl.load_workbook("xl/macro.xlsm", keep_vba=True)
book9["Sheet1"]["A1"].value = "Click the button!"
book9.save("macro_openpyxl.xlsm")
```

Like `XlsxWriter`, `OpenPyXL` covers a lot more functionality than I am able to cover in this section; it is therefore a good idea to have a look at the [official docs](#). We will also see more functionality at the end of this chapter when we pick up the reporting case study from the previous chapter again. Now, however, we'll move on to a few more packages that are helpful in more specific situations.

More Reader and Writer Packages

In this section, we'll explore a few additional packages that support reading and writing Excel files if you have specific needs. `Excelize` can write `PivotTables`, `pyxlsb` is a pure Python package for reading `xlsb` files, and `xlrd`/`xlwt`/`xlutils` can work with the legacy `xls` file format. Let's have a quick look at each of these packages!

Excelize

`Excelize` can read and write modern Excel files (`xlam`, `xlsm`, `xlsx`, `xltm`, `xltx`) and is implemented in the Go programming language. One use case for `Excelize` is to write Excel workbooks with `PivotTables` as none of the other packages currently support this—here is an example:

```
In [31]: import datetime as dt
import excelize

In [32]: book10 = excelize.new_file()
data = [
    ["Item", "Quantity", "Price"],
    ["Apple", 10, 1.20],
    ["Banana", 15, 0.75],
    ["Orange", 8, 1.50],
    ["Apple", 12, 1.25],
]

for row_ix, row_data in enumerate(data):
    book10.set_sheet_row("Sheet1", f"A{row_ix + 1}", row_data)

num_data_rows = len(data)

book10.add_pivot_table(
    excelize.PivotTableOptions(
        data_range="Sheet1!A1:C5",
        pivot_table_range="Sheet1!E1:G6",
        rows=[excelize.PivotTableField(data="Item")],
        data=[
            excelize.PivotTableField(
                data="Quantity", name="Total Quantity", subtotal="Sum"
            ),
        ],
    ),
```

```

        excelize.PivotTableField(
            data="Price", name="Average Price", subtotal="Average"
        ),
    ],
    row_grand_totals=True,
    col_grand_totals=True,
)
book10.save_as("excelize_pivot.xlsx")
book10.close()

```

This produces the pivot table shown in [Figure 15-2](#).

	A	B	C	D	E	F	G
1	Item	Quantity	Price			Values	
2	Apple	10	1.2		Row Labels	Total Quantity	Average Price
3	Banana	15	0.75		Apple	22	1.225
4	Orange	8	1.5		Banana	15	0.75
5	Apple	12	1.25		Orange	8	1.5
6					Grand Total	45	1.175

Figure 15-2. A pivot table produced by Excelize

To get a better idea of the rest of the Excelize syntax, have a look at cells 31–44 in *ch15.ipynb*, which you'll find in the companion repository. These cells produce the same Excel report that we created previously with *XlsxWriter* and *OpenPyXL* (see [Figure 15-1](#)).

Now that you know how to write PivotTables, let's move on to *pyxlsb*!

pyxlsb

pyxlsb can read Excel files in the binary *xlsb* format. In most cases, you should use *python-calamine* or *fastexcel* either directly or via *pandas* or *Polars* as they are faster and automatically convert date-formatted cells into *datetime* objects in Python. However, *pyxlsb* can still be useful if you need a pure Python package, so let's see how it works. You read sheets and cell values as follows:

```

In [45]: import itertools
         import pyxlsb

In [46]: # Helper function to read an Excel range with pyxlsb
         # first_cell is the top-left cell, last_cell is the bottom-right cell
         def read_pyxlsb(sheet, first_cell="A1", last_cell=None):
             errors = {
                 "0x0": "#NULL!",
                 "0x7": "#DIV/0!",
                 "0xf": "#VALUE!",
                 "0x17": "#REF!",
                 "0x1d": "#NAME?",

```

```

        "0x24": "#NUM!",
        "0x2a": "#N/A",
    }

    first_cell = address_to_indices(first_cell)
    first_cell = (first_cell[0] + 1, first_cell[1] + 1)

    if last_cell:
        last_cell = address_to_indices(last_cell)
        last_cell = (last_cell[0] + 1, last_cell[1] + 1)

    data = []
    # sheet.rows() is a generator that requires islice to slice it
    for row in itertools.islice(
        sheet.rows(),
        first_cell[0] - 1,
        last_cell[0] if last_cell else None,
    ):
        data.append(
            [errors.get(cell.v, cell.v) for cell in row][
                first_cell[1] - 1 : last_cell[1] if last_cell else None
            ]
        )
    return data

In [47]: # Loop through sheets. With pyxlsb, the workbook
# and sheet objects can be used as context managers.
# book.sheets returns a list of sheet names, not objects!
# To get a sheet object, use get_sheet() instead.
with pyxlsb.open_workbook("xl/stores.xlsb") as book12:
    for sheet_name in book12.sheets:
        with book12.get_sheet(sheet_name) as sheet:
            dim = sheet.dimension
            print(f"'{sheet_name}' has {dim.h} rows and {dim.w} cols")

Out[47]: '2019' has 7 rows and 5 cols
'2020' has 7 rows and 5 cols
'2019-2020' has 20 rows and 5 cols

In [48]: # Read in the values of a range of cells.
# Instead of "2019", you could also use its index (1-based).
with pyxlsb.open_workbook("xl/stores.xlsb") as book13:
    with book13.get_sheet("2019") as sheet:
        data = read_pyxlsb(sheet, "B2")
    data[:2] # Print the first two rows

Out[48]: [['Store', 'Employees', 'Manager', 'Since', 'Flagship'],
['New York', 10.0, 'Sarah', 43301.0, False]]

```

Since pyxlsb doesn't recognize cells with dates, you will have to manually convert values from date-formatted cells into datetime objects like so:

```
In [49]: from pyxlsb import convert_date
```

```
convert_date(data[1][3])
```

```
Out[49]: datetime.datetime(2018, 7, 20, 0, 0)
```

At this point, we have covered all libraries that deal with the modern version of Excel workbooks. Sometimes, however, you still need to work with the legacy version of Excel workbooks. In this case, you can use `xlrd`, `xlwt`, and `xlutils`.

xlrd, xlwt, and xlutils

`xlrd` reads, `xlwt` writes, and `xlutils` edits legacy *xls* files. These packages aren't actively developed anymore, but they can still be useful if you need to work with pure Python packages. If having a pure Python reader isn't a requirement, though, you should use `python-calamine` or `fastexcel` instead, either directly or via `pandas` or `Polars`. Let's now see how `xlrd` reads legacy Excel files!

Reading with xlrd

The following sample code shows you how to read the values from an Excel workbook with `xlrd`:

```
In [50]: import xlrd
         from xlrd.biffh import error_text_from_code
         from xlwt.Utills import cell_to_rowcol2
```

```
In [51]: # Open the workbook to read cell values. The file is
         # automatically closed again after loading the data.
         book14 = xlrd.open_workbook("xl/stores.xls")
```

```
In [52]: # Get a list with all sheet names
         book14.sheet_names()
```

```
Out[52]: ['2019', '2020', '2019-2020']
```

```
In [53]: # Loop through the sheet objects
         for sheet in book14.sheets():
             print(sheet.name)
```

```
Out[53]: 2019
         2020
         2019-2020
```

```
In [54]: # Get a sheet object by name or index (0-based)
         sheet = book14.sheet_by_index(0)
         sheet = book14.sheet_by_name("2019")
```

```
In [55]: # Dimensions
         sheet.nrows, sheet.ncols
```

```
Out[55]: (8, 6)
```

```
In [56]: # Read the value of a single cell
         # using "A1" notation and using cell indices (0-based).
```



```

# The "*" unpacks the tuple that cell_to_rowcol2 returns
# into individual arguments.
sheet.cell(*cell_to_rowcol2("B3")).value
sheet.cell(2, 1).value

```

Out[56]: 'New York'

In [57]: # Helper function to read an Excel range with xlrd
first_cell is the top-left cell, last_cell is the bottom-right cell
def read_xlrd(sheet, first_cell="A1", last_cell=None):

```

    if last_cell is None:
        last_cell = (sheet.nrows, sheet.ncols)
    else:
        last_cell = address_to_indices(last_cell)
        last_cell = (last_cell[0] + 1, last_cell[1] + 1)

    first_cell = address_to_indices(first_cell)
    first_cell = (first_cell[0] + 1, first_cell[1] + 1)

    values = []
    for r in range(first_cell[0] - 1, last_cell[0]):
        row = []
        for c in range(first_cell[1] - 1, last_cell[1]):
            if sheet.cell(r, c).ctype == xlrd.XL_CELL_DATE:
                value = xlrd.xldate.xldate_as_datetime(
                    sheet.cell(r, c).value, sheet.book.datemode
                )
            elif sheet.cell(r, c).ctype in [
                xlrd.XL_CELL_EMPTY,
                xlrd.XL_CELL_BLANK,
            ]:
                value = None
            elif sheet.cell(r, c).ctype == xlrd.XL_CELL_ERROR:
                value = error_text_from_code[sheet.cell(r, c).value]
            elif sheet.cell(r, c).ctype == xlrd.XL_CELL_BOOLEAN:
                value = bool(sheet.cell(r, c).value)
            else:
                value = sheet.cell(r, c).value
            row.append(value)
        values.append(row)
    return values

```

In [58]: # Read in a range of cell values
data = read_xlrd(sheet=sheet, first_cell="B2")
data[:2] # Print the first two rows

Out[58]: [['Store', 'Employees', 'Manager', 'Since', 'Flagship'],
['New York', 10.0, 'Sarah', datetime.datetime(2018, 7, 20, 0, 0),
False]]

With the basic functionality of xlrd covered, let's see what you have to do to read big files.

Reading large files with xlrd

When reading big files in the legacy *xls* format, xlrd allows you to load sheets on demand, like this:

```
In [59]: with xlrd.open_workbook("xl/stores.xls", on_demand=True) as book15:
         sheet = book15.sheet_by_index(0) # Only loads the first sheet
```

If you don't want to use the workbook as a context manager as we do here, you would need to call `book.release_resources()` manually to properly close the workbook again. To use xlrd in this mode with pandas, use it like this:

```
In [60]: with xlrd.open_workbook("xl/stores.xls", on_demand=True) as book16:
         with pd.ExcelFile(book16, engine="xlrd") as f:
             df = pd.read_excel(f, sheet_name=0)
```

Now that you know how to read legacy *xls* files with a pure Python package, let's see what it takes to write one!

Writing with xlwt

xlwt writes Excel files in the *xls* legacy format. The following code reproduces what we have done previously with XlsxWriter and OpenPyXL, as shown in [Figure 15-1](#). xlwt, however, cannot produce charts and only supports the bmp format for pictures:

```
In [61]: import datetime as dt
         import xlwt
         from xlwt.Utls import cell_to_rowcol2

In [62]: # Helper function to write a nested list with xlwt
         def write_xlwt(sheet, values, target_cell="A1", date_format="%m/dd/yy"):
             date_format = xlwt.easyxf(num_format_str=date_format)
             target_cell = address_to_indices(target_cell)

             for row_ix, row in enumerate(values):
                 for col_ix, cell in enumerate(row):
                     if isinstance(cell, (dt.datetime, dt.date)):
                         sheet.write(
                             row_ix + target_cell[0],
                             col_ix + target_cell[1],
                             cell,
                             date_format,
                         )
                     else:
                         sheet.write(
                             row_ix + target_cell[0],
                             col_ix + target_cell[1],
                             cell,
                         )

In [63]: # Instantiate a workbook
         book17 = xlwt.Workbook()
```

```

# Add a sheet and give it a name
sheet = book17.add_sheet("Sheet1")

# Writing individual cells using A1 notation
# and cell indices (0-based)
sheet.write(*cell_to_rowcol2("A1"), "Hello 1")
sheet.write(r=1, c=0, label="Hello 2")

# Formatting: fill color, alignment, border, and font
formatting = xlwt.easyxf(
    "font: bold on, color red;"
    "align: horiz center;"
    "borders: top_color red, bottom_color red,"
    "right_color red, left_color red,"
    "left thin, right thin,"
    "top thin, bottom thin;"
    "pattern: pattern solid, fore_color yellow;"
)
sheet.write(r=2, c=0, label="Hello 3", style=formatting)

# Number formatting (using Excel's formatting strings)
number_format = xlwt.easyxf(num_format_str="0.00")
sheet.write(3, 0, 3.3333, number_format)

# Date formatting (using Excel's formatting strings)
date_format = xlwt.easyxf(num_format_str="mm/dd/yyyy")
sheet.write(4, 0, dt.datetime(2012, 2, 3), date_format)

# Formula: you must use the English name of the formula
# with commas as delimiters
sheet.write(5, 0, xlwt.Formula("SUM(A4, 2)"))

# Two-dimensional list
data = [
    [None, "North", "South"],
    ["Last Year", 2, 5],
    ["This Year", 3, 6],
]
write_xlwt(sheet=sheet, values=data, target_cell="A10")

# Picture (only allows to add bmp format)
sheet.insert_bitmap("images/python.bmp", 0, 2)

# This writes the file to disk
book17.save("xlwt.xls")

```

Now that you know how writing legacy *xls* files works, the next section explains how you can edit them.

Editing with xlutils

`xlutils` acts as a bridge between `xlrd` and `xlwt`. This makes it explicit that this is not a true editing operation: `xlrd` reads the spreadsheet including the formatting (by setting `formatting_info=True`), and `xlwt` then writes it out again, including the changes that were made in between:

```
In [64]: import xlutils.copy

In [65]: book18 = xlrd.open_workbook("xl/stores.xls", formatting_info=True)
        book19 = xlutils.copy.copy(book18)
        book19.get_sheet(0).write(0, 0, "changed!")
        book19.save("stores_edited.xls")
```

At this point, you know how to read and write an Excel workbook in a specific format. The next section shows you how you can use `pandas` and the reader and writer packages together to produce nicer reports.

Writing Formatted DataFrames to Excel

To format `DataFrames` in Excel the way we want, we can write code that uses `pandas` together with `XlsxWriter` or `OpenPyXL`. We'll first use this combination to add a title to the exported `DataFrame`. We'll then format a `DataFrame`'s header and index before wrapping this section up by formatting the data part of a `DataFrame`.

When writing workbooks, you can combine `pandas` with `XlsxWriter`, allowing you to easily add a title to your `DataFrame` report:

```
In [66]: with pd.ExcelWriter(
        "pandas_and_xlsxwriter.xlsx",
        engine="xlsxwriter",
    ) as writer:
        df = pd.DataFrame({"col1": [1, 2, 3, 4], "col2": [5, 6, 7, 8]})
        # Write a DataFrame
        df.to_excel(writer, sheet_name="Sheet1", startrow=4, startcol=2)

        # Get the XlsxWriter workbook and sheet objects
        book20 = writer.book
        sheet = writer.sheets["Sheet1"]

        # From here on, it's XlsxWriter code
        sheet.write("A1", "This is a Title") # Write a single cell value
```

These samples use `XlsxWriter`, but it works conceptually the same with other engines like `OpenPyXL`. Let's now continue with formatting the index and headers of a `DataFrame`.

Formatting the Index and Headers of a DataFrame

To get complete control over the formatting of the index and column headers, you simply have to write them yourself. The following sample shows you how to do this with XlsxWriter. Let's start by creating a DataFrame:

```
In [67]: df = pd.DataFrame(  
        {"col1": [1, -2], "col2": [-3, 4]},  
        index=["row1", "row2"],  
        )  
df.index.name = "ix"  
df
```

```
Out[67]:
```

	col1	col2
ix		
row1	1	-3
row2	-2	4

To format the index and headers with XlsxWriter, do as follows:

```
In [68]: # Formatting index/headers with XlsxWriter  
with pd.ExcelWriter(  
    "formatting_xlsxwriter.xlsx",  
    engine="xlsxwriter",  
) as writer:  
    # Write out the df with the default formatting to A1  
    df.to_excel(writer, startrow=0, startcol=0)  
  
    # Write out the df with custom index/header formatting to F1  
    startrow, startcol = address_to_indices("F1")  
    # 1. Write out the data part of the DataFrame  
    df.to_excel(  
        writer,  
        header=False,  
        index=False,  
        startrow=startrow + 1,  
        startcol=startcol + 1,  
    )  
    # Get the book and sheet object and create a style object  
    book22 = writer.book  
    sheet = writer.sheets["Sheet1"]  
    style = book22.add_format({"bg_color": "#D9D9D9"})  
  
    # 2. Write out the styled column headers  
    for row_ix, col in enumerate(df.columns):  
        sheet.write(startrow, startcol + row_ix + 1, col, style)  
  
    # 3. Write out the styled index  
    index = [df.index.name if df.index.name else None] + list(df.index)  
    for row_ix, row in enumerate(index):  
        sheet.write(startrow + row_ix, startcol, row, style)
```

This code produces the output shown in [Figure 15-3](#). If you want to work with OpenPyXL instead, have a look at cell 69 in this chapter's notebook *ch15.ipynb*.

	A	B	C	D	E	F	G	H
1	ix	col1	col2			ix	col1	col2
2	row1	1	-3			row1	1	-3
3	row2	-2	4			row2	-2	4

Figure 15-3. A DataFrame with the default format (left) and with a custom format (right)

With the index and header formatted, let's see how we can style the data part!

Formatting the Data Part of a DataFrame

In the last chapter, we briefly looked at pandas Styler objects to format the data part of a DataFrame. In this section, we're doing the same, but instead of Styler objects, we're using the writer engine directly. To set the number format of the cells to three decimals and center-align the content as shown in [Figure 15-4](#), do the following with XlsxWriter:

```
In [70]: with pd.ExcelWriter(
            "data_format_xlsxwriter.xlsx",
            engine="xlsxwriter",
        ) as writer:
            # Write out the DataFrame
            df.to_excel(writer)

            # Get the book and sheet objects
            book23 = writer.book
            sheet = writer.sheets["Sheet1"]

            # Formatting the columns (individual cells can't be formatted)
            number_format = book23.add_format(
                {"num_format": "0.000", "align": "center"},
            )
            sheet.set_column(
                first_col=1,
                last_col=2,
                cell_format=number_format,
            )
```

If you want to use OpenPyXL instead, see cells 71–72 in this chapter's notebook *ch15.ipynb*.

	A	B	C
1	ix	col1	col2
2	row1	1.000	-3.000
3	row2	-2.000	4.000

Figure 15-4. A DataFrame with a formatted data part

If you need to customize the way date and time are shown in Excel, it's easiest to use the `date_format` and `datetime_format` options:

```
In [73]: df = pd.DataFrame(
    {
        "Date": [dt.date(2020, 1, 1)],
        "Datetime": [dt.datetime(2020, 1, 1, 10)],
    }
)
with pd.ExcelWriter(
    "date.xlsx",
    date_format="yyyy-mm-dd",
    datetime_format="yyyy-mm-dd hh:mm:ss",
) as writer:
    df.to_excel(writer)
```

The output of this code is shown in Figure 15-5.

	A	B	C
1		Date	Datetime
2	0	2020-01-01	2020-01-01 10:00:00

Figure 15-5. A DataFrame with formatted dates

Now that you know how to format DataFrames in Excel, it's time to pick up our reporting case study one last time and see how we can improve the Excel report with the knowledge we've gained in this chapter!

Reporting Case Study (Take 3)

Having made it to the end of this chapter, you know enough to be able to go back to the sales report from the last chapter and make it more visually appealing. If you like, open `sales_report_pandas.py` from the companion repository and try to turn it into the report shown in Figure 15-6.

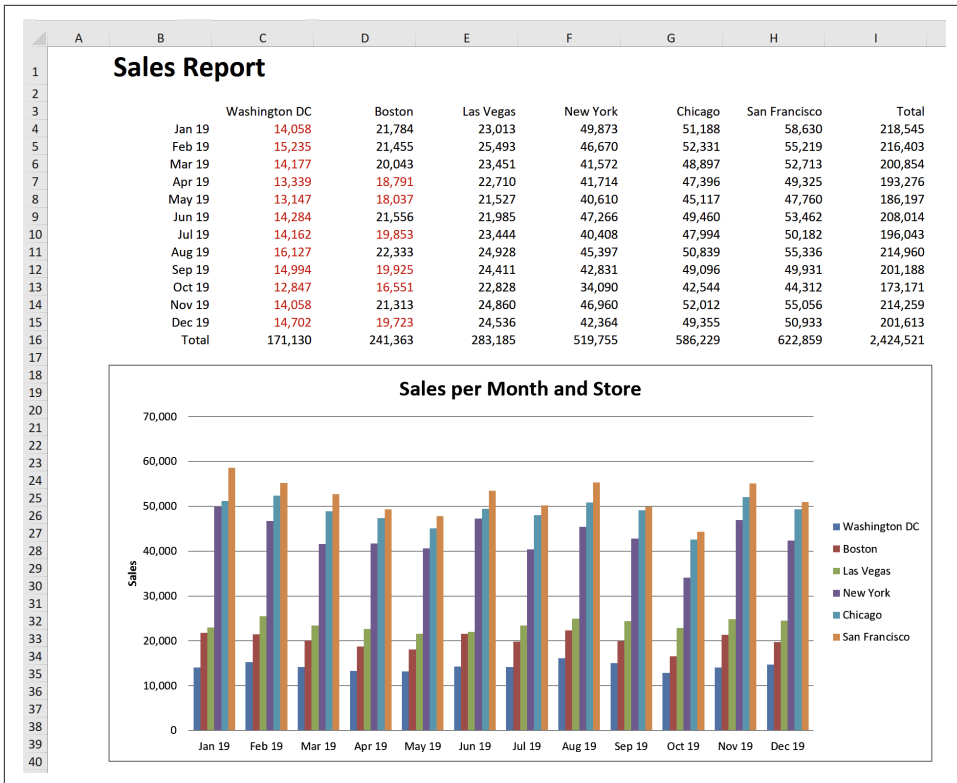


Figure 15-6. The sales report as created by `sales_report_xlsxwriter.py`

The red numbers are sales figures that are below 20,000. I haven't touched every aspect of formatting in this chapter (like how to apply conditional formatting), so you will have to use the documentation of the package you choose to work with. To compare your solution, I have included two versions of the script that produces this report in the companion repo. The first version is based on `XlsxWriter` (`sales_report_xlsxwriter.py`), and the other one is based on `OpenPyXL` (`sales_report_openpyxl.py`).

Conclusion

In this chapter, I introduced you to various reader and writer packages that allow you to read and write Excel files in Python without requiring an installation of Excel. You can use them on their own or in combination with `pandas` to customize your `DataFrame` reports.

We have touched on a lot of packages, so let me repeat my recommendation from the beginning of this chapter: use `pandas` or `Polars` with `engine="calamine"` to read

Excel files, and XlsxWriter to write Excel files. If you have additional needs or if you meet a limitation, turn to one of the other packages that I presented in this chapter.

And that's it—you have reached the end of the book! My main idea was to introduce you to the world of Python and its powerful scientific packages, to give you more power for your next Excel project. I hope this book will make your start easier as you now know how to:

- Replace an Excel workbook with a Jupyter notebook and pandas code
- Enhance Excel's analytical and visualization capabilities with Microsoft's Python in Excel solution
- Use Excel as a frontend and connect it to pretty much anything you want via xlwings or xlwings Lite, either by clicking a button or by writing a custom function
- Read Excel workbooks with pandas and produce a report by filling in an Excel template with xlwings
- Produce Excel reports without an installation of Excel by using XlsxWriter

Soon enough, however, you'll want to move beyond the scope of this book. I invite you to check the [book's home page](#) from time to time for updates and additional material. In this spirit, here are a few ideas that you could explore on your own:

- Schedule the periodic run of a Python script using either the Task Scheduler on Windows or a cron job on macOS or Linux. You could, for example, create an Excel report every Friday based on data you consume from a REST API or a database.
- Write a Python script that sends email alerts whenever the values in your Excel files satisfy a certain condition. Maybe that is when your account balance, consolidated from multiple workbooks, falls below a certain value, or when it shows a different value from what you expect based on your internal database.
- Write code that finds errors in Excel workbooks: check for cell errors like #REF! or #VALUE! or logical errors like making sure that a formula includes all the cells it should. If you start tracking your mission-critical workbooks with a professional version control system like Git, you can even run these tests automatically whenever you commit a new version.
- For further ideas that align with your own work, don't forget that AI tools such as ChatGPT and Copilot don't just write and review code but are also good sparring partners for exploring new approaches to your problems, brainstorming ideas, and prototyping your next spreadsheet solution!

If this book inspires you to automate your daily or weekly routine of downloading data and copy/pasting it into Excel, I couldn't be happier. Automation doesn't just give you back time; it also dramatically reduces the chance of committing errors. If you have any feedback, please let me know about it! You can contact me via O'Reilly, by opening an issue on the [companion repository](#) or on X at [@felixzumstein](#).

The uv Package Manager

In [Chapter 2](#), I instructed you to run `uv sync`, which took care of installing Python and all the packages that we used in this book. In this appendix, I'll show you how to create a new project with uv and how to add, remove, and upgrade dependencies. I'll also introduce the `--project` option that lets you work outside your project directory. I'll conclude with a brief overview of other package managers that you should know about.



Prerequisites

This appendix assumes that you have uv installed as described in [Chapter 2](#).

Creating a New Project

To create a new project with uv, open the Terminal and change into the directory where you'd like to create your project folder. For the purpose of this appendix, call the project *myproject* and create it in your user directory:

Terminal (Windows)

```
cd %USERPROFILE%  
uv init --python=3.14 myproject
```

Terminal (macOS)

```
cd ~  
uv init --python=3.14 myproject
```

Replace `--python=3.14` with the Python version you want to use. You can get a list of available Python versions by running `uv python list`.



Select a Local Folder

Make sure to create uv projects in a local folder on your machine. Since uv installs the Python environment into the project folder, using services like OneDrive or Dropbox will cause the virtual environment to be uploaded to the cloud, which will be very slow.

Now change into the `myproject` directory and sync the environment:

Terminal (Windows and macOS)

```
cd myproject
uv sync
```

This creates the virtual environment (`.venv` directory) and the `uv.lock` file—I will say more about them in a moment.



Sync Your Project Before Opening It in VS Code

Run `uv sync` in a Terminal *before* opening the project in VS Code, as this helps VS Code find the correct virtual environment without manual intervention.

After running `uv sync`, open the project in VS Code (remember, the dot in the command refers to the current directory):

Terminal (Windows and macOS)

```
code .
```

In the VS Code explorer, you'll see a couple of files that uv generated:

This is the folder with the virtual environment. For an explanation of what a virtual environment is, see [Chapter 2](#).

This contains a list of files that won't be tracked by Git, a version control system (only created if Git is installed).

This file pins your Python version according to `--python` in the first command. If needed, you can edit the file manually or by running `uv python pin 3.14`.

main.py

This is just a sample file so you can run `uv run main.py` immediately to see if everything works.

pyproject.toml

This is a general-purpose configuration file used by Python. uv uses it to store your top-level dependencies. *toml* is a file format designed for configuration files and is similar to *ini* files.

README.md

Here you could add a description of your project.

uv.lock

This is where `uv` pins the exact version of every package, including sub-dependencies. When you do `uv sync`, the dependencies are installed according to this file.

Now that we have created a new project, let's see how we can add and remove packages!

Adding and Removing Packages

You can add dependencies, i.e., packages, by using `uv add`. For example, to add Requests and xlwings, run the following command:

Terminal (Windows and macOS)

```
uv add requests xlwings
```

Running this command will do the following:

- Add the dependencies, i.e., packages, to the *pyproject.toml* file under `project.dependencies`
- Add the dependencies, including sub-dependencies, to *uv.lock*
- Install the packages in the virtual environment



Migrate from a requirements.txt File

If you want to migrate from a traditional `requirements.txt` file, you can run `uv add -r requirements.txt`.

To remove a dependency again, use `uv remove`. For example, to remove Requests again, run:

Terminal (Windows and macOS)

```
uv remove requests
```

This will update the *pyproject.toml* and *uv.lock* files and remove the package from the virtual environment.

Key Things to Remember

Here is a summary of the most fundamental concepts when using uv:

- Always run the commands from the project directory.
- Add packages: `uv add mypackage`.
- Remove packages: `uv remove mypackage`.
- Run the `main.py` script from the Terminal: `uv run main.py`.
- Start an interactive Python REPL: `uv run python`.

Now that you know how to create a new project and how to add and remove dependencies, let's see how you can keep your packages up-to-date.

Upgrading Packages and uv

In this section, I'll show you how to keep both the packages and uv updated. To update uv itself, run the following command:

Terminal (Windows and macOS)

```
uv self update
```

To upgrade all packages, run this:

Terminal (Windows and macOS)

```
uv sync --upgrade
```

If you want to upgrade only a specific package, e.g., `xlwings`, the command is as follows:

Terminal (Windows and macOS)

```
uv sync --upgrade-package xlwings
```

Note that these upgrade commands respect any version constraints you've specified in `pyproject.toml`. For example, if you've pinned a package to a specific version, uv won't upgrade it even if newer versions are available.

So far, we've always run the uv command in the project directory. In the next section, I'll show you how you can use the virtual environment of a project from outside that directory.

Working Outside the Project Directory

If you're used to activating environments (either Conda environments or virtual environments), you can achieve a similar workflow using the `--project` flag. This lets you use a specific project's virtual environment from any directory.

Terminal (Windows)

```
uv run --project %USERPROFILE%\myproject main.py
```

Terminal (macOS)

```
uv run --project ~/myproject main.py
```

If you want to use the same project for a couple of commands, you can also set the `UV_PROJECT` environment variable instead of providing the `--project` argument with every command.

This was a high-level introduction to uv. In the next section, I will show you a few other package managers that you should know about.

Other Package Managers

uv came out in 2024 but has taken the Python world by storm. In this section, I'll introduce you to three other package managers: pip, Conda, and Pixi. pip predates uv, while Conda and Pixi have an extended scope as they also manage non-Python dependencies. Here are a few more details:

pip

pip is the traditional Python package manager. Instead of running `uv add requests`, you would run `pip install requests`. For pip, you can manually add dependencies to a file that is—by convention—called *requirements.txt*, and that can be installed using the following command: `pip install -r requirements.txt`.

Conda

Conda is a general-purpose package manager. Apart from installing Python packages, it can install dependencies that are otherwise hard to install, e.g., the GLPK (GNU Linear Programming Kit) package that isn't available as a Python package on PyPI. While Conda is created by Anaconda (the company, not the Python distribution), Conda is open source, and there is a community-led channel, **conda-forge**, that doesn't have any usage restrictions.

Pixi

Pixi combines uv and the conda-forge channel under one roof. It provides a project-based workflow like uv, and it's a great option if you regularly need to install dependencies from outside the Python ecosystem.

Copilot in Excel

Ever since OpenAI announced ChatGPT on November 30, 2022, the world hasn't been quite the same. ChatGPT is a chatbot that relies on *artificial intelligence* (AI) and that has an answer to almost anything—although not always the correct one. More specifically, ChatGPT uses *generative artificial intelligence* (GenAI), meaning it can process and create content such as text, images, or sound. To work with text content, including code, GenAI relies on a so-called *large language model* (LLM). In simple terms, an LLM is a neural network that is trained on vast amounts of text data.

While Microsoft is one of OpenAI's investors, they maintain their own AI offering under the name *Copilot*. The first stable version of Copilot in Excel was released in September 2024, so it's still in the early stages, and Copilot in Excel will likely evolve at a rapid pace.

This appendix gives you an overview of the different Copilot versions from Microsoft and a general introduction to prompt engineering. We'll also see what the requirements are to be able to use Copilot in Excel and how to use it in agent mode.

The Copilot Family

Copilot is Microsoft's brand name for its suite of AI tools and consists of Microsoft Copilot, Microsoft 365 Copilot, and GitHub Copilot:

Microsoft Copilot

Microsoft Copilot is a web-based AI assistant. On Windows 11, it is also available as a preinstalled app. You can upload Excel workbooks and ask Copilot questions about its data. In this form, Microsoft Copilot has many competitors, including **ChatGPT**, **Gemini**, **Claude**, or **DeepSeek**.

Microsoft 365 Copilot

Microsoft 365 Copilot is integrated into Office products such as Word and Excel and can interact directly with the respective Office application. Excel's implementation is called *Copilot in Excel*.

GitHub Copilot

GitHub is a platform for hosting code (in fact, it hosts the companion repo for this book) and was acquired by Microsoft in 2018. Microsoft launched GitHub Copilot eight months before ChatGPT but didn't add a chat interface until July 2023. You can access GitHub Copilot directly from a repository or from [its home page](#). Most importantly, though, it is integrated into code editors like VS Code or PyCharm.

In the next section, we're going to look at a few guidelines that are helpful with all AI tools, including the various implementations of Copilot.

Prompt Engineering

The term *prompt engineering* is commonly used to describe the art of asking AI tools such as Copilot the right questions. The following guidelines will help you become a better prompt engineer:

Provide specific and explicit context

The more context you add to your prompt, the better the results will usually be. For example, if you want the output to be an Excel table, you should mention that in the prompt. Most AI tools have a way to store custom instructions that will be taken into account with every prompt. Copilot in Excel offers this under Settings > Personalization > Custom Instructions. A good structure of your prompt includes role, task, requirements, and output, e.g.: "You are an Excel assistant for a financial analyst (role). Add a new column called Total Revenue to the Excel table Sales (task). Use only built-in formulas but no VBA (requirements). Make sure that the new column is part of the existing table Sales (output)."

Feed the error back

Error messages are good prompts! If your own code or the one produced by AI causes an error, copy and paste the error message back to the chat, asking it to fix the code based on that error message. Agentic AI tools, i.e., AI tools that can take actions on your behalf, usually do this automatically.

Reference previous context

AI chats allow you to have a natural conversation without the need to repeat details from previous prompts. For example, you can ask a follow-up question that can be as easy as "Can you simplify this?" This also means that you can split

up complex tasks into many smaller prompts, which can yield better results than providing a single long prompt.

Provide examples

If you find a relevant example somewhere online (official documentation are a good place to look), you can copy and paste it into the chat (often just pasting the URL to the page also works), stating that it's the general methodology you want to follow. This is especially helpful if you are dealing with newer technology that AI tools haven't properly picked up yet.

Don't provide sensitive information

Most AI tools learn from your questions, so don't include anything sensitive in your prompts like an API key or a database password.

Clear previous context

As much as it's useful to reference previous context, you do want to start from scratch for new topics. Usually, AI chats offer a button or command to either start a new conversation or to clear the prompt history.

Position your cursor

For integrated tools like Copilot in Excel or GitHub Copilot in VS Code, make sure to activate or select the correct content, if it makes sense to do so. For example, in Excel, activate the proper worksheet and put the cursor in a relevant cell. In VS Code, you want to open the correct file or Jupyter notebook and select the relevant code.

Be aware that LLMs are nondeterministic

If you ask an AI assistant the same question multiple times, you'll get a different answer every time. This is due to several factors:

- Nondeterministic nature: even the same model will typically give different responses to identical prompts because they use probabilistic word selection instead of deterministic algorithms.
- Technical differences: models use different training methodologies, parameters, and training datasets. They also have different knowledge cutoff dates, meaning some models have access to more recent information than others.
- Different objectives: models have different goals and are fine-tuned accordingly—some prioritize creativity, while others focus on code generation.

Hallucinations

Despite its confident answers, AI regularly makes up things that don't exist, e.g., it might invent an Excel formula. In this case, let it know that the formula doesn't exist or copy and paste the error back into the chat. It also means that you should always have at least a basic knowledge about the things you're asking Copilot.

Otherwise, it's easy to get fooled into a solution that seems to work at first sight but in reality has subtle bugs.

With these guidelines in mind, we are now ready to start Copilot!

First Steps with Copilot in Excel

Copilot in Excel makes the chat interface available from within Excel so you don't need to switch context when interacting with AI. However, the real advantage compared to an external chat tool is the ability to manipulate your workbook directly.

In this section, I'll first go through the requirements for Copilot in Excel, and then we'll use it in agent mode to add a column to an existing table.

Requirements

To be able to use Copilot in Excel, you need to do the following:

- Ensure you have a Microsoft 365 subscription. Versions of Office that come with a permanent license (e.g., Office 2024) are not supported.
- Turn AutoSave on. This requires you to store your workbook in Microsoft's cloud, i.e., on OneDrive or SharePoint. Note that more recent versions of Copilot also work with Excel files that are stored on your local disk.
- This is optional, but most likely, you also need a separate Microsoft Copilot Pro subscription. While Microsoft 365 includes a free allowance for Copilot, you'll burn through it quickly if you start using it on a regular basis.

If you see the Copilot button in the Home tab of your Excel ribbon, as shown in **Figure B-1**, you are all set. If not, you may have to update your version of Excel or ask your IT administrator to do it.

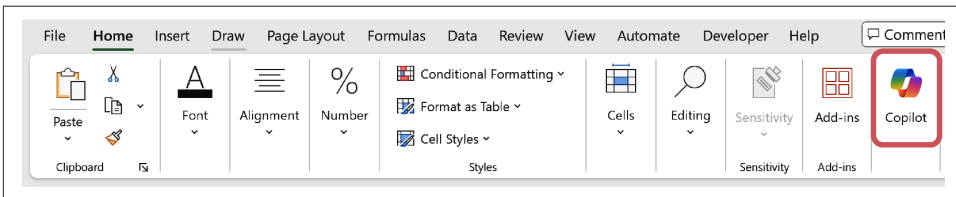


Figure B-1. The Copilot icon in the Home tab of the ribbon menu

With the requirements out of our way, let me show you how to start a conversation with Copilot.

Starting a Conversation

To start a conversation with Copilot in Excel, click the Copilot button, as shown in [Figure B-1](#). This opens the Copilot task pane on the right-hand side of Excel. While you can start chatting with it immediately, it gets a lot more powerful if you first click the Tools button and select Agent mode, as shown in [Figure B-2](#). Note that in more recent versions of Copilot, Agent mode is the default, so you can skip this step.

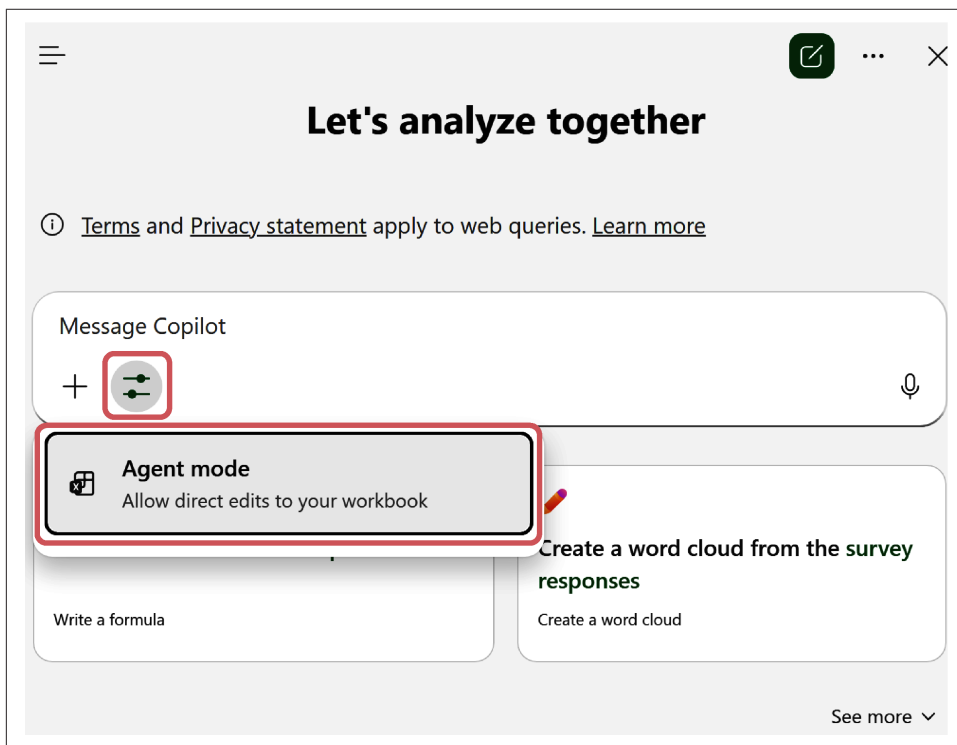


Figure B-2. Activating Copilot Agent Mode (not required in more recent versions of Excel)

When selecting Agent mode, you give Copilot in Excel permission to edit your workbook directly. To see how this works, let's copy *xl/ch07.xlsx* from the companion repository to OneDrive or SharePoint so you can enable AutoSave. With the sheet MSFT activated, I simply ask it to add the log return of the `adj_close` column to the `TableMSFT`, i.e., the table with the Microsoft stock history. For a refresher on log returns, you may want to have another look at [Chapter 7](#). After submitting my request in the chat, Copilot adds an additional column to the `TableMSFT` with the correct Excel formula (see [Figure B-3](#)).

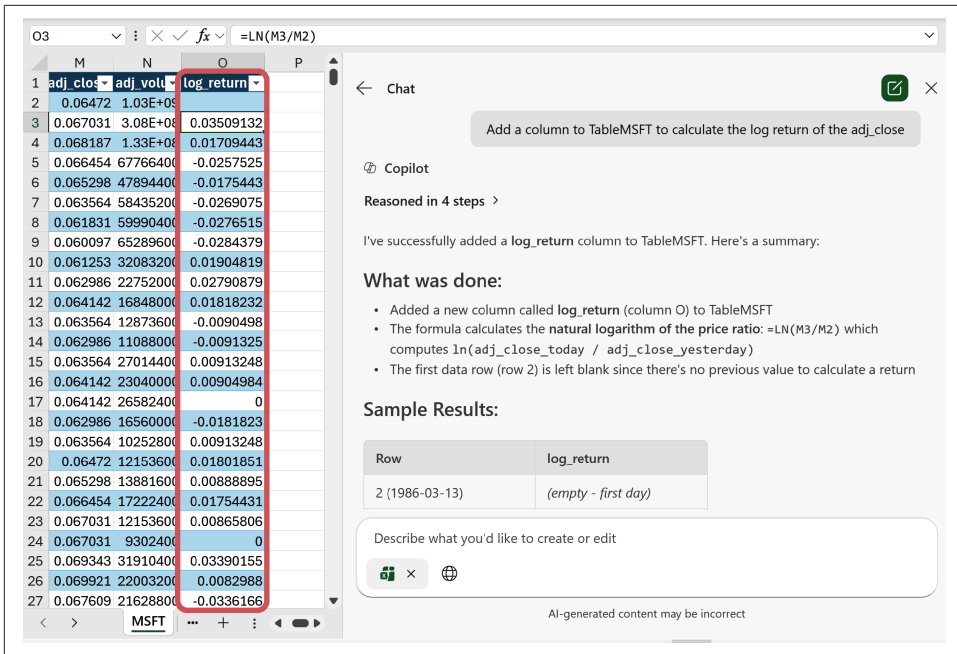


Figure B-3. Asking Copilot in Excel to add an additional column

The next logical step would have been to ask Copilot to write Python code. Unfortunately, at the time of this writing, Copilot in Excel doesn't support Python in Excel yet. While it was previously supported via *advanced analysis mode* and later via *Excel App Skills*, it is now planned to be integrated into the normal agent mode that we've just used. So when you're reading this, make sure to check if Copilot in Excel can insert Python in Excel cells!

Advanced Python Concepts

In this appendix, we're taking a closer look at the following three topics: classes and objects, time-zone-aware datetime objects, and mutable versus immutable objects. The topics are independent of one another, so you may read them in any order.

Classes and Objects

In this section, we will write our own class to get a better understanding of how classes and objects are related. Classes define new types of objects: a class behaves like a blueprint you use to build a car. Depending on the specifications you use, you get a different car—for example, a blue car or a red car. The process of getting a car (the object) from the blueprint (the class) is called *instantiation*, which is why objects are also called *class instances*. Classes allow you to define new data types that keep related data (*attributes*) and functions (*methods*) together and therefore help you structure and organize your code. Let's now define a Car class by using a data class. You turn a regular class into a *data class* with the `@dataclass` decorator. Data classes make it easier to write classes by taking care of boilerplate code that you would have to write with regular classes:

```
In [1]: from dataclasses import dataclass

In [2]: @dataclass
        class Car:
            # Data classes require type hints for attributes
            color: str
            speed: int = 0

        def accelerate(self, speed_increase):
            self.speed += speed_increase
```

This is a simple car class with two attributes (color and speed) and one method (accelerate). Methods are functions that are part of a class definition. The accelerate method will change the data (speed) of an instance of this class. First, let's instantiate two cars. You do this in the same way you call a function: call the class by adding parentheses and by providing the arguments that correspond to the attributes defined in the data class:

```
In [3]: # Let's instantiate two car objects
        car1 = Car("red")
        car2 = Car(color="blue")
```

Everything with regard to function arguments applies here as well: for car1, we provide the argument as positional argument, while for car2, we use keyword arguments. After instantiating the two car objects from the Car class, we'll have a look at their attributes and call their methods. As we will see, after accelerating car1, the speed of car1 is changed, but it is unchanged for car2 as the two objects are independent of each other:

```
In [4]: # Attributes give you access to the data of an object
        car1.color

Out[4]: 'red'

In [5]: car1.speed

Out[5]: 0

In [6]: # Calling the accelerate method on car1
        car1.accelerate(20)

In [7]: # The speed attribute of car1 changed
        car1.speed

Out[7]: 20

In [8]: # The speed attribute of car2 remained the same
        car2.speed

Out[8]: 0
```

Python also allows you to change attributes directly without having to use methods:

```
In [9]: car1.color = "green"

In [10]: car1.color

Out[10]: 'green'

In [11]: car2.color # unchanged

Out[11]: 'blue'
```

To summarize: classes define the attributes and methods of objects. Classes allow you to group related functions (“methods”) and data (“attributes”) so that they can conveniently be accessed via dot notation: `myobject.attribute` or `myobject.method()`.

Working with Time-Zone-Aware datetime Objects

In [Chapter 3](#), we briefly looked at time-zone-naive datetime objects. If time zone is of importance, you usually work in the *UTC* time zone and only transform to local time zones for display purposes. UTC stands for *Coordinated Universal Time* and is the successor of Greenwich Mean Time (GMT).



Windows Requires tzdata

Since Windows doesn't have the IANA time zone database that ZoneInfo uses, you need to install it via the Python package tzdata. We installed it during our initial setup in [Chapter 2](#), so there's nothing you need to do.

The following samples show a few common operations when working with datetime objects and time zones:

```
In [12]: import datetime as dt
        from zoneinfo import ZoneInfo

In [13]: # Time-zone-naive datetime object
        timestamp = dt.datetime(2020, 1, 31, 14, 30)
        timestamp.isoformat()

Out[13]: '2020-01-31T14:30:00'

In [14]: # Time-zone-aware datetime object
        timestamp_eastern = dt.datetime(
            2020, 1, 31, 14, 30, tzinfo=ZoneInfo("US/Eastern")
        )
        timestamp_eastern

Out[14]: datetime.datetime(2020, 1, 31, 14, 30,
        tzinfo=zoneinfo.ZoneInfo(key='US/Eastern'))

In [15]: # Printing in isoformat makes it easy to
        # see the offset from UTC
        timestamp_eastern.isoformat()

Out[15]: '2020-01-31T14:30:00-05:00'

In [16]: # Assign a time zone to a naive datetime object
        timestamp_eastern = timestamp.replace(tzinfo=ZoneInfo("US/Eastern"))
        timestamp_eastern.isoformat()

Out[16]: '2020-01-31T14:30:00-05:00'

In [17]: # Convert from one time zone to another
        timestamp_utc = timestamp_eastern.astimezone(ZoneInfo("UTC"))
        timestamp_utc.isoformat()

Out[17]: '2020-01-31T19:30:00+00:00'
```

```

In [18]: # From time-zone-aware to naive
         timestamp_eastern.replace(tzinfo=None)

Out[18]: datetime.datetime(2020, 1, 31, 14, 30)

In [19]: # Current time without time zone
         dt.datetime.now()

Out[19]: datetime.datetime(2026, 1, 29, 17, 46, 20, 485545)

In [20]: # Current time in UTC time zone
         dt.datetime.now(ZoneInfo("UTC"))

Out[20]: datetime.datetime(2026, 1, 29, 16, 46, 20, 508773,
                           tzinfo=zoneinfo.ZoneInfo(key='UTC'))

```

Mutable Versus Immutable Python Objects

In Python, objects that can change their values are called *mutable*, and those that can't are called *immutable*. Table C-1 shows how the most common data types qualify.

Table C-1. Mutable and immutable data types

Mutability	Data types
mutable	lists, dictionaries, sets
immutable	integers, floats, booleans, strings, datetime, tuples

Knowing the differences is important as mutable objects may behave differently from what you would expect, especially if you come from VBA. Have a look at the following VBA snippet:

```

Dim a As Variant, b As Variant
a = Array(1, 2, 3)
b = a
a(1) = 22
Debug.Print a(0) & ", " & a(1) & ", " & a(2)
Debug.Print b(0) & ", " & b(1) & ", " & b(2)

```

This prints the following:

```

1, 22, 3
1, 2, 3

```

Now let's do the same example in Python with a list:

```

In [21]: a = [1, 2, 3]
         b = a
         a[1] = 22
         print(a)
         print(b)

Out[21]: [1, 22, 3]
         [1, 22, 3]

```

What happened here? In Python, variables are names that you “attach” to an object. By doing `b = a`, you attach both names to the same object, the list `[1, 2, 3]`. All variables attached to that object will, therefore, show the changes to the list. This only happens with mutable objects, though: if you want to replace the list with an immutable object like a tuple, changing `a` would not change `b`. If you want a mutable object like `b` to be independent of changes in `a`, you have to explicitly copy the list:

```
In [22]: a = [1, 2, 3]
         b = a.copy()

In [23]: a
Out[23]: [1, 2, 3]

In [24]: b
Out[24]: [1, 2, 3]

In [25]: a[1] = 22 # Changing "a"...

In [26]: a
Out[26]: [1, 22, 3]

In [27]: b # ...doesn't affect "b"
Out[27]: [1, 2, 3]
```

By using a list’s `copy` method, you are creating a *shallow copy*: you will get a copy of the list, but if the list contains mutable elements (like nested lists), those inner objects will still be shared. If you want to copy all elements recursively to get a completely independent copy, you need to make a *deep copy* by using the `copy` module from the standard library:

```
In [28]: import copy

         b = copy.deepcopy(a)
```

For further details, please consult the [Python docs](#). Let’s now look at what happens when you use mutable objects as function arguments.

Calling Functions with Mutable Objects as Arguments

If you come from VBA, you are probably used to marking function arguments as pass-by-reference (ByRef) or pass-by-value (ByVal): when you pass a variable to a function as an argument, the function will have the ability to change it (ByRef) or will work on a copy of the values (ByVal), thus leaving the original variable untouched. ByRef is the default in VBA. Consider the following function in VBA:

```
Function increment(ByRef x As Integer) As Integer
    x = x + 1
    increment = x
End Function
```

Then, call the function like this:

```
Sub call_increment()  
    Dim a As Integer  
    a = 1  
    Debug.Print increment(a)  
    Debug.Print a  
End Sub
```

This will print the following:

```
2  
2
```

However, if you change ByRef in the increment function to ByVal, it will print:

```
2  
1
```

How does this work in Python? When you pass around variables, you pass around names that point to objects. This means that the behavior depends on whether the object is mutable or not. Let's first use an immutable object:

```
In [29]: def increment(x):  
         x = x + 1  
         return x  
  
In [30]: a = 1  
         print(increment(a))  
         print(a)  
  
Out[30]: 2  
         1
```

Now let's repeat the sample with a mutable object:

```
In [31]: def increment(x):  
         x[0] = x[0] + 1  
         return x  
  
In [32]: a = [1]  
         print(increment(a))  
         print(a)  
  
Out[32]: [2]  
         [2]
```

If the object is mutable and you would like to leave the original object unchanged, you will need to copy that object in the function before doing anything else.

Another case to watch out for is the use of mutable objects as default arguments in function definitions—let's see why!

Functions with Mutable Objects as Default Arguments

When you write functions, you normally shouldn't use mutable objects as default arguments. The reason is that the value of default arguments is evaluated only once as part of the function definition, not every time when the function is called. Therefore, using mutable objects as default arguments can lead to unexpected behavior:

```
In [33]: # Don't do this:
def add_one(x=[]):
    x.append(1)
    return x
```

```
In [34]: add_one()
```

```
Out[34]: [1]
```

```
In [35]: add_one()
```

```
Out[35]: [1, 1]
```

If you want to use an empty list as a default argument, do this instead:

```
In [36]: def add_one(x=None):
        if x is None:
            x = []
        x.append(1)
        return x
```

```
In [37]: add_one()
```

```
Out[37]: [1]
```

```
In [38]: add_one()
```

```
Out[38]: [1]
```


Symbols

%%time magic command, 199
%%timeit magic command, 199
%smagic magic command, 199
%magic magic command, 199

A

absolute cell references, 135
absolute paths, 19, 241
activating virtual environments, 365
ADBC (Arrow Database Connectivity) drivers, 226
adding dependencies, 363-364
address_to_indices helper function, 335
add_package function (Python), 239, 242
Agent mode (Copilot in Excel), 371-372
agg method (pandas), 114
AI (artificial intelligence), 191, 367
 (see also Copilot)
AI assistants, 32
AI function, 301, 304-307
aliases, 67
Altair, 120
Anaconda, 213
antivirus software, 205
Apache Parquet, 281
API (application programming interface), 220
 (see also web APIs)
api attribute (xlwings), 200
API keys
 obtaining, 301-302
 storing as environment variable, 302-304
app object (xlwings), 178, 182-184
AppleScript, 200

application programming interface (API), 220
 (see also web APIs)
arange function, 85
@arg decorator, 252
arguments, mutable objects as, 377-379
arithmetic operations on DataFrames, 106-108
arrays
 in NumPy
 broadcasting, 81-82
 constructors, 85
 data types, 79-81
 getting and setting elements, 84-85
 Python in Excel and, 149
 ufuncs, 83
 vectorization, 81-82
 views versus copies, 86
 VBA versus Python, 8-9
Arrow Database Connectivity (ADBC) drivers, 226
artificial intelligence (AI), 191, 367
 (see also Copilot)
async functions, 287
Asyncio library, 287-288
attributes, 373
 help information, 50
 methods versus, 103
 purpose of, 45
augmented assignment, 63
autocomplete, 31
await keyword, 287

B

backend
 purpose of, 234

- in Python Package Tracker case study, 238-242
- "batteries included", 5
- Binder, 27
- Black package, 73
- Bokeh, 120
- book object (xlwings), 178-179, 184
- book parameter (xlwings Lite), 279
- boolean data types, 48-49
- boolean indexing, 99-101, 102
- boolean operators, 99
- bound parameters, 230
- break statement, 62
- broadcasting, 81-82, 106
- built-in converters (xlwings), 188
- built-in exceptions, list of, 233
- built-in options (xlwings), 188-190
- buttons
 - creating, 209
 - running scripts from, 279-280
- ByRef function arguments (VBA), 377
- ByVal function arguments (VBA), 377

C

- caching, 265-267
- calamine engine, 90, 322, 333, 335
- calling functions, 45, 65-66, 377-379
- CamelCase, 72
- capitalize method (pandas), 108
- cd command, 19-21
- cells
 - Excel
 - absolute versus relative references, 135
 - converting to Python cells, 130-132
 - Python cells reading from, 134-137
 - in Jupyter notebooks, 27-29
 - Python
 - comments in, 139-140
 - creating, 130-132
 - execution order, 144
 - output types, 132-134
 - reading Excel cells, 134-137
- chaining, 53
- changing
 - DataFrame data, 101-104
 - directories, 19-21
- charts (see plots)
- charts collection (xlwings), 190
- ChatGPT, 367
- class instances, 69, 373-374
- classes, 45, 373-374
- CLASSIFY function, 309-310
- cloud providers of Jupyter notebooks, 27
- cmd command, 17
- code blocks, 58-59
- code cells, 28
- code examples, running, 28
- collections, 178
- colors, finding hex values, 340
- columns
 - adding to DataFrames, 103-104
 - in DataFrames, 94-96
 - selecting, 97
- COM (Component Object Model), 200
- command mode in Jupyter notebooks, 29
- Command Prompt (Windows) (see Terminal)
- commands (Terminal), 17-21
- comments, 47-48, 139-140
- companion repository
 - changing to directory of, 20
 - downloading, 15-17
- complex data type, 46
- composite keys, 225
- concat function (pandas), 109-110
- concatenating
 - DataFrames, 109-110
 - lists, 53
 - strings, 50
 - tuples, 57
- Conda package manager, 365
- conditional expressions, 59-60
- conditional formatting, 237
- configuration settings (xlwings), 214-215
- configuring custom functions, 251
- connection-only Power Query, 166-170
- constant_memory option (XlsxWriter), 340
- constructors, 46, 85
- context managers, 227
- continue statement, 62
- control flow
 - code blocks, 58-59
 - for loops, 60-63
 - if statement, 59-60
 - list comprehensions, 63-64
 - while loops, 62
- conversations, starting in Copilot in Excel, 371-372
- converters, built-in (xlwings), 188

- converting
 - custom functions to DataFrames, 251-257
 - data structures, 58
 - data types
 - Excel/Python conversion, 136
 - with type hints, 295-298
 - Excel cells to Python cells, 130-132
- Coordinated Universal Time (UTC), 375
- copies versus views, 86
- Copilot
 - prompt engineering, 368-370
 - versions of, 367-368
- COPILOT function, 301
- Copilot in Excel, 368
 - requirements, 370
 - starting conversation, 371-372
- corr method (pandas), 162
- correlation in time series analysis, 160-163
- CORS (Cross-Origin Resource Sharing), 288
- CPU time, 199
- create_plot function (pandas), 286
- credentials
 - obtaining API keys, 301-302
 - storing as environment variable, 302-304
- cross-application calls, minimizing, 264-265
- Cross-Origin Resource Sharing (CORS), 288
- cross-platform, Python as, 6
- CSE arrays, 253
- CSV files
 - exporting DataFrames as, 121-122
 - importing to DataFrames, 122-124
- Cursor, 41
- custom add-ins (xlwings), 214
- custom functions
 - importing, 248-251
 - purpose of, 245
 - Python in Excel versus, 248
- xlwings
 - configuring and deploying, 251
 - converting to DataFrames, 251-257
 - date formatting, 260
 - debugging, 262-263
 - downloading web data, 257-260
 - Hello World example, 247-251
 - performance optimization, 264-267
 - plotting with, 260-262
 - requirements, 246
- xlwings Lite
 - AI function example, 304-307

- allowing plots as images, 314
- date formatting, 311-313
- error handling, 298-299
- Hello World example, 294
- organizing, 299-301
- plotting with, 311-316
- sentiment analysis example, 307-310
- troubleshooting, 294
- type hints, 295-298

D

- data alignment (pandas), 107
- data analysis
 - with pandas (see pandas)
 - with xlwings Lite, 280-286
- data classes, 373
- data structures
 - converting, 58
 - dictionaries, 55-56
 - lists, 53-55
 - purpose of, 53
 - sets, 57-58
 - tuples, 56-57
- data types
 - boolean, 48-49
 - comments, 47-48
 - converting
 - Excel/Python conversion, 136
 - with type hints, 295-298
 - DataFrame columns, 91
 - mutable versus immutable, 376
 - numeric, 45-47
 - NumPy arrays, 79-81
 - purpose of, 44
 - strings, 49-51
- data validation, 236
- data viewer, Excel as, 176
- databases in Python Package Tracker case study, 223-231
 - connections, 225-226
 - SQL injection, 229-231
 - SQL queries, 226-229
 - structure of, 224-225
- @dataclass decorator, 373
- DataFrames
 - adding columns, 103-104
 - arithmetic operations, 106-108
 - column data types, 91
 - columns, 94-96

- concatenating, 109-110
- converting custom functions to, 251-257
- creating, 91-92
- datetime attributes, 157
- descriptive statistics, 113-114
- duplicate data, 105-106
- Excel spreadsheets versus, 90-91
- exporting
 - as CSV file, 121-122
 - methods, 121
- formatting, 354-357
- grouping, 114-115
- importing
 - as CSV file, 122-124
 - methods, 121
- index for, 92-94
- joining and merging, 110-113
- methods, 93
- missing data, 104-105
- pivoting data, 115-116
- plotting data, 117-120
 - list of plotting libraries, 120
 - with Matplotlib, 117-118
 - with Plotly, 118-119
- in Python in Excel, 137-139, 145-146
- reading/writing, 186-187
- selecting data, 96-101
- Series versus, 89, 97
- setting data, 101-104
- SQL queries versus, 227
- text manipulation, 108-109
- date serial number (Excel), 68
- dates, formatting in custom functions, 260, 311-313
- datetime attributes, accessing, 157
- datetime module, 68-70, 375-376
- DatetimeIndex
 - creating, 154-157
 - filtering, 157-158
- date_range function (pandas), 154-157
- debuggers, 32, 38-39
- debugging
 - custom functions, 262-263
 - Python Package Tracker case study, 242-243
- decimal data type, 46
- decorating functions
 - list of decorators, 268
 - order of decorators, 256
 - purpose of, 248
 - with script decorator, 278
- deep copies, 377
- default values in dictionaries, 56
- defined names, 192-194
- defining functions, 64-65
- deleting list elements, 54
- dependencies, 10, 363-364
- deploying
 - custom functions, 251
 - xlwings tools, 212-215
- describe method (pandas), 122
- descriptive statistics, 113-114
- deterministic functions, 265
- development environment setup
 - companion repository download, 15-17
 - Jupyter notebooks, 24-31
 - cells in, 27-29
 - edit versus command mode, 29
 - run order, 30-31
 - running, 25-27
 - Python and packages installation, 22-23
 - Python REPL startup, 23-24
 - Terminal usage, 17-21
 - uv installation, 21
 - VS Code, 31-41
 - advantages of, 31-33
 - alternatives to, 40-41
 - debugger, 38-39
 - installing, 33-35
 - running Jupyter notebooks, 39-40
 - running Python scripts, 35-38
- df method (DuckDB), 282, 284
- dictionaries, 55-56
- dir command, 18
- directories
 - changing, 19-21
 - compiling into report, 194-198
 - listing files, 18
- DistilBERT Base Uncased Finetuned SST-2
 - English model, 308
- docstrings, 71
- documentation, 50
- domain-specific languages, 4
- downloading
 - companion repository, 15-17
 - web data with custom functions, 257-260
- downsampling, 163
- drivers, 225
- dropna method (pandas), 104

- drop_duplicates method (pandas), 106
- dt accessor, 157
- DuckDB, 125, 280-286
- duplicate data, 105-106
- duplicated method (pandas), 106
- dynamic arrays
 - Excel support for, 253, 256
 - minimizing cross-application calls with, 264-265
 - spilling, 133
- dynamic typing, 44

E

- edit mode in Jupyter notebooks, 29
- editing files
 - with OpenPyXL, 346
 - with xlutils, 354
- editor in Python in Excel, 140-141
- enabling macros, 185, 208, 218, 252
- endpoints, 222
- enumerate function, 61
- environment variables, 215, 302-304
- error handling
 - conditional formatting and, 237
 - in Python Package Tracker case study, 231-234
 - VBA versus Python, 9-10
 - xlwings Lite custom functions, 298-299
- escaping characters in strings, 50
- Excel
 - advantages of Python, 4-12
 - cross-platform, 6
 - general-purpose language, 4
 - modern language features, 8-10
 - package management, 10-11
 - readability, 4-5
 - scientific computing, 6-7
 - standard library, 5
 - testing framework, 11
 - version control, 11-12
 - array-based calculations, 82
 - cells
 - absolute versus relative references, 135
 - converting to Python cells, 130-132
 - Python cells reading from, 134-137
 - charts in xlwings, 190
 - Copilot in Excel, 368
 - requirements, 370
 - starting conversation, 371-372

- as data viewer, 176
- date serial number, 68
- float data type, 46
- formula bar, expanding, 138
- as frontend, 203
- history of, 3
- Jupyter notebooks versus, 25
- language and regional settings, 178
- macros, enabling, 185, 208, 218, 252
- object model, 177-184
- pausing, 263
- Python in Excel (see Python in Excel)
- running multiple instances, 178
- as sandboxed in macOS, 183
- spreadsheets versus DataFrames, 90-91
- values, as output type, 133-134

- ExcelFile class (pandas), 323-327
- Excelize, 334, 347-348
- ExcelWriter class (pandas), 329-331
- exceptions (see error handling)
- execution order of Python cells, 144
- expand method (xlwings), 187
- expanding formula bar, 138
- explore function (DuckDB), 281
- exporting DataFrames
 - as CSV file, 121-122
 - methods, 121
- extensions for VS Code, installing, 35

F

- f-strings, 50
- False data type, 48-49
- fastexcel, 333, 336-337
- ffill method (pandas), 164
- figure object (xlwings), 191
- file extensions, viewing, 19
- file paths, raw strings for, 122
- files
 - editing
 - with OpenPyXL, 346
 - with xlutils, 354
 - listing in directories, 18
 - loading with Power Query, 166-170
 - naming, 68
 - opening in VS Code, 38
 - reading
 - in calamine, 322
 - with Excelize, 347-348
 - with fastexcel, 336-337

- limitations in pandas, 331
- with OpenPyXL, 341-343
- in pandas, 323-329
- in parallel, 327-329
- with python-calamine, 335-336
- with pyxlsb, 348-350
- with xlrd, 350-352
- running, 35-38, 278-280
- saving, 250
- writing
 - with Excelize, 347-348
 - limitations in pandas, 331
 - with OpenPyXL, 343-346
 - in pandas, 329-331
 - with XlsxWriter, 337-341
 - with xlwt, 352-353
- fillna method (pandas), 105
- filtering DatetimeIndex, 157-158
- float data type, 45-47
- floating-point inaccuracies, 46
- floating-point numbers, 45-47
- for loops, 60-63
- foreign keys (databases), 225
- formatted string literals (f-strings), 50
- formatting
 - DataFrames, 354-357
 - dates in custom functions, 260, 311-313
 - datetime objects, 69
 - PEP 8 style guide examples, 70-72
 - Ruff package examples, 72-73
- formula bar, expanding, 138
- forward-filling, 164
- fraction data type, 46
- frequency strings, 163
- frontend, 203
 - purpose of, 234
 - in Python Package Tracker case study, 234-238
- frozen executables, 213
- full outer joins, 111
- @func decorator, 248, 249
- Function Wizard, 258, 299
- functions
 - calling, 45, 65-66, 377-379
 - custom (see custom functions)
 - decorating
 - list of decorators, 268
 - order of decorators, 256
 - purpose of, 248

- with script decorator, 278
- defining, 64-65
- deterministic, 265
- as methods, 45
- purpose of, 64
- recalculating, 249
- universal, 83
- volatile, 249

G

- GenAI (generative artificial intelligence), 367
 - (see also Copilot)
- general-purpose languages, 4
- generative artificial intelligence (GenAI), 367
 - (see also Copilot)
- GET requests, 220
- Git, 11-12
- GitHub Copilot, 368
- globbing, 197
- Google Colab, 27
- GoTo statements (VBA), 10
- graphical user interface (GUI), 203
- gridlines, hiding, 237
- groupby method (pandas), 114-115
- grouping DataFrames, 114-115
- GUI (graphical user interface), 203

H

- hallucinations, 369
- head method (pandas), 122
- heatmaps, 163
- Hello World example
 - xlwings custom functions, 247-251
 - xlwings Lite custom functions, 294
- hexadecimal, 339
- hiding gridlines, 237
- HoloViews, 120
- homogeneous data, 80
- horizontal one-dimensional arrays, 134
- HTTP status codes, 222
- HTTPX, 222
- Hugging Face, 307-310
- hvPlot, 120

I

- IDEs (integrated development environments), 32, 40-41
- if statement, 59-60, 241

- `iloc` attribute (pandas), 98
- images, allowing plots as, 314
- immutability, 56, 376-379
- implicit line breaks, 54
- importing
 - custom functions, 248-251
 - DataFrames
 - as CSV file, 122-124
 - methods, 121
 - modules, 66-68
- indentation rules (Python), 5
- indexes for DataFrames, 92-94
- indexing, 51-52
- `info` method (pandas), 91, 122
- initialization script in Python in Excel, 144
- inner joins, 111
- `insert` function (pandas), 93
- inserting
 - key-value pairs, 55
 - list elements, 54
- installing
 - extensions for VS Code, 35
 - packages, 22-23, 289-290
 - Python, 22-23, 212-213
 - Ruff extension, 74
 - uv package manager, 21
 - VS Code, 33-35
 - xlwings add-in, 204-206
 - xlwings Lite, 275-276
- instantiation, 45, 69, 373
- `int` data type, 45-47
- integers, 45-47
- integrated development environments (IDEs), 32, 40-41
- IntelliSense, 31
- interactive Python sessions, starting, 23-24
- `isna` method (pandas), 104
- `is_unique` method (pandas), 105

J

- JavaScript Object Notation (JSON), 220
- `join` function (pandas), 110-113
- joining DataFrames, 110-113
- JSON (JavaScript Object Notation), 220
- Jupyter notebooks, 15, 24-31
 - cells in, 27-29
 - cloud providers, 27
 - comments, 48
 - edit versus command mode, 29

- magic commands, 199
- restarting, 67
- run order, 30-31
- running, 25-27, 39-40

JupyterLab, 41

K

- Kaggle, 27
- key-value pairs, 55-56
- keyboard shortcuts
 - in Jupyter notebooks, 29
 - in Python in Excel, 147
- keyword arguments, 65, 91

L

- labels (pandas)
 - selecting data via, 96-97
 - setting data type, 101
- labels (VBA), 10
- lambda expressions, 115
- language settings, 178
- large language models (LLMs), 367, 369
- left joins, 111
- legacy CSE arrays, 253
- line continuation, 54
- linting, 73-74
- Linux, 6
- list comprehensions, 63-64
- list separator, 178
- listing files in directories, 18
- lists, 53-55, 149
- literals, 57, 69
- LLMs (large language models), 367, 369
- LN function, 159
- loading data with Power Query, 166-170
- `load_parquet` function (DuckDB), 282, 287
- `loc` attribute (pandas), 96-97
- LOG function, 159
- `log` function (NumPy), 159
- log returns, 158
- `log10` function (NumPy), 159
- loops
 - for, 60-63
 - while, 62
- Lotus 1-2-3, 3
- `ls` command, 18
- lxml package, 346

M

- machine learning, 308
- macOS
 - AppleScript, 200
 - changing to companion repository direc-
tory, 20
 - companion repository download, 16
 - Excel sandboxed in, 183
 - Jupyter notebooks, 25
 - opening Terminal, 18
 - permissions, 20
 - Python installation, 22
 - running Python scripts, 37
 - starting Python REPL, 24
 - starting VS Code, 35
 - uv installation, 21
 - viewing file extensions, 19
 - VS Code extensions, 35
 - VS Code installation, 33
 - xlwings ribbon add-in, 206
 - xlwings, permissions/preferences for, 177
- macros, enabling, 185, 208, 218, 252
- magic commands, 199
- marimo notebooks, 31
- Markdown cells, 28, 48
- mathematical operators, 47
- Matplotlib, 117-118
 - with Python in Excel, 142-144
 - in xlwings, 190-191
- matrixes, 53, 82
- mean method (pandas), 113
- memoization, 266
- merge function (pandas), 110-113
- merging
 - DataFrames, 110-113
 - dictionaries, 56
- method chaining, 93
- methods, 373
 - attributes versus, 103
 - in DataFrames, 93
 - help information, 50
 - importing and exporting DataFrames, 121
 - purpose of, 45
 - for strings, 50
- micropip package manager, 290
- Microsoft 365 Copilot, 368
- Microsoft Copilot, 367
- minimizing cross-application calls, 264-265
- missing data, 104-105

- module search path (Python), 210

- modules

- datetime, 68-70, 375-376
- importing, 66-68

- moving average, 165
- Multiplan, 3
- multiple Excel instances, running, 178
- multiplying matrixes, 82
- mutable objects, 376-379

N

- named arguments, 65
- named ranges, 192-194, 235
- namespaces, 299-301
- naming
 - with CamelCase, 72
 - files, 68
 - projects, 206
 - variables, 44
- NaN (Not a Number), 104-105
- ndim option (xlwings), 189-190
- nested lists, 53
- None constant, 49
- normalization, 223
- Not a Number (NaN), 104-105
- notebooks (see Jupyter notebooks)
- Notepad++, 40
- numeric data types, 45-47
- NumFOCUS, 7
- NumPy
 - arrays
 - broadcasting, 81-82
 - constructors, 85
 - data types, 79-81
 - getting and setting elements, 84-85
 - in Python in Excel, 149
 - ufuncs, 83
 - vectorization, 81-82
 - views versus copies, 86
 - release of, 3
- NYC taxi dataset, analyzing with xlwings Lite,
280-286

O

- objects
 - attributes and methods, 45
 - as class instances, 69, 373-374
 - help information, 50
 - mutable versus immutable, 376-379

- as output type, 132
- purpose of, 44
- Office.js, 274
- one-based indices, 182
- one-dimensional arrays, horizontal, 134
- one-to-many relationships, 225
- open source software (OSS), 7
- Open-Meteo, 311-316
- OpenAI API key
 - obtaining, 301-302
 - storing as environment variable, 302-304
- OpenAI models, list of, 304
- opening
 - files in VS Code, 38
 - Terminal, 17
- OpenPyXL, 334, 341-347, 356-357
- operators
 - boolean, 48-49, 99
 - mathematical, 47
 - ternary, 60
- optimizing
 - custom functions, 264-267
 - in xlwings, 199-200
- options method (xlwings), 187, 188
- options, built-in (xlwings), 188-190
- order of function decorators, 256
- organizing xlwings Lite custom functions, 299-301
- OSS (open source software), 7
- outer joins, 111
- output types for Python cells, 132-134
- output, viewing in Python in Excel, 156

P

- package management in Python, 10-11, 365
- packages
 - adding/removing, 217-220, 363-364
 - installing, 22-23, 289-290
 - reader/writer packages
 - Excelize, 347-348
 - fastexcel, 336-337
 - OpenPyXL, 341-347
 - python-calamine, 335-336
 - pyxlsb, 348-350
 - when to use, 333-335
 - xlrd, 350-352
 - XlsxWriter, 337-341
 - xlutils, 354
 - xlwt, 352-353

- upgrading, 364
- pandas, 3
 - alternatives to, 125
 - DataFrames
 - adding columns, 103-104
 - arithmetic operations, 106-108
 - column data types, 91
 - columns, 94-96
 - concatenating, 109-110
 - converting custom functions to, 251-257
 - creating, 91-92
 - datetime attributes, 157
 - descriptive statistics, 113-114
 - duplicate data, 105-106
 - Excel spreadsheets versus, 90-91
 - formatting, 354-357
 - grouping, 114-115
 - importing and exporting, 121-124
 - index for, 92-94
 - joining and merging, 110-113
 - methods, 93
 - missing data, 104-105
 - pivoting data, 115-116
 - plotting data, 117-120
 - in Python in Excel, 137-139, 145-146
 - reading/writing, 186-187
 - selecting data, 96-101
 - Series versus, 89, 97
 - setting data, 101-104
 - SQL queries versus, 227
 - text manipulation, 108-109
- Excel files
 - limitations of, 331
 - reading, 323-329
 - writing, 329-331
- limitations of, 125
- reader/writer packages
 - Excelize, 347-348
 - fastexcel, 336-337
 - OpenPyXL, 341-347
 - python-calamine, 335-336
 - pyxlsb, 348-350
 - when to use, 333-335
 - xlrd, 350-352
 - XlsxWriter, 337-341
 - xlutils, 354
 - xlwt, 352-353
- time series analysis
 - correlation, 160-163

- creating DatetimeIndex, 154-157
 - Excel limitations, 154
 - filtering DatetimeIndex, 157-158
 - percentage changes, 158-160
 - rebasng, 160-163
 - resampling, 163-164
 - rolling windows, 165
 - shifting, 158-160
- Parquet, 281
- pass statement, 58-59
- pausing Excel, 263
- pct_change function (pandas), 160
- PEP 8 style guide
 - examples, 70-72
 - Ruff package for, 72-74
- percentage changes in time series analysis, 158-160
- performance optimization
 - of custom functions, 264-267
 - in xlwings, 199-200
- permissions (macOS), 20
- pictures collection (xlwings), 190-191
- pip package manager, 290, 365
- pivoting DataFrame data, 115-116
- PivotTables, 347-348
- pivot_table function (pandas), 115-116
- Pixi package manager, 365
- plot method (pandas), 117
- Plotly, 118-119
- Plotnine, 120, 143
- plots, 117-120
 - allowing as images, 314
 - list of plotting libraries, 120
 - in Matplotlib, 117-118
 - in Plotly, 118-119
 - types of, 119
 - in xlwings, 190-191
- plotting
 - with custom functions, 260-262, 311-316
 - DataFrame data, 117-120
 - list of plotting libraries, 120
 - with Matplotlib, 117-118
 - with Plotly, 118-119
 - with Python in Excel, 142-144
 - tips for, 143
- Polars, 125, 337
- portfolio variance example, 6
- position
 - selecting data by, 98
 - setting data by, 101
- positional arguments, 65
- Positron, 40
- POST requests, 220
- Power Query
 - loading data with, 166-170
 - xlwings versus, 198
- pretrained models, 308
- preview mode, 38
- primary keys, 225
- project flag, 365
- projects
 - creating in uv package manager, 361-363
 - naming, 206
 - renaming, 210
 - working outside project directory, 365
- prompt engineering, 368-370
- pseudorandom numbers, 85
- pull requests, 12
- pwd command, 18, 20
- pycache folder, 68
- PyCharm, 40
- PyDev, 41
- pyfetch, 288
- Pylance, 75
- Pyodide, 274, 288
- PyPI, 10, 221
- PyPy, 10
- Python
 - advantages for Excel, 4-12
 - cross-platform, 6
 - general-purpose language, 4
 - modern language features, 8-10
 - package management, 10-11
 - readability, 4-5
 - scientific computing, 6-7
 - standard library, 5
 - testing framework, 11
 - version control, 11-12
 - chaining, 53
 - classes, 45, 373-374
 - control flow
 - code blocks, 58-59
 - for loops, 60-63
 - if statement, 59-60
 - list comprehensions, 63-64
 - while loops, 62
 - data structures
 - converting, 58

- dictionaries, 55-56
- lists, 53-55
- purpose of, 53
- sets, 57-58
- tuples, 56-57
- data types
 - boolean, 48-49
 - comments, 47-48
 - mutable versus immutable, 376
 - numeric, 45-47
 - purpose of, 44
 - strings, 49-51
- files
 - naming, 68
 - running, 35-38
 - saving, 250
- functions
 - calling, 45, 65-66, 377-379
 - defining, 64-65
 - purpose of, 64
- history of, 3
- indentation rules, 5
- indexing, 51-52
- installing, 22-23, 212-213
- module search path, 210
- modules
 - datetime, 68-70, 375-376
 - importing, 66-68
- objects
 - attributes and methods, 45
 - as class instances, 69, 373-374
 - help information, 50
 - mutable versus immutable, 376-379
 - as output type, 132
 - purpose of, 44
- as open source, 7
- PEP 8 style guide
 - examples, 70-72
 - Ruff package for, 72-74
- slicing, 52-53
- type hints
 - for book parameter, 279
 - converting data types, 295-298
 - purpose of, 74-76
 - in Python Package Tracker case study, 238
- variables, 44
- Python cells
 - comments in, 139-140
 - creating, 130-132
 - execution order, 144
 - output types, 132-134
 - reading Excel cells, 134-137
- Python editor, 140-141
- Python in Excel
 - add-on, 150
 - custom functions versus, 248
 - DataFrames in, 137-139, 145-146
 - initialization script, 144
 - keyboard shortcuts, 147
 - limitations of, 148-150
 - loading data with Power Query, 166-170
 - output, 156
 - plotting with, 142-144
- Python cells
 - comments in, 139-140
 - creating, 130-132
 - execution order, 144
 - output types, 132-134
 - reading Excel cells, 134-137
- Python editor in, 140-141
- requirements, 129-130
- settings, 147
- xl function, 134-137
- Python Package Tracker case study (xlwings)
 - application structure, 234-243
 - backend, 238-242
 - debugging, 242-243
 - frontend, 234-238
 - databases in, 223-231
 - connections, 225-226
 - SQL injection, 229-231
 - SQL queries, 226-229
 - structure of, 224-225
 - error handling, 231-234
 - packages, adding, 217-220
 - web APIs in, 220-223
- Python REPL, 23-24
- Python Software Foundation, 7
- python-calamine, 333, 335-336
- PYTHONPATH environment variable (xlwings), 210, 215
- PyXLL, 269
- pyxlsb, 348-350

Q

- Quandl, 122
- quickstart command (xlwings), 206, 247

- with RunPython function, 208-211
- RunPython function without, 211-212
- standalone workbooks, 213
- quotation marks in strings, 49

R

- random numbers, 85
- range function, 60
- range object (xlwings), 179-182, 188
- raw strings, 122
- readability of Python, 4-5
- reader/writer packages
 - Excelize, 347-348
 - fastexcel, 336-337
 - OpenPyXL, 341-347
 - python-calamine, 335-336
 - pyxlsb, 348-350
 - when to use, 333-335
 - xlrd, 350-352
 - XlsxWriter, 337-341
 - xlutils, 354
 - xlwt, 352-353
- reading
 - DataFrames, 186-187
 - Excel cells with Python, 134-137
 - files
 - in calamine, 322
 - with Excelize, 347-348
 - with fastexcel, 336-337
 - limitations in pandas, 331
 - with OpenPyXL, 341-343
 - in pandas, 323-329
 - in parallel, 327-329
 - with python-calamine, 335-336
 - with pyxlsb, 348-350
 - with xlrd, 350-352
- read_csv function (pandas), 122-124
- read_excel function (pandas), 90, 323-327
- read_excel_parallel function (pandas), 328
- read_json method (pandas), 223
- rebasings in time series analysis, 160-163
- recalculating functions, 249
- recursive globbing, 197
- regional settings, 178
- reindex method (pandas), 93
- relational API for DuckDB, 282
- relational database systems, 223
- relations (DuckDB), 282
- relative cell references, 135

- relative paths, 19, 241
- removing
 - dependencies, 363-364
 - Python cells, 131
 - xlwings Lite, 276
- renaming
 - projects, 210
 - tables, 236
- repeating strings, 50
- REPL (read-eval-print loop), 23
- replace method (pandas), 102-103
- report function (DuckDB), 284-285
- reporting case study (xlwings), 194-198, 319-322, 357
- resample method (pandas), 163
- resampling in time series analysis, 163-164
- reset_index function (pandas), 92
- REST APIs (see web APIs)
- restarting Jupyter notebooks, 67
- @ret decorator, 255-256
- return decorator, 255-256
- right joins, 111
- rolling method (pandas), 165
- rolling windows in time series analysis, 165
- row-major order of cell execution, 144
- Ruff package
 - formatting examples, 72-73
 - installing extension, 74
 - linting examples, 73-74
- Run main button (xlwings), 206-207
- run order in Jupyter notebooks, 30-31
- Run Python File button, 38
- running
 - code examples, 28
 - files, 35-38, 278-280
 - Jupyter notebooks, 25-27, 39-40
 - multiple Excel instances, 178
 - VBA code in xlwings, 184-186
- RunPython function (xlwings), 208-212, 215
- Rust, 335

S

- sandboxing, 183
- saving files, 250
- scalar, 82
- scientific computing in Python, 6-7
- @script decorator, 267-268, 278
- scripts (see files)
- scrolling previous commands, 21

- seaborn, 120, 143-144
- seeds, 85
- selecting DataFrame data, 96-101
- sentiment analysis, 307-310
- Series versus DataFrames, 89, 97
- sets, 57-58
- setting DataFrame data, 101-104
- set_index function (pandas), 92
- shallow copies, 377
- shared drives, 213
- sheet buttons, running scripts from, 279-280
- sheet object (xlwings), 178, 181
- shift method (pandas), 159
- shifting in time series analysis, 158-160
- Show Console checkbox (Windows), 243
- show method (DuckDB), 282
- significant white space, 58
- slicing, 52-53
- sort_index method (pandas), 94
- sort_values method (pandas), 94
- spaghetti code, 10
- spilling, 133
- spreadsheets (Excel) versus DataFrames, 90-91
 - (see also files)
- Spyder, 40
- SQL injection, 229-231
- SQL queries, 226-229
- SQLAlchemy, 225-226
- SQLite, 223, 225
- squeeze method (DuckDB), 284
- standalone workbooks, 213-214
- standard library in Python, 5
- starting
 - Copilot in Excel conversation, 371-372
 - interactive Python sessions, 23-24
- startswith method (pandas), 109
- state, 44
- status codes (HTTP), 222
- strings, 8, 49-51
- strip method (pandas), 108
- structured references, 137
- Styler objects, 321
- Sublime, 40
- syntax highlighting, 28, 31

T

- tables, renaming, 236
- tail method (pandas), 122
- Terminal

- commands overview, 17-21
 - keeping open, 243
 - long file paths, 37
 - running Python scripts, 36-37
 - starting VS Code, 35
 - in VS Code, 37
- ternary operators, 60
- testing framework in Python, 11
- text editors
 - list of, 40-41
 - VS Code (see VS Code)
- text handling, VBA versus Python, 8
- text manipulation (pandas), 108-109
- time series analysis
 - correlation, 160-163
 - DatetimeIndex
 - creating, 154-157
 - filtering, 157-158
 - Excel limitations, 154
 - percentage changes, 158-160
 - rebasng, 160-163
 - resampling, 163-164
 - rolling windows, 165
 - shifting, 158-160
- time series, purpose of, 153
- time zones, 375-376
- to_csv method (pandas), 121-122
- to_excel function (pandas), 329-331
- transformers, 308
- troubleshooting
 - custom functions, 294
 - Python installation, 23
- True data type, 48-49
- try/except statements, 231-233
- tuples, 56-57
- Ty, 75
- type annotations (see type hints)
- type hints
 - for book parameter, 279
 - converting data types, 295-298
 - purpose of, 74-76
 - in Python Package Tracker case study, 238
- tzdata package, 375

U

- UDF Server, RunPython function with, 215
- UDFs (see custom functions)
- ufuncs (universal functions), 83
- UI (user interface), 203, 276-278

- uninstalling (see removing)
- UNIQUE function, 256
- unique method (pandas), 105
- unit tests in Python, 11
- universal functions (ufuncs), 83
- updating
 - packages, 364
 - xlwings, 204
- upsampling, 163
- user interface (UI), 203, 276-278
- user-defined functions (see custom functions)
- UTC (Coordinated Universal Time), 375
- uv package manager, 289
 - adding/removing dependencies, 363-364
 - creating projects, 361-363
 - installing, 21
 - upgrading, 364
 - working outside project directory, 365

V

- values (Excel), as output type, 133-134
- variables, 44
 - as attributes, 45
 - keyword arguments and, 91
 - strings and, 50
 - xl function, 136
- VBA (Visual Basic for Applications)
 - ByRef and ByVal function arguments, 377
 - Excel object model, 177-184
 - Python advantages over, 4-12
 - cross-platform, 6
 - general-purpose language, 4
 - modern language features, 8-10
 - package management, 10-11
 - readability, 4-5
 - scientific computing, 6-7
 - standard library, 5
 - testing framework, 11
 - version control, 11-12
 - running code in xlwings, 184-186
 - xlwings (see xlwings)
- VCS (version control system), 11-12, 32
- vectorization, 81-82
- version control system (VCS), 11-12, 32
- view function (xlwings), 176
- viewing file extensions, 19
- views versus copies, 86
- virtual environments, 23, 365
- VisiCalc, 3

- Visual Basic for Applications (see VBA)
- Visual Studio, 33
- Visual Studio Code (see VS Code)
- volatile functions, 249
- VS Code, 15, 31-41
 - advantages of, 31-33
 - alternatives to, 40-41
 - debugger, 38-39
 - installing, 33-35
 - opening files, 38
 - Ruff extension, 74
 - running Jupyter notebooks, 39-40
 - running Python scripts, 35-38

W

- waiting on applications pop-up, 263
- wall time, 199
- Wasm, 274
- web APIs
 - in Python Package Tracker case study, 220-223
 - VBA versus Python, 9
 - xlwings Lite requests, 286-289
- web data, downloading with custom functions, 257-260
- while loops, 62
- Windows
 - changing to companion repository directory, 20
 - COM (Component Object Model), 200
 - companion repository download, 16
 - connection-only Power Query, 166-170
 - frozen executables, 213
 - Jupyter notebooks, 25
 - opening Terminal, 17
 - Python installation, 22
 - raw strings for file paths, 122
 - running Python scripts, 37
 - RunPython function with UDF Server, 215
 - Show Console checkbox, 243
 - starting Python REPL, 23
 - starting VS Code, 35
 - uv installation, 21
 - viewing file extensions, 19
 - VS Code extensions, 35
 - VS Code installation, 33
 - xlwings ribbon add-in, 205
- Wing Python IDE, 41
- WINGMAN function, 305

with statement (Python), 227

writing

 DataFrames, 186-187

 files

 with Excelize, 347-348

 limitations in pandas, 331

 with OpenPyXL, 343-346

 in pandas, 329-331

 with XlsxWriter, 337-341

 with xlwt, 352-353

X

xl function (Python in Excel), 134-137, 141, 149

xlOil, 269

xlrd, 334, 350-352

xlSlim, 269

XLSTART folder, 205

XlsxWriter, 334, 337-341, 354-356

xlutils, 354

xlwings

 add-in, 204-206

 alternatives to, 269

 built-in converters, 188

 built-in options, 188-190

 configuration settings, 214-215

 custom add-ins, 214

 custom functions

 configuring and deploying, 251

 converting to DataFrames, 251-257

 date formatting, 260

 debugging, 262-263

 downloading web data, 257-260

 Hello World example, 247-251

 performance optimization, 264-267

 plotting with, 260-262

 requirements, 246

 DataFrames, reading/writing, 186-187

 defined names, 192-194

 deploying tools, 212-215

 Excel as data viewer, 176

 Excel charts in, 190

 Excel object model, 177-184

 function decorators, 248, 268

 macOS permissions/preferences, 177

 Matplotlib plots in, 190-191

 missing functionality, 200-201

 performance optimization, 199-200

Power Query versus, 198

Python Package Tracker case study

 application structure, 234-243

 databases in, 223-231

 error handling, 231-234

 packages, adding, 217-220

 web APIs in, 220-223

quickstart command, 206, 247

 with RunPython function, 208-211

 RunPython function without, 211-212

 standalone workbooks, 213

reporting case study, 194-198, 319-322

Run main button, 206-207

running VBA code, 184-186

RunPython function, 208-212

standalone workbooks, 213-214

updating, 204

xlwings Lite

 custom functions

 AI function example, 304-307

 allowing plots as images, 314

 date formatting, 311-313

 error handling, 298-299

 Hello World example, 294

 organizing, 299-301

 plotting with, 311-316

 sentiment analysis example, 307-310

 troubleshooting, 294

 type hints, 295-298

 data analysis with, 280-286

 installing, 275-276

 limitations of, 290-291

 operational overview, 274-275

 Python package installation, 289-290

 removing, 276

 requirements, 273

 running scripts, 278-280

 user interface, 276-278

 web API requests, 286-289

xlwings Server, 291

xlwt, 334, 352-353

Z

Zed, 40

The Zen of Python, 4

zero-based indices, 182

About the Author

Felix Zumstein is an independent software developer and the founder of Zoomer Analytics GmbH. He pioneered the integration of Python and Excel with his open source project *xlwings*.

His work, which predates Microsoft's own "Python in Excel" feature by a decade, has enabled countless companies across the globe to connect Excel with Python. Felix is also a LinkedIn instructor and runs the *xlwings* meetup in London.

Colophon

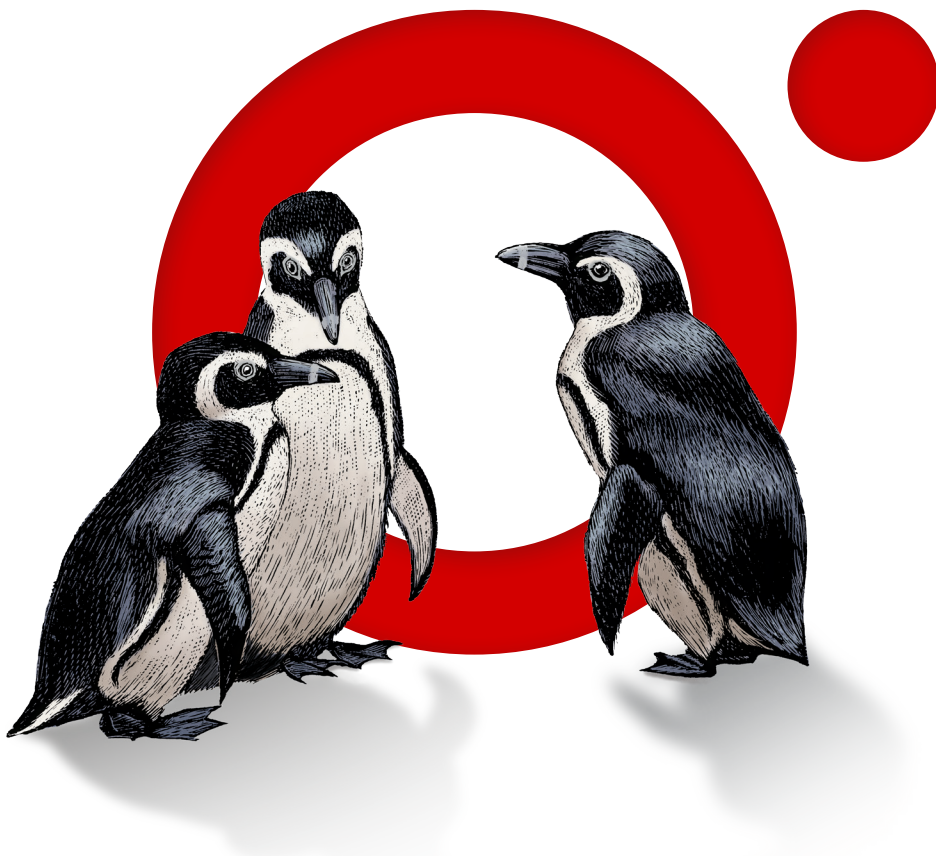
The animal on the cover of *Python for Excel* is a false coral snake (*Anilius scytale*). This brightly colored snake, also known as the American pipe snake, is found in the Guianas region of South America, the Amazon rainforest, and Trinidad and Tobago.

The false coral snake grows up to around 70 cm in length and has bright red- and black bands. Its banded appearance is similar to that of the coral snake, from whom it derives one of its common names; however, the false coral snake is missing the distinctive yellow bands of the "true" coral snake. Its body has roughly the same diameter for most of its length with a very short tail, giving it a tube-like appearance. Its small eyes are covered by large head scales.

This burrowing snake has been observed to be ovoviviparous. It feeds on beetles, amphibians, lizards, fish, and other small snakes. The false coral snake also retains spurs, or small protruding bits of bone near its vent, which are vestigial traces of hips and occasionally upper leg bones. This feature, along with the thick bones and distinctive shape of its skull, makes the false coral snake closely resemble snakes' ancestral, lizard-like condition.

The conservation status of the false coral snake is "Data Deficient," meaning that there is not yet enough information to judge the threat of its extinction. Many of the animals on O'Reilly covers are endangered; all of them are important to the world.

The cover illustration is by Karen Montgomery, based on a black-and-white engraving from *English Cyclopedia Natural History*. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.



O'REILLY®

**Learn from experts.
Become one yourself.**

60,000+ titles | Live events with experts | Role-based courses
Interactive learning | Certification preparation | Verifiable skills

Try the O'Reilly learning platform free for 10 days.

